

**Денисов К.М.**

**Конспект лекций по курсу  
«Основы цифровой схемотехники»**

**Санкт-Петербург**

**2025**

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ

УНИВЕРСИТЕТ ИТМО

**К. М. Денисов**

**Конспект лекций по курсу  
«Основы цифровой схемотехники»**

Учебное пособие



**Санкт-Петербург**

2025

## Лекция №1

### **“Аналоговые и цифровые методы обработки информации. Их сопоставление. Достоинства и недостатки. Области применения. Современное состояние вопроса. История развития микропроцессоров и их роль в современной преобразовательной технике”.**

Практически любое инженерное устройство имеет целью своего функционирования то или иное преобразование энергии или преобразование информации. Задачей любой системы управления в самом общем смысле является обработка информации о текущем режиме работы управляемого объекта и выработка на основе этого управляющих сигналов с целью приближения текущего режима работы объекта к заданному. Под обработкой информации в данном случае подразумевается решение тем или иным способом уравнений состояния системы.

В электронных устройствах существуют два основных способа обработки информации: аналоговый и цифровой.

При аналоговом способе обработки информации каждой переменной величине в системе ставится в соответствие один из плавно меняющихся параметров определенного участка электрической цепи (ток, напряжение, частота и т.д.). Функциональные зависимости между различными переменными в системе реализуются путем построения соответствующих электрических цепей. Простейшие функциональные зависимости можно реализовать на основе законов функционирования элементарных пассивных элементов электрической цепи:

- для резистора  $e = i * R$ ;
- для конденсатора  $e = \left( \frac{1}{C} \right) * \int i dt$ ;
- для индуктивности  $e = L * \left( \frac{di}{dt} \right)$ .

Принципиальной особенностью аналогового способа обработки информации является возможность плавного (в известных пределах) изменения величин электрических сигналов, соответствующих переменным системы. Все преобразования осуществляются практически мгновенно.

При цифровом способе обработки информации каждой переменной величине в системе ставится в соответствие ее цифровой код. Функциональные зависимости в системе реализуются путем непосредственного решения уравнений системы теми или иными численными методами по заранее заложенной программе. Устройство, реализующее это решение называется процессором.

Первый процессор, как программно функционирующее устройство, способное выполнять арифметические и логические операции, а также осуществлять ветвление алгоритма своего функционирования в зависимости от результата предыдущих вычислений, был создан в 40-е годы нашего столетия в США специалистами фирмы ИВМ. Он представлял собой устройство на электромеханических реле, занимал несколько этажей здания, имел крайне низкое быстродействие и надежность, и был пригоден лишь для очень узкого класса специфических вычислений. По мере прогресса электронной техники совершенствовалась и элементная база для построения процессоров. Появлялись процессоры на электронных лампах, транзисторах, дискретных логических микросхемах малой степени интеграции. По мере совершенствования процессоры имели все меньшие габаритные размеры, потребляли все меньше энергии, обладали все большей производительностью и надежностью. Однако они все еще были мало пригодны для выполнения операций управления в реальном масштабе времени, а по тому использовались в основном только для определенного класса вычислительных задач.

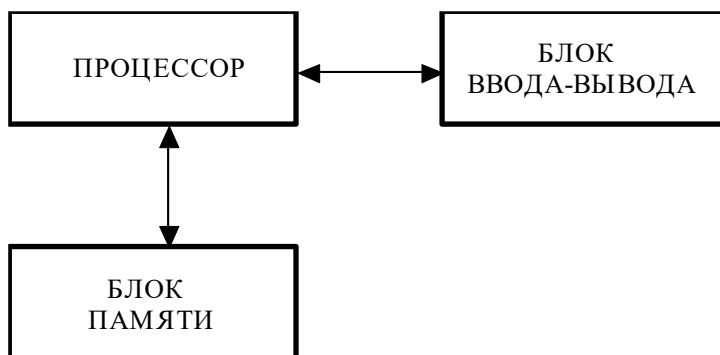
Настоящая революция в вычислительной технике произошла после появления первого т.н. “микропроцессора”, т.е. процессора, выполненного в виде одной микросхемы большой степени интеграции. Это был 4-разрядный микропроцессор 4004 фирмы INTEL (1971 г.). В 1973 г. фирма INTEL выпускает 8-разрядный микропроцессор 8080, а в 1978 г. - 16-разрядный микропроцессор 8086, имеющий 29 тысяч транзисторов на кристалле и начальную стоимость 360\$.

В 1965 году один из основателей фирмы Intel Гордон Мур (Gordon Earle Moore) сформулировал т.н. «закон Мура», согласно которому число транзисторов на кристалле будет удваиваться каждые два года. Дальнейшая эволюция микропроцессоров подтвердила его правоту. В 1993 г. микропроцессор INTEL PENTIUM имел уже 3.2. млн. транзисторов на кристалле и начальную стоимость 878\$, а появившиеся на рынке в 2011 г. Микропроцессоры семейства Intel Core I имеют на кристалле порядка 1 млрд. транзисторов. Основными направлениями эволюции микропроцессоров являлись (и являются) увеличение разрядности одновременно производимых вычислений и уменьшение времени выполнения вычислений.

**Микропроцессор** - программно-управляемое устройство, предназначенное для обработки цифровой информации и управления процессом этой обработки, выполненное в виде одной (или нескольких) интегральной схемы с высокой степенью интеграции электронных элементов.

Уменьшение стоимости, потребляемой мощности и габаритных размеров, повышение надежности и производительности микропроцессоров способствовали значительному расширению сферы их использования. Наряду с традиционными вычислительными системами они все чаще стали использоваться в задачах управления. При этом перед микропроцессором ставились задачи программного управления различными периферийными объектами в реальном масштабе времени.

Упрощенная структурная схема микропроцессорной системы управления имеет вид.



На процессор возлагается задача выполнения всех программных действий, необходимых в соответствии с алгоритмом работы устройства. В блоке памяти хранятся команды программы функционирования процессора, а также значения констант и переменных величин, участвующих в вычислениях. Блок ввода-вывода выполняет функцию сопряжения микропроцессорной системы с объектом управления.

Широкое использование микропроцессорной техники именно для задач управления привело к появлению на рынке специализированных микропроцессорных устройств, ориентированных на подобного рода применения. Особенностью этих микросхем является то, что помимо собственно процессора, на этом же кристалле расположен блок памяти и блок ввода-вывода, что позволяет снизить функциональную сложность и габаритные размеры микропроцессорной системы управления. Подобные устройства называются микроконтроллерами.

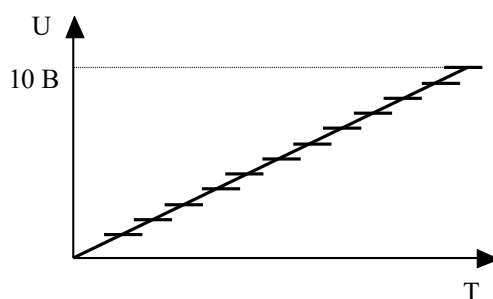
**Микроконтроллер** - вычислительно-управляющее устройство, предназначенное для выполнения функций логического контроля и управления периферийным

оборудованием, выполненное в виде одной БИС и сочетающее в себе микропроцессорное ядро, блок электронной памяти и набор встроенных устройств ввода-вывода.

Первыми микроконтроллерами явились микросхемы семейства MCS-48 фирмы INTEL, выпущенные в 1976 году. В 1981 году фирма INTEL выпустило новое семейство 8-разрядных микроконтроллеров MCS-51, которые получили огромное распространение во всем мире и дали толчок бурному развитию микроконтроллеров. вслед за фирмой INTEL микроконтроллеры начинают выпускать и другие ведущие производители микропроцессорной техники. Фирма MOTOROLA выпустила самый популярный в мире 8-разрядный микроконтроллер M68HC05. Эволюция микроконтроллеров соответствовала общему прогрессу микропроцессорной техники. Увеличивалась разрядность микроконтроллеров, их быстродействие, усовершенствовалась встроенная система ввода-вывода. Появились 16-разрядные (MCS-96 фирмы INTEL, M68HC12, M68HC16 фирмы MOTOROLA и др.), а затем и 32-разрядные (M68HC32 фирмы MOTOROLA) микроконтроллеры. С 1984 года начало развиваться направление т.н. RISC ( Reduced Instruction Set Computer) микроконтроллеров, обладающих пониженным энергопотреблением. Представителями подобных устройств являются семейства SAB 80C166 фирмы Infineon, MPC500 (на базе PowerPC ядра) фирмы MOTOROLA и др.

Дальнейшая интеграция позволила объединить на одном кристалле несколько процессорных ядер, оснащенных совместными и индивидуальными устройствами ввода-вывода и пространствами памяти. Такие устройства стали называться «**система на кристалле**» (SoC (System on Chip)). Примером таких устройств может служить 32-разрядное семейство F28M35 фирмы Texas Instruments, сочетающее в себе RISC-ядро ARM Cortex M3 с соответствующей периферией для решения коммуникационных задач и специализированное DSP-ядро C2000 для решения задач управления в реальном времени.

Отличительной особенностью цифровых систем управления является дискретизация сигнала по уровню, величина которой определяется разрядностью производимых вычислений. Так, в случае 8-разрядной системы, весь диапазон изменения значения сигнала делится на 256 участков и цифровой код, соответствующий этому сигналу может принимать лишь одно из 256 значений. Это, очевидно, накладывает ограничение на точность цифровой системы управления. Вследствие этого, долгое время в прецизионных системах продолжали (и в ряде случаев продолжают) использовать аналоговые методы обработки информации. Проведем сравнительный анализ. Пусть в аналоговой системе некоторый сигнал, в амплитуде которого заложена информация, может изменяться в пределах от 0 до 10 В. Уровень шума при этом не превышает 1 мВ. Для достоверной передачи информации, исключающей влияние шумов, минимальное приращение сигнала должно составлять как минимум 1 мВ. Т.о., с помощью подобного сигнала можно передать  $\frac{10}{0.001} = 10000$  единиц информации.



Для передачи такого же количества информации в цифровом коде необходимо иметь разрядность как минимум 14 двоичных разрядов. Следовательно, цифровые системы с меньшей разрядностью будут уступать по точности описанной аналоговой системе. Однако, при наличии разрядности, большей чем 14 бит цифровая система может не только

не уступать, но и превосходить по точности аналоговую поскольку ее параметры не изменяются с течением времени и не таких внешних факторов как температура, влажность и т.п., что в большой степени присуще практически всем аналоговым системам.

Второй отличительной особенностью микропроцессорных систем является последовательное во времени выполнение команд процессором. Все команды, каждая из которых имеет конечное время выполнения, исполняются последовательно, одна за другой. Следовательно, от момента начала выполнения алгоритма до получения конечного результата проходит некоторый интервал времени. Это отрицательно сказывается на быстродействии цифровых систем, и, в частности, на области устойчивости и полосе пропускания цифровых систем управления. Однако, процесс повышения быстродействия микропроцессорных устройств идет неуклонно вперед и в настоящее время существуют микропроцессоры, минимальное время выполнения команды у которых меньше 1 нс. С помощью современных микропроцессоров уже сегодня возможно создавать системы управления с полосой пропускания в десятки и даже сотни КГц. В свою очередь, аналоговые системы, несмотря на практически мгновенное протекание сигналов, также обладают конечным быстродействием из-за неидеальности компонентов и наличия паразитных реактивных связей в системе. Временные параметры цифровых систем, в отличие от аналоговых, не изменяются с течением времени и не зависят от внешних факторов.

Т.о., в настоящее время, благодаря всему вышеперечисленному идет полномасштабное внедрение микропроцессорной техники практически во все сферы деятельности, где еще вчера господствовали аналоговые методы обработки информации.

В современной преобразовательной технике микроконтроллеры выполняют не только роль непосредственного управления полупроводниковым преобразователем за счет встроенных специализированных периферийных устройств, но и роль цифрового регулятора, системы защиты и диагностики, а также системы связи с технологической сетью высшего уровня.

В последнее время появился ряд микроконтроллеров, специализированных для задач управления полупроводниковыми преобразователями. Их вычислительное ядро, построенное, как правило, на базе т.н. «процессоров цифровой обработки сигналов» (DSP – Digital Signal Processor), адаптировано на выполнение рекуррентных полиномиальных алгоритмов цифрового регулирования. Встроенные периферийные устройства включают в себя многоканальные генераторы ШИМ-сигналов, аналого-цифровые преобразователи, блоки векторных преобразований координат, таймеры-счетчики, Watcdog-таймеры и т.д. Примерами таких устройств могут служить микроконтроллеры TMS320C2000 фирмы Texas Instruments, 56800 фирмы Motorola, SAB 80C166 фирмы Infineon.

## Лекция № 2

### **“Цифровые методы представления информации. Цифровые коды. Двоичная и шестнадцатиричная системы счисления. Перевод чисел из одной системы счисления в другую. Двоичная арифметика. Формы представления чисел с фиксированной и плавающей точкой”.**

Двоичная система счисления (система счисления с основанием 2) является позиционной системой, аналогичной десятичной системе счисления, в которой положение разряда определяется степенью основания, используемого в качестве множителя данного разряда.

В общем любое число можно представить в виде выражения

$$\sum_{j=m}^{j=n} a_j x^{j-1}$$

где m и n - пределы изменения показателя. m определяет точность представления числа, n определяет максимальный диапазон представляемых чисел.

В случае двоичной системы счисления число представляется в виде

$$\sum_{j=m}^{j=n} a_j 2^{j-1}$$

Пример:  $28 = 1 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0$

Будем называть каждую цифру в представлении числа разрядом.

С помощью n-разрядного двоичного кода можно представить  $2^n$  последовательных чисел с дискретностью  $2^{m-1}$ .

Для преобразования числа из десятичной системы счисления в двоичную необходимо последовательно делить заданное число на 2 и формировать результат из остатков деления начиная с младшего разряда.

Пример:  $(125)_{10} = 125 / 2 = 62 + 1$

$$62 / 2 = 31 + 0$$

$$31 / 2 = 15 + 1$$

$$15 / 2 = 7 + 1$$

$$7 / 2 = 3 + 1$$

$$3 / 2 = 1 + 1$$

$$1 / 2 = 0 + 1$$

$$= (1111101)_2$$

Для преобразования числа из двоичной системы счисления в десятичную необходимо сложить десятичные веса всех ненулевых разрядов числа.

Правило двоичного сложения.

$$0+0=0$$

$$0+1=1$$

$$1+0=1$$

$1+1=0$  плюс перенос в следующий старший разряд

Пример:  $(13)_{10}$

$$(00001101)_{(2)}$$

$$\begin{array}{r} +(10)_{(10)} \\ \hline (23)_{(10)} \end{array}$$

$$\begin{array}{r} +(00001010)_{(2)} \\ \hline (00010111)_{(2)} \end{array}$$

Правило двоичного вычитания.

$$\begin{array}{l} 0 - 0 = 0 \\ 1 - 0 = 1 \\ 1 - 1 = 0 \\ 0 - 1 = 1 \text{ заем 1 из следующего} \\ \text{старшего разряда} \end{array}$$

Самостоятельно изучить двоичное умножение и деление.

Для представления как положительных, так и отрицательных чисел вводится дополнительный знаковый разряд, который является старшим разрядом числа. У положительных чисел он равен 0, а у отрицательных - 1. Число нуль имеет в прямом коде два представления: 00..0 и 10..0.

Прямой код используется для умножения чисел, так как для этого необходимо перемножить модули чисел сомножителей и вычислить знак произведения исходя из знаков сомножителей.

**Обратный код** (или дополнение до 1) двоичного числа получают заменой всех его единиц на нули, а нулей на единицы.

$$\begin{array}{ll} \text{Пример:} & X = (11010010)_{(2)} - \text{прямой код;} \\ & \bar{X} = (00101101)_{(2)} - \text{обратный код.} \end{array}$$

Пусть требуется выполнить вычитание двух целых положительных n-разрядных чисел  $X = \sum_{j=1}^n x_j * 2^{j-1}$  и  $Y = \sum_{j=1}^n y_j * 2^{j-1}$ . Разность S можно вычислить как

$$\begin{aligned} S &= X - Y = X + (-Y) = X - 2^n + (2^n - Y) = X - 2^n + W; \text{ где } W = 2^n - Y. \\ W &= 2^n - Y = \left( \sum_{j=1}^n 2^{j-1} + 1 \right) - Y = \sum_{j=1}^n 2^{j-1} - \sum_{j=1}^n y_j * 2^{j-1} + 1 = \sum_{j=1}^n (1 - y_j) * 2^{j-1} + 1 \\ &= \sum_{j=1}^n \bar{y}_j * 2^{j-1} + 1 = \bar{Y} + 1. \end{aligned}$$

Число  $W = 2^n - Y$  называется дополнением числа Y до  $2^n$ . Таким образом нахождение разности сводится к сложению с дополнением вычитаемого. Аналогичным образом получим:

$$S = X - 2^n + W = (0 * 2^n + X) + (1 * 2^n + W) - 2^{n+1}.$$

Код  $0 * 2^n + X = 0 * 2^n + \sum_{j=1}^n x_j * 2^{j-1}$  называется дополнительным кодом положительного числа X. Как видно, он совпадает с прямым кодом.



Код  $1 * 2^n + W = \sum_{j=1}^n \overline{y_j} * 2^{j-1} + 1$  называется дополнительным кодом отрицательного

числа  $-Y$ . Он образуется сложением 1 с обратным кодом числа  $Y$  и добавлением старшего единичного знакового разряда.

Пример:  $(5)_{(10)} = (00000101)_{(2)}$

$(-5)_{(10)} = (11111011)_{(2)}$

Дополнения двоичных чисел просто получаются посредством схем инвертирования.

Сложение двоичных чисел  $X$  и  $Y$ , представленных в дополнительном коде, основывается на теореме: Дополнительный код арифметической суммы  $S$  двух чисел  $X$  и  $Y$  любых знаков равен арифметической сумме дополнительных кодов чисел. При этом при сложении чисел одинакового знака должно выполняться условие  $|X| + |Y| \leq 2^n - 1$ , т.е. разрядная сетка не должна переполняться. Если  $X > 0$  и  $Y > 0$  и  $|X| + |Y| \geq 2^n$ , то происходит потеря значения  $+2^n$  и изменение знака остатка суммы на противоположный. Если же  $X < 0$  и  $Y < 0$ , то потеря значения  $-2^n$  и изменение знака остатка суммы на противоположный происходит при  $|X| + |Y| \geq 2^n + 1$ .

Использование дополнительного кода позволяет создавать схемы, выполняющие сложение, вычитание, умножение и деление посредством обычного сложения.

Пример:  $(16)_{(10)}$

$(00010000)_{(2)}$

$(00010000)_{(2)}$

$- (10)_{(10)}$

$- (00001010)_{(2)}$

$= + (11110110)_{(2)}$

$(6)_{(10)}$

$(00000110)_{(2)}$

$(00000110)_{(2)}$

В дополнительном коде старший разряд кода является знаковым.

$$(01111111)_{(2)} = (127)_{(10)} = 1 * 2^0 + 1 * 2^1 + 1 * 2^2 + 1 * 2^3 + 1 * 2^4 + 1 * 2^5 + 1 * 2^6 + 0 * 2^7$$

$$(01111110)_{(2)} = (126)_{(10)} = 0 * 2^0 + 1 * 2^1 + 1 * 2^2 + 1 * 2^3 + 1 * 2^4 + 1 * 2^5 + 1 * 2^6 + 0 * 2^7$$

...

$$(00000001)_{(2)} = (1)_{(10)} = 1 * 2^0 + 0 * 2^1 + 0 * 2^2 + 0 * 2^3 + 0 * 2^4 + 0 * 2^5 + 0 * 2^6 + 0 * 2^7$$

$$(00000000)_{(2)} = (0)_{(10)} = 0 * 2^0 + 0 * 2^1 + 0 * 2^2 + 0 * 2^3 + 0 * 2^4 + 0 * 2^5 + 0 * 2^6 + 0 * 2^7$$

$$(11111111)_{(2)} = (-1)_{(10)} = 1 * 2^0 + 1 * 2^1 + 1 * 2^2 + 1 * 2^3 + 1 * 2^4 + 1 * 2^5 + 1 * 2^6 - 1 * 2^7$$

$$(11111110)_{(2)} = (-2)_{(10)} = 0 * 2^0 + 1 * 2^1 + 1 * 2^2 + 1 * 2^3 + 1 * 2^4 + 1 * 2^5 + 1 * 2^6 - 1 * 2^7$$

...

$$(10000001)_{(2)} = (-127)_{(10)} = 1 * 2^0 + 0 * 2^1 + 0 * 2^2 + 0 * 2^3 + 0 * 2^4 + 0 * 2^5 + 0 * 2^6 - 1 * 2^7$$

$$(10000000)_{(2)} = (-128)_{(10)} = 0 * 2^0 + 0 * 2^1 + 0 * 2^2 + 0 * 2^3 + 0 * 2^4 + 0 * 2^5 + 0 * 2^6 - 1 * 2^7$$

**Смещенный код** применяется в блоках АЦП и ЦАП.

$$(11111111)_{(2)} = (127)_{(10)}$$

$$(11111110)_{(2)} = (126)_{(10)}$$

...

$$(10000001)_{(2)} = (1)_{(10)}$$

$$(10000000)_{(2)} = (0)_{(10)}$$

$$(01111111)_{(2)} = (-1)_{(10)}$$

...

$$(00000001)_{(2)} = (-127)_{(10)}$$

$$(00000000)_{(2)} = (-128)_{(10)}$$

**Двоично-десятичный код** применяется в системах индикации и т.п. Каждый десятичный разряд кодируется группой из 4 двоичных разрядов.

$$\text{Пример: } (9137)_{(10)} = (1001000100110111)_{(2/10)}$$

Существуют алгоритмы двоично-десятичной арифметики, аналогичные алгоритмам двоичной арифметики. Для упрощения алгоритмов арифметических операций в двоично-десятичном коде используются “код с избытком 3” и код “2421” (код Айкена), в которых каждая десятичная цифра также записывается в виде 4 двоичных разрядов.

| Цифра | Код с избытком 3 | Код 2421 |
|-------|------------------|----------|
| 0     | 0011             | 0000     |
| 1     | 0100             | 0001     |
| 2     | 0101             | 0010     |
| 3     | 0110             | 0011     |
| 4     | 0111             | 0100     |
| 5     | 1000             | 1011     |
| 6     | 1001             | 1100     |
| 7     | 1010             | 1101     |
| 8     | 1011             | 1110     |
| 9     | 1100             | 1111     |

В коде 2421 первые пять десятичных чисел повторяют двоичный эквивалент, а последние пять чисел имеют избыток +6.

Эти коды обладают важным свойством: для того, чтобы для какой-либо цифры получить ее дополнение до 9, достаточно взять ее обратный код. Это позволяет упростить десятичную арифметику точно так же, как применение дополнительного кода упрощает двоичную арифметику. Вместо вычитания какого-либо числа достаточно произвести сложение с числом, которое представляет собой дополнение до 9 исходного числа плюс 1.

$$\begin{array}{r} \text{Пример: } 27 - 15 = 27 \\ \quad \quad \quad + 85 \text{ (дополнение до 9 числа 15 = 84, затем прибавили 1)} \\ \hline \quad \quad \quad 12 \end{array}$$

**Код Грея** используется в механических шифраторах угла поворота вала. При переходе из любого его состояния к следующему изменяется лишь один разряд. Это позволяет предотвратить ошибки, так как в данном случае при переходе между двумя соседними значениями все разряды не могут измениться одновременно.

|        |        |
|--------|--------|
| (0000) | (1100) |
| (0001) | (1101) |
| (0011) | (1111) |
| (0010) | (1110) |
| (0110) | (1010) |
| (0111) | (1011) |
| (0101) | (1001) |
| (0100) | (1000) |

Представление чисел в соответствии с формулой

$$\sum_{j=m}^{j=n} a_j 2^j$$

называется **представлением чисел с фиксированной точкой**. Такой способ дает возможность при фиксированных значениях  $m$  и  $n$  представить числа в диапазоне от  $2^m$  до  $2^{n+1} - 1$ . Например при  $m=0$  и  $n=7$ , т.е. с помощью 8-разрядного двоичного кода можно представить числа в диапазоне от 0 до 255, а в случае необходимости представления отрицательных чисел, от -127 до +127. Такой диапазон представления чисел в ряде случаев оказывается недопустимо маленьким, что влечет за собой усложнение алгоритмов обработки данных с большим диапазоном изменения.

Для представления чисел в большом диапазоне изменения величин применяется т.н. **формат с плавающей точкой**. В соответствии с международным стандартом IEEE754 число с плавающей точкой одинарной точности представляется в виде

$$(-1)^S * M * (2)^{E-127},$$

где  $M$  называется мантиссой, а  $E$  - порядком числа. Число записывается в формате

$$SE_7E_7E_7...E_0M_{22}M_{21}...M_0$$

Для записи мантиссы отводится 23 двоичных разряда, а для записи порядка – 8 двоичных разрядов. Мантиссу принято записывать в т.н. нормированном виде, в котором она лежит в пределах от 1 до 2. При этом единственный целый разряд в двоичной записи мантиссы всегда равен единице и его можно не записывать. Поэтому, в записи мантиссы используются только 23 старших дробных двоичных разряда.

Например, число с плавающей точкой в формате IEEE754 имеет вид

0.01111100.010000000000000000000000

Бит знака  $S=0$ . Значит число положительное.

Порядок равен  $(01111100)_2 - (127)_{10} = 124 - 127 = -3$ .

Мантисса равна  $(1.0100000000 0000000000 000)_{2} = (1.25)_{10}$ . (Старшая единица в записи мантиссы – неявная.)

Таким образом, число равно  $1.25 * 2^{-3} = 0.15625$ .

В данном формате предусмотрены специальные форматы для записи таких категорий, как ноль, бесконечность и «не число» (NaN – Not a Number). Для них

используются крайние значения порядка (все нули и все единицы), которые не разрешается использовать для представления обычных чисел.

Ноль записывается как S.00000000.000000000000000000000000. Бит знака S может быть равен 0 или 1. Это дает возможность задавать +0 и -0.

Бесконечность записывается как S.11111111.000000000000000000000000.

Неопределенность NaN записывается аналогично бесконечности, но с ненулевой мантиссой.

Такой формат представления данных дает возможность записывать числа со знаком, модуль которых изменяется в диапазоне от  $2^{-126}$  до  $(2 - 2^{-23}) * 2^{127}$ . Налицо значительное увеличение диапазона изменения чисел, однако для их хранения необходим больший объем памяти.

Как и для формата с фиксированной точкой, при представлении чисел с плавающей точкой используются специальные алгоритмы арифметических операций над такими числами.

**Шестнадцатиричная система счисления** применяется для сокращения записи двоичных чисел. Каждые соседние 4 разряда в двоичной записи числа представляются в виде одного 16-ричного разряда. Для представления чисел от 10 до 15 применяются буквы от A до F.

$$\text{Пример: } (41667)_{(10)} = (1010001011000011)_{(2)} = (A2C3)_{(16)}$$

Шестнадцатиричное представление наилучшим образом соответствует байтовой структуре ЭВМ (1 байт = 8 бит). Один байт представляется в виде 2 шестнадцатиричных цифр. Чтобы преобразовать число из двоичной кода в шестнадцатиричный необходимо разбить код числа на группы из 4 бит начиная с младшего разряда и представить каждую группу в виде одной шестнадцатиричной цифры. Чтобы преобразовать число из шестнадцатиричного кода в двоичный необходимо последовательно записать каждую шестнадцатиричную цифру в виде 4 двоичных разрядов.

### Лекция № 3

## “Основные логические операции над двоичными числами. Синтез логических устройств. Таблицы истинности”.

Логические функции оперируют логическими переменными, т.е. переменными, принимающими только два значения - ИСТИНА и ЛОЖЬ (на математическом языке - 0 и 1). Результат логической функции может принимать тоже только эти два значения. Для представления логических функций используют аппарат логических уравнений и таблиц истинности.

**Таблица истинности** представляет собой таблицу, в которой каждой комбинации входных логических переменных ставится в соответствие требуемое значение данной логической функции.

| $x_1$ | $x_2$ | L  | $x_n$ | Y          |
|-------|-------|----|-------|------------|
| 0     | 0     | .. | 0     | $y_{00.0}$ |
| 1     | 0     | .. | 0     | $y_{10.0}$ |
| ..    | ..    | .. | ..    | ..         |
| 1     | 1     | .. | 1     | $y_{11.1}$ |

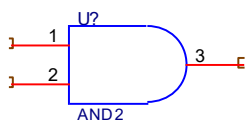
Логические функции могут быть одной, двух и более логических переменных. Элементарными логическими функциями являются следующие.

**Функция “Логическое И” (конъюнкция или логическое произведение)**

| $x_1$ | $x_2$ | y |
|-------|-------|---|
| 0     | 0     | 0 |
| 0     | 1     | 0 |
| 1     | 0     | 0 |
| 1     | 1     | 1 |

Логическое уравнение:  $y = x_1 * x_2$ .

Схематическое изображение функции

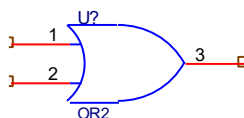


**Функция “Логическое ИЛИ” (дизъюнкция или логическое сложение)**

| $x_1$ | $x_2$ | y |
|-------|-------|---|
| 0     | 0     | 0 |
| 0     | 1     | 1 |
| 1     | 0     | 1 |
| 1     | 1     | 1 |

Логическое уравнение:  $y = x_1 + x_2$ .

Схематическое изображение функции

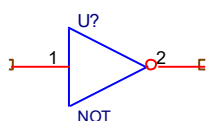


### Функция “Логическое НЕ” (отрицание)

| $x$ | $y$ |
|-----|-----|
| 0   | 1   |
| 1   | 0   |

Логическое уравнение:  $y = \bar{x}$ .

Схематическое изображение функции



### Логические теоремы.

|                                                                                     |                                                                                     |                               |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------|
| $A + A = A$                                                                         | $A * A = A$                                                                         | (теорема идемпотентности).    |
| $A + 1 = 1$                                                                         | $A * 0 = 0$                                                                         | (теорема о нулевом элементе). |
| $A + 0 = A$                                                                         | $A * 1 = A$                                                                         | (теорема идентичности).       |
| $\overline{\overline{A}} = A$                                                       |                                                                                     | (теорема инволюции).          |
| $A + \overline{A} = 1$                                                              | $A * \overline{A} = 0$                                                              | (теорема дополнительности).   |
| $A * B = B * A$                                                                     | $A + B = B + A$                                                                     | (теорема коммутативности).    |
| $(A + B) + C = A + (B + C)$                                                         | $(A * B) * C = A * (B * C)$                                                         | (теорема ассоциативности).    |
| $A * (B + C) = A * B + A * C$                                                       | $A + (B * C) = (A + B) * (A + C)$                                                   | (теорема дистрибутивности).   |
| $A + A * B = A$                                                                     | $A * (A + B) = A$                                                                   | (теорема поглощения).         |
| $A + B + C + \dots = \overline{\overline{A} * \overline{B} * \overline{C} * \dots}$ | $A * B * C * \dots = \overline{\overline{A} + \overline{B} + \overline{C} + \dots}$ | (теорема де Моргана).         |

Используя данные теоремы, любую логическую функцию можно реализовать посредством блоков “И” и “НЕ” или “ИЛИ” и “НЕ” (это доказал Шеффер в 1913г.). Для перехода от одного базиса к другому используются две последние теоремы, которые называются **теоремами де Моргана** в честь шотландского математика Огастеса де Моргана.

Схемы элементов «И-НЕ» (штрих Шеффера), «ИЛИ-НЕ» (Стрелка Пирса) и «НЕ», реализованные на комплементарных МОП-транзисторах (КМОП-технология), представлены на рисунке.



3. Составить прямоугольную таблицу (карту Карно), размерность которой определяется максимально возможным количеством комбинаций в выделенных группах входных переменных функции. Вдоль осей таблицы напротив последовательных строк и столбцов отложить последовательные состояния выделенных групп переменных в коде Грея.
4. Заполнить ячейки, расположенные на пересечении строк и столбцов таблицы значениями функции, соответствующими текущей комбинации состояний входных переменных.
5. Выделить в таблице все правильные группы, содержащие 1 (либо 0).

Свойства правильных групп:

- правильная группа может состоять только из  $2^n$  ячеек, где  $n$  – целое неотрицательное число;
- правильная группа может иметь только прямоугольную (или квадратную) форму;
- в правильную группу могут входить только ячейки, содержащие единицы (или только нули);
- любая ячейка, содержащая единицу, может входить в любое количество правильных групп, т.е. группы могут пересекаться между собой;
- правильную группу можно «сшивать» по краям таблицы.

6. Записать уравнение функции по выделенным группам. Это уравнение будет состоять из нескольких «подчастей» по количеству выделенных правильных групп. Отдельные подчасти объединяются между собой с помощью функции «ИЛИ». В каждую из подчастей входят только те переменные функции, которые не меняют своего значения во всех ячейках карты Карно, которые образуют данную правильную группу. Причем, если в этих ячейках данная переменная равна единице, то она входит в выражение в прямом виде, а если она равна нулю – то в инверсном. Отдельные переменные объединяются между собой в выражении данной подчасти с помощью функции «И».

Пример: Функция задана таблицей истинности.

| A | B | C | Q |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

Составим карту Карно для данной функции.

|   | AB |    |    |    |
|---|----|----|----|----|
| C | 00 | 01 | 11 | 10 |
| 0 | 0  | 0  | 1  | 0  |
| 1 | 0  | 1  | 1  | 1  |

Выделяем группы, содержащие “1”.

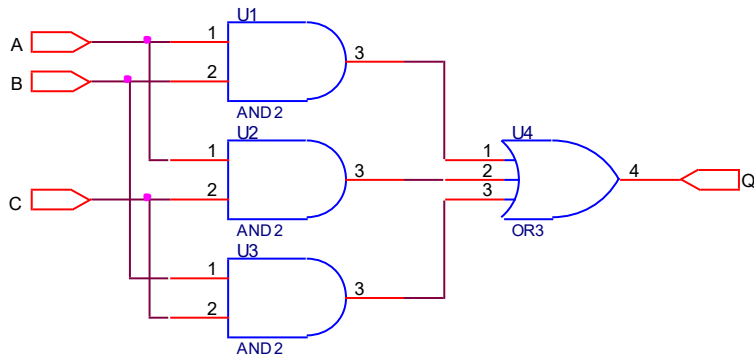


|   | AB |    |    |    |
|---|----|----|----|----|
| C | 00 | 01 | 11 | 10 |
| 0 | 0  | 0  | 1  | 0  |
| 1 | 0  | 1  | 1  | 1  |

Согласно выделенным группам записываем уравнение логической функции.

$$Q = A * B + B * C + A * C$$

Схематическое изображение функции:



Рассмотрим основные виды комбинационных логических устройств.

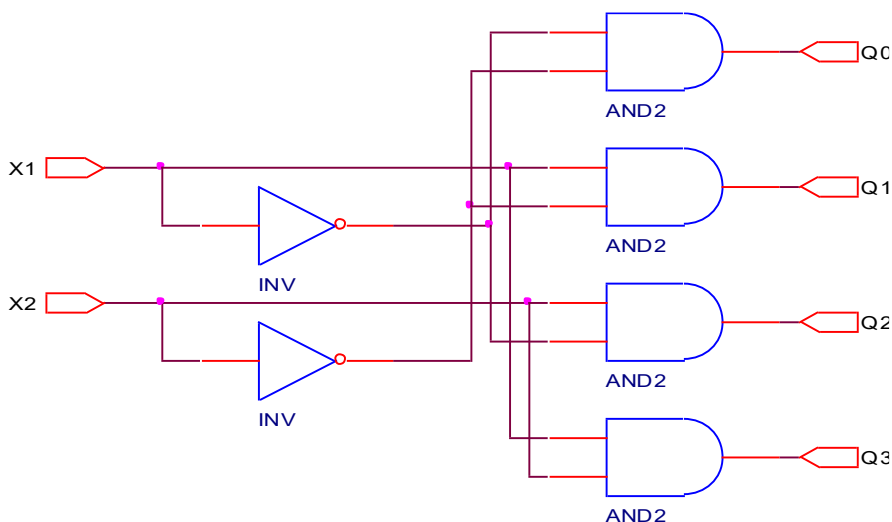
**Полным дешифратором** называется устройство, имеющее  $n$  входов и  $2^n$  выходов, причем каждой комбинации значений входных сигналов соответствует сигнал, равный 1, только на одном выходе. В качестве примера рассмотрим дешифратор на 2 входа. Его таблица истинности имеет вид.

| $X_1$ | $X_2$ | $Q_0$ | $Q_1$ | $Q_2$ | $Q_3$ |
|-------|-------|-------|-------|-------|-------|
| 0     | 0     | 1     | 0     | 0     | 0     |
| 0     | 1     | 0     | 1     | 0     | 0     |
| 1     | 0     | 0     | 0     | 1     | 0     |
| 1     | 1     | 0     | 0     | 0     | 1     |

Т.о., логические уравнения для соответствующих выходных разрядов дешифратора имеют вид:

$$Q_0 = \overline{X_1} * \overline{X_2}; Q_1 = \overline{X_1} * X_2; Q_2 = X_1 * \overline{X_2}; Q_3 = X_1 * X_2.$$

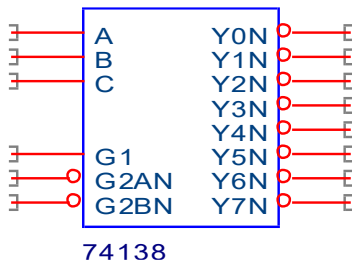
Схема, реализующая подобное устройство имеет вид.



Полный дешифратор, дополненный общим разрешающим входом называется дешифратором-демультиплексором. Если добавить в рассмотренный дешифратор общий разрешающий вход Е, то его уравнения примут вид:

$$Q_0 = \overline{E} * \overline{X_1} * \overline{X_2}; Q_1 = \overline{E} * \overline{X_1} * X_2; Q_2 = \overline{E} * X_1 * \overline{X_2}; Q_3 = \overline{E} * X_1 * X_2.$$

Примером подобного дешифратора может служить микросхема 1533 ИД7 (74ALS138), представляющая собой дешифратор 3\*8 с тремя входами общего разрешения: единичным G1, нулевыми G2AN и G2BN. Работа микросхемы разрешена если G1=1 & G2AN=0 & G2BN=0.



Т.о., дешифраторы осуществляют преобразование двоичного кода в унитарный код, т.е. код, только один из разрядов которого равен 1.

Шифраторы выполняют функцию обратную дешифраторам, т.е. преобразуют унитарный код в двоичный.

Таблица истинности шифратора на 4 входа имеет вид:

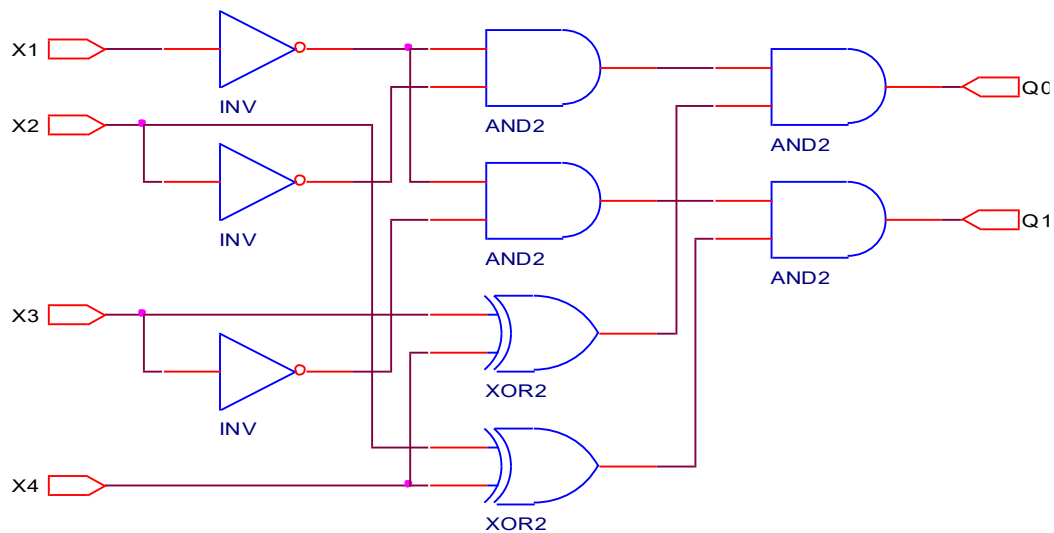
| $X_1$ | $X_2$ | $X_3$ | $X_4$ | $Q_0$ | $Q_1$ |
|-------|-------|-------|-------|-------|-------|
| 1     | 0     | 0     | 0     | 0     | 0     |
| 0     | 1     | 0     | 0     | 0     | 1     |
| 0     | 0     | 1     | 0     | 1     | 0     |
| 0     | 0     | 0     | 1     | 1     | 1     |

Уравнения для соответствующих выходов шифратора имеют вид:

$$Q_0 = \overline{X_1} * \overline{X_2} * (X_3 * \overline{X_4} + \overline{X_3} * X_4) = \overline{X_1} * \overline{X_2} * (X_3 \oplus X_4);$$

$$Q_1 = \overline{X_1} * \overline{X_3} * (X_2 * \overline{X_4} + \overline{X_2} * X_4) = \overline{X_1} * \overline{X_3} * (X_2 \oplus X_4).$$

Схема, реализующая указанные уравнения имеет вид.



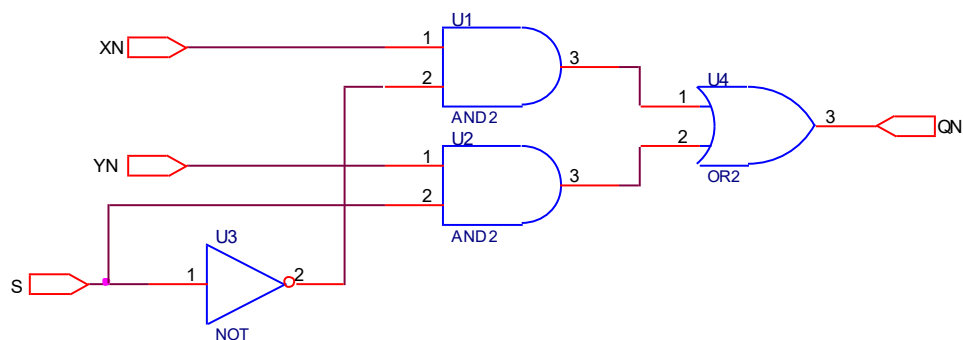
**Мультиплексором** называется комбинационная схема, имеющая  $m$  адресных и  $2^m$  информационных входов, а также один выход, на который подается сигнал с того информационного входа, чей адрес в данный момент присутствует на адресных входах. Рассмотрим мультиплексор на 2 входа  $X$  и  $Y$  с адресным входом  $S$ . Таблица истинности такого устройства имеет вид:

| X | Y | S | Q |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 0 | 1 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 1 |

Этой таблице истинности соответствует следующее уравнение выхода

$$Q = \bar{S} * X + S * Y$$

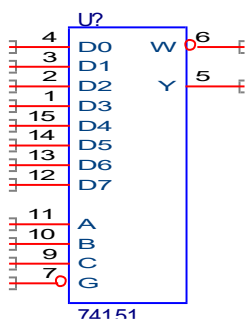
Этому уравнению соответствует схема.



При добавлении общего разрешающего входа  $E$  получим устройство, называемое селектор-мультиплексор. При этом его уравнение примет вид.

$$Q = \bar{E} * (\bar{S} * X + S * Y)$$

Примером подобного устройства может служить микросхема 1533КП7 (74ALS151), представляющая собой селектор-мультиплексор на 8 каналов с общим входом разрешения G.



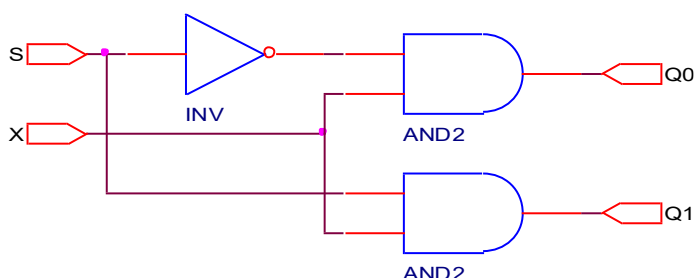
Демультимплексоры выполняют функцию, обратную мультиплексорам, т.е. производят коммутацию одного информационного входного сигнала на  $2^m$  выходов, где  $m$  - число адресных входов. Рассмотрим демультимплексор на 2 выхода. Его таблица истинности имеет вид:

| X | S | Q <sub>0</sub> | Q <sub>1</sub> |
|---|---|----------------|----------------|
| 0 | 0 | 0              | 0              |
| 1 | 0 | 1              | 0              |
| 0 | 1 | 0              | 0              |
| 1 | 1 | 0              | 1              |

Этой таблице истинности соответствуют уравнения выходов:

$$Q_0 = \bar{S} * X; Q_1 = S * X.$$

Этим уравнениям соответствует схема.



**Компаратор** выполняет функцию сравнения двоичных чисел. Его выход равен нулю если числа равны и единице в противном случае. Рассмотрим компаратор двухбитных чисел X и Y. Его таблица истинности имеет следующий вид.

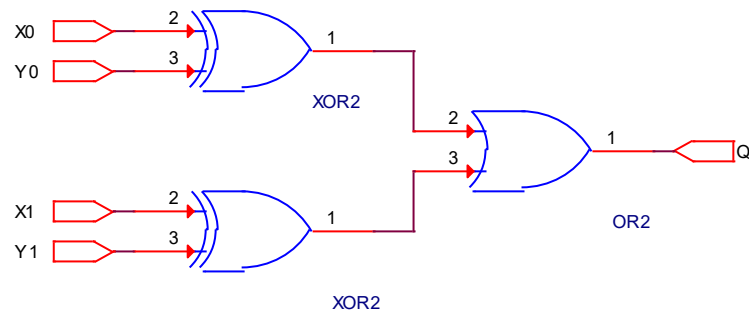
| X <sub>0</sub> | X <sub>1</sub> | Y <sub>0</sub> | Y <sub>1</sub> | Q |
|----------------|----------------|----------------|----------------|---|
| 0              | 0              | 0              | 0              | 0 |
| 0              | 0              | 0              | 1              | 1 |
| 0              | 0              | 1              | 0              | 1 |
| 0              | 0              | 1              | 1              | 1 |

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 1 |
| 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 0 | 1 | 1 |
| 1 | 0 | 1 | 0 | 0 |
| 1 | 0 | 1 | 1 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 | 0 |

По таблице истинности находим уравнение функции выхода.

$$Q = X_0 * \overline{Y_0} + \overline{X_0} * Y_0 + X_1 * \overline{Y_1} + \overline{X_1} * Y_1 = (X_0 \oplus Y_0) + (X_1 \oplus Y_1)$$

Схема компаратора имеет вид.



При увеличении разрядности компаратора параллельно добавляется соответствующее количество элементов «исключающее или» для дополнительных разрядов сравниваемых чисел.

Создадим устройство для суммирования двух двоичных чисел  $X_1$  и  $X_2$ . Опишем закон формирования младшего разряда суммы  $S_0$  и переноса в следующий разряд  $c_1$  с помощью таблицы истинности.

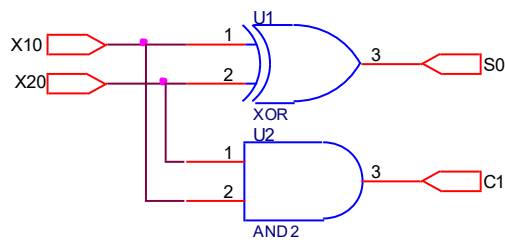
| $x_{1_0}$ | $x_{2_0}$ | $s_0$ | $c_1$ |
|-----------|-----------|-------|-------|
| 0         | 0         | 0     | 0     |
| 0         | 1         | 1     | 0     |
| 1         | 0         | 1     | 0     |
| 1         | 1         | 0     | 1     |

Т.о. получим уравнения для младшего разряда суммы и переноса:

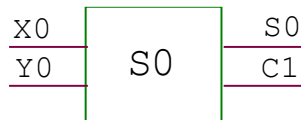
$$s_0 = \overline{x_{1_0}} * x_{2_0} + x_{1_0} * \overline{x_{2_0}} = x_{1_0} \oplus x_{2_0}$$

$$c_1 = x_{1_0} * x_{2_0}$$

По полученным уравнениям получаем схему т.н. полусумматора.



Обозначим разработанное устройство блоком S0.



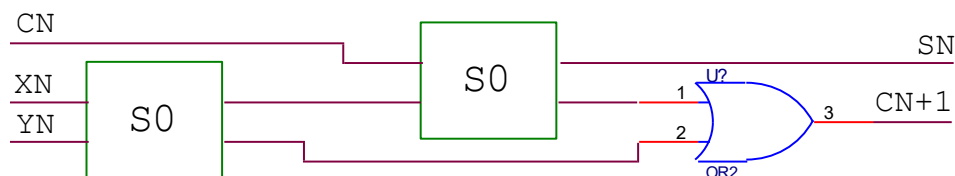
Закон формирования любого следующего разряда суммы и разряда переноса в следующий разряд описывается таблицей истинности.

| $x_{1_n}$ | $x_{2_n}$ | $c_n$ | $s_n$ | $c_{n+1}$ |
|-----------|-----------|-------|-------|-----------|
| 0         | 0         | 0     | 0     | 0         |
| 0         | 0         | 1     | 1     | 0         |
| 0         | 1         | 0     | 1     | 0         |
| 0         | 1         | 1     | 0     | 1         |
| 1         | 0         | 0     | 1     | 0         |
| 1         | 0         | 1     | 0     | 1         |
| 1         | 1         | 0     | 0     | 1         |
| 1         | 1         | 1     | 1     | 1         |

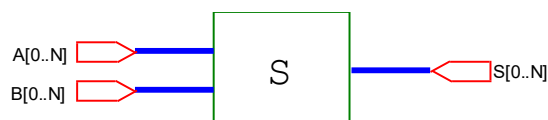
Т.о. получаем уравнения для произвольного разряда суммы и переноса:

$$\begin{aligned}
 s_n &= \overline{x_{1_n}} * \overline{x_{2_n}} * c_n + \overline{x_{1_n}} * x_{2_n} * \overline{c_n} + x_{1_n} * \overline{x_{2_n}} * c_n + x_{1_n} * x_{2_n} * \overline{c_n} = \\
 &= \overline{c_n} * (\overline{x_{1_n}} * \overline{x_{2_n}} + \overline{x_{1_n}} * x_{2_n}) + c_n * (\overline{x_{1_n}} * x_{2_n} + x_{1_n} * \overline{x_{2_n}}) = \\
 &= \overline{c_n} * (\overline{x_{1_n}} \oplus \overline{x_{2_n}}) + c_n * (x_{1_n} \oplus x_{2_n}) = c_n \oplus (x_{1_n} \oplus x_{2_n}) \\
 c_{n+1} &= c_n * x_{1_n} + c_n * x_{2_n} + x_{1_n} * x_{2_n} = c_n * (x_{1_n} + x_{2_n}) + x_{1_n} * x_{2_n} = \\
 &= c_n * (x_{1_n} \oplus x_{2_n}) + x_{1_n} * x_{2_n}
 \end{aligned}$$

Т.о. получаем схему суммирования чисел произвольной разрядности.



Включая параллельно несколько подобных схем, можно получить устройство для суммирования произвольного числа разрядов.



Разработанный сумматор называется «сумматор с последовательным переносом» (Ripple Carry Adder -RCA). Его главным достоинством является простота. Недостатком подобного сумматора является низкое быстродействие, обусловленное последовательным вычислением переносов в отдельных разрядах. Для повышения быстродействия сумматора введем функции распространения переноса  $p_n$  и генерации переноса  $g_n$ .

$$p_n = x_n \oplus y_n$$

$$g_n = x_n * y_n$$

С учетом введенных функций сигналы суммы и переноса записываются следующим образом.

$$s_n = p_n \oplus c_n$$

$$c_{n+1} = g_n + p_n * c_n$$

Таким образом, сигналы переносов отдельных разрядов можно записать как

$$c_1 = g_0$$

$$c_2 = g_1 + p_1 * c_1 = g_1 + p_1 * g_0$$

$$c_3 = g_2 + p_2 * c_2 = g_2 + p_2 * g_1 + p_2 * p_1 * g_0$$

$$c_4 = g_3 + p_3 * c_3 = g_3 + p_3 * g_2 + p_3 * p_2 * g_1 + p_3 * p_2 * p_1 * g_0$$

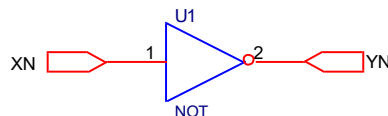
В этих уравнениях все функции переносов зависят только от входных сигналов сумматора и не зависят от сигналов переноса предыдущих разрядов. Поэтому сигналы сумм всех разрядов вычисляются одновременно. Такой сумматор называется «сумматор с параллельным переносом» или «сумматор с предсказанием переноса» (Carry Lookahead Adder - CLA). Это самый быстрый и самый сложный вид сумматора.

Создадим схему, реализующую преобразование прямого кода числа в его дополнительный код. Как известно, для этого необходимо найти обратный код числа и прибавить к нему 1.

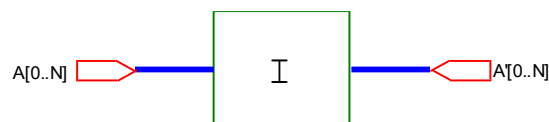
Схема, вычисляющая обратный код числа, описывается уравнением

$$y_n = \overline{x_n}$$

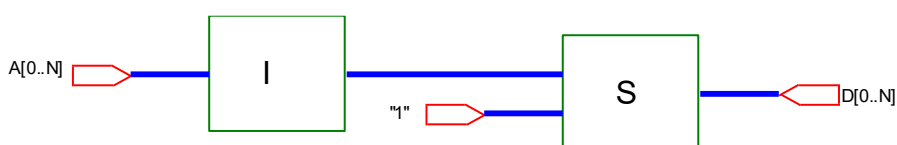
Эта схема называется «инвертор».



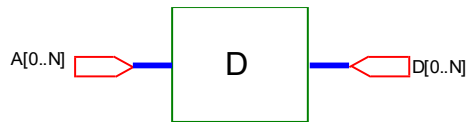
Обозначим это устройство как



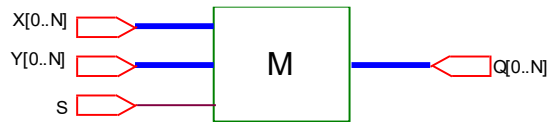
Подав на один вход сумматора код с выхода инвертора, а на другой код числа “1”, на выходе сумматора получим дополнительный код числа.



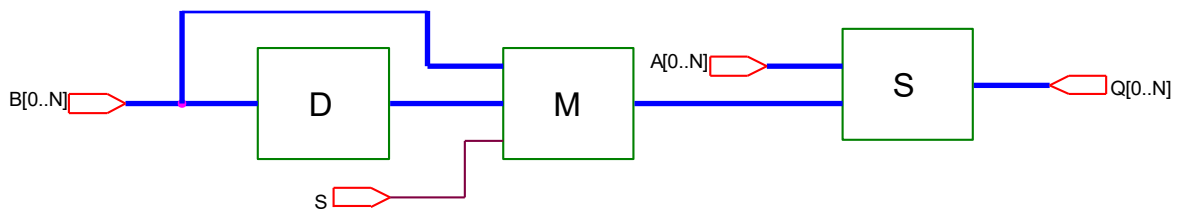
Обозначим это устройство



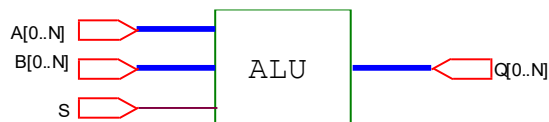
Включив параллельно несколько схем вышеописанного мультиплексора, получим мультиплексор на требуемое число разрядов.



Создадим устройство, способное складывать или вычитать два числа, в зависимости от состояния управляющего сигнала  $s$ . Для этого необходимо подать на один вход сумматора код одного из чисел, а на другой его вход подать либо прямой либо дополнительный код второго из чисел, в зависимости от состояния управляющего входа  $s$ . Такую коммутацию сигналов можно осуществить с помощью мультиплексора.



Разработанное устройство представляет собой примитивный прообраз арифметического устройства (АЛУ) процессора.



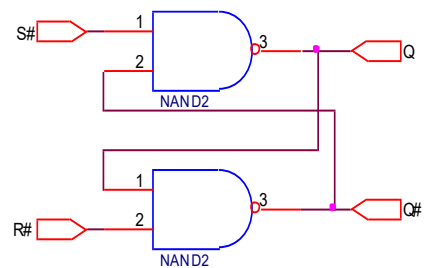
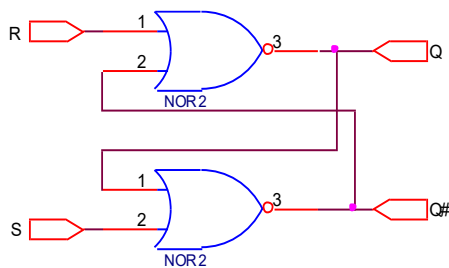


## Лекция № 4

### “Последовательностные логические устройства.”

Все ранее рассмотренные схемы являются схемами мгновенного действия. Информация на их выходах меняется мгновенно с изменением информации на входах. Поэтому, они называются **комбинационными**. Однако, зачастую бывает необходим элемент, способный запоминать информацию о том, что какое-то событие имело место. Назовем его **запоминающим устройством (ЗУ)**.

Элементарной запоминающей ячейкой является триггер. Простейший, т.н. “**R-S**” триггер может быть построен с помощью двух логических элементов путем введения обратных связей.



При подаче активного уровня на вход R на выходе Q устанавливается низкий уровень, а на выходе  $\bar{Q}$  - высокий уровень сигнала, и не меняется при дальнейших изменениях уровня сигнала на входе R. Аналогично, при подаче активного уровня на вход S, на выходе Q устанавливается высокий уровень, а на выходе  $\bar{Q}$  - низкий. Одновременная подача активных уровней на оба входа запрещена. Простейший R-S триггер фиксирует информацию, поступившую на его входы в любой момент времени.

Как можно заметить, состояние выходных сигналов подобных схем зависит не только от входных, но и от выходных сигналов, запоминаемых устройством до прихода новой совокупности входных сигналов. Такие устройства называются **последовательностными схемами** или **конечными автоматами**.

Работу триггерных схем можно описать с помощью таблиц, аналогичных ранее применявшимся таблицам истинности. Однако, в данном случае они называются таблицами переключений, поскольку в них текущее состояние выхода определяется не только текущим состоянием входов, но и предыдущим состоянием самого выхода. Для простейшего RS-триггера Таблица переключений имеет вид:

| $S_n$ | $R_n$ | $Q_n$ | $Q_{n+1}$ |
|-------|-------|-------|-----------|
| 0     | 0     | 0     | 0         |
| 0     | 0     | 1     | 1         |
| 0     | 1     | 0     | 0         |
| 0     | 1     | 1     | 0         |
| 1     | 0     | 0     | 1         |
| 1     | 0     | 1     | 1         |
| 1     | 1     | 0     | X         |
| 1     | 1     | 1     | X         |

Представленной таблице переключений соответствует следующая карта Карно:

| $Q_n$ | $S_n R_n$ | 00 | 01 | 11 | 10 |
|-------|-----------|----|----|----|----|
| 0     |           | 0  | 0  | X  | 1  |
| 1     |           | 1  | 0  | X  | 1  |

Если позиции, соответствующие неопределенным состояниям заполнить единицами, то получим функцию RS-триггера, который при подаче на входы комбинации R=1, S=1 переключает выход Q в состояние 1 независимо от его предыдущего состояния:

$$Q_{n+1} = S_n + Q_n * \overline{R_n} = \overline{\overline{S_n} * \overline{Q_n} * \overline{R_n}} ;$$

Если неопределенные позиции заполнить нулями, то получим уравнение RS-триггера, который при подаче на входы комбинации R=1, S=1 переключает выход Q в состояние 0 независимо от его предыдущего состояния:

$$Q_{n+1} = S_n * \overline{R_n} + Q_n * \overline{R_n} = \overline{R_n} * (S_n + Q_n) = \overline{R_n + \overline{S_n} + \overline{Q_n}} ;$$

Если заполнить неопределенные позиции комбинацией 01, то получим уравнение т.н. Е-триггера, который при подаче на входы комбинации R=1, S=1 оставляет выход Q в его предыдущем состоянии:

$$Q_{n+1} = S_n * \overline{R_n} + S_n * Q_n + \overline{R_n} * Q_n ;$$

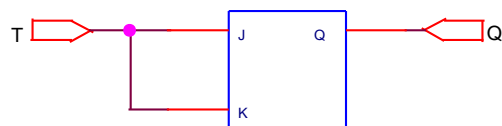
Если заполнить неопределенные позиции комбинацией 10, то получим уравнение т.н. **JK-триггера**, который при подаче на входы комбинации R=1, S=1 переключает выход Q в состояние, инверсное его предыдущему состоянию:

$$Q_{n+1} = S_n * \overline{Q_n} + \overline{R_n} * Q_n ;$$

JK-триггер является часто используемым, поэтому для отличия от RS-триггера его входы не S и R, а J и K соответственно. Поэтому уравнение JK-триггера можно переписать так:

$$Q_{n+1} = J_n * \overline{Q_n} + \overline{K_n} * Q_n ;$$

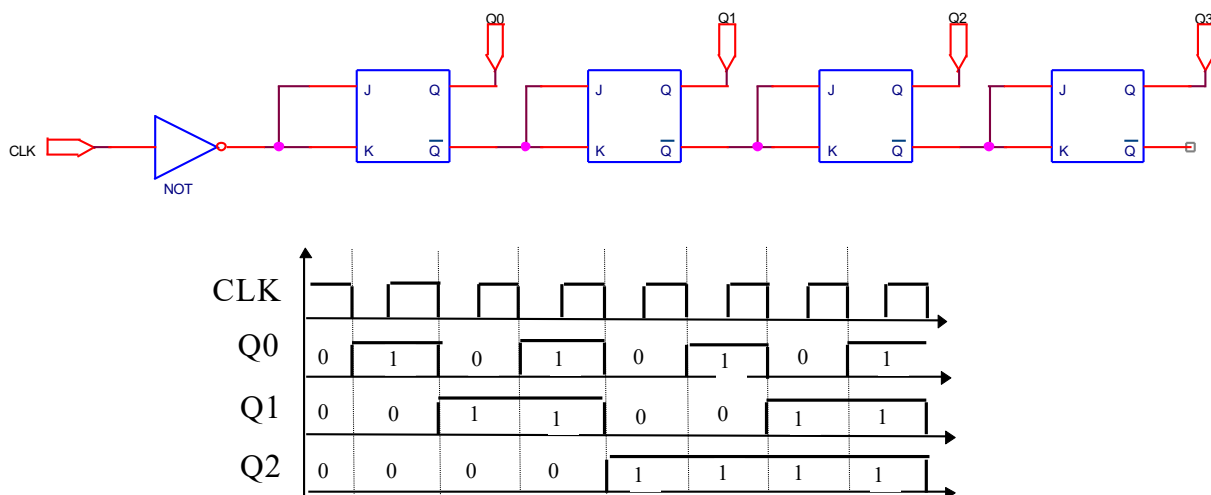
Этот триггер характерен тем, что комбинация входных сигналов 11 приводит к инвертированию выходных сигналов. Данное свойство этого триггера позволяет ему работать в т.н. “счетном” режиме. Для этого необходимо объединить входы J и K. При этом каждый входной импульс будет приводить к инвертированию выходного сигнала. Триггер, работающий в счетном режиме, называют **Т-триггером**.



Уравнение Т-триггера получим, произведя замену J=K=T в последнем уравнении:

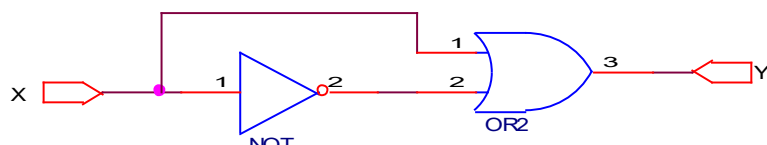
$$Q_{n+1} = T * \overline{Q_n} + \overline{T} * Q_n = T \oplus Q_n ;$$

Объединив последовательно несколько Т-триггеров можно получить простейший двоичный счетчик или делитель частоты.

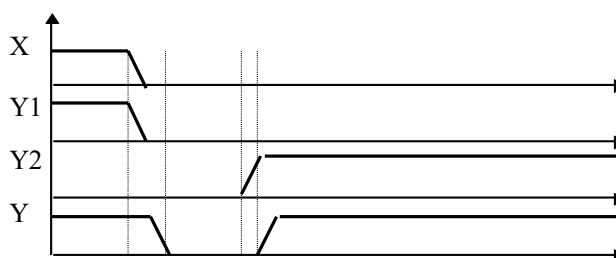


Подобные счетчики обладают пониженным быстродействием т.к. каждый триггер вносит некоторую задержку при прохождении счетных импульсов.

Из-за конечного времени переключения логических элементов и задержки срабатывания в логических схемах возможны т.н. “гонки сигналов” - переходные режимы, вызванные тем, что из-за задержек поступления информации выходной сигнал на некоторое время принимает ложные значения. Рассмотрим эти режимы на примере схемы, реализующей функцию  $Y = X + \bar{X} = 1$ .



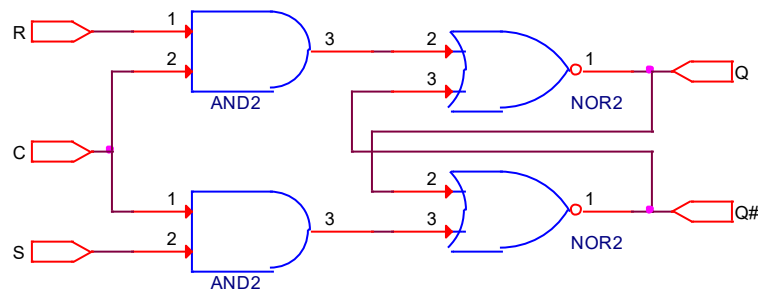
Теоретически выходной сигнал такой схемы Y должен быть всегда равен 1 и не изменяться при изменении входного сигнала X. Однако в реальной схеме инвертор имеет конечное время распространения сигнала, из-за чего сигнал на второй вход элемента “И-НЕ” приходит несколько позже, чем на первый. Т.о., существует короткий интервал времени, в течение которого комбинация входных сигналов логического элемента соответствует нулевому уровню выходного сигнала.



Подобные ситуации могут приводить к ложным срабатываниям триггеров и нарушению алгоритма работы устройства. Устранить ошибки, возникающие за счет “гонок сигналов” в логических цепях можно за счет временного стробирования. Вырабатываемый логическим устройством сигнал в этом случае передается на последующие устройства не непрерывно и не в произвольные моменты времени, а только в такие моменты времени, когда искажение правильных значений выходного сигнала за счет “гонок” заведомо исключено.

Временное стробирование обеспечивается с помощью схем совпадения (схем “И” или “И-НЕ”). В течение времени стробирования на один вход схемы “И” действует стробирующий импульс, а на второй - сигнал логического устройства. Обычно стробирующие сигналы действуют периодически. Их называют **тактовыми** или

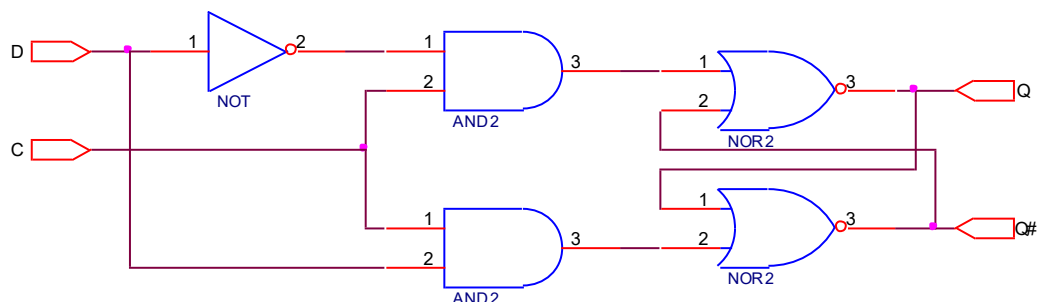
**синхронизирующими**. Период следования тактовых импульсов называют **тактом** логического устройства. Триггеры, работающие с использованием синхронизирующих сигналов, называют **синхронными**. Схема синхронного RS-триггера имеет вид:



Основой схемы служит асинхронный RS-триггер. Сигнал на каждый вход триггера подается через схему “И”. При активном единичном уровне синхросигнала С сигналы со входов S и R передаются на соответствующие входы триггера, а при нулевом уровне синхросигнала на обоих входах триггера устанавливается неактивный нулевой уровень сигналов. Т.о., синхронный R-S триггер, в отличие от него, обеспечивает фиксацию информации только при активном состоянии на входе синхронизации. Логическая функция синхронного RS-триггера имеет вид:

$$Q_{n+1} = \overline{(R_n * C_n) + ((S_n * C_n) + Q_n)};$$

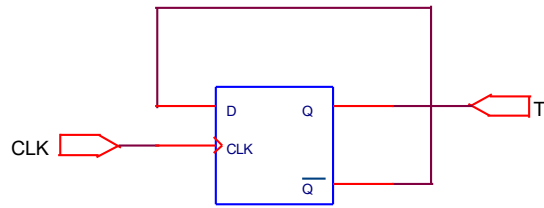
Состояние в синхронных триггерах может измениться только во время действия синхроимпульса. В промежутках между синхроимпульсами выходные напряжения постоянны. Это свойство позволяет создать на основе синхронных RS-триггеров триггеры задержки - **D-триггеры** (Delay Trigger). D-триггер фиксирует информацию, приходящую на один вход данных. У D-триггера значение выходного сигнала в (n+1) такте равно значению входного сигнала в n такте. Т.о., D-триггер запоминает состояние входного сигнала на период одного такта квантования. Схема D-триггера имеет вид:



Он выполняет логическую функцию:

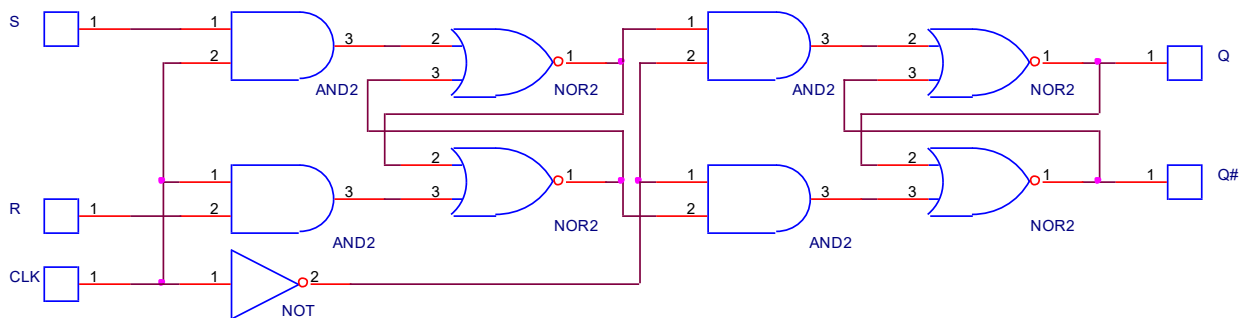
$$Q_{n+1} = C * \overline{D} + \overline{C} * D + Q_n;$$

Из D-триггера можно получить счетный Т-триггер, подав на вход D сигнал с инверсного выхода.

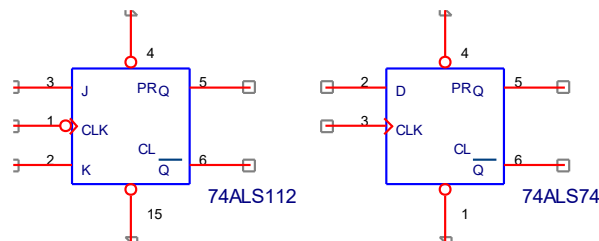


Описанные типы триггеров пропускают входной сигнал на выход в течение всего активного периода сигнала синхронизации.

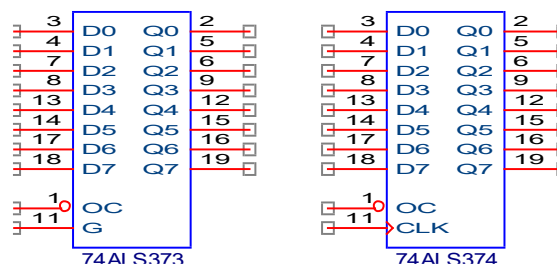
**Двухступенчатый триггер** никогда непосредственно не пропускает входной сигнал на выход.



Синхронные триггеры выпускаются в виде отдельных микросхем, примерами которых могут служить КР1533ТВ9 (74ALS112) - двойной JK-триггер, КР1533ТМ2 (74ALS74) - двойной D-триггер и др.



Объединив несколько D-триггеров с единым входом синхронизации можно получить **регистр**, способный фиксировать многоразрядную информацию. В соответствии со структурой внутренних триггеров существуют регистры-защелки и регистры с синхронным стробированием. **Регистры-защелки** пропускают входную информацию на выход при одном уровне сигнала синхронизации и фиксируют ее при переходе сигнала синхронизации в другое состояние. **Регистры с синхронным стробированием** фиксируют информацию по фронту сигнала синхронизации. Примерами таких регистров могут служить микросхемы КР1533ИР22 (74ALS373) - 8-разрядный регистр-защелка и КР1533ИР23 (74ALS374) - 8-разрядный регистр, фиксирующий информацию по переднему фронту сигнала синхронизации.



**Функции переходов** рассмотренных типов триггеров можно получить из уравнений этих триггеров если в них положить  $C=1$ . Они имеют вид:

$$\begin{aligned} Q_{n+1} &= D_n - \text{для D-триггера;} \\ Q_{n+1} &= J_n * \overline{Q_n} + \overline{K_n} * Q_n - \text{для JK-триггера;} \\ Q_{n+1} &= Q_n \oplus T_n - \text{для Т-триггера.} \end{aligned}$$

Отсюда можно вывести **функции возбуждения** каждого из перечисленных типов триггеров, которые определяют, какие значения должны быть установлены на управляющих входах в предыдущем такте квантования для того, чтобы в следующем такте на выходе триггера установился сигнал заданного значения. Эти функции имеют вид:

$$\begin{aligned} D_n &= Q_{n+1} - \text{для D-триггера;} \\ J_n &= Q_{n+1} \\ K_n &= \overline{Q_{n+1}} - \text{для JK-триггера;} \\ T_n &= Q_{n+1} \oplus Q_n - \text{для Е-триггера.} \end{aligned}$$

Представленные функции используются для синтеза конечных автоматов. Продемонстрируем процесс синтеза на примере создания двоичного счетчика. Составим таблицу переключений счетчика, в которой представим последовательные комбинации выходов счетчика и комбинации, следующие за текущей.

| $Q_{2_n}$ | $Q_{1_n}$ | $Q_{0_n}$ | $Q_{2_{n+1}}$ | $Q_{1_{n+1}}$ | $Q_{0_{n+1}}$ |
|-----------|-----------|-----------|---------------|---------------|---------------|
| 0         | 0         | 0         | 0             | 0             | 1             |
| 0         | 0         | 1         | 0             | 1             | 0             |
| 0         | 1         | 0         | 0             | 1             | 1             |
| 0         | 1         | 1         | 1             | 0             | 0             |
| 1         | 0         | 0         | 1             | 0             | 1             |
| 1         | 0         | 1         | 1             | 1             | 0             |
| 1         | 1         | 0         | 1             | 1             | 1             |
| 1         | 1         | 1         | 0             | 0             | 0             |

После этого можно составить карты Карно и определить уравнения зависимостей последующих состояний разрядов счетчика от предыдущих.

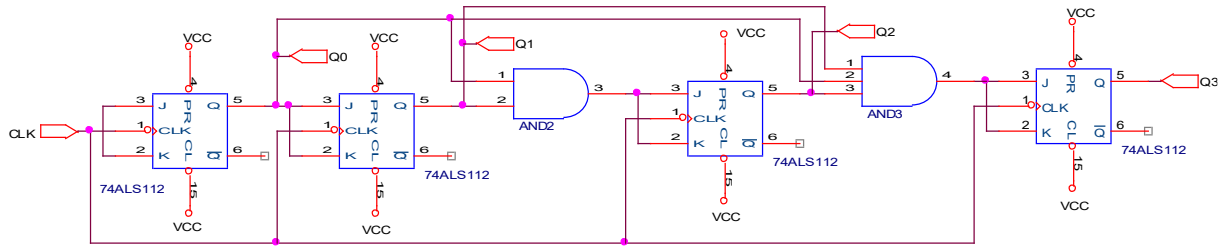
$$\begin{aligned} Q_{0_{n+1}} &= \overline{Q_{0_n}} ; \\ Q_{1_{n+1}} &= Q_{0_n} * \overline{Q_{1_n}} + \overline{Q_{0_n}} * Q_{1_n} = Q_{0_n} \oplus Q_{1_n} ; \\ Q_{2_{n+1}} &= Q_{2_n} * \overline{Q_{1_n}} + Q_{2_n} * \overline{Q_{0_n}} + Q_{0_n} * Q_{1_n} * \overline{Q_{2_n}} = Q_{2_n} * (\overline{Q_{0_n}} + \overline{Q_{1_n}}) + \overline{Q_{2_n}} * Q_{0_n} * Q_{1_n} = ; \\ &= Q_{2_n} * \overline{Q_{0_n}} * \overline{Q_{1_n}} + \overline{Q_{2_n}} * Q_{0_n} * Q_{1_n} = Q_{2_n} \oplus (Q_{0_n} * Q_{1_n}) \end{aligned}$$

Подставляя теперь эти уравнения в выражения для функций возбуждения, получим уравнения для цепей, формирующих управляющие сигналы триггеров соответствующих разрядов счетчика. В случае реализации счетчика на Т-триггерах получим:

$$\begin{aligned} T_{0_n} &= Q_{0_n} \oplus \overline{Q_{0_n}} = 1 ; \\ T_{1_n} &= Q_{1_n} \oplus Q_{0_n} \oplus Q_{1_n} = Q_{0_n} ; \end{aligned}$$

$$T_{2_n} = Q_{2_n} \oplus (Q_{2_n} \oplus (Q_{0_n} * Q_{1_n})) = Q_{0_n} * Q_{1_n}.$$

Эти уравнения соответствуют следующей схеме.



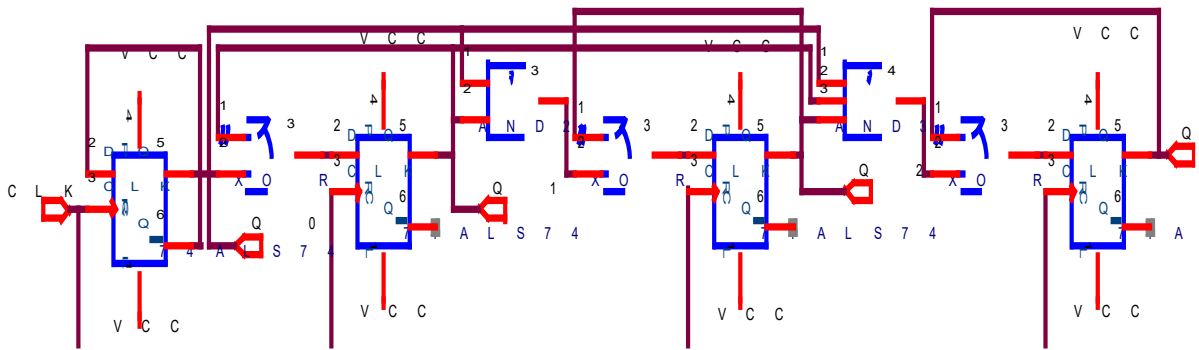
В случае реализации счетчика на D-триггерах получим уравнения:

$$D_{0_n} = \overline{Q_{0_n}};$$

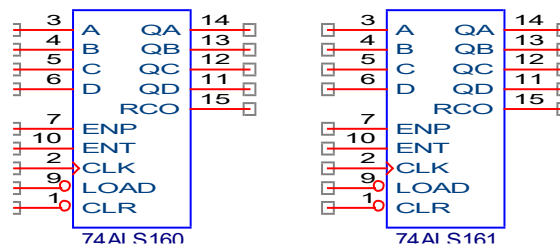
$$D_{1_n} = Q_{0_n} \oplus Q_{1_n};$$

$$D_{2_n} = Q_{2_n} \oplus (Q_{0_n} * Q_{1_n}).$$

Эти уравнения соответствуют следующей схеме.

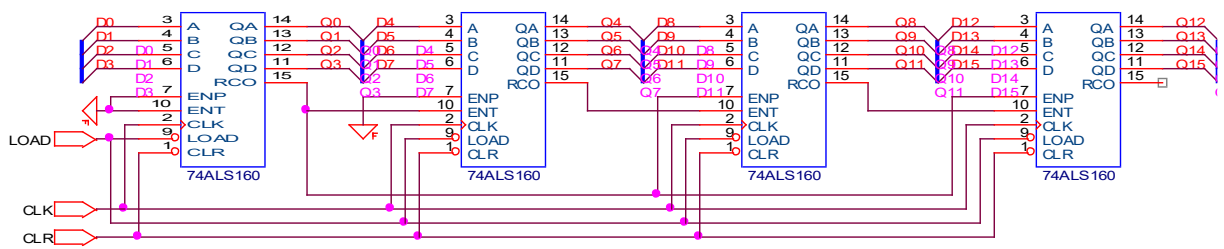


Аналогичным образом можно осуществить синтез двоично-десятичных, вычитающих, реверсивных и других типов счетчиков. Подобные счетчики называются синхронными, т.к. сигнал синхронизации подается одновременно на все триггеры. Такие устройства выпускаются серийно в виде отдельных микросхем, примерами которых могут служить КР1533ИЕ9 (74ALS160) - 4-разрядный двоично-десятичный синхронный счетчик, КР1533ИЕ10 (74ALS161) - 4 разрядный двоичный синхронный счетчик и др.

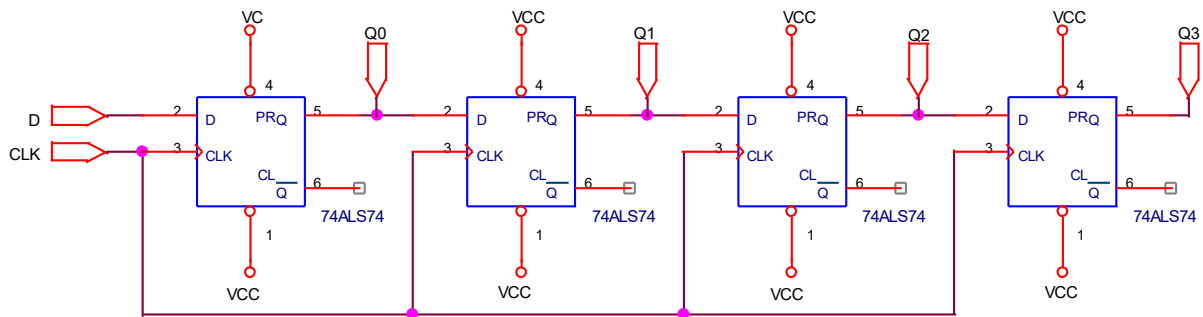


Данные устройства имеют параллельную предзагрузку данных через входы А, В, С, D и строб загрузки LOAD, а также сброс в нулевое состояние по сигналу CLR. Выход переноса в следующий разряд RCO устанавливается в нулевое состояние если текущее состояние счетчика является последним (1111 - для двоичных и 1001 - для двоично десятичных).

Включая последовательно несколько таких устройств можно создавать многоразрядные синхронные счетчики.



Если объединить несколько D-триггеров по цепочке, соответствующей уравнению  $D_{n+1} = Q_n$ , то получим т.н. **регистр сдвига**.



Они используются как устройства с последовательным вводом данных (для ввода N-разрядного двоичного слова необходимо n тактов квантования), а также для выполнения операций умножения или деления на  $2^n$  путем сдвига двоичного слова на n разрядов в лево или вправо соответственно.



## Лекция № 5

### “Устройства электронной памяти.”

Основными операциями в памяти, в общем случае, являются:

- **запись** - занесение информации в память;
- **считывание** - выборка информации из памяти.

Основными характеристиками систем памяти являются:

- **информационная емкость** - максимально возможное количество байтов хранимой информации;
- **ширина выборки (разрядность)** - количество разрядов информации, записываемых в ЗУ или извлекаемых из него за одно обращение;
- **время обращения (быстродействие)** - характеризуется временем цикла обращения к ЗУ, т.е. интервалом времени от момента поступления кода адреса до установления информации на выходе.
- способность сохранения информации при отключения питания.

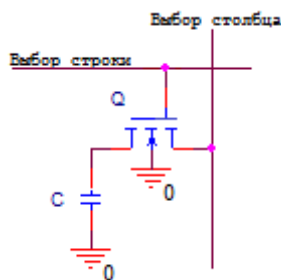
По способу организации доступа к памяти различают ЗУ с произвольным доступом (адресные) и ЗУ с последовательным доступом (безадресные).

По способу хранения информации различают статические и динамические ЗУ.

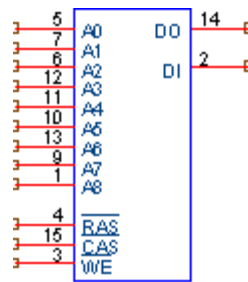
По функциональному назначению различают сверхоперативные ЗУ (СОЗУ), оперативные ЗУ (ОЗУ), постоянные ЗУ (ПЗУ), перепрограммируемые постоянные ЗУ (ППЗУ), стековые ЗУ.

**СОЗУ** представляют собой набор регистров, содержание которых непосредственно используется при обработке информации в микропроцессоре. СОЗУ встроено в БИС микропроцессора и предназначено для хранения команд, операндов и результатов промежуточных вычислений. Разрядность СОЗУ соответствует разрядности микропроцессора.

**ОЗУ (RAM - Random Access Memory)** предназначено для хранения оперативной (переменной) информации, требующейся в процессе обработки. ОЗУ разделяют на динамические и статические. Элементарная ячейка динамического ОЗУ базируется на конденсаторе.



Подобные запоминающие ячейки расположены в виде матрицы на пересечении линий связи, называемых соответственно столбцами и строками. Во время цикла записи конденсатор заряжается до “0” или “1” от шины выбора столбца при активном состоянии сигнала выбора ряда. Во время цикла чтения заряд конденсатора передается через открытый транзистор и шину выбора столбца на выходные усилители, и дальше, на выход. Утечки емкости, а также ее разряд во время цикла чтения обуславливают необходимость регенерации информации, которая заключается в последовательном переборе адресов строк. Примером динамического ОЗУ может служить микросхема 41256 (K565РУ7).



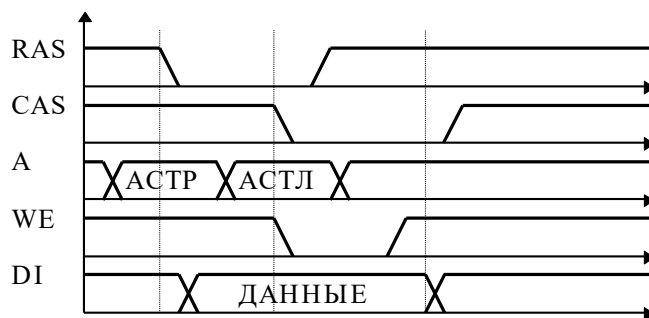
Ее основные характеристики:

Емкость - 256 Кбит.

Разрядность - 1 бит.

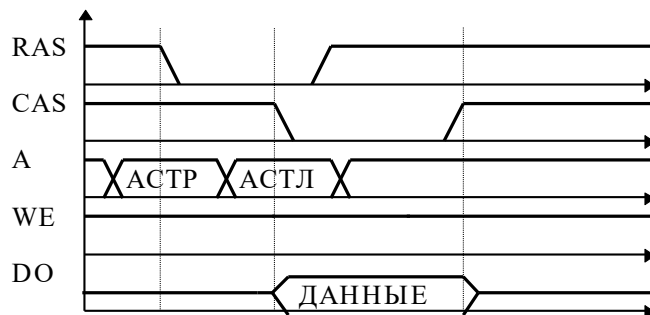
Время цикла “запись/чтение” - около 300 нс.

Диаграмма цикла записи данных имеет вид.



Сначала на адресные входы подается код адреса необходимой строки, который фиксируется внутри микросхемы по отрицательному фронту сигнала RAS. Затем, аналогичным образом, по отрицательному фронту сигнала CAS фиксируется код адреса необходимого столбца. После этого, по отрицательному фронту сигнала WR данные, находящиеся на входе DI, записываются в выбранную запоминающую ячейку.

Диаграмма цикла чтения имеет вид:



Адрес требуемой запоминающей ячейки фиксируется аналогично циклу записи. При нулевом уровне сигнала на входе CAS и единичном на входе WR выход DO выходит из высокоимпедансного состояния и на нем появляются данные из требуемой запоминающей ячейки.

Регенерация информации осуществляется обращением к каждой из 512 строк не реже, чем один раз каждые 8 мс. При этом регенерируется сразу вся строка.

Следующим шагом эволюции динамических ОЗУ стало появление т.н. «синхронных динамических ОЗУ» (SDRAM - Synchronous Dynamic Random Access Memory). В отличие от обычного динамического ОЗУ, эти устройства синхронизируют прием входных сигналов, выдачу выходных сигналов и свое внутреннее состояние со специальным

внешним тактовым сигналом. Помимо этого, весь массив памяти в таких устройствах разбит на несколько независимых банков, которые могут находиться в различных состояниях одновременно. Например, можно совмещать выборку данных из одного банка с установкой адреса в другом банке. Доступ к этим банкам чередуется, и соответственно устраняются задержки, что обеспечивает создание непрерывного потока данных. Это позволяет реализовать конвейерную обработку подаваемых извне запросов записи и считывания данных, что приводит к увеличению частоты поступления таких запросов без необходимости уменьшения времени срабатывания запоминающей ячейки.

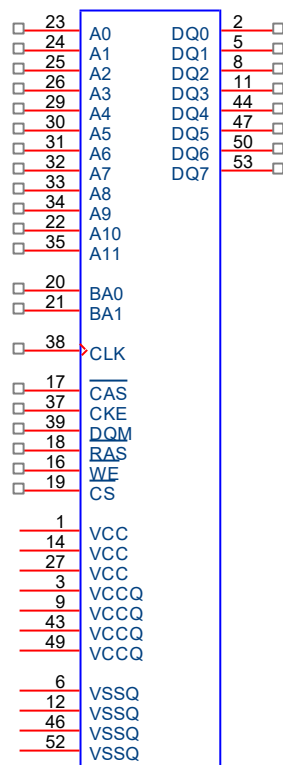
В памяти SDRAM организована пакетная обработка данных, что позволяет производить обращение по новому адресу столбца ячейки памяти на каждом тактовом цикле. Смысл пакетной обработки заключается в том, при активированной строке задание адреса одного столбца позволяет получить доступ сразу к последовательности нескольких столбцов (пакету столбцов) без дополнительного указания их адресов.

В памяти SDRAM ядро и буферы обмена работают в синхронном режиме на одной и той же частоте (100 - 175 МГц). Передача каждого бита из буфера происходит с каждым тактом работы ядра памяти.

Примером синхронного динамического ОЗУ может служить микросхема MT48LC8M8.

Ее основные характеристики:

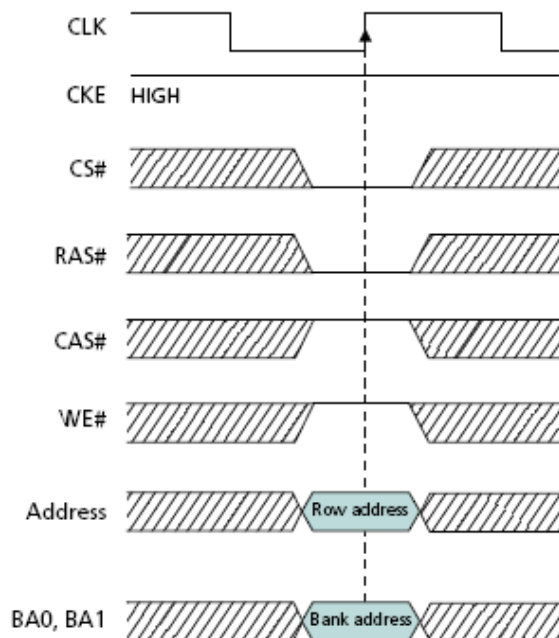
- Емкость – 64 Мбита;
- Разрядность – 8 бит;
- Организация – 4 банка по 2 Мбайта;
- частота тактового сигнала – 133 МГц.



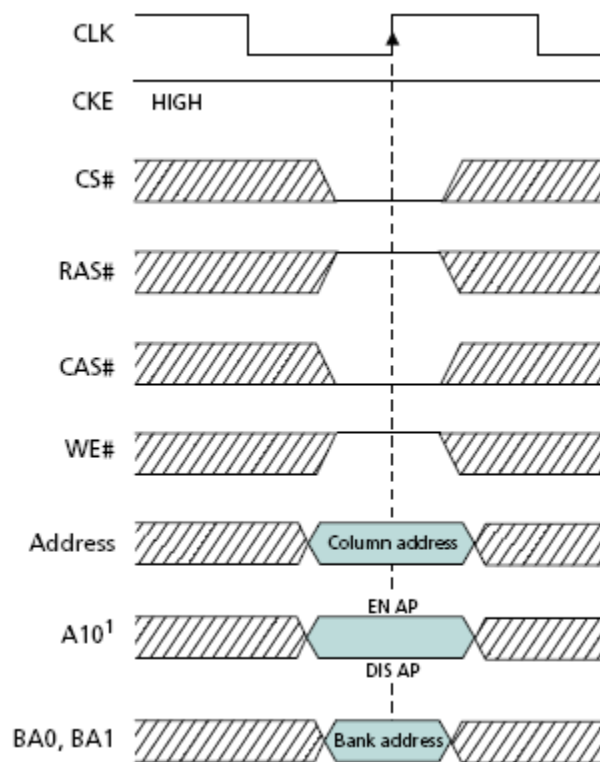
сигнала CLK с помощью других управляющих сигналов. Возможные команды приведены в таблице.

| Команда                             | CS# | RAS# | CAS# | WE# | DQM | BA/A        | DQ     |
|-------------------------------------|-----|------|------|-----|-----|-------------|--------|
| Блокировка                          | 1   | *    | *    | *   | *   | *           | *      |
| Ожидание (NOP)                      | 0   | 1    | 1    | 1   | *   | *           | *      |
| Выбор банка и строки (ACTIVE)       | 0   | 0    | 1    | 1   | *   | Банк/строка | *      |
| Чтение (READ)                       | 0   | 1    | 0    | 1   | 0/1 | Банк/строка | *      |
| Запись (WRITE)                      | 0   | 1    | 0    | 0   | 0/1 | Банк/строка | Данные |
| Прерывание пакета (BURST TERMINATE) | 0   | 1    | 1    | 0   | *   | *           | *      |
| Деактивация строки (PRECHARGE)      | 0   | 0    | 1    | 0   | *   | Код         | *      |
| Регенерация (REFRESH)               | 0   | 0    | 0    | 1   | *   | *           | *      |
| Загрузка регистра состояния         | 0   | 0    | 0    | 0   | *   | Код         | *      |

Команда ACTIVE используется для активации строки в конкретном банке. Ее диаграмма представлена на рисунке.

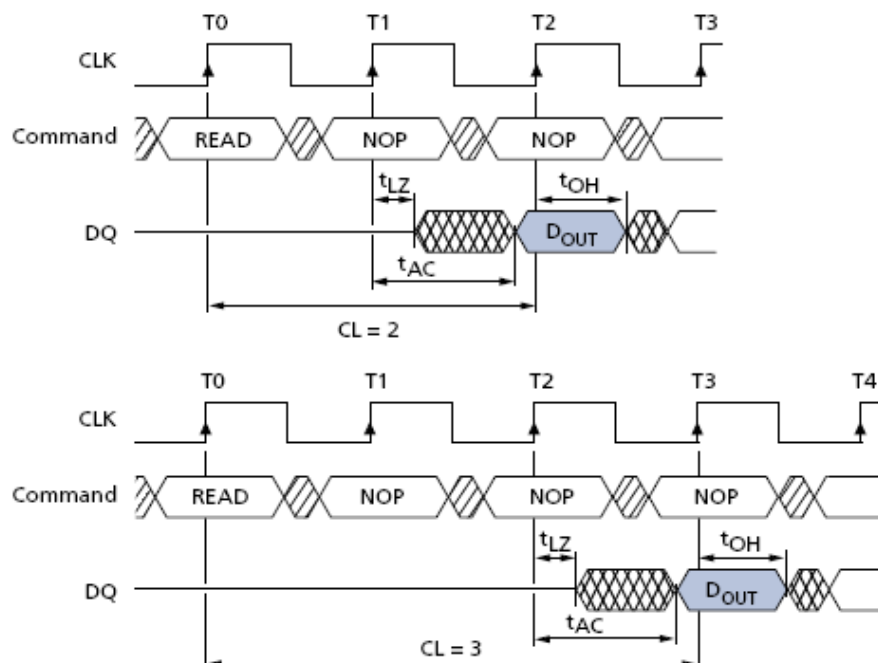


Команда READ используется для инициирования пакетного чтения из активной строки активного банка. Ее диаграмма представлена на рисунке.

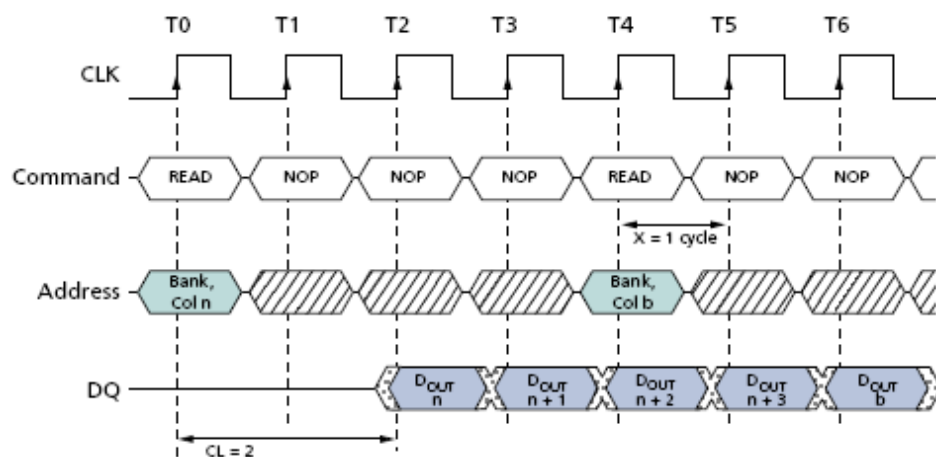


Если сигнал A10 равен 1, то после завершения пакетного чтения активная строка закрывается. Если A10 равен 0, то она остается открытой для последующих чтений.

Запрашиваемые данные могут быть прочитаны на выводах DQ через один или два такта ожидания NOP, в зависимости от их количества CL, запрограммированного в регистре состояния.

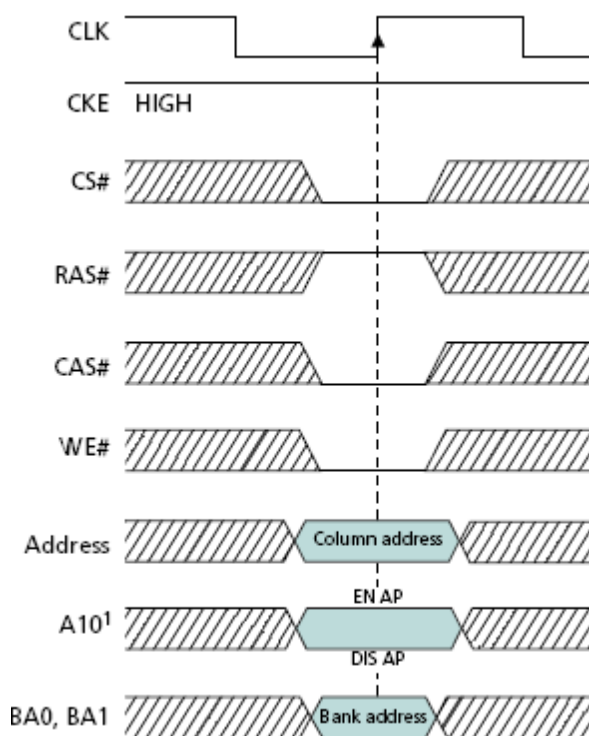


При пакетном чтении в последующих тактах ожидания NOP считываются данные из следующих столбцов. Пример такого чтения представлен на рисунке.

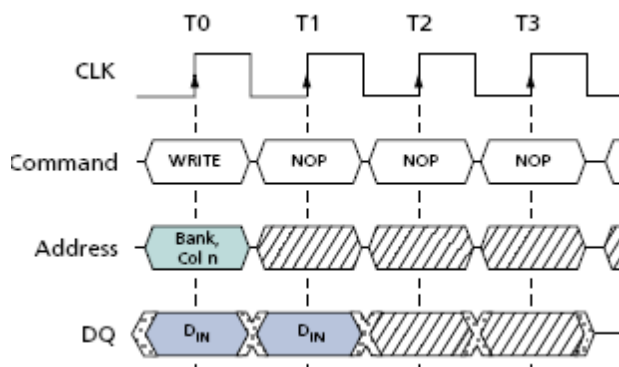


При этом DQM должен быть равен нулю во время выполнения команды READ и последующих тактов ожидания в зависимости от длины считываемого пакета данных.

Команда WRITE используется для инициирования пакетной записи в активную строку активного банка. Ее диаграмма представлена на рисунке.



Данные могут быть записаны в пакетном режиме во время последующих тактов ожидания, в соответствии с приведенной на рисунке диаграммой.



При этом DQM должен быть равен нулю во время выполнения команды WRITE и последующих тактов ожидания в зависимости от длины считываемого пакета данных.

Команда BURST TERMINATE используется для преждевременного прерывания пакетного чтения.

Команда PRECHARGE используется для выключения активной строки в одном или во всех банках сразу. Если сигнал A10 равен 0, то выключается активная строка в банке, определяемом сигналами BA0 и BA1. Если A10=1, то выключается активная строка во всех банках.

Команда REFRESH используется для регенерации содержимого всего массива памяти.

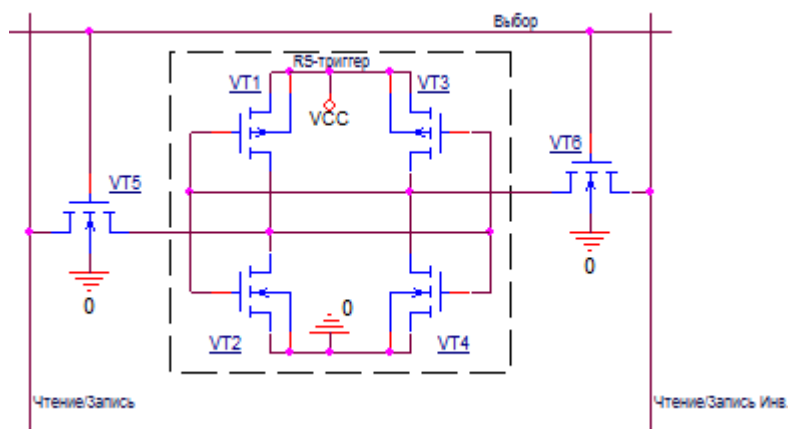
Команда загрузки регистра состояния используется для задания количества циклов ожидания CL при чтении и записи данных, и размера пакета при реализации пакетного чтения и записи.

Следующим эволюционным шагом на пути повышения быстродействия динамического ОЗУ стало появление памяти типа DDR SDRAM (Double Data Rate Synchronous Dynamic Random Access Memory). Ее принципиальное усовершенствование заключается в том, что обмен данными с микросхемой осуществляется как по положительному, так и по отрицательному перепаду сигнала CLK. Это позволяет удвоить частоту записи и считывания информации. При этом ядро памяти работает на той же частоте, что и в SDRAM, и синхронизируется только по положительным перепадам сигнала CLK. На удвоенной частоте работают только буферы ввода-вывода микросхемы памяти, которые синхронизируются по обоим фронтам CLK. Для этого необходимо, чтобы на каждом периоде CLK ядро передавало буферу в 2 раза больше данных, чем в обычной SDRAM. Поэтому данные между ядром и буфером ввода-вывода передаются по двум независимым каналам связи одновременно. Таким образом, ядро памяти работает в режиме т.н. «двойной предвыборки» (2n-Prefetch), и передает на каждом периоде CLK в 2 раза больше информации. Поэтому, в частности, минимальная длина пакета данных не может быть меньше 2.

Для повышения надежности обмена информацией на таких высоких частотах ввод сигнала CLK сделан в этих устройствах дифференциальным.

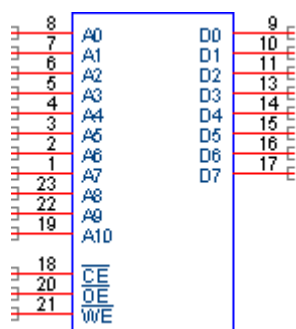
Развитие этой технологии привело к появлению на рынке устройств динамической памяти типа DDR2 SDRAM и DDR3 SDRAM, принципиальное отличие которых состоит в том, что их ядро работает в режиме «учетверенной» (4n-Prefetch) и «увосмеренной» (2n-Prefetch) предвыборки соответственно. Для этого между ядром и буфером ввода-вывода реализуется 4 и 8 независимых каналов связи соответственно. Это позволяет в соответствующее количество раз повысить частоту обмена информацией с устройством.

Элементарной запоминающей ячейкой статического ОЗУ является триггер. Триггер, как схема с двумя устойчивыми состояниями не требует регенерации информации. Он способен хранить информацию как угодно долго при включенном питании. На рисунке представлена схема стандартной шеститранзисторной запоминающей ячейки статического ОЗУ.



Ее основу составляет RS-триггер на транзисторах VT1 – VT4, дополненный транзисторами доступа VT5 и VT6. Для записи единицы информации по линиям чтения/записи подаются инверсные значения (0,1 или 1,0). Тот транзистор, на чей исток подан ноль, оказывается подключен параллельно с нижним транзистором соответствующей стойки, а тот, на чей исток подана единица – параллельно с верхним транзистором своей стойки. При подаче отпирающего сигнала на линию выбора такая комбинация обеспечивает переключение триггера в требуемое состояние. После снятия сигнала выбора обратные связи удерживают триггер в данном состоянии.

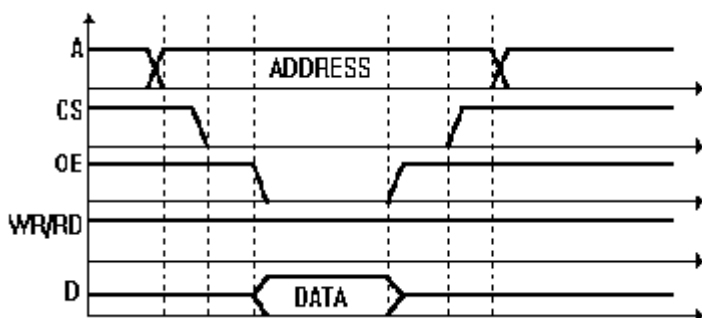
Примером статического ОЗУ может служить микросхема 6216 (KP537PY10).



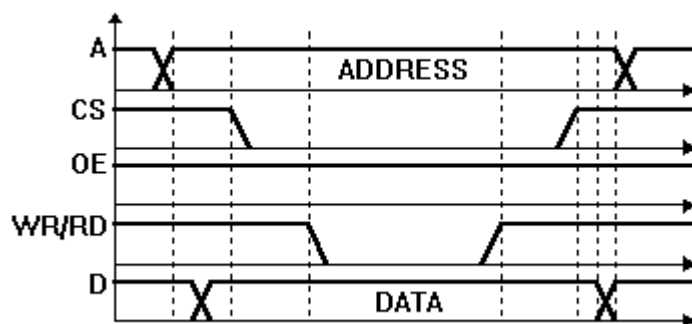
Ее основные характеристики:

- Емкость - 16384 бита;
- Разрядность - 8 бит (2048\*8);
- Время цикла записи/чтения - 200 нс.

Диаграммы циклов чтения и записи имеют вид:







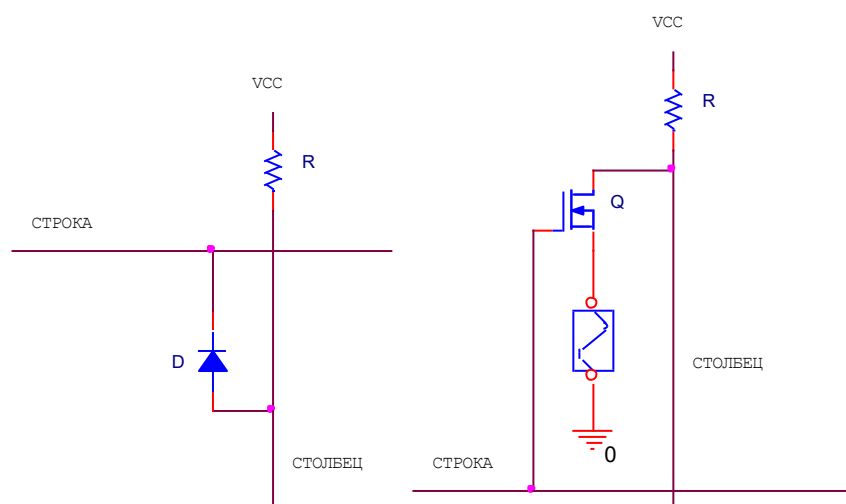
На адресные входы подается код полного адреса требуемой запоминающей ячейки (а не отдельные адреса строки и столбца как в предыдущем случае) и удерживается на протяжении всего цикла. Вход CS является входом выбора кристалла. Микросхема осуществляет цикл записи или чтения только при нулевом уровне сигнала на этом входе. Вход OE является входом разрешения выхода. При подаче на этот вход сигнала нулевого уровня выходы данных микросхемы выходят из высокоимпедансного состояния и на них появляются данные из запоминающей ячейки, адрес которой в данный момент присутствует на адресных входах. Запись данных в микросхему осуществляется по положительному перепаду сигнала на входе WR/RD при единичном уровне сигнала на входе OE и нулевом - на входе CS.

Статические ОЗУ, как правило, быстрее чем динамические и не требуют, как последние регенерации информации в процессе работы. Однако, они сложнее динамических ОЗУ по своей организации. Поэтому статические ОЗУ как правило имеют меньший объем и большую стоимость чем динамические ОЗУ.

ПЗУ - запоминающее устройство, содержимое которого не может быть изменено микропроцессором в процессе работы и сохраняется при выключении питания. Эти запоминающие устройства в составе системы работают только в режиме чтения.

По способу программирования ПЗУ делятся на масочные и перепрограммируемые.

Масочные ПЗУ реализуются на базе диодных или транзисторных матриц.

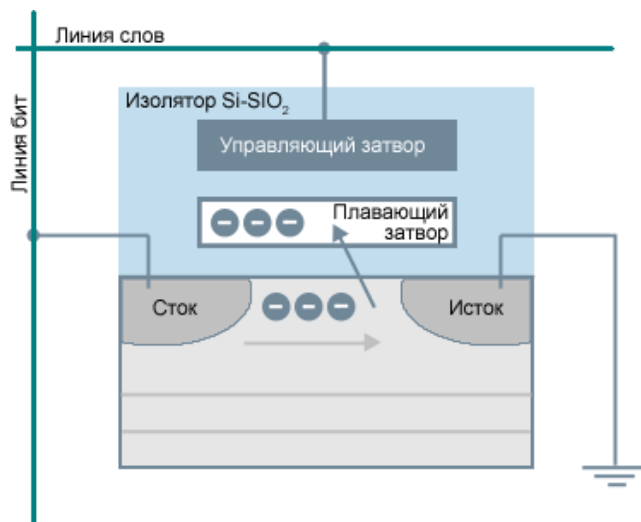


Выбор необходимой информации осуществляется подачей нуля на соответствующую линию строки. Если диод есть, то на линии столбца окажется ноль, если нет - то единица. Запись информации осуществляется путем выжигания ненужных диодов лазерным или электронным лучом на заводе в процессе изготовления микросхемы. В дальнейшем записанная информация изменена быть не может.

Более быстрое ПЗУ реализуется на биполярных и МОП - транзисторах. Информация записывается путем нанесения или отсутствия металлизации истока.

Перепрограммируемые ПЗУ также являются энергонезависимыми, однако позволяют многократно изменять свое содержимое с помощью особых циклов очистки и программирования, реализуемых в специальных устройствах, называемых программаторами ПЗУ.

В качестве элементарной запоминающей ячейки, расположенной на каждом пересечении строк и столбцов матрицы, у таких устройств используется т.н. полевой транзистор с плавающим затвором. Его структура представлена на рисунке.



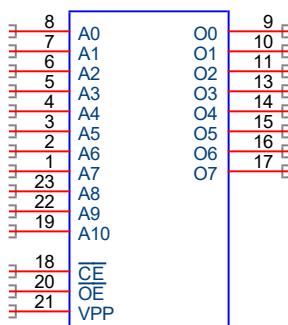
Программирование или занесение информации в такую запоминающую ячейку осуществляется подачей повышенного напряжения между управляющим затвором и истоком. При этом между стоком и истоком начинает протекать ток. Повышенное управляющее напряжение приводит к возникновению т.н. «туннельного эффекта», из-за которого часть электронов с повышенной энергией проникает сквозь тонкий слой изолятора и оседают на плавающем затворе. После снятия повышенного управляющего напряжения заряд, накопленный на плавающем затворе, может храниться очень долгое время. В зависимости от величины заряда на плавающем затворе изменяется пороговое напряжение отпирающего транзистора. На этом основан принцип считывания информации из такой ячейки.

Для изменения содержимого подобной запоминающей ячейки необходимо сначала осуществить ее «очистку», которая заключается в стекании заряда с плавающего затвора. Она осуществляется подачей на управляющий затвор высокого напряжения обратной полярности. При этом электроны с плавающего затвора «туннелируют» т.е. стекают в исток.

Перепрограммируемые ПЗУ бывают двух типов: электрически стираемые и с ультрафиолетовым стиранием. Это объясняется тем, что схемы обеспечения режима очистки, реализующие формирование повышенного напряжения обратной полярности и подачу его на управляющие затворы транзисторов запоминающих ячеек, занимают много места на кристалле микросхемы, что препятствует реализации подобной памяти большого объема. Поэтому в микросхемах с ультрафиолетовым стиранием схема очистки не реализуется. Вместо этого, в корпусе микросхемы делается прозрачное окно, закрытое кварцевым стеклом. При попадании сквозь это окно ультрафиолетового излучения на запоминающие ячейки происходит эффект, аналогичный подаче повышенного обратного напряжения на их затворы, в результате чего, заряд с плавающих затворов стекает и ячейка очищается. Таким образом, для очистки подобной микросхемы ППЗУ ее необходимо подвергнуть ультрафиолетовому облучению в течение 10 -20 минут, после чего в нее можно заносить новую информацию.

В электрически стираемых ППЗУ схема очистки реализована. Поэтому, как очистка, так и перепрограммирование таких устройств осуществляется непосредственно в программаторе. Однако, информационная емкость таких устройств, как правило, меньше по сравнению с ППЗУ с ультрафиолетовым стиранием.

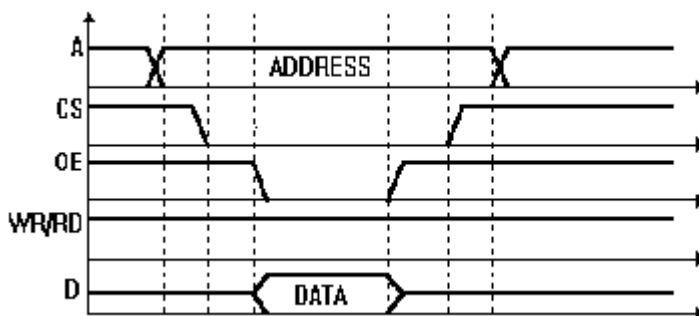
Примером ППЗУ с УФ стиранием может служить микросхема К573РФ2.



Ее основные характеристики:

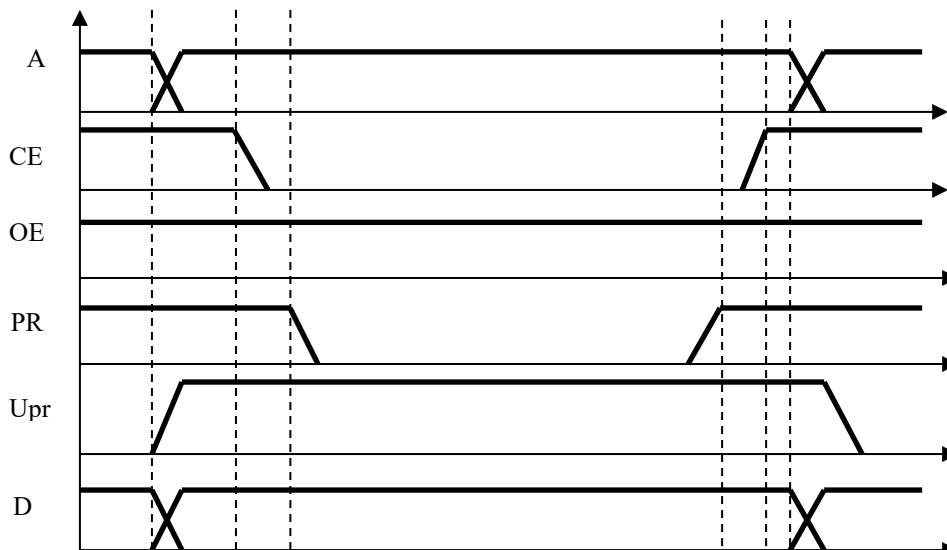
- емкость - 16384 Бит;
- разрядность - 8 Бит;
- время цикла чтения - 200 нс.

Диаграмма цикла чтения имеет вид.



Данные из запоминающей ячейки, адрес которой присутствует на адресных входах в течение всего цикла, считываются при подаче сигналов нулевого уровня на вход выбора кристалла CS и вход разрешения выхода CEO. На входах PR и U<sub>PR</sub> при этом должны быть единичные уровни сигналов.

Диаграмма цикла программирования имеет вид.



Программирование запоминающей ячейки осуществляется при единичном уровне сигнала на входе CEO и нулевом уровне сигнала на входе CS путем подачи повышенного напряжения программирования на вход  $U_{PR}$  и нулевого импульса программирования длительностью 50 мс на вход PR . Как видно из приведенных диаграмм, режим программирования существенно отличается от режима чтения и в общем случае не может быть реализован в работающей системе. Программирование подобных микросхем осуществляется в специальных устройствах, называемых программаторами ППЗУ.

## Лекция № 6

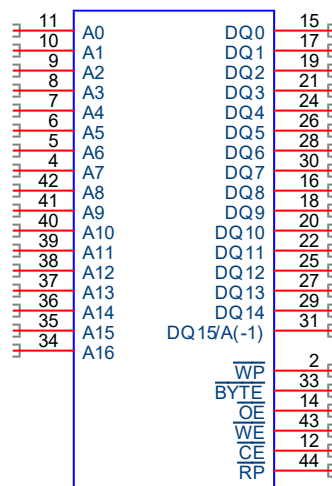
### «Flash-память».

Flash-память - электрически перепрограммируемое интеллектуальное ПЗУ, которое может быть перепрограммировано непосредственно в работающей системе.

В качестве элементарной запоминающей ячейки в таких устройствах памяти, аналогично обычным ППЗУ, используется полевой транзистор с плавающим затвором. Однако, наряду с матрицей запоминающих ячеек, на кристалле Flash-памяти реализованы схемы формирования и подачи повышенного напряжения прямой и обратной полярности, необходимые для осуществления программирования и очистки массива памяти, а также встроенный управляющий микроконтроллер. Он реализует все временные алгоритмы очистки и программирования массива памяти, а также командный интерфейс с внешним управляющим устройством.

Flash-технология была изобретена инженером компании Toshiba Фудзиро Масуокой в 1984 году. Первая микросхема Flash-памяти была выпущена фирмой Intel в 1988 году. В обычном режиме чтения, в который Flash-память переходит сразу после включения питания, она ничем не отличается от обычного ПЗУ. Для того, чтобы очистить блок памяти или записать байт необходимо сначала задать последовательность команд внутреннему контроллеру. Процедуры очистки и записи внутренний контроллер осуществляет самостоятельно.

Примером микросхемы Flash-памяти может служить микросхема 28F200BV-T/V из семейства Boot Block Flash Memory фирмы Intel.



Характеристики микросхемы:

- Емкость - 2 Мбита;
- Разрядность - 8/16 бит (128К\*16 или 256К\*8);
- Время цикла чтения - 60 - 120 нс;
- Количество циклов перезаписи - 100000;
- Потребление в активном режиме - 60 мА;
- Потребление в режиме Power Down - 0.2 мкА.

Выводы микросхемы имеют следующее функциональное назначение:

- \* A0 - A16 - шина адреса;
- \* D0 - D15 - шина данных;
- \*  $\overline{CE}$  - вход выбора кристалла;
- \*  $\overline{OE}$  - вход разрешения выхода;
- \*  $\overline{WE}$  - вход разрешения записи (строб записи);

\*  $\overline{RP}$  - сигнал сброса микросхемы и ввода ее в режим пониженного энергопотребления;

\*  $\overline{BYTE}$  - вход выбора разрядности (8 или 16 бит).

Микросхема может работать в режимах с разрядностью 8 или 16 бит в зависимости от уровня сигнала на входе  $\overline{BYTE}$ . Если на этом выводе единичный уровень сигнала, то микросхема работает с разрядностью 16 бит. Данные, в этом случае, передаются и принимаются через выводы DQ0 - DQ15. Если на выводе  $\overline{BYTE}$  нулевой уровень сигнала, то микросхема работает с разрядностью 8 бит. В этом случае данные передаются и принимаются через выводы DQ0 - DQ7, выводы DQ8 - DQ14 находятся в высокоимпедансном состоянии, а вывод DQ15/A-1 используется как вход дополнительного младшего адресного разряда (т.к. при разрядности 8 бит объем адресуемых ячеек микросхемы возрастает в два раза).

Организационно весь объем памяти микросхемы разбит на пять блоков:

- блок загрузки (Boot Block), объемом 16 Кбайт;
- два блока параметров (Parameter Block), объемом по 8 Кбайт каждый;
- два основных блока (Main Block), объемом 96 и 128 Кбайт.

Адресное пространство распределено среди указанных блоков следующим образом.

| 28F200BV-T                                  | Блок                    | 28F200BV-B                                  |
|---------------------------------------------|-------------------------|---------------------------------------------|
| 1FFFFH (3FFFFH)<br>...<br>1E000H (3C000H)   | Boot Block<br>16 KB     | 00000H<br>...<br>01FFFFH (03FFFFH)          |
| 1DFFFFH (3BFFFFH)<br>...<br>1D000H (3A000H) | Parameter Block<br>8 KB | 02000H (04000H)<br>...<br>02FFFFH (05FFFFH) |
| 1CFFFFH (39FFFFH)<br>...<br>1C000H (38000H) | Parameter Block<br>8 KB | 03000H (06000H)<br>...<br>03FFFFH (07FFFFH) |
| 1BFFFFH (37FFFFH)<br>...<br>10000H (20000H) | Main Block<br>96 KB     | 04000H (08000H)<br>...<br>0FFFFH (1FFFFH)   |
| 0FFFFH (1FFFFH)<br>...<br>00000H            | Main Block<br>128 KB    | 10000H (20000H)<br>...<br>1FFFFH (3FFFFH)   |

Микросхема может работать в режимах с разрядностью 8 и 16 бит. В скобках указаны адреса для режима с разрядностью 8 бит. У микросхем с индексом В Boot Block расположен в области младших адресов, а у микросхем с индексом Т - в области старших адресов. Это позволяет использовать микросхемы в системах с микропроцессорами, начинающими работу в области старших и младших адресов.

Интерфейс между процессором и внутренним контроллером Flash-памяти называется Command User Interface (CUI). Взаимодействие между процессором и внутренним контроллером осуществляется путем записи в CUI определенного набора команд и чтения регистра состояния микросхемы. Текущий режим работы микросхемы определяется записью соответствующей команды в CUI. Возможны следующие команды:

- \* Чтение массива/Сброс программирования и стирания. Код команды - 0FFH.
- \* Предустановка программирования. Код команды - 40H или 10H.
- \* Предустановка очистки. Код команды - 20H.
- \* Подтверждение очистки/Возобновление очистки. Код команда - 0D0H.
- \* Приостановка очистки. Код команды - 0B0H.
- \* Чтение регистра состояния. Код Команды - 70H.

- \* Очистка регистра состояния. Код команды - 50H.
  - \* Чтение идентификатора. Код команды - 90H.
- Возможные режимы работы микросхемы представлены в следующей таблице.

| Режим                              | 1-й цикл шины |       |        | 2-й цикл шины |       |        |
|------------------------------------|---------------|-------|--------|---------------|-------|--------|
|                                    | Цикл          | Адрес | Данные | Цикл          | Адрес | Данные |
| Чтение массива                     | Запись        | х     | 0FFH   |               |       |        |
| Чтение регистра состояния          | Запись        | х     | 70H    | Чтение        | х     | SRD    |
| Очистка регистра состояния         | Запись        | х     | 50H    |               |       |        |
| Запись байта или слова             | Запись        | WA    | 40H    | Запись        | WA    | WD     |
| Очистка блока                      | Запись        | BA    | 20H    | Запись        | BA    | 0D0H   |
| Приостановка/Возобновление очистки | Запись        | х     | 0B0H   | Запись        | х     | 0D0H   |

где:

SRD - содержимое регистра состояния;

WA - адрес записываемой ячейки;

WD - записываемые данные;

BA - любой адрес внутри стираемого блока.

Регистр состояния - это внутренний регистр микросхемы, биты которого определяют текущее состояние микросхемы. Регистр состояния микросхемы имеет следующую структуру.

| D7   | D6  | D5 | D4 | D3   | D2 | D1 | D0 |
|------|-----|----|----|------|----|----|----|
| WSMS | ESS | ES | PS | VPPS | R  | R  | R  |

Отдельные биты регистра состояния имеют следующее значение:

- WSMS (Write State Machine Status) - статус операции:
  - 1 - готово;
  - 0 - занято;
- ESS (Erase Suspend Status) - статус приостановки очистки:
  - 1 - очистка приостановлена;
  - 0 - очистка продолжается/завершена;
- ES (Erase Status) - статус очистки:
  - 1 - ошибка при очистке блока;
  - 0 - блок очищен успешно;
- PS (Program Status) - статус программирования:
  - 1 - ошибка при записи слова/байта;
  - 0 - байт/слово записан успешно;
- VPPS - статус напряжения программирования:
  - 1 - пониженное напряжение программирования, операция прервана;
  - 0 - напряжение программирования в норме;
- R - зарезервировано.

Рассмотрим отдельные режимы работы микросхемы.

#### Режим чтения массива.

Микросхема переходит в этот режим после включения питания и выхода из состояния сброса (положительный перепад сигнала на входе  $\overline{RP}$ ) без дополнительных команд. В этом режиме микросхема подобна обычному ПЗУ.

Если микросхема не находится в режиме чтения массива, то перейти в него можно записав в CUI команду “Чтение массива” (0FFH) по любому адресу.

#### Режим чтения регистра состояния.

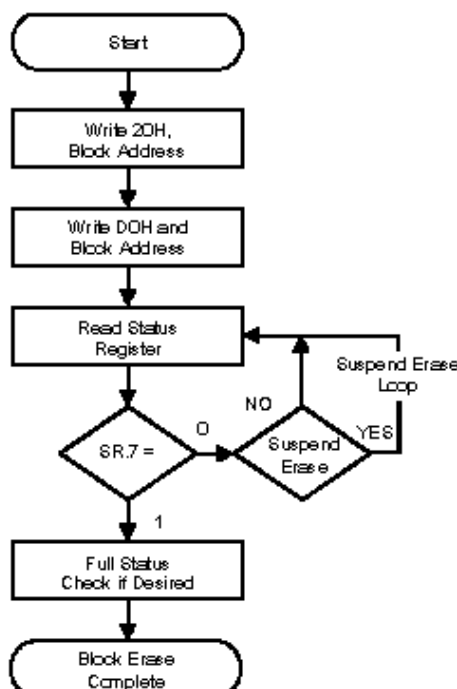
Для того, чтобы прочитать регистр состояния необходимо записать команду “Чтение регистра состояния” (70H) в CUI по любому адресу. После этого цикл чтения по любому адресу приведет к считыванию содержимого регистра состояния. Содержимое регистра состояния выдается всегда на выходы Q0 - Q7. В 16-разрядном режиме выходы Q8 - Q14 при этом находятся в высокоимпедансном состоянии, а вывод Q15/A-1 используется как дополнительный адресный вход. Для возврата в режим чтения массива после чтения регистра состояния необходимо записать команду “Чтение массива” (0FFH) в CUI по любому адресу.

#### Режим очистки регистра состояния.

Для очистки регистра состояния необходимо записать команду “Очистка регистра Состояния” (50H) в CUI по любому адресу. После этого все биты регистра состояния устанавливаются в 0. Для перехода в режим чтения массива после этого необходимо записать команду “Чтение массива” (0FFH) в CUI по любому адресу.

#### Режим очистки блока.

Для того, чтобы очистить любой из блоков, входящих в состав микросхемы необходимо последовательно записать команду “Предустановка очистки” (20H) и “Подтверждение очистки” (0D0H) в CUI по любому адресу, входящему в очищаемый блок. При очистке блока все биты во всех байтах, входящих в очищаемый блок устанавливаются в 1. В течение выполнения операции очистки бит D7 регистра состояния микросхемы равен 0, что свидетельствует о продолжении операции. После окончания операции очистки D7 становится равным 1. Бит D5 регистра состояния индицирует результат выполнения операции. Во время операции очистки уровень напряжения программирования на выводе VPP должен быть на уровне 5В. Если уровень напряжения VPP понижен, то операция выполняться не будет и бит D3 регистра состояния будет установлен в 1. После завершения операции необходимо очистить регистр состояния. Т.о. алгоритм операции очистки блока имеет следующий вид.



Для перехода в режим чтения массива после этого необходимо записать команду “Чтение массива” (0FFH) в CUI по любому адресу. Одновременно можно очистить только один блок. Очистить можно только весь блок целиком. Время очистки Boot Block - 0.3с, Parameter Block и Main Block - 0.6с.

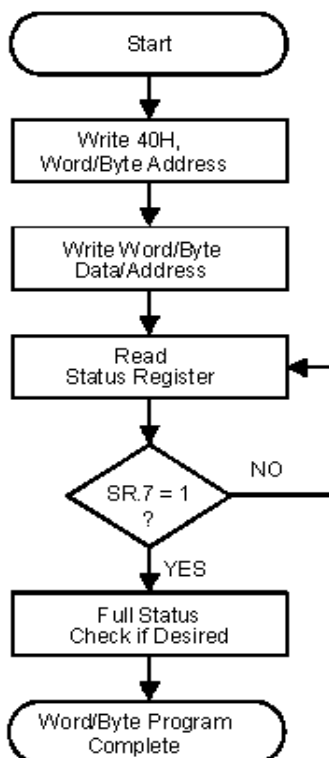
#### Режим приостановки - возобновления очистки.



Т.к. операция очистки блока требует большого количества времени, ее можно приостановить с целью считать информацию из другого блока микросхемы. Если операция очистки блока была запущена, то запись команды “Приостановка очистки” (0B0H) в CUI по любому адресу приостанавливает ее. При этом текущее состояние микросхемы запоминается. Бит D6 регистра состояния индицирует приостановку процесса очистки. После этого в CUI может быть записана команда “Чтение массива” для чтения информации из любого другого блока микросхемы. Можно, также записать команду “Чтение регистра состояния” для считывания информации о состоянии микросхемы. Для продолжения процесса очистки блока с того места, где он был прерван необходимо записать команду “Возобновление очистки” (0D0H) в CUI по любому адресу.

#### Режим программирования байта/слова.

Для записи информации по определенному адресу в предварительно очищенную ячейку необходимо сначала записать в CUI по этому адресу команду “Предустановка программирования” (40H), а затем записать по этому адресу требуемую информацию. В процессе выполнения операции бит D7 регистра состояния равен 0. Во время процесса программирования можно задавать лишь команду “Чтение регистра состояния”. После окончания процесса программирования бит D4 регистра состояния показывает успешно ли она была завершена. В процессе программирования любой бит можно лишь изменить из 1 в 0. Т.о. алгоритм программирования байта/слова имеет следующий вид.



При попытке записать 1 в ячейку, в которой до этого был записан 0, информация в ячейке не изменится, и ошибка не будет зафиксирована. Если во время процесса программирования напряжения программирования на выводе VPP будет ниже нормы (5В), то операция не будет произведена и бит D3 регистра состояния будет установлен в 1. Время записи байта/слова - бмкс. Для перехода в режим чтения массива после операции программирования необходимо записать команду “Чтение массива” (0FFH) в CUI по любому адресу.

#### Защита блоков от стирания информации.

Блоки могут быть защищены от стирания и записи новой информации во избежание случайной потери информации. Существуют несколько режимов защиты блоков, которые отражены в следующей таблице.

| VPP  | $\overline{RP}$ | $\overline{WP}$ | Статус защиты                          |
|------|-----------------|-----------------|----------------------------------------|
| “0”  | x               | x               | Все блоки защищены.                    |
| >“0” | “0”             | x               | Все блоки защищены (“Сброс”).          |
| >“0” | 12В             | x               | Все блоки открыты.                     |
| >“0” | “1”             | “0”             | Boot Block защищен, остальные открыты. |
| >“0” | “1”             | “1”             | Все блоки открыты.                     |

Т.о. в активном состоянии для защиты всех блоков необходимо выполнение условия  $VPP=0$ . Если напряжение на выводе  $VPP>0$ , а на выводе  $\overline{RP}$  - меньше 12В, то может быть защищен только Boot Block если напряжение на выводе  $\overline{WP}$  равно 0. Если напряжение на выводе  $\overline{RP}$  равно 12В, то эта защита не действует. Если блок защищен, то при попытке очистить его или записать в него информацию эти операции выполняться не будут и будет индцироваться соответствующая ошибка в регистре состояния.

#### Режим пониженного энергопотребления Deep Power Down Mode.

Для уменьшения потребления энергии предусмотрен режим пониженного энергопотребления, в который микросхема входит при  $\overline{RP}=0$ . При этом все выходы микросхемы переходят в высокоимпедансное состояние. Микросхема потребляет в этом режиме ток от источника питания около 0.2 мкА. Для выхода из этого состояния необходимо установить единичный уровень сигнала на выводе  $\overline{RP}$ . после этого микросхема возвращается в активное состояние в режим “Чтение Массива”, но только через 0.5 мкс.

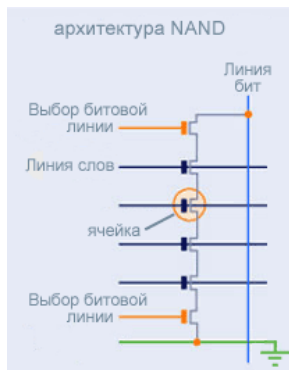
На сегодняшний день самыми распространенными технологиями построения массивов Flash-памяти являются т.н. NOR и NAND технологии.

Технология NOR использует классическую двумерную матрицу проводников, в которой на пересечении каждой строки и каждого столбца установлено по одной элементарной запоминающей ячейке. При этом проводник строк подключается к стоку транзистора, а столбцов к управляющему затвору. Исток подключается к общей для всех подложке. В такой конструкции можно легко считать состояние конкретного транзистора, подав положительное напряжение на один столбец и одну строку. Недостатком такой технологии является плохая масштабируемость матрицы запоминающих ячеек. Поскольку при записи нового значения необходимо подать высокое напряжение на стоки всех транзисторов строки, то транзисторы должны располагаться относительно далеко друг от друга для исключения ложных записей в соседних ячейках.



Технология NAND подразумевает трехмерный массив элементарных запоминающих ячеек. В основе та же самая матрица, что и у NOR-устройств, но вместо одного

транзистора в каждом пересечении устанавливается столбец из последовательно включенных ячеек. Каждый столбец изолируется от соседних крайними ключами на базе обычных МОП транзисторов. Это исключает влияние на соседние столбцы при подаче на данный столбец высокого напряжения. За счет этого плотность компоновки транзисторов можно резко увеличить. Однако алгоритм доступа к ячейкам для чтения и записи заметно усложняется.



Технология NOR позволяет получить быстрый доступ индивидуально к каждой ячейке, однако площадь ячейки велика. Наоборот, NAND имеют малую площадь ячейки, но относительно длительный доступ сразу к большой группе ячеек. Соответственно различается область применения: NOR используется как непосредственная память программ микропроцессоров и для хранения небольших вспомогательных данных. Максимальные значения объемов памяти микросхем NOR — 64 МБайт. NAND – устройства имеют максимальные значения объема памяти, измеряемые единицами гигабайт. Они используются для создания массовых устройств хранения данных.

Названия NOR и NAND произошли от ассоциации схемы включения ячеек в массив со схемотехникой микросхем КМОП логики.

Запоминающие устройства, в которых элементарная ячейка способна хранить один бит информации, называются устройствами типа SLC (Single Level Cell). На сегодняшний день выпускаются устройства Flash-памяти, в которых в одной ячейке может храниться два, три, и даже четыре бита информации. Это достигается множественной градацией уровней заряда, хранящегося на плавающем затворе. Так, для обеспечения хранения двух бит информации в одной ячейке необходимо 4 уровня градации заряда, для 3-х бит – 8, а для 4-х – 16 уровней градации величины заряда. Такие устройства называются устройствами типа MLC (Multi Level Cell). Они являются более емкими по сравнению с SLC-устройствами, однако обладают большим временем доступа и меньшим максимальным количеством циклов перезаписи.

Помимо Flash-памяти исследуются и развиваются другие технологии энергонезависимой памяти. Одной из таких альтернативных технологий, по которой на сегодняшний день серийно выпускаются устройства памяти, является FRAM – Ferroelectric RAM. Принцип хранения информации здесь основан на эффекте гистерезиса поляризации в сегнетоэлектрике. Ячейка памяти FRAM похожа на ячейку DRAM, в которой вместо линейного изолятора между затвором и каналом используется сегнетоэлектрик. Сегнетоэлектрик имеет нелинейную зависимость заряда от приложенного электрического поля. Эта зависимость имеет вид петли гистерезиса. При приложении к такой ячейке памяти электрического поля в ней меняется направление поляризации доменов, переходя на другой участок петли гистерезиса. При снятии напряжения питания поляризация доменов

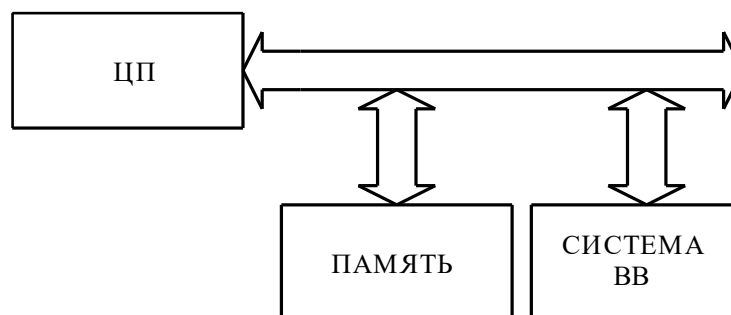
сохраняется. За счет этого можно получить два хорошо различимых по энергии состояния, а это достаточно для создания памяти на основе такой ячейки. По быстродействию он сопоставим со статическим ОЗУ. Главными недостатками такого типа памяти являются плохая масштабируемость и относительно высокая стоимость. Устройства памяти по технологии FRAM выпускаются серийно на объемы до единиц мегабит. Схожую структуру запоминающей ячейки имеют устройства MRAM (Magnetoresistive RAM), в которой вместо сегнетоэлектрика используются ферромагнитные материалы. Информация в такой ячейке хранится не в виде электрических зарядов, а в виде изменения электрического сопротивления ячейки, которое изменяется в зависимости от ориентации магнитных доменов в материале.

Ещё одним перспективным направлением развития энергонезависимой памяти является PRAM - Phase-change RAM. Принцип хранения информации у таких устройств основан на резком изменении электрического сопротивления халькогенидного стекла при переходе из аморфного в кристаллическое состояние. Переход из одного состояния в другое осуществляется нагревом ячейки с помощью электрических импульсов. При записи новой информации не требуется стирание записанных ранее данных, что сокращает время. По скорости записи PRAM сопоставима со статическим ОЗУ, при том, что она является энергонезависимой.

## Лекция № 7

### Особенности организации микропроцессорной системы.

Базовая структура микропроцессорной системы имеет вид



Задача управления системой возлагается на **центральный процессор (ЦП)**, который связан с **памятью** и **системой ввода-вывода** через каналы памяти и ввода-вывода соответственно. ЦП считывает из памяти команды, которые образуют программу, и декодирует их. В соответствии с результатом декодирования команд он осуществляет выборку данных из памяти и портов ввода, обрабатывает их и пересылает обратно в память или порты вывода. Существует также возможность ввода-вывода данных из памяти на внешние устройства и обратно, минуя ЦП. Этот механизм называется **прямым доступом к памяти (ПДП)**. Каждая составная часть микропроцессорной системы имеет достаточно сложную внутреннюю структуру.

С точки зрения пользователя при выборе микропроцессора целесообразно располагать некоторыми обобщенными комплексными характеристиками возможностей микропроцессора. Разработчик нуждается в уяснении и понимании лишь тех компонентов микропроцессора, которые явно отражаются в программах и должны быть учтены при разработке схем и программ функционирования системы. Такие характеристики определяются понятием архитектуры микропроцессора.

**Архитектура микропроцессора** - это его логическая организация, рассматриваемая с точки зрения пользователя; она определяет возможности микропроцессора по аппаратной и программной реализации функций, необходимых для построения микропроцессорной системы. Понятие архитектуры микропроцессора отражает:

- его структуру, т.е. совокупность компонентов, составляющих микропроцессор, и связей между ними; для пользователя достаточно ограничиться регистровой моделью микропроцессора;
- способы представления и форматы данных;
- способы обращения ко всем программно-доступным для пользователя элементам структуры (адресация к регистрам, ячейкам постоянной и оперативной памяти, внешним устройствам);
- набор операций, выполняемых микропроцессором;
- характеристики управляющих слов и сигналов, вырабатываемых микропроцессором и поступающих в него извне;
- реакцию на внешние сигналы (система обработки прерываний и т.п.).

По способу организации пространства памяти микропроцессорной системы различают два основных типа архитектур.

Организация, при которой для хранения программ и данных используется одно пространство памяти, называется **фон Неймановской архитектурой** (по имени математика Джона фон Неймана (John von Neumann), предложившего кодирование программ в формате, соответствующем формату данных). Она ещё известна как **Принстонская архитектура**. Программы и данные хранятся в едином пространстве, и нет никаких признаков, указывающих на тип информации в ячейке памяти. Преимуществами такой архитектуры являются более простая внутренняя структура микропроцессора и меньшее количество управляющих сигналов. Примером микропроцессоров с такой архитектурой могут служить микропроцессоры для персональных компьютеров семейства x86.

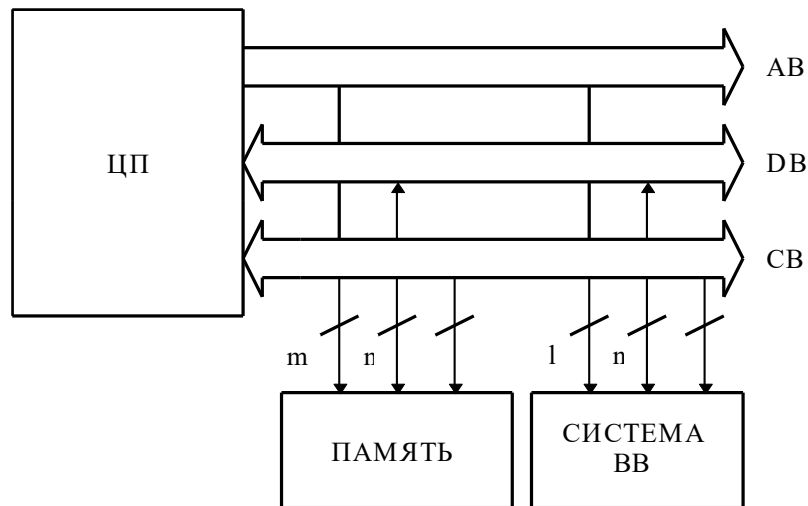
Организация, при которой память программ CSEG (Code Segment) и память данных DSEG (Data Segment) разделены и имеют свои собственные адресные пространства и способы доступа к ним, называется **Гарвардской архитектурой** (по имени лаборатории Гарвардского Университета, предложившей ее). Такая архитектура является более сложной и требует дополнительных управляющих сигналов. Однако, она позволяет осуществлять более гибкие манипуляции информацией, реализовывать компактно кодируемый набор машинных команд и, в ряде случаев, ускорять работу микропроцессора. Примером микропроцессоров с такой архитектурой могут служить микроконтроллеры семейства MCS-51.

В настоящее время выпускаются микропроцессоры со смешанной архитектурой, в которых CSEG и DSEG имеют единое адресное пространство, однако различные механизмы доступа к ним. Конкретным примером являются микропроцессоры семейства TMS320F2000 фирмы Texas Instruments.

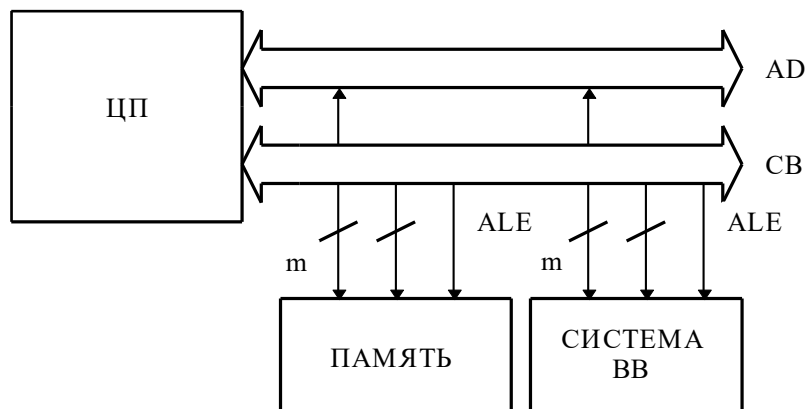
На физическом уровне микропроцессор взаимодействует с памятью и системой ввода-вывода через единый набор системных шин - **внутрисистемную магистраль**. Она, в общем случае состоит из:

- шины данных DB (Data Bus), по которой производится обмен данными между ЦП, памятью и системой ВВ;
- шины адреса AB (Address Bus), используемой для передачи адресов ячеек памяти и портов ВВ, к которым осуществляется обращение;
- шины управления CB (Control Bus), по которой передаются управляющие сигналы, реализующие циклы обмена информацией и управляющие работой системы.

Этот же набор шин применяется для организации канала ПДП. Магистраль такого типа носит название **демультиплексной** или **трехшинной с отдельными шинами адреса и данных**.



В некоторых микропроцессорах с целью сокращения ширины физической магистрали вводят **совмещенную шину адреса-данных AD** (Address/Data Bus), по которой передаются как адреса, так и данные. Этап передачи адресной информации отделен по времени от этапа передачи данных и стробируется специальным сигналом ALE (Address Latch Enable), который включен в состав CB. Данную магистраль обычно называют **мультиплексной** или **двухшинной с совмещенными шинами адреса и данных**.

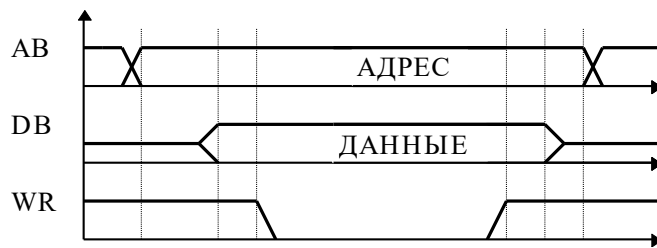


Физический обмен данными через магистраль выполняется словами или байтами в виде следующих друг за другом обращений к каналу. За один цикл обращения к магистрали между ЦП, памятью и системой ВВ передается одно слово или байт. Существуют несколько типовых циклов обмена. Среди них **чтение памяти** и **запись в память**.

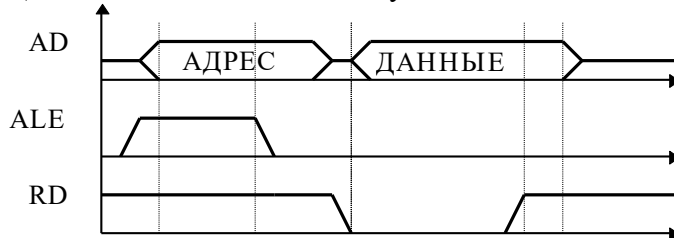
Цикл чтения памяти по демультиплексной магистрали.



Цикл записи в память по демультиплексной магистрали.



Цикл чтения из памяти по мультиплексной магистрали.



Цикл записи в память по мультиплексной магистрали.



При изолированном пространстве ВВ добавляются циклы **чтения порта ВВ** и **записи порта ВВ**.

Цикл чтения порта ВВ по демultipлексной магистрали



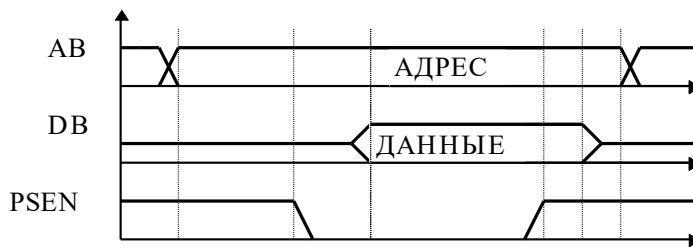
Цикл записи в порт ВВ по демultipлексной магистрали.



В случае архитектуры Гарвардского типа, когда память программ и память данных разделены, вводится также **цикл чтения памяти программ**.

Цикл чтения памяти программ по демultipлексной магистрали.





В некоторых случаях, когда на магистрали работают устройства, быстродействие которых уступает быстродействию ЦП, длительности стробов RD, WR и т.п. могут оказаться недостаточными для правильного выполнения операции обмена со стороны периферийного модуля. Тогда для организации надежного завершения магистральной операции в состав СВ вводят специальный сигнал READY. В каждом цикле обращения к каналу перед окончанием строба RD или WR ЦП проверяет состояние сигнала READY. Если он к этому моменту еще не сброшен, то ЦП продлевает соответствующий строб, вставляя в него т.н. такты ожидания WS (Wait State). Максимальное количество WS может быть ограничено либо не ограничено в зависимости от конкретной модели микропроцессора и режима его работы.



В обычном режиме работы на магистрали присутствует единственное активное устройство в лице ЦП, который инициирует все циклы обмена данных на магистрали. Однако возможны случаи, когда на одной и той же магистрали присутствуют несколько активных устройств, которые должны работать с одуим и тем же блоком памяти и блоком ВВ. Для того, чтобы другое активное устройство могло передать данные по магистрали, необходимо дезактивировать на это время ЦП. Для этих целей большинство современных микропроцессоров поддерживают т.н. режим **“прямого доступа к памяти” (ПДП)**. Для осуществления этого режима в СВ вводят дополнительные сигналы HOLD и HLDA. При поступлении активного уровня на вход HOLD микропроцессор прерывает выполнение своей программы, переводит выходы всех своих шин в высокоимпедансное состояние и выставляет активный уровень на выходе HLDA, что должно служить сигналом для другого активного устройства о том, что оно может начинать свои циклы обмена на магистрали. Когда это устройство заканчивает свои циклы обмена, оно сбрасывает сигнал HOLD, после чего ЦП переходит в свое обычное состояние и продолжает выполнять программу.

Другим режимом работы ЦП, требующим от него изменения нормального хода выполнения программы, является т.н. режим **“прерывания”**. Практически все современные микропроцессоры имеют один или несколько т.н. входов внешних прерываний INT0, INT1, и т.д., на которые поступают сигналы, свидетельствующие о некоторых событиях в системе, на которые ЦП должен отреагировать определенным образом. При поступлении активного уровня сигнала на один из таких входов микропроцессор прерывает нормальное выполнение программы, запоминает адрес команды, на которой он прервал работу, и переходит к выполнению т.н. **“подпрограммы обработки прерывания” (ПОП)**, записанной в CSEG по определенному адресу. Адрес этой

подпрограммы записан в специальной ячейке памяти, называемой “**вектором прерывания**”. Каждый отдельный источник прерывания имеет свой собственный вектор прерывания. После выполнения ПОП, по специальной команде, которой должна заканчиваться ПОП, процессор возвращается к выполнению прерванной программы по запомненному адресу. Источники прерываний могут быть как внешними (т.е. поступать на один из входов микросхемы, которые называются “входами запроса прерывания”), так и внутренними (т.е. генерироваться внутри процессора по определенным условиям). Т.к. одновременно могут придти несколько различных запросов прерываний, то существует определенная дисциплина, задающая последовательность обслуживания отдельных прерываний. Эту дисциплину обеспечивает система “**приоритетного арбитража прерываний**”, реализованная либо внутри ЦП, либо с помощью специального контроллера. В соответствии с этой системой каждый источник прерывания имеет свой заданный приоритет (постоянный или переменный), определяющий очередность его обслуживания. При одновременном приходе нескольких запросов прерываний вначале обслуживается прерывание с более высоким приоритетом, а затем с более низким. Прерывание с более высоким приоритетом может прервать уже начавшуюся подпрограмму обработки прерывания, имеющего более низкий приоритет, точно так же, как оно прерывает основную программу. При этом образуются т.н. “вложенные прерывания”.

Кроме CSEG и DSEG практически все современные микропроцессоры имеют специально выделенное пространство данных небольшого объема, называемое **набором программно-доступных регистров RSEG (Register Segment)**. В отличие от CSEG и DSEG регистры RSEG располагаются внутри ЦП в непосредственной близости от его АЛУ, что обеспечивает быстрый физический доступ к информации, хранящейся в них. В них, как правило, хранятся промежуточные результаты вычислений, часто используемые ЦП. Область RSEG может быть полностью изолирована от пространства данных DSEG, может частично пересекаться с ней, и может полностью являться подчастью DSEG. Внутренняя логическая организация RSEG очень разнообразна и играет важную роль при классификации архитектур микропроцессоров.

Регистры микропроцессора функционально неоднородны: одни служат для хранения данных или адресной информации, другие - для управления работой ЦП. В соответствии с этим все регистры можно разделить на **регистры данных, указатели и регистры специального назначения**. Регистры данных участвуют в арифметических и логических операциях в качестве источников операндов и приемников результата, адресные регистры или указатели используются для вычисления адресов данных и команд, расположенных в основной памяти. Специальные регистры служат для индикации текущего состояния ЦП и управления работой его составных частей. Возможна архитектура, при которой одни и те же регистры используются для хранения как данных, так и адресной информации. Такие регистры называются **регистрами общего назначения (РОН)**. Способы использования того или иного вида регистров определяют конкретные особенности архитектур микропроцессора.

Среди регистров данных часто выделяют один регистр, называемый **аккумулятором А (Accumulator)**, с которым связывают большинство команд арифметической и логической обработки данных. Это означает, что арифметические и логические команды используют в качестве одного из своих операндов содержимое аккумулятора и сохраняют в нем результат операции. Ссылка на него производится неявно с помощью кода операции. При этом нет необходимости в коде команды выделять специальную область для адресов операнда и результата. Такой тип архитектуры микропроцессора называется **аккумуляторным**. К недостаткам такой архитектуры можно отнести относительно низкое быстродействие, объясняемое тем, что аккумулятор является “узким местом”, в которое каждый раз необходимо сначала занести операнд перед выполнением операции. Примером такой архитектуры могут служить микроконтроллеры семейства MCS-51 фирмы Intel.

Другим примером организации регистров данных являются т.н. **“регистровый банк”**, состоящий из нескольких регистров R0, R1, и т.д. В этом случае операнды и результаты арифметических и логических операций могут храниться в любом из этих регистров, что расширяет возможности по манипуляции данными. В отличие от аккумулятора, используемые регистры адресуются явно в коде команды. Примером такой организации могут служить микропроцессоры PowerPC фирмы Motorola. В ряде микропроцессоров, предназначенных для работы в реальном масштабе времени, предусмотрены не один, а несколько регистровых банков. В каждый момент времени доступен лишь один из них, выбор которого осуществляется записью соответствующей информации в определенный служебный регистр. Примером таких устройств могут служить микропроцессоры семейства ARM7TDMI.

Архитектура, при которой процессор способен использовать в качестве адресов операндов и результатов операции ячейки основной памяти, называется архитектурой типа **“память - память”**. При этом исключаются временные затраты на перепись содержимого рабочих регистров при переходе от одной процедуры к другой. Однако при этом теряется быстрый доступ к промежуточным данным, т.к. они хранятся не во внутренних регистрах, а в DSEG. Решением этой проблемы может служить размещение часть DSEG на одном кристалле с ЦП и использование в качестве рабочих областей этого внутреннего сегмента ОЗУ. Примером такой организации могут служить микропроцессоры семейства 80x86 фирмы Intel.

Практически во всех современных микропроцессорах выделяют отдельную область памяти под так называемый **“стек”**, используемый, в общем случае, для передачи параметров процедурам и сохранения адресов возврата из них. Стек может быть расположен внутри микропроцессора или вне его. Он может занимать часть адресного пространства DSEG или RSEG, а может быть расположен и отдельно от них. В последнем случае говорят о т.н. **“аппаратном стеке”**. Передача функций аккумулятора вершине стека приводит к т.н. **“стековой архитектуре”**. Стековая организация дает возможность использовать безадресные команды, код которых имеет наименьшую длину. Безадресные команды оперируют данными, находящимися на вершине стека и непосредственно под ней. При выполнении операции исходные операнды извлекаются из стека, а результат передается на вершину стека. Стековая архитектура обладает высокой вычислительной эффективностью. Существует специальный язык высокого уровня FORTH, построенный на основе безадресных команд. Такая архитектура используется в специализированных процессорах высокой производительности и, в частности в RISC-процессорах.

Служебные регистры, расположенные внутри микропроцессора, предназначены для различных функций управления его работой и индикации состояния его составных частей. Их состав и организация зависят от конкретной архитектуры процессора и различаются в каждом конкретном случае. Наиболее часто встречающимися регистрами специальных функций являются **“программный счетчик”** PC (Program Counter), **“указатель стека”** SP (Stack Pointer) и **“слово состояния программы”** PSW (Program Status Word). Программный счетчик PC в каждый конкретный момент времени содержит адрес команды, следующей в CSEG за той, которая в данный момент выполняется. Указатель стека SP хранит текущий адрес вершины стека. Слово состояния программы PSW содержит набор текущих признаков результата выполнения операции. С каждым признаком результата связывается одноразрядная переменная-флажок, соответствующая определенному биту PSW. К типовым флажкам-признакам относятся:

- **CF** (Carry Flag) - флажок переноса из старшего разряда АЛУ. Равен 1, если в результате выполнения арифметической операции или операции сдвига произошел перенос из старшего разряда результата;
- **ZF** (Zero Flag) - флажок признака нуля. Равен 1, если результат операции равен 0;
- **SF** (Sign Flag) - флажок знака результата. Дублирует знаковый разряд результата операции;

- **AF** (Auxiliary Carry Flag) - флажок дополнительного переноса. Равен 1, если в результате выполнения арифметической операции или операции сдвига произошел перенос из младшей тетрады результата в старшую. Часто используется в двоично-десятичной арифметике;
- **OF** (Overflow Flag) - флажок переполнения. Равен 1, если в результате выполнения арифметической операции произошло переполнение разрядной сетки результата;
- **PF** (Parity Flag) - флажок четности. Равен 1, если число 1 в результате операции нечетно и наоборот.
- **IF** (Interrupt Flag) - флажок разрешения прерывания. Индицирует, разрешены ли прерывания в системе.

Конкретные флаги используются программой для анализа результата предшествующей команды и принятия решения о дальнейшем ходе выполнения программы. Специальные регистры могут занимать часть адресного пространства DSEG или RSEG, а могут быть расположены и отдельно от них.

Адресные регистры или указатели используются для реализации тех или иных методов адресации операндов, используемых в конкретных командах микропроцессора. Их конкретный набор и функции зависят от того, какие методы адресации реализованы в данной модели микропроцессора.

Под **методом адресации** понимается метод кодирования адреса операнда или результата операции в коде команды.

В общем случае код команды микропроцессора можно представить в следующем виде

|     |      |      |     |    |
|-----|------|------|-----|----|
| КОП | АОП1 | АОП2 | ... | АР |
|-----|------|------|-----|----|

где,

- КОП - код операции;
- АОП1 - поле адреса первого операнда;
- АОП2 - поле адреса второго операнда;
- АР - поле адреса результата.

Наличие отдельных полей, кроме КОП, определяется конкретной командой и типом микропроцессора. Информация в полях АОП и АР определяется конкретным методом адресации, используемым в данной команде.

Наиболее распространенными методами адресации, используемыми в современных моделях микропроцессоров, являются:

- **Регистровая адресация.** Операнд находится в регистре. Адрес регистра включен в код операции. Поле адреса в команде отсутствует;
- **Прямая адресация.** Физический адрес операнда расположен в соответствующем поле адреса.
- **Непосредственная адресация.** Непосредственное значение операнда расположено в соответствующем поле адреса.
- **Косвенная регистровая адресация.** Физический адрес операнда расположен в регистре косвенного адреса DP (Data Pointer). Адрес регистра включен в код операции. Поле адреса в команде отсутствует. В качестве DP может выступать РОН или специальный адресный регистр;
- **Косвенная автоинкрементная/автодекрементная адресация.** Физический адрес операнда расположен в регистре косвенного адреса DP. Адрес регистра включен в код операции. Поле адреса в команде отсутствует. После (либо до) выполнения операции содержимое DP автоматически инкрементируется/декрементируется, чтобы указывать на следующий элемент таблицы.

- **Адресация по базе со смещением.** Базовый адрес операнда расположен в регистре базы ВР (Base Pointer). Адрес регистра включен в код операции. Смещение адреса операнда относительно базового адреса расположено в соответствующем поле адреса. В качестве ВР может выступать РОН или специальный адресный регистр;
- **Индексная адресация.** Базовый адрес операнда расположен в соответствующем поле адреса. Смещение адреса операнда относительно базового адреса расположено в индексном регистре Х (Index). В качестве Х может выступать РОН или специальный адресный регистр;
- **Адресация по базе с индексированием.** Базовый адрес операнда расположен в регистре базы ВР, смещение адреса операнда относительно базового адреса расположено в индексном регистре Х. Адреса регистров включены в код операции. Поле адреса в команде отсутствует; В качестве Х и ВР могут выступать РОН или специальные адресные регистры;
- **Сегментная адресация.** Вся память разбита на сегменты определенного объема. Адрес сегмента хранится в сегментном регистре SR (Segment Register), смещение адреса относительно начала сегмента расположено в соответствующем поле адреса либо в индексном регистре Х. В качестве Х может выступать РОН или специальный адресный регистр;

В зависимости от того, какие методы адресации реализованы в конкретном процессоре, в нем имеются те или иные адресные регистры. Более сложные методы адресации требуют большего времени для вычисления адреса операнда.

На ранних этапах развития микропроцессорной техники наряду с повышением разрядности и тактовой частоты, основной тенденцией являлось стремление сделать новые микропроцессоры более удобными для программирования. Это проявлялось в реализации больших наборов методов адресации, включении в набор команд сложных комплексных инструкций (иногда даже прямо реализующих конструкции языков высокого уровня). Позднее такие микропроцессоры были объединены под общим термином CISC-микропроцессоры (Complex (Complete) Instruction Set Computer). Характерными особенностями CISC-микропроцессоров являются:

- небольшое количество регистров общего назначения;
- большое количество различных команд процессора;
- семантически нагруженные команды, которые выполняются за много машинных циклов;
- большое количество реализуемых методов адресации;
- большое количество форматов команд различной разрядности;
- двухадресный формат команд. Результат операции записывается по адресу одного из операндов;
- наличие вычислительных команд типа «регистр-память».

Однако многие компиляторы не задействовали все возможности таких наборов команд, а на сложные методы адресации уходит много времени из-за дополнительных обращений к медленной памяти. При этом, из-за сложной схемной организации, такие микропроцессоры потребляли много энергии. Было показано, что такие сложные функции лучше исполнять последовательностью более простых команд, если при этом процессор упрощается и в нём остаётся место для большего числа регистров, за счёт которых можно сократить количество обращений к памяти. Эти идеи явились основой для развития нового направления организации микропроцессоров, которое стало называться RISC-микропроцессоры (Restricted (Reduced) Instruction Set Computer). Основными принципами организации RISC-микропроцессоров являются:

- одинаковая длина кода для большинства команд;

- реализация минимально-достаточного набора методов адресации путем разделения команд для обработки данных и обращения к памяти. Обращение к памяти идёт только через команды load и store, а все прочие команды ограничены внутренними регистрами;
- большое количество регистров общего назначения;
- каждая команда выполняется за один машинный цикл;
- трехадресный формат команд;
- вычислительные команды только тапа «регистр-регистр».

Это упростило организацию процессоров: позволило командам иметь фиксированную длину, упростило конвейеры и изолировало логику, имеющую дело с задержками при доступе к памяти, только в двух командах. В результате подобные микропроцессоры стали потреблять гораздо меньше энергии в пересчете на одну вычислительную операцию. На сегодняшний день RISC-микропроцессоры широко применяются, особенно в автономных и портативных системах. Конкретным примером RISC-архитектуры может служить архитектура фирмы ARM, по которой выпускаются микропроцессоры многими ведущими производителями, такими, как Analog Devices (ADUC7XXX), Atmel (SAM3, SAM7, SAM9), Qualcomm (MSM72XX, MSM82XX), Texas Instruments (Stellaris), nVidia (Tegra, Tegra2, Tegra3) и др.

Процесс исполнения команды любым микропроцессором подразумевает последовательное прохождение нескольких стадий, включающих в себя:

- выборку команды из памяти;
- декодирование команды;
- выборку из памяти данных, используемых в команде;
- выполнение самой вычислительной операции;
- сохранение результата операции.

Суммарное время выполнения команды равно сумме времен реализации каждого из перечисленных этапов.

В ранних микропроцессорах все перечисленные операции выполнялись одним большим устройством, состоящим из нескольких блоков. Каждый из блоков отвечал за реализацию одного из перечисленных этапов выполнения команды. Однако, поскольку эти этапы выполняются принципиально последовательно, то во время работы одного из блоков остальные простаивали.

На определенном этапе развития архитектуры микропроцессоров для повышения их вычислительной производительности было решено все вышеуказанные блоки реализовать в виде отдельных устройств и обеспечить их параллельное функционирование. Такая концепция получила название «**конвейера команд**» микропроцессора. Это позволило кардинальным образом повысить вычислительную производительность микропроцессоров без уменьшения времени выполнения отдельных команд. В микропроцессорах фирмы Intel конвейер появился на стадии 486. В более ранних версиях микропроцессоров линейки x86 его не было. На сегодняшний день конвейером оснащаются практически все быстродействующие микропроцессоры. Например, микроконтроллер TMS320F28xx фирмы Texas Instruments оснащен 8-стадийным конвейером, включающим следующие стадии выполнения команды

- F1 (Fetch 1). Адрес команды выставляется на шине адреса памяти программ;
- F2 (Fetch 2). Одна или две команды считываются из памяти программ и помещаются в очередь команд;
- D1 (Decode 1). Определяется размер команды. Проверяется легальность команды.
- D2 (Decode 2). Команда декодируется. Если в команде требуется чтение данных из памяти или запись данных в память, то их адрес декодируется. Производится модификация указателя стека и адресных регистров, если таковая необходима. Если требуется изменение последовательности выполнения команд (переход или программное прерывание), оно выполняется. Если команда доходит до этой фазы, она выполняется до конца;

- R1 (Read 1). Если в команде требуется чтение данных из памяти, то их адрес выставляется на соответствующей шине адреса;
- R2 (Read 2). Если в предыдущей фазе были адресованы данные, то они считываются из памяти;
- E (Execute). Выполняется вычислительная операция, указанная в команде. Если в команде используются данные, хранящиеся в регистрах, то они считываются в начале этой стадии и записываются обратно в конце этой стадии;
- W (Write). Результат выполнения операции записывается в память.

[illegible]

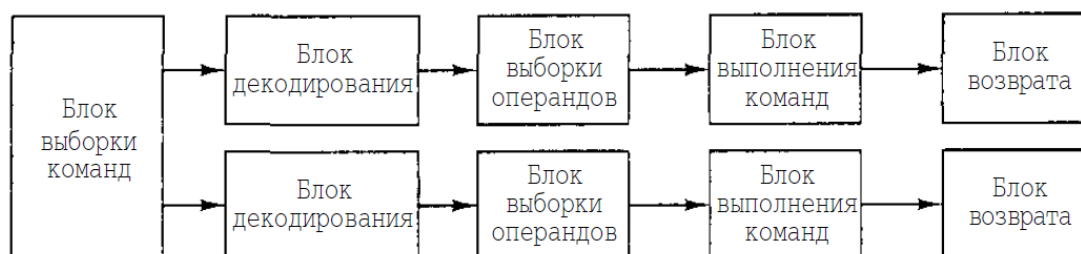
Как видно, несмотря на то что, время выполнения каждой команды равно восьми машинным циклам, благодаря наличию конвейера команды выполняются каждый машинный цикл. Это равносильно увеличению вычислительной производительности в восемь раз. Однако, такой выигрыш достигается только при последовательном выполнении команд. Если в последовательности команд встречается команда перехода, то стадии конвейера F1 – D1 очищаются. Команды, дошедшие до стадии D2 и дальше, при этом продолжают выполняться. Поэтому, первая команда после команды перехода выполнится на 3 машинных цикла дольше. Эта ситуация иллюстрируется ниже. Имеется в виду, что команда I2 выполняет переход на команду I10.

[illegible]

Как видно, между командами  $I1$  и  $I10$  образуются три пустых цикла, которые тратятся впустую. Из данного примера можно сделать вывод о том, что количество команд ветвления алгоритма влияет на эффективность работы процессора. Для исключения подобных необоснованных задержек в состав систем команд современных микропроцессоров входят т.н. команды «**поздних переходов**». Их смысл заключается в том, что при выполнении команды перехода, независимо от того, будет ли действительно выполнен этот переход, всегда выполняются несколько (в данном случае - три) команд, непосредственно следующих за командой перехода. При этом задача адекватного расположения команд в коде программы, не нарушающего логической последовательности операций, предусмотренных алгоритмом, при написании программы на языке высокого уровня возлагается на компилятор, а при написании на языке ассемблера – на программиста.

Традиционный микропроцессор, способный в каждый момент времени выполнять одну какую-либо элементарную операцию, называется «**скалярным**» микропроцессором.

Затруднения на пути увеличения тактовой частоты работы процессоров в последние годы вынудили разработчиков на поиски дальнейших путей усовершенствования архитектуры микропроцессоров. Это привело на определенном этапе к появлению т.н. «**суперскалярной архитектуры**». Первые суперскалярные микропроцессоры имели не один, а два конвейера команд, работающих параллельно, с одним блоком выборки команд из памяти.



Блок выборки команд считывает из памяти сразу две команды и распределяет их между двумя, подключенными к нему конвейерами. Возможность одновременного и параллельного функционирования двух конвейеров подразумевает наличие у микропроцессора нескольких вычислительных устройств, способных работать параллельно. В качестве подобных вычислительных устройств могут, например, служить:

- арифметико-логическое устройство, выполняющее операции сложения и вычитания чисел, а также логические операции над ними;
- умножитель, выполняющий операции умножения чисел;
- устройство сдвига, осуществляющее различные операции арифметического и логического сдвига двоичных чисел.

Таким образом, обеспечивается параллельное одновременное выполнение двух команд. Для того, чтобы команды могли выполняться параллельно, они не должны конфликтовать друг с другом при использовании ресурсов процессора (например, регистров). Помимо этого, ни одна из них не должна зависеть от результата выполнения другой. Возможные конфликты выявляются и устраняются либо компилятором, либо в реальном времени с помощью специальной схемы анализа конфликтов, которой оснащаются суперскалярные процессоры. Первым суперскалярным микропроцессором линейки x86 фирмы Intel был процессор Pentium. Он содержал два пятистадийных конвейера. Современные микроконтроллеры с архитектурой ARM Cortex M7 имеют двойной 6-ступенчатый конвейер с динамическим предсказанием переходов.

Сложность аппаратного согласования параллельных команд затрудняет увеличение количества конвейеров. Поэтому, в более поздних версиях суперскалярных микропроцессоров, вместо увеличения количества конвейеров, стал использоваться т.н. метод «**статпланирования**», согласно которому параллельные инструкции объединяются компилятором в длинную команду, в которой все инструкции заведомо параллельные. Подобный процессор должен обладать одним конвейером и достаточно большим



количеством вычислительных устройств, способных работать параллельно. Такая архитектура получила название VLIW (Very Long Instruction Word). Примером такой архитектуры может служить семейство процессоров Intel Core.

Следующим шагом на пути увеличения быстродействия микропроцессоров стало объединение на одном кристалле нескольких полноценных процессорных ядер. При этом все процессорные ядра являются полностью независимыми. Их объединяет лишь общее пространство памяти, с которой они работают. Поэтому в подобных системах приходится решать сложные задачи распределения и арбитража доступа к памяти со стороны отдельных ядер. Задача распределения вычислительной загрузки отдельных ядер решается совместно компилятором и соответствующих дополнительных аппаратных блоков. Примером подобных микропроцессоров может служить современное семейство Intel Core I.

Наличие нескольких аппаратных ядер существенно увеличивает вычислительную производительность, однако отрицательно сказывается на цене и потребляемой мощности вычислительной системы. Компромиссным вариантом, позволяющим повысить производительность процессора без значительного повышения его цены и потребляемой мощности, может служить реализация нескольких (обычно - двух) т.н. «**виртуальных**» ядер на базе одного физического ядра. Примером такого подхода может служить технология Hyper-Threading, разработанная фирмой Intel. Эта технология реализует идею т.н. «мультипоточности» на одном физическом ядре процессора. Для того, чтобы процессор был способен реализовывать квазипараллельную обработку двух потоков данных (т.е. решение двух независимых задач), он должен иметь по одному регистру состояния, набору регистров общего назначения и контроллеру прерываний на каждое виртуальное ядро. В первую очередь физическим ядром процессора обрабатывается основной поток (Thread) данных, т.е. решается первая – основная задача. Второй – дополнительный поток данных может быть обработан в те интервалы времени, когда по каким-либо причинам обработка основного потока данных простаивает. В качестве примеров подобных причин можно привести циклы ожидания конвейера команд при ветвлении алгоритма и циклы ожидания при доступе к медленной памяти. Увеличение вычислительной производительности за счет нескольких виртуальных ядер сильно зависит от типов реализуемых алгоритмов в основном и дополнительном потоке. Технология Hyper-Threading реализована в процессорах семейства Core I фирмы Intel.

Существуют микропроцессоры, архитектура которых адаптирована для выполнения вычислений определенного рода. К числу таких процессоров относятся т.н. “**процессоры цифровой обработки сигналов**” DSP (Digital Signal Procesor). Их архитектура имеет особенности, позволяющие им с наибольшей производительностью осуществлять алгоритмы рекуррентной обработки данных, которые используются во многих задачах, требующих их выполнения в масштабе “реального времени”, таких как аудио- и видео-кодирование, регулирование, цифровая фильтрация, цифровая связь и т.п. Все эти процессоры построены, как правило, по Гарвардской архитектуре. Современные DSP имеют отдельные шины адреса/данных для CSEG и DSEG, что позволяет им с помощью одной команды осуществить доступ к различным видам памяти и произвести несколько операций над данными. Основной особенностью DSP является то, что кроме обыкновенного АЛУ, которое присутствует во всех процессорах, они имеют еще несколько вычислительных устройств. К числу таких устройств в первую очередь относится т.н. “умножитель-аккумулятор” MAU (Multiple-Accumulator Unit), способный с помощью одной команды умножить два многоразрядных числа и сложить результат удвоенной разрядности с результатом предыдущей команды. Подобная операция “умножения-сложения” используется во всех рекуррентных алгоритмах. Наличие MAU в сочетании с вышеуказанными особенностями организации шин процессора позволяет DSP за одну команду полностью выполнить один шаг рекуррентного алгоритма и подготовить исходные данные для следующего шага. Другим дополнительным вычислительным устройством

является “многоразрядный регистр сдвига” S (Shifter), способный выполнять операции сдвига над числами, разрядность которых превышает разрядность АЛУ. Совместная работа этих вычислительных устройств позволяет достичь на выполнении рекуррентных алгоритмов вычислительной производительности, несравнимой с любыми другими процессорами. Примерами современных DSP могут служить:

- семейство ADSP-21XX фирмы Analog Devices - 16-разрядные DSP с фиксированной точкой, производительность до 30 MIPS;
- семейство TMS320C5000 фирмы Texas Instruments - 16-разрядные DSP с плавающей точкой, производительность до 600 MIPS.
- TMS320F2000 фирмы Texas Instruments - 32-разрядный DSP с фиксированной и плавающей точкой, адаптированный для задач управления приводом, производительность до 300 MIPS, 18-канальный встроенный ШИМ, два 8-канальных 10-разрядных АЦП.

В составе системы ВВ также можно выделить ряд функционально законченных устройств, которые оформляются в виде модулей, подключаемых непосредственно к единой магистрали системы. В простейшем случае это адресуемые ЦП буферные регистры - **порты ВВ**. Более сложные программно-управляемые подсистемы ВВ, содержащие блоки портов, получили название **периферийных адаптеров**. В случае, когда средства ВВ предназначаются для управления специальным внешним оборудованием и реализации специальных функций ВВ, их называют **периферийными контроллерами**. Наиболее сложными из современных средств обмена с внешними устройствами ВВ считают **сопроцессоры ВВ**, которые работают по собственным программам, хранящимся в собственной памяти, и по сути дела представляют собой отдельные микропроцессорные системы. Однако, независимо от сложности конкретной подсистемы ВВ, со стороны ЦП все они представляются тем или иным набором адресуемых регистров, который, как правило, является частью DSEG.

**Разрядностью микропроцессорной системы** принято считать количество бит информации, которое ее ЦП может обработать с помощью одной команды. Разрядность микропроцессора определяется разрядностью его АЛУ, внутренних регистров данных и внешней шины данных. На сегодняшний день существуют 8-, 16-, 32- и 64-разрядные микропроцессоры. Для того, чтобы обрабатывать информацию с разрядностью большей, чем разрядность микропроцессора необходимо реализовывать специальные алгоритмы вычислений с повышенной разрядностью. Эти алгоритмы требуют дополнительного времени для своего выполнения. Поэтому повышение разрядности микропроцессора при заданной разрядности вычислений, напрямую связано с увеличением быстродействия системы.

В зависимости от того, в каком формате процессор способен воспринимать и обрабатывать данные, различают **микропроцессоры с фиксированной точкой** и **микропроцессоры с плавающей точкой**.

При представлении чисел в формате с фиксированной точкой число записывается в ячейку памяти, имеющую фиксированное значение двоичных разрядов, в ненормированном виде. При этом положение в этой записи двоичной точки, разделяющей целую и дробную части числа, аппаратно не отслеживается, и может мысленно задаваться программистом в любом из возможных положений. Например, число 15.375 может быть представлено в 32-разрядной ячейке памяти в виде 0000000000000000000000001111.011. Надо напомнить, что числа со знаком представляются, в большинстве случаев, в дополнительном коде.

При записи числа в формате с плавающей точкой, оно записывается в виде  $M \cdot 2^P$ , где М- мантисса числа, а Р – порядок. В языках программирования высокого уровня принят т.н. IEEE-стандарт представления чисел с плавающей точкой. В соответствии с ним, число записывается в ячейке памяти в формате, представленном на рисунке.

|        |        |        |        |   |
|--------|--------|--------|--------|---|
| 31(63) | 30(62) | 23(52) | 22(51) | 0 |
| S      | P      |        | M      |   |

Здесь: S – знак числа (0 — для положительных, 1 — для отрицательных);

P — сдвинутый порядок числа;

M — мантисса числа.

Мантисса хранится в памяти в нормализованном по уровню 2 виде. При нарушении нормализации мантиссу сдвигают влево до тех пор, пока она не станет меньше 2. Каждая операция сдвига сопровождается уменьшением порядка на 1. Но если мантисса всегда нормализована, то старшую единицу (целую часть числа) можно и не хранить в памяти. Это экономит один бит и, следовательно, увеличивает точность представления вещественных чисел. Эта единица присутствует неявно и называется неявной единицей (*implicit one*). Отбрасывание старшей цифры мантиссы выполняется для форматов **float** и **double**, но не выполняется для **long double**.

Порядок числа хранится "сдвинутым", т.е. к нему прибавляется число так, чтобы порядок был всегда неотрицательным. Для чисел формата **float** прибавляется 127, для чисел формата **double** — 1023. Всегда неотрицательный порядок избавляет от необходимости выделять один бит для хранения знака порядка и упрощает операции над ними.

Например, число 15.375 (1111.011 в двоичной системе счисления) в формате **float IEEE**-стандарта записывается так:

$$1.111011 \cdot 2^3$$

Учитывая отбрасывание неявной единицы и сдвиг порядка, получаем внутреннее представление числа:

$$S=0;$$

$$P = 3 + 127 = 130 \text{ (1000.0010 в двоичной системе счисления);}$$

$$M = 1110110...0.$$

Теперь запишем внутреннее представление числа 15.375 в формате **float**:

|    |          |                          |    |   |
|----|----------|--------------------------|----|---|
| 31 | 30       | 23                       | 22 | 0 |
| 0  | 10000010 | 111011000000000000000000 |    |   |

При заданной точности вычислений и разрядности, диапазон чисел, представимых в формате с плавающей точкой значительно превышает диапазон чисел в формате с фиксированной точкой. Поэтому вычисления с плавающей точкой используются для обеспечения повышенной точности результата. Реализация подобных алгоритмов на процессорах с фиксированной точкой влечет за собой большое время вычислений и, следовательно, снижение быстродействия системы. Процессоры с плавающей точкой способны выполнять арифметические операции над числами с плавающей точкой с помощью одной команды. Поэтому они выполняют подобные вычисления значительно быстрее, чем процессоры с фиксированной точкой.

## Лекция № 8

# Микроконтроллеры – отдельный класс микропроцессорных устройств для встроенных применений.

Микроконтроллер – вычислительно-управляющее устройство, предназначенное для выполнения функций контроля и управления периферийным оборудованием.

Уклон в сторону управления накладывает отпечаток на особенность архитектуры микроконтроллеров. Основной из этих особенностей является то, что наряду с процессорным ядром микроконтроллера имеют в своём составе подсистему ввода-вывода и, возможно, подсистему памяти. В последнем случае принято говорить об однокристальных микро-ЭВМ.

Рассмотрим особенности организации каждой из подсистем микроконтроллеров.

### Процессорное ядро

Современные микроконтроллеры могут быть построены как по Гарвардской (MCS-51 Intel), так и по Фон Неймановской архитектуре (80C166 Siemens). Все они имеют внешнюю системную магистраль для обмена данными с внешней памятью и дополнительными периферийными устройствами. Классические семейства микроконтроллеров (MCS-51) имеют, как правило, мультиплексные шины адреса/данных, что было обусловлено необходимостью минимизировать размер микросхемы. Однако современные быстродействующие микроконтроллеры используют уже демultipлексную шину, что ускоряет работу системы. Некоторые модели микроконтроллеров имеют возможность работать либо мультиплексной, либо с демultipлексной шиной, в зависимости от требуемой конфигурации системы. В случае демultipлексной шины контроллер быстрее обменивается данными по магистрали. При работе с мультиплексной шиной, освободившиеся выводы используются как порты ввода-вывода. (MCS251 Intel, 80C166 Siemens).

Большинство микроконтроллеров выполняют только операции с фиксированной точкой. Однако, в последнее время появились и микроконтроллеры с плавающей точкой, примером которых может служить семейство TMS320F2833X фирмы Texas Instruments. Существуют 8-разрядные (MCS-51 Intel, MC6805 Motorola), 16-разрядные (80C166 Siemens, MC6816 Motorola), 32-разрядные (MC683XX Motorola, MPC500 PowerPC, TMS320F2000) микроконтроллеры. Системы команд микроконтроллеров поддерживает, как правило, широкий набор методов адресации в т.ч. бытовую адресацию.

### Подсистема памяти.

Существуют микроконтроллеры с аккумуляторной (MCS-51) и регистровой (MCS-96) организацией. Количество регистров и их разрядность зависит от конкретной модели. Зачастую микроконтроллеры имеют несколько банков регистров (MCS-51, 80C166).

Память данных определенного объема присутствует на простом микроконтроллере практически всегда. Она обменивается данными с процессорным ядром по внутренней магистрали микроконтроллера, которая может быть организована иначе, чем внешняя. Поэтому, обмен данными с внутренней памятью данных, как правило, осуществляется быстрее, чем с внешней.

Варианты реализации внутренней памяти программ могут быть различными:

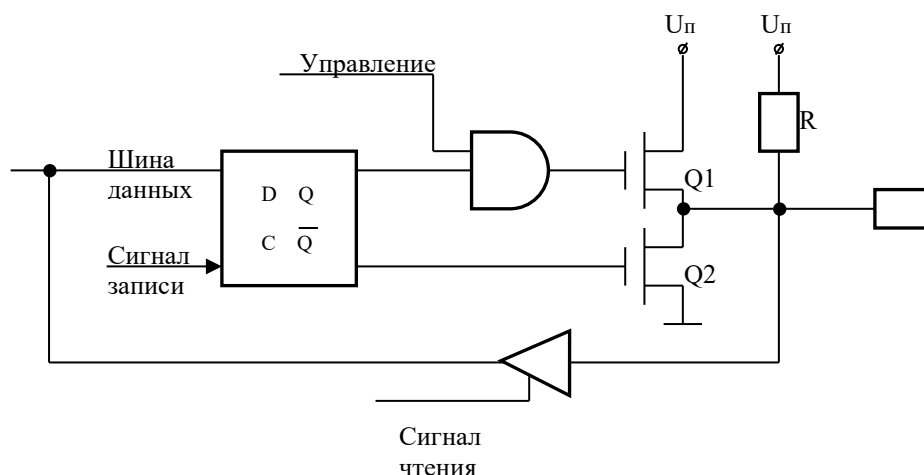
1. Она может отсутствовать. В этом случае микроконтроллер выполняет программу, считывая команды из внешней памяти программ через системную магистраль.
2. Она может быть выполнена в виде масочного ПЗУ. В этом случае микроконтроллер не нуждается во внешней памяти программ. Однако в этом случае программа во внутреннюю память записывается однократно на этапе изготовления кристалла и не может быть изменена в дальнейшем. Как правило, программа, записанная во внутреннюю память, выполняется быстрее, чем из внешней памяти.
3. Она может быть выполнена в виде однократно программируемого ППЗУ. В этом случае пользователь сам может записать программу во внутреннюю память, но лишь однажды. Для записи программы, как правило, необходим специальный программатор. Однако существуют микроконтроллеры, способные программировать сами себя (MCS-96) (программатор реализован внутри кристалла).
4. Она может быть выполнена в виде ППЗУ с УФ стиранием. В этом случае память программ может быть многократно перепрограммирована с помощью программатора. Перед очередным программированием она должна быть очищена с помощью УФ излучения.

5. Она может быть выполнена в виде Flash – памяти. В этом случае память программ может быть многократно перепрограммирована в процессе работы системы.
6. Она может быть реализована в виде ОЗУ. В этом случае для загрузки программы после включения питания используется так называемый Boot Strep загрузчик. Это механизм, позволяющий после включения питания загрузить начальную программу функционирования по последовательному каналу связи, либо по системной магистрали.

#### Подсистема ввода-вывода.

Подсистем ввода-вывода состоит из набора разнообразных устройств, выполняющих специфические функции управления и контроля. К их числу наиболее часто относятся:

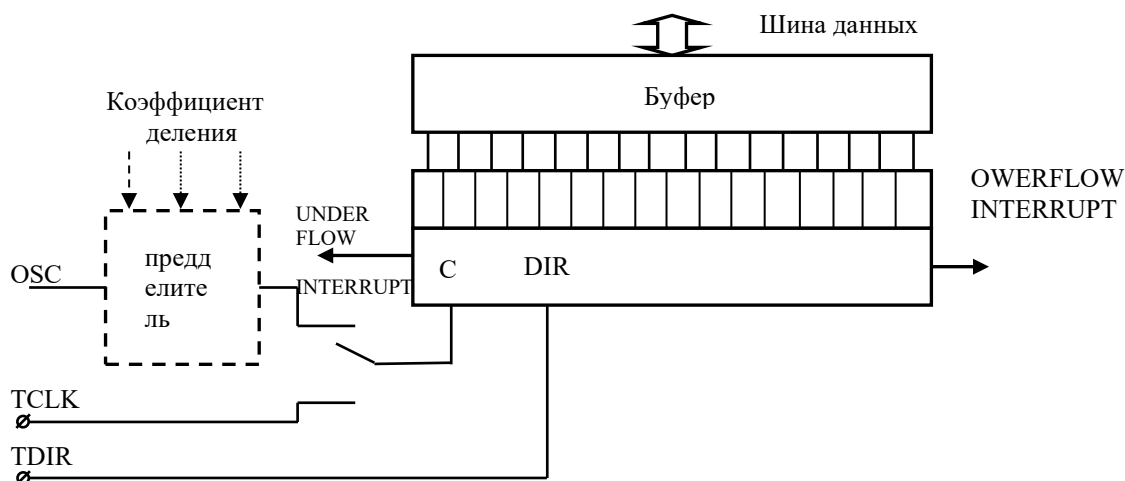
1. Порты ввода-вывода. Они могут быть либо однонаправленными (выполняя функции входа или выхода соответственно), либо квази-двунаправленными. Такие порты могут выполнять функции как входа, так и выхода (в каждый конкретный момент времени либо вход, либо выход). Упрощенная структура квази - двунаправленного порта:



Если порт работает как выход, то один из выходных транзисторов открыт, а другой закрыт, в соответствии с уровнем выходного сигнала. Чтобы порт работал как вход необходимо записать в выходной триггер «1», тогда транзистор Q2 будет закрыт, а сигнал управления блокирует открытие Q1. Таким образом, образуется высокоимпедансный вход. Микроконтроллеры, поддерживающие бытовую адресацию способны управлять состоянием каждого вывода порта отдельно.

2. Таймеры/счетчики. Они представляют собой специализированные счетчики с коммутируемыми источниками синхронизации.

Обобщенная структурная схема таймера/счетчика.



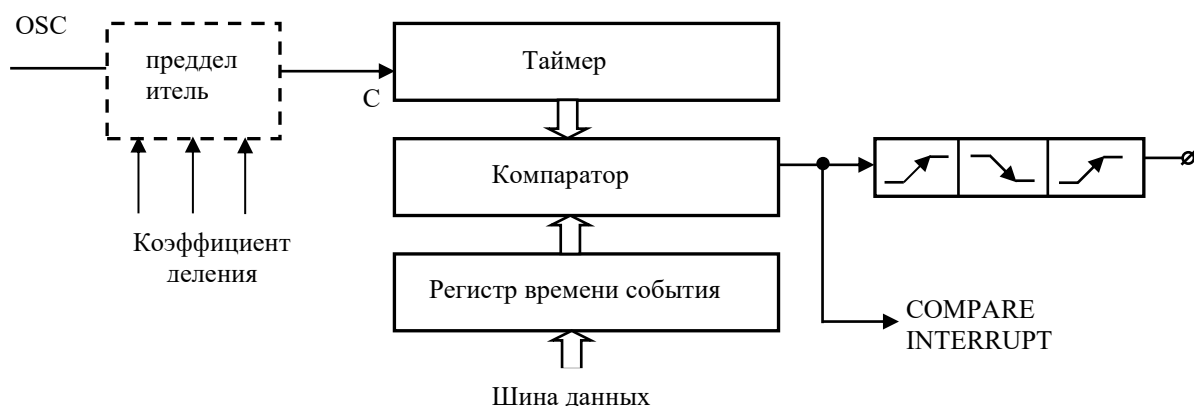
Источником счетных импульсов может быть либо системный синхросигнал с фиксированной частотой (возможно предварительно деленной в заданное число раз), либо один из входов микроконтроллера. В первом случае устройство выполняет функции таймера, так как фактически считает интервалы времени постоянной длительности.

Во втором случае устройство выполняет функции счетчика событий (отрицательных или положительных перепадов) на входе микросхемы. Счет может производиться либо в одном, либо в обоих направлениях. В последнем случае направление счета определяется уровнем сигнала на соответствующем входе микросхемы. При переходе содержимого счетчика из наибольшего состояния в наименьшее и наоборот могут генерироваться соответствующие внутренние запросы на прерывание. Конкретные режимы работы и структура таймеров зависят от конкретной модели микроконтроллера.

### 3. Модули «Захват/Сравнение». (Capture/Compare).

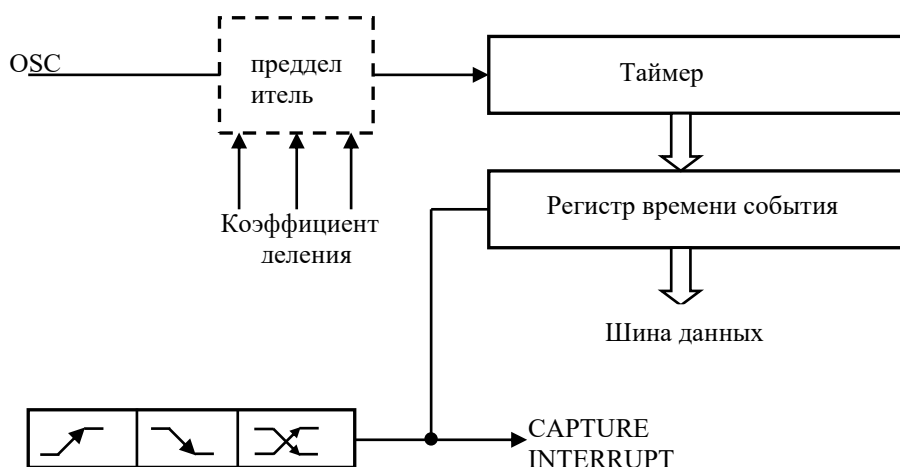
Эти модули, называемые ещё модулями быстрого ввода-вывода, предназначены для быстрой генерации события на выходе микросхемы, либо быстрой реакции на событие на её входе без участия процессора. Они имеют два основных режима работы: входной и выходной.

Структурная схема модуля в выходном режиме (Compare).



Таймер модуля считает импульсы, поступающие с постоянной частотой, соответствующей сигналу синхронизации (возможно деленной на предделителе). В регистр времени события предварительно записывается по команде определенное число. Компаратор сравнивает каждый момент времени содержимое таймера и содержимое регистра времени события. В момент, когда их содержимое становится равным друг другу, происходит заданное событие на соответствующем выходе микросхемы и выставляется запрос на прерывание. Таким образом, можно запрограммировать определённое выходное событие в заданное время и осуществить его без участия процессора. Этот режим используется, в частности, для генерации ШИМ.

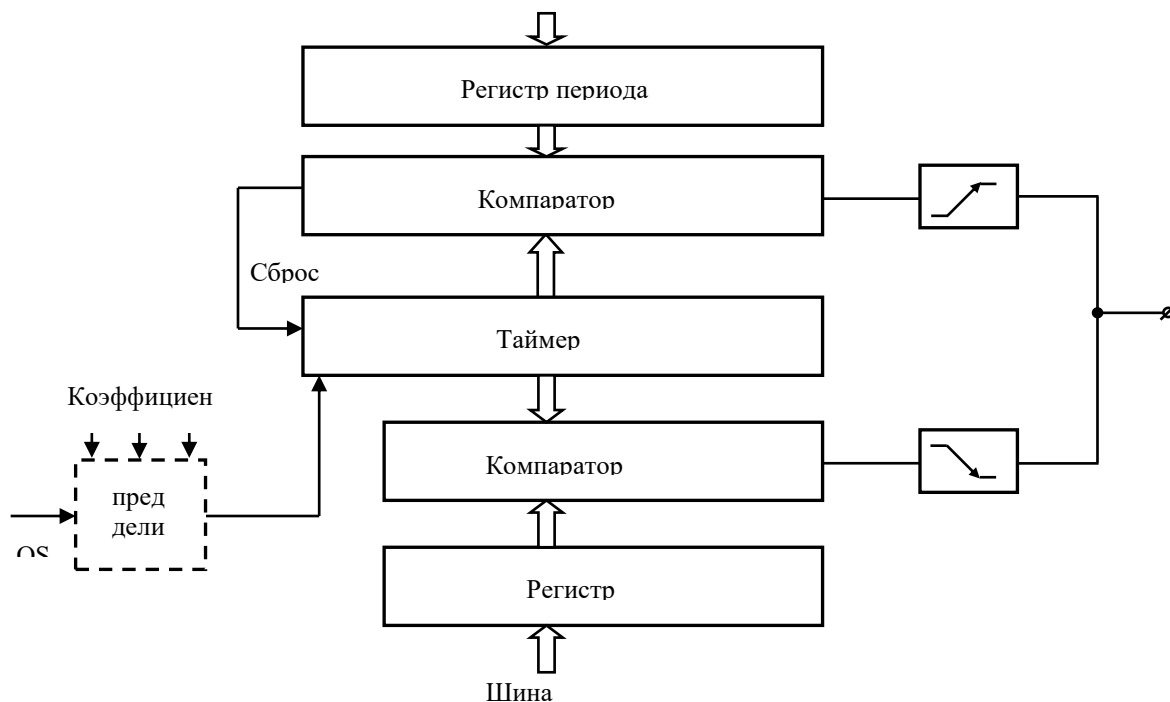
Структурная схема модуля во входном режиме (Capture)



Таймер модуля считает импульсы системного синхросигнала, проходящие, возможно, через предделитель частоты. В момент, когда на соответствующем входе микросхемы происходит заданное событие, содержимое таймера фиксируется в регистре времени события и вырабатывается запрос на прерывание. Таким образом, запоминается время события, которое затем может быть считано процессором по соответствующей команде. Этот режим используется, в частности, для измерения частоты и длительности входного сигнала.

4. Широтно-импульсный модулятор. Встроенный ШИМ предназначен для генерации широтно - модулированного сигнала на выходе микросхемы без участия процессора.

Структурная схема блока ШИМ.



На таймер блока подается системный синхросигнал (возможно через предделитель). В регистр периода процессор записывает число, соответствующее периоду ШИМ. При совпадении содержимого этого регистра и таймера последний сбрасывается в нулевое состояние и на соответствующем выходе микросхемы формируется положительный перепад. В регистре длительности процессор записывает число, соответствующее длительности импульса. При совпадении его содержимого с содержимым таймера, на выходе микросхемы формируется отрицательный перепад.

При такой организации возможная разрядность ШИМ зависит от частоты сигнала.

В некоторых моделях микроконтроллеров используются специализированные многоканальные модули ШИМ, в частности для управления приводом (80C196MC Intel).

5. Модуль последовательного ввода-вывода.

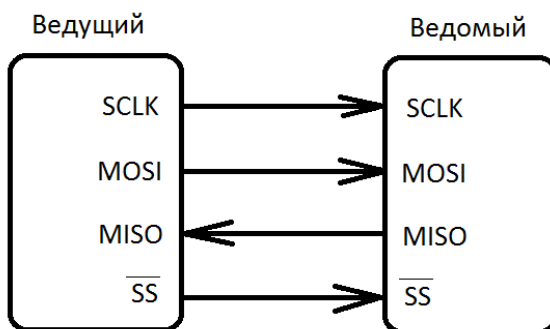
Модуль последовательного ввода-вывода используется для обмена данными между микроконтроллером и удаленным периферийным узлом. Данные передаются в последовательном коде, то есть биты передаются во временной последовательности друг за другом по одному каналу связи. Это даёт возможность использовать один канал связи вместо нескольких в случае параллельного кода. Однако при этом снижается быстродействие канала связи. Различают полудуплексные и дуплексные каналы последовательной связи. В случае полудуплексного канала данные передаются по одному и тому же каналу в обе стороны, но в каждый конкретный момент времени только в одну сторону.

В случае дуплексного канала данные передаются по двум каналам (в одну сторону – по одному, в другую – по-другому). При этом появляется возможность передавать информацию в обе стороны одновременно.

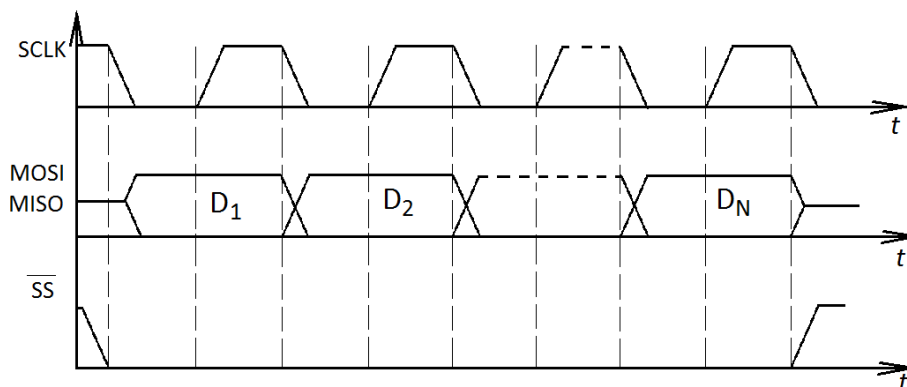
По методу синхронизации различают синхронные и асинхронные каналы связи. При синхронном методе передачи каждый передаваемый бит данных синхронизируется импульсом в канале синхронизации.

При этом по каналу данных передаются только собственно данные. Такой метод передачи позволяет обеспечить скорость передачи порядка 1Гбит/сек. Однако, для его организации необходим дополнительный канал синхросигнала.

Примером синхронного последовательного канала связи может служить интерфейс SPI (Serial Peripheral Interface), разработанный фирмой Motorola. Он используется для подключения к микроконтроллерам таких периферийных устройств, как АЦП, ЦАП, Flash-память и т.д. Интерфейс SPI организован по принципу «ведущий-ведомый» (master-slave). Ведущим на шине может быть только одно устройство, в качестве которого чаще всего используется микроконтроллер. К одному ведущему устройству может быть подключено несколько ведомых.



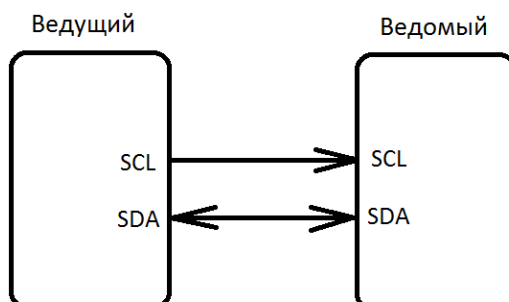
Интерфейс состоит из четырех сигналов: SCLK (Serial Clock), MOSI (Master Output – Slave Input), MISO (Master Input – Slave Output), SS (Slave Select). Ведущее устройство инициирует циклы обмена информацией по шине, вырабатывая сигнал синхронизации SCLK, сигнал выбора ведомого устройства SS и выставляя побитно данные на линии MOSI. Ведомое устройство, выбранное активным уровнем сигнала SS, синхронно с поступающими по линии SCLK импульсами принимает побитно входные данные по линии MOSI и выставляет побитно выходные данные на линию MISO. При пассивном уровне сигнала SS выход MISO ведомого устройства находится в высокоимпедансном состоянии. Это позволяет подключать к одному ведущему устройству несколько ведомых, выбираемых разными сигналами SS (SS1, SS2, и т.д.). Пример диаграммы цикла обмена по шине SPI представлена на рисунке.



Интерфейс SPI является дуплексным, т.е. обмен данными осуществляется одновременно в двух направлениях. Данные передаются старшим битом вперед.

Основным недостатком интерфейса SPI является большое количество сигналов, входящих в него (для организации двусторонней передачи данных необходимо четыре линии связи).

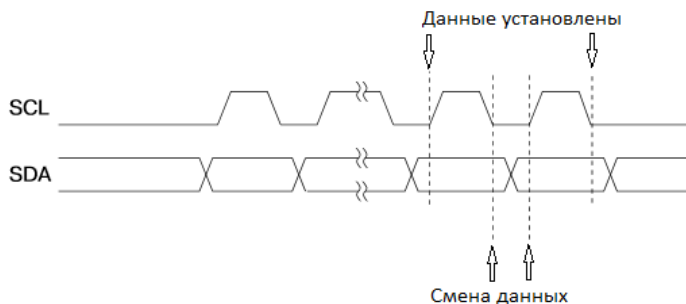
Еще одним примером синхронного канала связи может служить интерфейс I<sup>2</sup>C, разработанный компанией Philips. Этот интерфейс обеспечивает двустороннюю полудуплексную передачу данных с возможностью адресации до 128 устройств. Данный канал связи организуется с помощью всего двух сигналов. Сигнал SCL является каналом синхронизации, а сигнал SDA – каналом данных. Аналогично шине SPI интерфейс I<sup>2</sup>C организован по принципу «ведущий – ведомый» (master – slave).



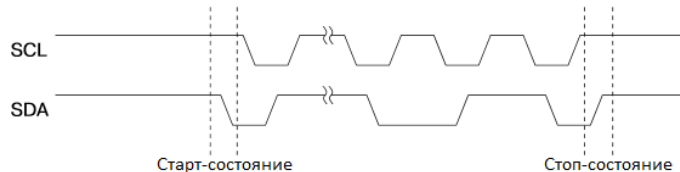


Ведущее устройство инициирует цикл обмена данными, вырабатывая сигнал синхронизации, передавая по каналу данных адрес целевого ведомого устройства и тип цикла обмена (запись или чтение). Ведомые устройства, обнаружив на шине запрос по своему адресу, реализуют требуемую операцию обмена данными. В отличие от шины SPI, здесь возможен режим, когда несколько различных устройств (каждое в свое время) могут выполнять роль ведущего.

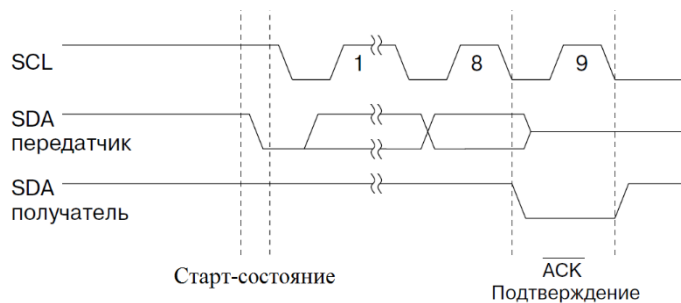
В исходном состоянии, при отсутствии передачи данных, оба сигнала всеми устройствами, подключенными к шине, поддерживаются в единичном состоянии. Во время передачи каждый бит данных на линии SDA стробируется положительным импульсом на линии SCL. Каждая смена очередного бита данных следующим осуществляется при нулевом состоянии сигнала SCL.



Признаком начала передачи пакета данных является т.н. «Старт – состояние» - отрицательный перепад сигнала SDA в то время, когда сигнал SCL находится в единичном состоянии. Такой перепад невозможен при передаче самих данных. Поэтому, он интерпретируется ведомыми устройствами как сигнал начала передачи. Аналогично, сигналом конца передачи пакета данных является т.н. «Стоп – состояние» - положительный перепад сигнала SDA при единичном уровне сигнала SCL.



Каждый байт данных передается за 9 периодов сигнала SCL. В девятом периоде устройство, являющееся приемником данного байта, выставляет «подтверждение приема» (Acknowledgement) в виде нулевого состояния линии SDA.

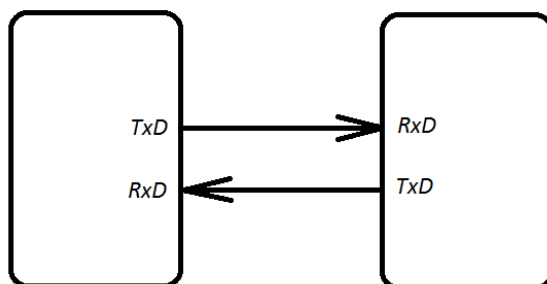


Пакет передаваемой информации начинается «старт – состоянием», за которым следует 7-битный адрес целевого ведомого устройства и бит типа операции, который равен нулю при передаче от ведущего в ведомому и единице в противном случае. Ведомое устройство, идентифицировавшее свой адрес, выставляет подтверждение, сигнализирующее о том, что оно готово к обмену данными. Далее могут следовать один или несколько байт данных в одном, либо в другом направлении, квитируемых подтверждением на приемной стороне. Отсутствие очередного подтверждения служит сигналом к завершению цикла обмена информацией. Завершается каждый передаваемый пакет данных «Стоп – состоянием». Все байты передаются старшими битами вперед.

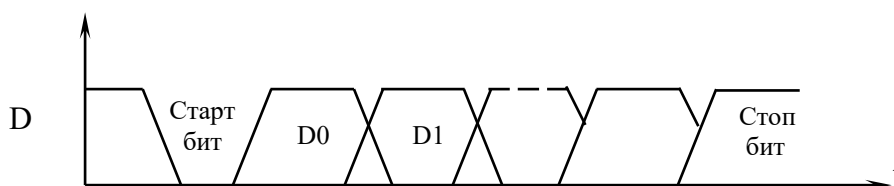
При асинхронном методе передачи канал синхронизации отсутствует. Это вызывает необходимость передавать по каналу данных дополнительную информацию для синхронизации приемной стороны.

Примером асинхронного канала связи может служить интерфейс UART (Universal Asynchronous Receiver – Transmitter). Он используется для связи микроконтроллерных систем с компьютерами. Данный

интерфейс предусматривает наличие у каждого из подключенных к шине устройств, всего двух сигналов RxD и TxD, с помощью которых осуществляется дуплексный обмен данными.



Все, подключенные к шине устройства являются равноправными и могут инициировать передачу данных другому устройству. Диаграмма одного пакета передаваемой информации представлена на рисунке.



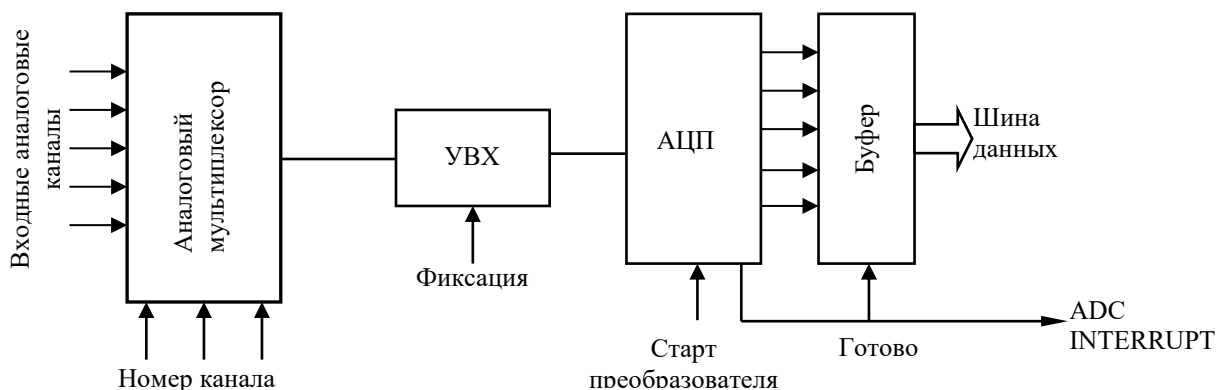
Первым в посылке передается «СТАРТ – бит» нулевого уровня. Приняв его, приемная сторона подстраивает фазу своих синхросигналов в соответствии с моментом прихода СТАРТ – бита. После него один за другим следуют биты данных. В конце цепочки данных может следовать бит четности для проверки правильности приема. Заканчивается посылка СТОП – битом единичного уровня. Частота передачи, то есть время передачи одного бита заранее должны быть согласованы на передающей и приемной сторонах. Такой метод передачи позволяет обеспечить скорость передачи до 200 Kbit/сек.

Микроконтроллер может содержать либо отдельные синхронные и асинхронные порты ввода-вывода, либо универсальный порт, способный работать в обоих режимах.

## 6. Модуль АЦП.

Модуль АЦП предназначен для преобразования входной аналоговой информации в цифровую и передачи ее в процессор для дальнейшей обработки. Микроконтроллеры имеют, как правило, несколько каналов АЦП. Преобразование информации на том или ином канале инициализируются соответствующей командой.

Структурная схема блока АЦП.



Преобразования начинаются выбором требуемого канала, который осуществляется записью номера канала в специальный служебный регистр. Аналоговый мультиплексор пропускает на выход сигнал выбранного канала. После этого подается сигнал фиксации на устройство выборки – хранения. Оно фиксирует уровень выбранного аналогового сигнала на весь период преобразования. Затем подается сигнал начала преобразования. АЦП представляет собой, как правило, АЦП последовательных приближений. Время преобразования составляет обычно около 10μs. После окончания преобразования АЦП выдает сигнал готовности, по которому его выходная информация записывается в буферный регистр, с которого она затем может быть считана процессором по соответствующей команде. При этом вырабатывается запрос на прерывание. Некоторые микроконтроллеры имеют блоки АЦП, способные работать в автоматическом режиме, опрашивая несколько каналов последовательно без участия процессора(80C166, Intel).

#### 7. Контроллер периферийных событий.

Этот контроллер предназначен для обеспечения некоторого блока событий в заданное время без участия процессора. Заданные события выполняются на микропрограммном уровне. Такими блоками событий могут быть:

- передача блока информации из одного места памяти (или устройства ВВ) в другое;
- последовательный опрос нескольких каналов АЦП;
- передача информации по последовательному каналу связи.

Использование такого механизма обработки событий позволяет снизить загрузку процессора и распараллелить процесс обработки информации.

Конкретный набор устройств ВВ, их характеристики и возможные режимы работы зависят от модели микроконтроллера.

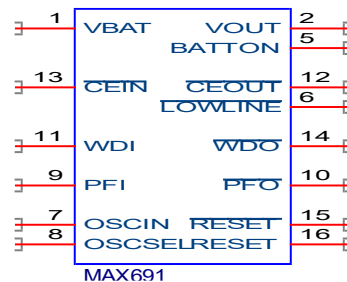
## Лекция № 9

### “Особенности питания микропроцессорной системы. Супервизоры питания”.

Системам питания микропроцессорных узлов в преобразовательной технике предъявляются особые требования, связанные с тем, что микропроцессорам приходится работать в условиях повышенных электромагнитных помех, создаваемых силовыми преобразователями. Особенностью микропроцессорной техники является ее склонность к невосстанавливаемым сбоям. Даже единичная помеха на шинах адреса или данных или изменение напряжения питания вне допустимых пределов могут привести к полной потере работоспособности устройства. Поэтому необходимо применять дополнительные меры для исключения подобных ситуаций. К этим мерам относятся:

- экранирование микропроцессорной части системы во избежание электромагнитных помех;
- гальваническая развязка и стабилизация цепей питания на плате контроллера во избежание наводок и падения напряжения на длинных линиях кабелей питания;
- резервное питание от аккумуляторов во избежание пропадания питания системы при авариях блока питания;
- контроль напряжений питания во время работы системы и возможность аварийного сброса системы при пропадании питания;
- контроль выполнения программного алгоритма функционирования системы и возможность аварийного сброса системы при нарушении алгоритма функционирования.

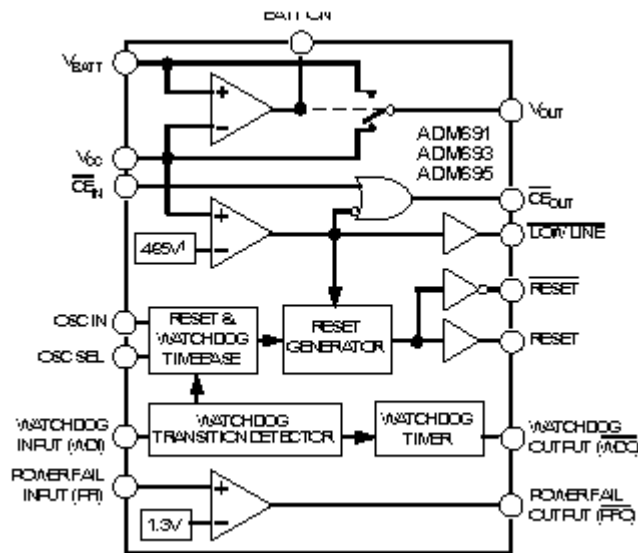
Описанные меры реализуются специальными конструкционными и схемотехническими средствами. Для реализации комплексных схемотехнических и программных мер предотвращения сбоев работы микропроцессорных систем были разработаны специальные микросхемы, называемые микропроцессорными супервизорами. Примером такого устройства может служить микросхема MAX691 фирмы Maxim или ее аналог ADM691 фирмы Analog Devices.



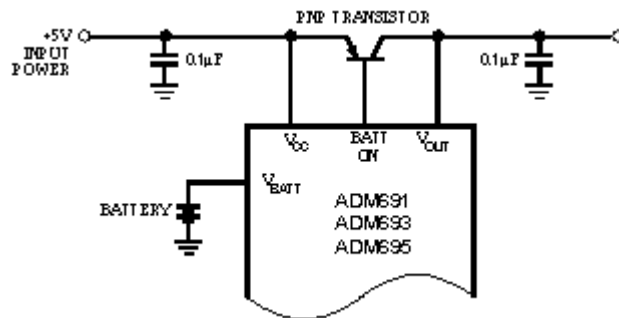
Микросхемы выполняет следующие функции:

- вырабатывает сигнал системного сброса при включении и выключении питания, а также при понижении напряжения питания ниже порогового уровня;
- переключает питание системы на аварийное питание от аккумулятора при понижении уровня основного напряжения питания ниже порогового уровня;
- вырабатывает сигнал системного сброса если внутренний сторожевой таймер микросхемы не был сброшен в течение определенного времени;
- осуществляет блокировку сигнала записи в ОЗУ при понижении напряжения питания ниже порогового уровня;
- вырабатывает сигнал запроса прерывания при понижении напряжения питания ниже номинального уровня.

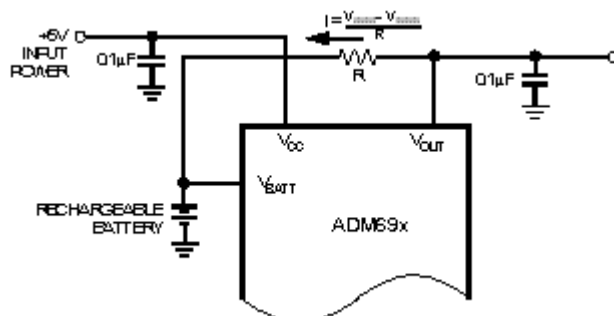
Внутренняя структура микросхемы имеет следующий вид.



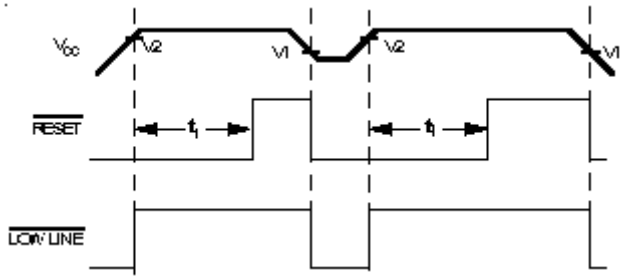
Основной источник питания подключается к входу  $V_{CC}$  микросхемы. К выводу  $V_{BATT}$  подключается резервный аккумуляторный источник питания. Микросхема постоянно сравнивает уровни сигналов, подаваемых на эти входы и подключает к выводу  $V_{OUT}$  тот, напряжение на котором в данный момент выше. переключение происходит когда напряжение на выводе  $V_{CC}$  снижается до уровня  $V_{BATT} + 50$  мВ, или когда оно повышается до уровня  $V_{BATT} + 70$  мВ. Гистерезис шириной в 20 мВ предусмотрен для предотвращения колебаний коммутатора в окрестностях точки переключения. Выводы питания части системы, критичной к уровню напряжения питания должны быть подключены к выводу  $V_{OUT}$ , что обеспечит их переключение на резервный источник питания при пропадании основного напряжения питания. В нормальном состоянии микросхема способна пропустить через себя ток цепи питания системы (вывод  $V_{OUT}$ ) величиной не более 100 мА. В случаях, когда питающаяся от вывода  $V_{OUT}$  система требует большего тока, необходимо подключить внешний PNP транзистор, который может управляться непосредственно от вывода BATT ON.



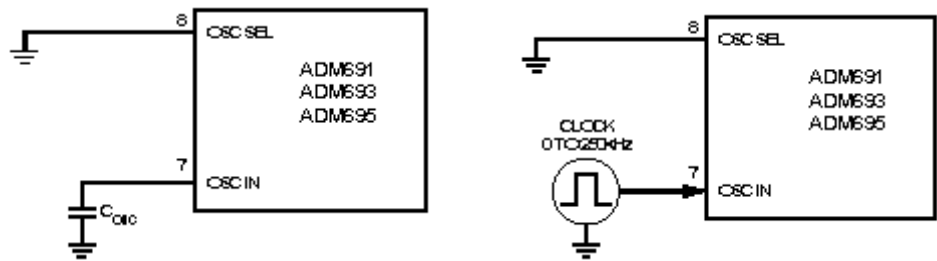
Если в качестве резервного источника питания используется конденсатор большой емкости или перезаряжаемые аккумуляторы, то необходимо обеспечить путь подзаряда накопителя энергии. Это реализуется путем подключения резистора между выводами  $V_{OUT}$  и  $V_{BATT}$ .



Допустимое напряжение аккумулятора резервного питания может изменяться в пределах от 2 В до 4.25 В. Вывод **RESET** микросхемы должен быть подключен ко входу системного сброса микропроцессора. На этом выходе формируется сигнал системного сброса с активным низким уровнем при понижении напряжения питания ниже порогового уровня сброса (4.65 В). После превышения напряжением питания порогового уровня сигнал на выводе **RESET** остается в активном состоянии еще в течение 50 мсек., что обеспечивает удержание системы в состоянии сброса до стабилизации напряжения питания. Уровень сигнала на выходе **LOWLINE** индицирует текущее состояние основного источника питания. Если основное напряжение питание ниже порогового уровня, то выход **LOWLINE** находится в нулевом состоянии.



Этот выход может быть использован для визуальной индикации уровня напряжения питания. Вывод WDI является входом сторожевого таймера (Watchdog Timer) микросхемы. Сторожевой таймер используется для контроля функционирования программы. Для использования сторожевого таймера в рабочей программе процессора должно быть предусмотрено периодическое инвертирование уровня сигнала, подаваемого на вход WDI. Если в течение определенного периода времени сигнал на входе WDI не будет инвертирован, будет выработан сигнал системного сброса на выводе **RESET**. Интервал времени, допустимый для инвертирования сигнала на входе WDI, а также длительность импульса системного сброса могут быть заданы путем комбинации сигналов на входах OSC IN и OSC SEL. На эти входы могут быть поданы сигналы единичного или нулевого уровня, внешний синхросигнал (частотой от 0 до 250 КГц), а также подключен внешний конденсатор.

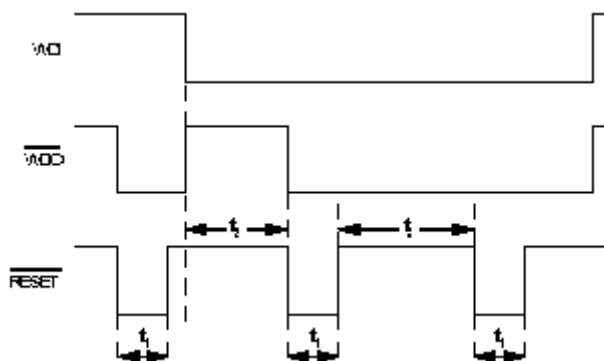


При этом длительность периода срабатывания сторожевого таймера и длительность импульса системного сброса определяются следующим образом.

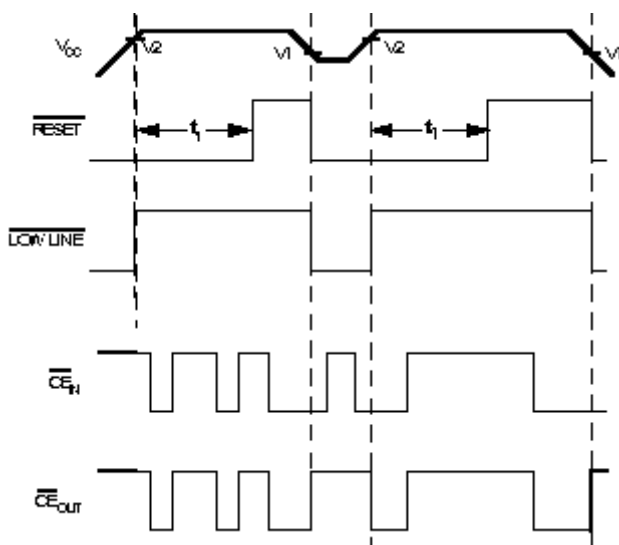
| OSC SEL            | OSC IN               | Период сторожевого таймера       |                                  | Длительность импульса сброса     |
|--------------------|----------------------|----------------------------------|----------------------------------|----------------------------------|
|                    |                      | обычно                           | после сброса                     |                                  |
| 0                  | внешний синхросигнал | $1024 * T_{CLK}$                 | $4096 * T_{CLK}$                 | $512 * T_{CLK}$                  |
| 0                  | внешний конденсатор  | $260\text{мс} * C / 47\text{пФ}$ | $1.04\text{с} * C / 47\text{пФ}$ | $130\text{мс} * C / 47\text{пФ}$ |
| 1 или не подключен | 0                    | 100 мс                           | 1.06 с                           | 50 мс                            |
| 1 или не подключен | 1 или не подключен   | 1.06 с                           | 1.06 с                           | 50 мс                            |

Сразу после импульса системного сброса период сторожевого таймера увеличен для того, чтобы система могла нормально инициализироваться.

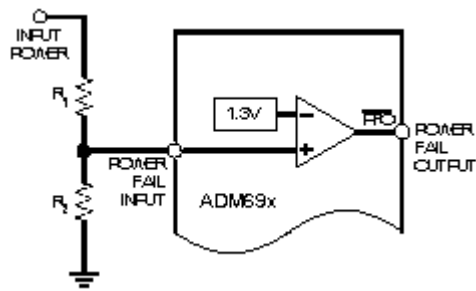
Вывод  $\overline{WDO}$  может быть использован для визуальной индикации состояния сторожевого таймера. При срабатывании сторожевого таймера и вызванном этим импульсом системного сброса на выводе  $\overline{RESET}$  вывод  $\overline{WDO}$  переходит в нулевое состояние и удерживается в нем до инвертирования сигнала на входе  $\overline{WDI}$ .



Выводы  $\overline{CEIN}$  и  $\overline{CEOUT}$  используются для блокировки циклов записи при ненормальном уровне напряжения питания. К выводу  $\overline{CEIN}$  подключается сигнал выбора кристалла памяти или строб записи, формируемые микропроцессорной системой при осуществлении циклов записи. При нормальном уровне напряжения питания этот сигнал передается без изменения на вывод  $\overline{CEOUT}$ , к которому подключается соответствующий вывод микросхем памяти. При снижении уровня напряжения на выводе  $V_{CC}$  ниже порогового уровня (4.65 В) или уровня на выводе  $V_{BATT}$ , вывод  $\overline{CEOUT}$  переводится в единичное состояние независимо от уровня сигнала на выводе  $\overline{CEIN}$ . Тем самым предотвращается повреждение данных, записанных в памяти при снижении напряжения питания.

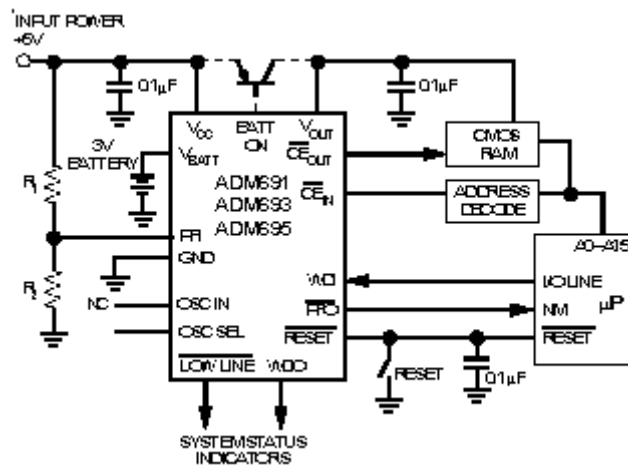


Выводы  $\overline{PFI}$  и  $\overline{PFO}$  используются для предварительного предупреждения микропроцессора о понижении напряжения питания. Напряжения на входе  $\overline{PFI}$  постоянно сравнивается с внутренним источником напряжения 1.3 В. При снижении напряжения на входе  $\overline{PFI}$  ниже уровня 1.3 В вывод  $\overline{PFO}$  переводится в нулевое состояние. Вывод  $\overline{PFO}$  подсоединяется к входу запроса прерывания микропроцессора. По этому прерыванию микропроцессор должен сохранить все оперативные данные и перейти в режим ожидания восстановления напряжения питания. Вход  $\overline{PFI}$ , как правило подключается к делителю напряжения, рассчитанному таким образом, чтобы при снижении напряжения питания до определенного уровня, выше 4.65 В, напряжения на входе  $\overline{PFI}$  достигало уровня 1.3 В.



Т.о., микросхема супервизора питания ADM691 способна реализовать полный спектр схемотехнических мер контроля и резервирования напряжения питания микропроцессорной системы, а также контролировать ситуации сбоев программы функционирования.

Типичная конфигурация системы с использованием супервизора питания ADM691 выглядит следующим образом.



Вывод  $\overline{\text{RESET}}$  имеет внутренний токоограничительный резистор. Поэтому он может быть объединен с другими цепями сброса, имеющими выходные цепи с открытым коллектором (в данном случае - кнопка сброса) и подключен непосредственно на вход сброса микропроцессора. Вывод  $\overline{\text{RESET}}$  выполняет те же функции, что и  $\overline{\text{RESET}}$ , однако имеет активный единичный уровень. Он предназначен для использования микросхемы с микропроцессорами, имеющими вход системного сброса с активным единичным уровнем.

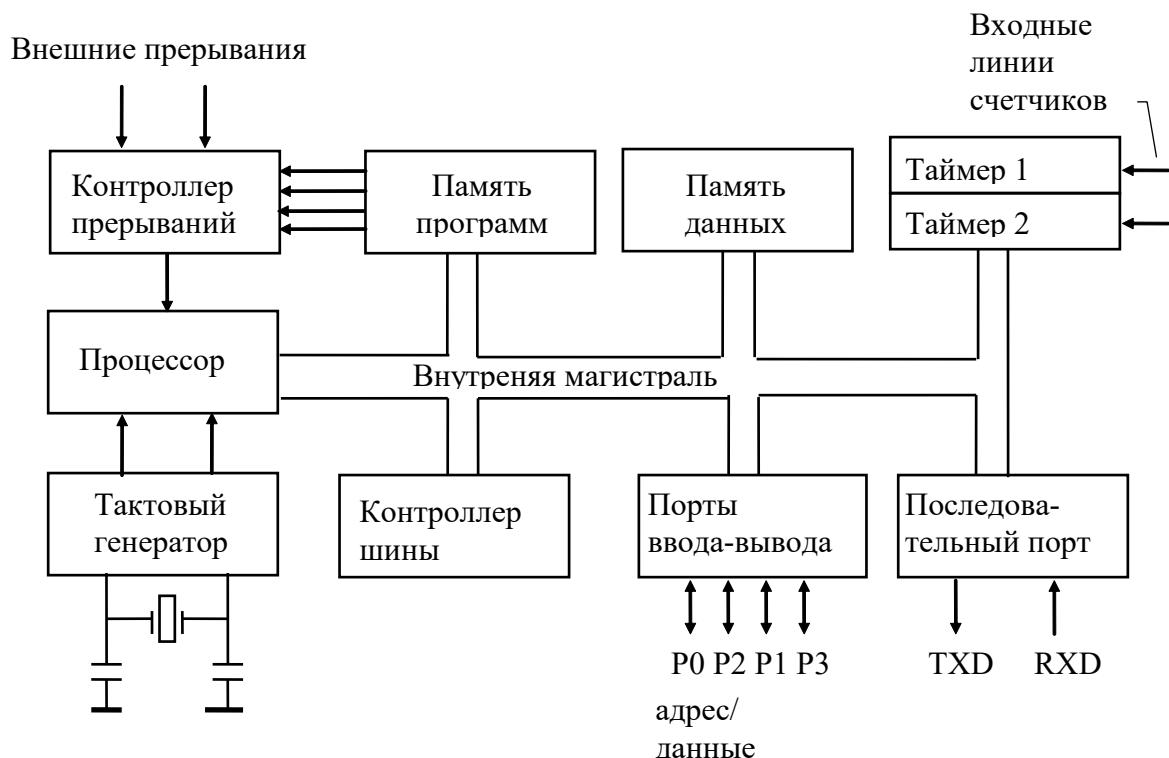


## Лекция № 10

### “Семейство MCS-51 фирмы Intel - как представитель 8-разрядных микроконтроллеров. Обобщенная архитектура семейства. Состав семейства. Типовая схема включения микроконтроллера.”

Семейство 8-разрядных микроконтроллеров MCS-51 было выпущено фирмой Intel в начале 80-х годов. Микроконтроллеры MCS-51 являются функционально завершенными однокристальными микроЭВМ Гарвардской архитектуры, содержащими все необходимые узлы для работы в автономном режиме, и предназначены для реализации различных цифровых алгоритмов управления. На сегодняшний день семейство MCS-51 содержит несколько десятков типов микросхем, отличающихся конкретной реализацией отдельных узлов и условиями эксплуатации. Все микросхемы семейства обладают аналогичной архитектурой и имеют целый ряд общих узлов.

Структурная схема микроконтроллера MCS-51.



Общими узлами всех микроконтроллеров семейства являются:

- 8-разрядный центральный процессор (ЦП), ориентированный на управление исполнительными устройствами. ЦП имеет встроенную схему 8-разрядного аппаратного умножения и деления чисел. Наличие в наборе команд большого числа операций для работы с прямоадресуемыми битами дает возможность говорить о т.н. “булевом процессоре”;
- внутренняя память программ масочного или репрограммируемого типа, имеющая для различных кристаллов объем от 4 до 32 Кбайт; в некоторых версиях она отсутствует;
- не менее чем 128 байтное резидентное ОЗУ данных, которое используется для организации регистровых банков, стека и хранения пользовательских данных;
- не менее 32 двунаправленных интерфейсных линий (портов), индивидуально настраиваемых на ввод или вывод информации;
- два 16-битных многорежимных счетчика/таймера, используемых для подсчета внешних событий, организации временных задержек и тактирования коммуникационного порта;
- двунаправленный дуплексный последовательный коммуникационный порт, предназначенный для организации каналов связи между микроконтроллером и внешними устройствами;
- двухуровневая приоритетная система прерываний, поддерживающая не менее 5 векторов прерываний от внутренних и внешних источников;
- встроенный тактовый генератор.

Центральный процессор микроконтроллеров MCS-51 имеет следующие технические характеристики:

Разрядность АЛУ - 8 бит.

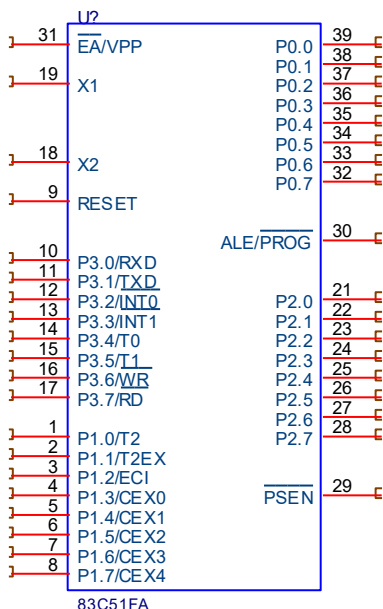
Число выполняемых команд - 111.  
 Длина команд - 1, 2, или 3 байта.  
 Число регистров общего назначения (РОН) - 32.  
 Число прямоадресуемых битовых переменных - 128.  
 Число прямоадресуемых битов в области регистров специальных функций - 128.  
 Максимальный объем памяти программ - 64 Кбайта.  
 Максимальный объем памяти данных - 64 Кбайта.  
 Максимальный объем внутренней памяти данных - 256 байт.  
 Времы выполнения команд при тактовой частоте 12 МГц:  
     сложение - 1 мкс;  
     пересылки "регистр- внешняя память данных" - 2 мкс;  
     умножение/деление - 4 мкс;  
 Методы адресации операнда - регистровый, косвенный, прямой, непосредственный.

Различные микроконтроллеры семейства содержат широкий набор дополнительных средств для эффективной обработки асинхронных событий, измерения частот и временных интервалов, обработки аналоговых сигналов, управления двигателями, организации мультипроцессорных систем.

В частности, подсемейство 80XC51FX, в которое входит ряд микросхем имеет следующие дополнительные возможности:

- 3 16-битных таймера счетчика.
- Программируемый частотный выход.
- Таймер/счетчик с возможностью прямого и обратного счета.
- Матрица программируемых счетчиков, реализующих режимы:
  - сторожевого таймера (Watch Dog Timer);
  - широтно-импульсного модулятора;
  - захвата/сравнения;
  - высокоскоростного выхода.
- Трехуровневая система защиты памяти.
- 256-байтовое резидентное ОЗУ.
- Четырехуровневая система прерываний.
- 7 внешних источников прерываний.
- 2 режима уменьшенного энергопотребления.

Типичным представителем данного подсемейства является микросхема 83C51FA.



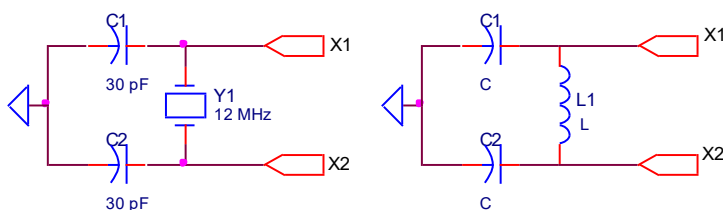
Синхронизация микроконтроллера.

Базовым интервалом времени, на котором основана синхронизация работы всего микроконтроллера является машинный цикл. Машинный цикл имеет фиксированную длительность и содержит 6 состояний S1-S6, каждое из которых по длительности соответствует такту, и, в свою очередь, состоит из двух временных интервалов, определяемых фазами P1 и P2. Длительность фазы равна периоду следования внешнего сигнала синхронизации.



Таким образом каждый машинный цикл состоит из 12 периодов входного тактового сигнала и при частоте последнего 12 МГц составляем 1 мкс. Дважды за один машинный цикл формируется сигнал ALE.

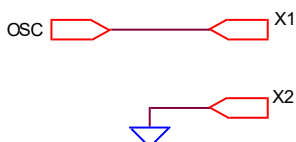
Входной тактовый сигнал вырабатывается либо встроенным тактовым генератором микроконтроллера при подключении к его выводам X1 и X2 кварцевого резонатора или LC-цепочки, либо внешним источником тактовых сигналов.



В случае LC-цепочки частота опорного синхросигнала определяется выражением

$$f = \frac{1}{2\pi\sqrt{LC}}$$

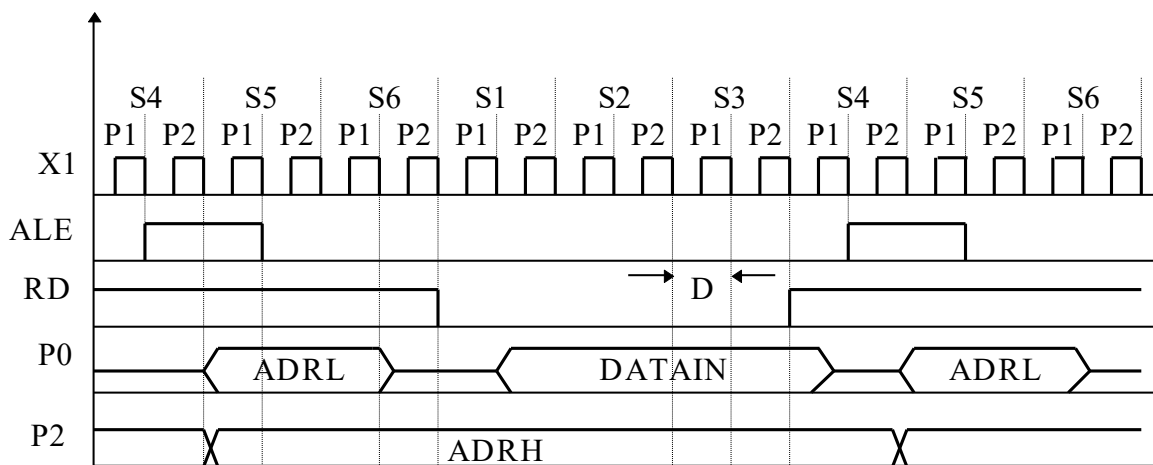
В случае внешнего тактового генератора его выход подключается ко входу X1, а вывод X2 подключается к общей шине.



Внешний интерфейс микроконтроллера.

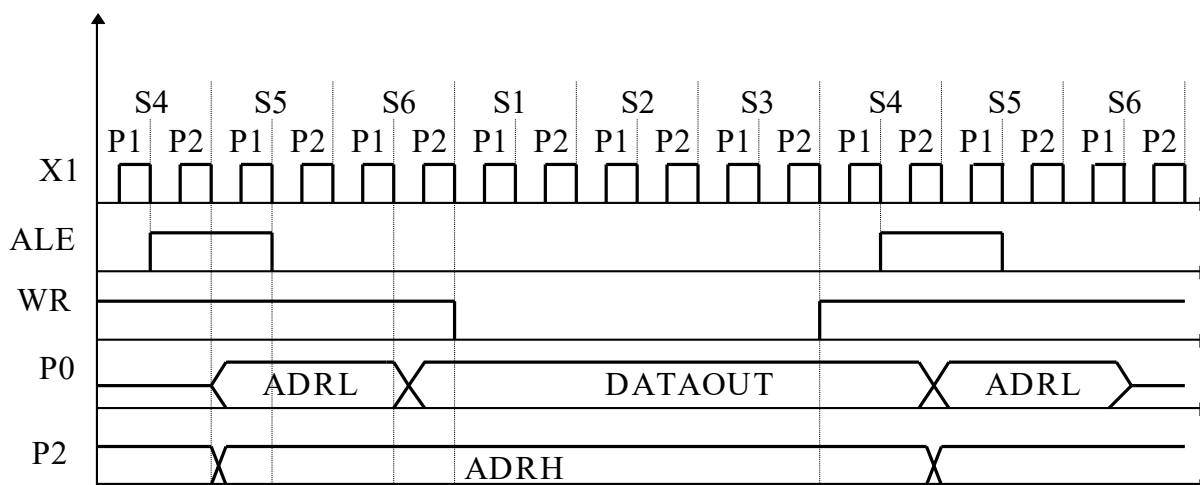
Внешний интерфейс микроконтроллера предназначен для организации взаимодействия его с внешней памятью программ и данных и устройствами ввода-вывода. Обмен данными осуществляется по внешней мультиплексной магистрали микроконтроллера. Эта магистраль использует выводы порта P0 для выдачи младшего байта адреса и чтения/записи байта данных, а выводы порта P2 - для выдачи старшего байта адреса. Обращение к внешней памяти данных требует двух машинных циклов. Поэтому второй импульс ALE внутри машинного цикла при обращении к внешней памяти данных не генерируется.

Цикл чтения из внешней памяти данных.



По заднему фронту сигнала ALE младший байт адреса, поступающий с выводов порта P0 должен быть зафиксирован во внешнем регистре-защелке. Старший байт адреса держится на выводах порта P2 в течение всего цикла и не требует внешней фиксации. Данные считываются в фазе P1 состояний S3 текущего машинного цикла. К этому моменту они должны быть заведомо установлены на шине данных.

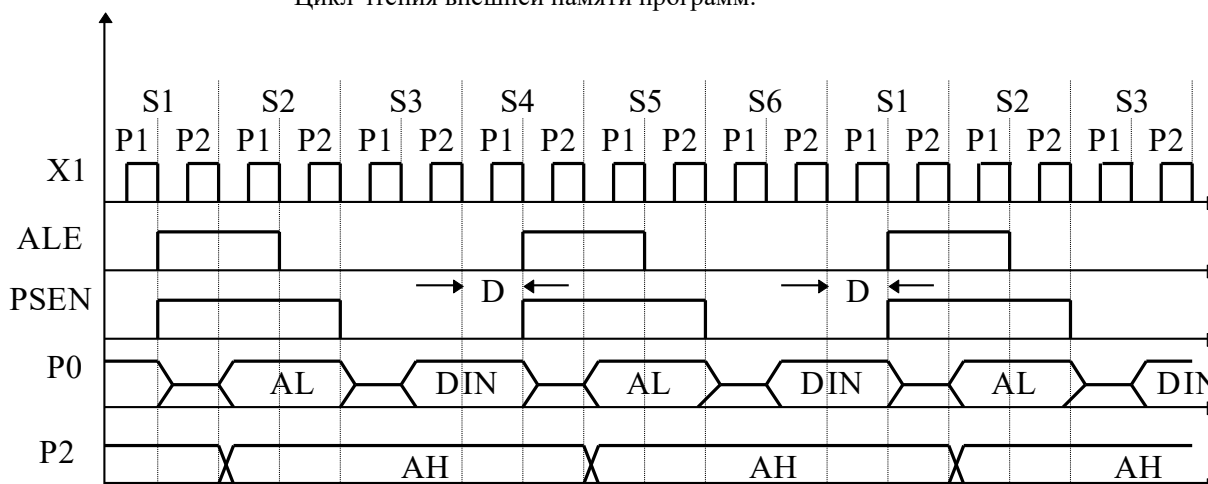
Цикл записи во внешнюю память данных.



При обращении к внутренней памяти данных сигналы RD и WR не вырабатываются. Однако сигнал ALE вырабатывается всегда.

Для обращения ко внешней памяти программ используется механизм отличный от механизма обращения к памяти данных. Для фиксации слов программы используется специальный сигнал PSEN. При этом сигналы ALE и PSEN вырабатываются дважды за машинный цикл. Если второй выбираемый байт в команде не используется то он игнорируется. При переходе к выполнению следующей команды он будет считан вторично.

Цикл чтения внешней памяти программ.



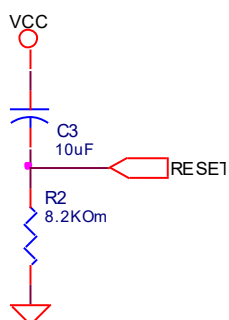
Так же, как и в случае памяти данных, младший байт адреса должен быть зафиксирован во внешнем регистре по заднему фронту сигнала ALE. Байты программы считываются в фазе P1 состояний S4 и S1 текущего машинного цикла. При обращении к внутренней памяти программ сигнал PSEN не генерируется.

Работа с внешней памятью программ разрешена лишь при низком уровне на входе EA. При этом микроконтроллер считает, что вся память программ расположена во внешней памяти. При высоком уровне сигнала на этом входе микроконтроллер реализует цикл обращения к внешней памяти программ лишь в случае, если требуемый адрес выходит за пределы внутреннего сегмента памяти программ. Если в микросхеме прошит бит секретности, то уровень сигнала на входе EA фиксируется при сбросе микроконтроллера во внутренней защелке и дальнейшее его изменение не влияет на работу системы.

### Начальная установка микроконтроллера.

Начальная установка (аппаратный сброс) производится с целью запуска или перезапуска микроконтроллера после подачи на него напряжения питания. Сброс осуществляется подачей на вход RESET высокого уровня и удержания его в течение не менее двух машинных циклов. Этот сигнал может подаваться асинхронно по отношению к внутреннему тактированию. Вход RESET постоянно опрашивается микроконтроллером в момент S5P2 каждого машинного цикла. После сигнала сброса порты ввода-вывода находятся в неизменном состоянии в течение 19-ти периодов тактирования, после чего в промежутке между 19-м и 31-м тактом переводятся в начальное “единичное” состояние. При этом сигналы ALE и PSEN находятся в неактивном высоком состоянии. По сигналу сброса микроконтроллер устанавливает все служебные регистры в начальное состояние. На содержимое внутренней памяти данных сигнал RESET не влияет. При включении питания она устанавливается в произвольное состояние. Длительность сигнала RESET должна быть не меньше времени необходимого для запуска внутреннего генератора плюс 2 машинных цикла. В свою очередь время установки генератора зависит от частоты синхронизации и добротность кварцевого резонатора. При частоте 10 МГц оно обычно составляет около 1 мкс.

Для автоматического рестарта микросхемы после подачи напряжения питания, к выводу RESET необходимо подключить RC-цепочку, обеспечивающую требуемую задержку, позволяющую генерировать одиночный импульс сброса.



После снятия сигнала RESET проходит от 1 до 2 тактовых периодов до их активизации. При этом микроконтроллер начинает выполнять программу с адреса 0000h внутренней либо внешней памяти программ (в зависимости от уровня сигнала EA).

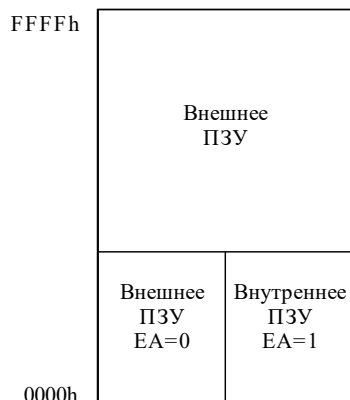
Удержание выводов ALE и PSEN в активном (нулевом) состоянии при активном сигнале RESET приводит к переводу всех выводов микросхемы в высокоимпедансное состояние (режим “ONCE”). Этот режим используется для отладки системы.

### Организация системы памяти микроконтроллера 8xC51Fx.

Все микроконтроллеры семейства MCS-51 имеют Гарвардскую архитектуру, т.е. раздельное адресное пространство программ и данных. Такое разделение позволяет осуществлять доступ к памяти данных по 8-битным адресам, что способствует более быстрой обработке данных в ЦП. Тем не менее, возможно использование и 16-битных адресов данных.

Программная память доступна исключительно по чтению. Общий объем ее адресного пространства составляет 64 Кбайта. В различных версиях кристаллов может присутствовать до 32 Кбайт внутренней памяти программ. При обращении к этим областям памяти используются циклы обмена по внутренней магистрали микроконтроллера. При необходимости можно расширить память программ за счет внешнего ПЗУ.

### Карта памяти программ.



После сигнала RESET микроконтроллер начинает выполнение программы с адреса 0000h либо внешней либо внутренней памяти программ, в зависимости от состояния сигнала EA. У микроконтроллеров MCS-51 имеется два адресных пространства памяти данных. Общий объем внешней памяти данных составляет 64 Кбайта. Доступ к ней осуществляется только с использованием косвенной адресации. Адрес может иметь размер 1 или 2 байта, для чего используются адресные регистры соответствующей разрядности. Однобайтная адресация часто используется при страничной организации ОЗУ. Внутренняя память данных имеет объем 256 байт и доступна как с помощью прямой, так и косвенной адресации. Память данных доступна как по чтению, так и по записи.

Внутренняя память данных может быть разделена на три условных блока - нижний, верхний и пространство Регистров Специальных Функций (SFR).

Карта внутренней памяти данных.

|     |                                                                                        |                                                                                     |
|-----|----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| FFh | Верхняя область<br>внутреннего ОЗУ<br>(доступна в режиме<br>косвенной адресации)       | Область регистров<br>специальных функций<br>(доступна в режиме<br>прямой адресации) |
| ... |                                                                                        |                                                                                     |
| ... |                                                                                        |                                                                                     |
| 80h |                                                                                        |                                                                                     |
| 7Fh | Нижняя область внутреннего ОЗУ<br>(доступна в режимах<br>прямой и косвенной адресации) |                                                                                     |
| ... |                                                                                        |                                                                                     |
| ... |                                                                                        |                                                                                     |
| 00h |                                                                                        |                                                                                     |
|     | косвенная адресация                                                                    | прямая адресация                                                                    |

Внутренняя память данных всегда адресуется байтом, что подразумевает 256 байт адресного пространства. Однако, фактически можно обращаться к 384-м байтам. Прямая адресация ячейки с адресом свыше 7Fh осуществляет доступ к области регистров специальных функций, а косвенная - к верхней области внутренней памяти данных. Т.о. эти две области занимают одно и то же адресное пространство, хотя и разделены физически. Все байты в нижней 128-байтной половине внутренней памяти данных могут адресоваться как прямо, так и косвенно.

Первые 32 байта в нижней области внутреннего ОЗУ сгруппированы в 4 банка по 8 регистров. Инструкции программы могут оперировать с ними как с регистрами общего назначения R0-R7. Два бита регистра PSW определяют, какой из банков используется в текущий момент. Это позволяет более эффективно использовать память программ, поскольку регистровые инструкции короче, чем прямая адресация. Переключение регистровых банков эффективно заменяет операции сохранения регистров в стеке на время обработки прерывания.

Следующие за регистровыми банками 16 байт образуют блок побитно-адресуемого пространства. Набор инструкций MCS-51 содержит широкий набор операций над битами, а 128 бит (16 байт по 8) в этом блоке могут быть прямо адресованы. Битовые адреса имеют значения от 00h до 7Fh.

Карта нижней области внутреннего ОЗУ.

|     |                                             |     |     |     |     |     |     |     |
|-----|---------------------------------------------|-----|-----|-----|-----|-----|-----|-----|
| 7Fh | Побайтно-адресуемая область внутреннего ОЗУ |     |     |     |     |     |     |     |
| ... |                                             |     |     |     |     |     |     |     |
| 30h |                                             |     |     |     |     |     |     |     |
|     | Побитно-адресуемая область внутреннего ОЗУ  |     |     |     |     |     |     |     |
| 2Fh | 7Fh                                         | 7Eh | 7Dh | 7Ch | 7Bh | 7Ah | 79h | 78h |
| 2Eh | 77h                                         | 76h | 75h | 74h | 73h | 72h | 71h | 70h |
| 2Dh | 6Fh                                         | 6Eh | 6Dh | 6Ch | 6Bh | 6Ah | 69h | 68h |
| 2Ch | 67h                                         | 66h | 65h | 64h | 63h | 62h | 61h | 60h |
| 2Bh | 5Fh                                         | 5Eh | 5Dh | 5Ch | 5Bh | 5Ah | 59h | 58h |
| 2Ah | 57h                                         | 56h | 55h | 54h | 53h | 52h | 51h | 50h |

|                   |                    |     |     |     |     |     |     |     |
|-------------------|--------------------|-----|-----|-----|-----|-----|-----|-----|
| 29h               | 4Fh                | 4Eh | 4Dh | 4Ch | 4Bh | 4Ah | 49h | 48h |
| 28h               | 47h                | 46h | 45h | 44h | 43h | 42h | 41h | 40h |
| 27h               | 3Fh                | 3Eh | 3Dh | 3Ch | 3Bh | 3Ah | 39h | 38h |
| 26h               | 37h                | 36h | 35h | 34h | 33h | 32h | 31h | 30h |
| 25h               | 2Fh                | 2Eh | 2Dh | 2Ch | 2Bh | 2Ah | 29h | 28h |
| 24h               | 27h                | 26h | 25h | 24h | 23h | 22h | 21h | 20h |
| 23h               | 1Fh                | 1Eh | 1Dh | 1Ch | 1Bh | 1Ah | 19h | 18h |
| 22h               | 17h                | 16h | 15h | 14h | 13h | 12h | 11h | 10h |
| 21h               | 0Fh                | 0Eh | 0Dh | 0Ch | 0Bh | 0Ah | 09h | 08h |
| 20h               | 07h                | 06h | 05h | 04h | 03h | 02h | 01h | 00h |
| 1Fh<br>...<br>18h | Регистровый банк 3 |     |     |     |     |     |     |     |
| 17h<br>...<br>10h | Регистровый банк 2 |     |     |     |     |     |     |     |
| 0Fh<br>...<br>08h | Регистровый банк 1 |     |     |     |     |     |     |     |
| 07h<br>...<br>00h | Регистровый банк 0 |     |     |     |     |     |     |     |

Для 16 адресов в пространстве SFR имеется возможность как битовой, так и байтовой адресации. Побитно-адресуемыми регистрами являются те, чей шестнадцатеричный адрес заканчивается на “0” или “8”. Битовые адреса в этой области располагаются со значения 80h по FFh.

Наиболее часто используемыми регистрами специальных функций являются:

- Аккумулятор (байтовый адрес 0E0h). Используется во всех арифметических и логических операциях в качестве операнда;
- Регистр В (байтовый адрес - 0F0h). Используется при операциях умножения и деления, а также как сверхоперативный регистр;
- Слово состояния программы PSW (байтовый адрес - 0D0h). Содержит информацию о состоянии программы;
- Указатель стека (байтовый адрес -081h). Используется для указания на вершину стека в операциях записи в стек и чтения из него. По аппаратному сбросу устанавливается в значение 07h (область стека в этом случае начинается с адреса 08h) и инкрементируется при каждой записи в стек. Запись в SP производится для предопределения положения стека во внутренней памяти данных микроконтроллера;
- Указатель данных DPTR (байтовые адреса 082h, 083h). Состоит из двух байт: старшего - DPH и младшего - DPL. Используется как 16-ти или 8-битовый указатель адреса при обращении к внешней памяти или выполнении команды перехода по косвенному адресу.

Слово состояния программы PSW.

Слово состояния программы PSW содержит ряд статусных битов, отражающих текущее состояние процессора.

Формат PSW.

| PSW.7 | PSW.6 | PSW.5 | PSW.4 | PSW.3 | PSW.2 | PSW.1 | PSW.0 |
|-------|-------|-------|-------|-------|-------|-------|-------|
| CY    | AC    | F0    | RS1   | RS0   | OV    | -     | P     |

Назначение отдельных битов PSW.

| Бит         | Назначение                                                                                                    |
|-------------|---------------------------------------------------------------------------------------------------------------|
| CY          | Флаг переноса.                                                                                                |
| AC          | Флаг дополнительного переноса.                                                                                |
| F0          | Флаг пользователя 0.                                                                                          |
| RS1,<br>RS0 | Биты выбора используемого регистрового банка:<br>00 - банк 0;<br>01 - банк 1;<br>10 - банк 2;<br>11 - банк 3. |

|    |                                              |
|----|----------------------------------------------|
| OV | Флаг переполнения.                           |
| -  | Зарезервирован. Доступен по чтению и записи. |
| P  | Флаг четности.                               |

Бит переноса кроме арифметических, используется также и в булевых операциях. Биты RS0 и RS1 используются для выбора одного из 4-х регистровых банков. Бит четности отражает число установленных бит в аккумуляторе: P=1, если аккумулятор содержит нечетное число единиц, и P=0, если четное. Т.о., количество ненулевых битов в аккумуляторе плюс P является всегда четным числом. Два бита в PSW ни с чем не связаны и могут использоваться как флаги состояний.

#### Система команд микроконтроллеров семейства MCS-51.

Система команд MCS-51 поддерживает единый набор инструкций, который предназначен для выполнения 8-битовых алгоритмов управления исполнительными устройствами. Существует возможность использования быстрых методов адресации к внутреннему ОЗУ, осуществления битовых операций над небольшими структурами данных. Имеется развернутая система адресации однобитовых переменных, как самостоятельного типа данных, позволяющая использовать отдельные биты в логических и управляющих командах булевой алгебры.

#### Режимы адресации.

Набор команд MCS-51 поддерживает следующие режимы адресации:

**Прямая адресация.** Операнд определяется 8-битовым адресом в инструкции. Прямая адресация используется только для младшей половины внутренней памяти данных и регистров SFR.

**Косвенная адресация.** Инструкция адресует регистр, содержащий адрес операнда. Данный вид адресации используется для внешнего и внутреннего ОЗУ. Для указания 8-битовых адресов могут использоваться регистры R0 и R1 выбранного регистрового банка или указатель стека SP. Для 16-битовой адресации используется только регистр указателя данных DPTR.

**Регистровые инструкции.** Регистры R0-R7 текущего регистрового банка могут быть адресованы через конкретные инструкции, содержащие 3-хбитовое поле, указывающее номер регистра в самой инструкции. В этом случае соответствующее поле адреса в команде отсутствует.

**Операции с использованием специальных регистров.** Некоторые инструкции используют индивидуальные регистры. Например операции с аккумулятором, DPTR, и т.д. В данном случае адрес операнда вообще не указывается в команде. Он предопределяется кодом операции.

**Непосредственные константы.** Константа может находиться прямо в команде за кодом операции.

**Индексная адресация.** Индексная адресация может использоваться только для доступа к программной памяти и только в режиме чтения. В этом режиме осуществляется просмотр таблиц в памяти программ. 16-битовый регистр (DPTR или программный счетчик) указывает базовый адрес требуемой таблицы, а аккумулятор указывает на точку входа в нее.

#### Арифметические и логические инструкции.

Пример арифметической команды: Операция сложения может быть выполнена одной из следующих команд:

```
ADD A,7Fh
ADD A,@R0
ADD A,R7
ADD A,#127
```

Все арифметические инструкции выполняются за 1 машинный цикл за исключением команды INC DPTR, - требующей 2 машинных циклов, а также операций умножения и деления, выполняемых за 4 машинных цикла. Любой байт во внутренней памяти данных может быть инкрементирован и декрементирован без использования аккумулятора.

Инструкция MUL AB производит умножение данных в аккумуляторе на данные, находящиеся в регистре B, помещая произведение в регистры A (младшая половина) и B (старшая половина).

Инструкция DIV AB делит содержимое аккумулятора на значение в регистре B, оставляя остаток в B, а частное в аккумуляторе.

Инструкция DA A предназначена для двоично-десятичных арифметических операций. Она не делает преобразования двоичного числа в двоично-десятичное, а лишь обеспечивает правильный результат при сложении 2-х двоично-десятичных чисел.

Пример логической команды: Операция логического И может быть выполнена одной из следующих команд:

```
ANL A,7Fh
```



ANL A,@R1

ANL A,R6

ANL A,#53h

Все логические операции над содержимым аккумулятора выполняются за один машинный цикл, остальные - за 2. Логические операции могут производиться над любым из нижних 128 байтов внутренней памяти данных или над любым регистром SFR в режиме прямой адресации без использования аккумулятора.

Операции циклического сдвига (RL A, RLC A и т.д.) перемещают содержимое аккумулятора на 1 бит вправо или влево. В случае левого циклического сдвига младший бит перемещается в старшую позицию. В случае правого циклического сдвига происходит обратное.

Операция SWAP A осуществляет обмен младшей и старшей тетрад в аккумуляторе.

### **Команды передачи данных.**

Команда MOV dest,src позволяет пересылать данные между ячейками внутреннего ОЗУ или SFR без использования аккумулятора. При этом работа с верхней половиной внутреннего ОЗУ может осуществляться только в режиме косвенной адресации, а обращение к регистрам SFR - только в режиме прямой адресации.

Во всех микросхемах MCS-51 стек размещается непосредственно в резидентной памяти данных и увеличивается вверх. Инструкция PUSH вначале увеличивает значение указателя стека SP, а затем записывает в стек байт данных. Команда PUSH и POP используются только в режиме прямой адресации, записывая или восстанавливая байт, но стек является всегда доступным при косвенной адресации через регистр SP. Таким образом, стек может использовать и верхние 128 байт памяти данных. Эти же соображения исключают возможность использования стековых команд для адресации регистров SFR.

Инструкции передачи данных включают в себя 16-битовую операцию пересылки MOV DPTR,#data16, которая используется для инициализации регистра указателя данных DPTR, при просмотре таблиц в программной памяти или для доступа к внешней памяти данных.

Операция XCH A,byte применяется для обмена данными между аккумулятором и адресуемым байтом. Команда XCHD A,@Ri аналогична предыдущей, но выполняется только для младших тетрад участвующих в обмене операндов.

Для доступа к внешней памяти данных используется только косвенная адресация. В случае однобайтных адресов используются регистры R0 или R1 текущего регистрового банка, а для 16-разрядных - регистр DPTR. При любом методе доступа к внешней памяти данных аккумулятор играет роль источника либо приемника информации.

Для доступа к таблицам, размещенным в программной памяти, используются команды:

MOVC A,@A+DPTR

MOVC A,@A+PC

В качестве базового адреса таблицы используется содержимое соответственно DPTR или PC, а смещение берется из A. Эти команды используются исключительно для чтения данных из программной памяти, но не для записи в нее.

### **Булевы операции.**

Микросхемы MCS-51 содержат в своем составе “булевый” процессор. Внутреннее ОЗУ имеет 128 прямоадресуемых бит. Пространство SFR может также поддерживать до 128 битовых полей. Битовые инструкции осуществляют условные переходы, пересылки, сброс, инверсии, операции “И”, “ИЛИ”. Все указанные биты доступны в режиме прямой адресации.

Бит переноса CF в PSW используется как 1-битный аккумулятор булевого процессора.

### **Инструкции переходов.**

Адреса операций переходов обозначаются на языке ассемблера меткой либо реальным значением в пространстве памяти программ. Адреса условных переходов ассемблируются в относительное смещение - знаковый байт, прибавляемый к программному счетчику PC в случае выполнения условия перехода. Границы таких переходов лежат в пределах между -128 и +127, относительно первого байта, следующего за инструкцией. В PSW отсутствует флажок нуля, поэтому инструкции JZ и JNZ проверяют условие “равно нулю” тестирование данных в аккумуляторе.

Существует три вида команды безусловного перехода: SJMP, LJMP, AJMP, различающиеся форматом адреса назначения. Инструкция SJMP кодирует адрес как относительное смещение, и занимает 2 байта. Дальность перехода ограничена диапазоном от -128 до +127 байт относительно инструкции, следующей за SJMP. В инструкции LJMP используется адрес назначения в виде 16-битной константы. Длина команды составляет 3 байта. Адрес назначения может располагаться в любом месте памяти программ. Команда AJMP использует 11-битную константу адреса. Команда состоит из 2-х байт. При выполнении этой инструкции младшие 11 бит адресного счетчика замещаются 11-битным адресом из команды. 5 старших бит PC остаются

неизменными. Т.о., переход может производиться внутри 2К-байтного блока, в котором располагается инструкция, следующая за командой AJMP.

Существует два вида команды вызовы подпрограммы: LCALL и ACALL. Инструкция LCALL использует 16-битный адрес вызываемой подпрограммы. В данном случае подпрограмма может быть расположена в любом месте памяти программ. Инструкция ACALL использует 11-битный адрес подпрограммы. В этом случае вызываемая подпрограмма должна быть расположена в одном 2К-байтном блоке с инструкцией, следующей за ACALL. Оба варианта команды кладут на стек адрес следующей команды и загружают в PC соответствующее новое значение.

Подпрограмма завершается инструкцией RET, позволяющей вернуться на инструкцию, следующую за командой CALL. Эта инструкция снимает со стека адрес возврата и загружает его в PC. Инструкция RETI используется для возврата из подпрограмм обработки прерываний. Единственное отличие RETI от RET состоит в том, что RETI информирует систему о том, что обработка прерывания завершилась. Если в момент выполнения RETI нет других прерываний, то она идентична RET.

Инструкция DJNZ предназначена для управления циклами. Для выполнения цикла N раз надо загрузить в счетчик байт со значением N и закрыть тело цикла командой DJNZ, указывающей на начало цикла.

Команда CJNE сравнивает два своих операнда как беззнаковые целые и производит переход по указанному в ней адресу, если сравниваемые операнды не равны. Если первый операнд меньше, чем второй, то бит переноса CF устанавливается в "1".

Все команды в ассемблированном виде занимают 1, 2 или 3 байта. Структура кода команды может иметь один из следующих видов:

|    | $D_7K D_0$         | $D_7K D_0$ | $D_7K D_0$ |
|----|--------------------|------------|------------|
| 1  | КОП                |            |            |
| 2  | КОП                | #data8     |            |
| 3  | КОП                | adr8       |            |
| 4  | КОП                | bitadr     |            |
| 5  | КОП                | reladr     |            |
| 6  | $a_{10}a_9a_8$ КОП | $a_7K a_0$ |            |
| 7  | КОП                | adr8       | #data8     |
| 8  | КОП                | adr8       | reladr     |
| 9  | КОП                | adr8s      | adr8d      |
| 10 | КОП                | #data8     | reladr     |
| 11 | КОП                | bitadr     | reladr     |
| 12 | КОП                | adr16h     | adr16l     |
| 13 | КОП                | #data16h   | #data16l   |

#### Система прерываний.

Система прерываний предназначена для организации программной обработки событий, происходящих в различных частях системы программных циклов ожидания этих событий.

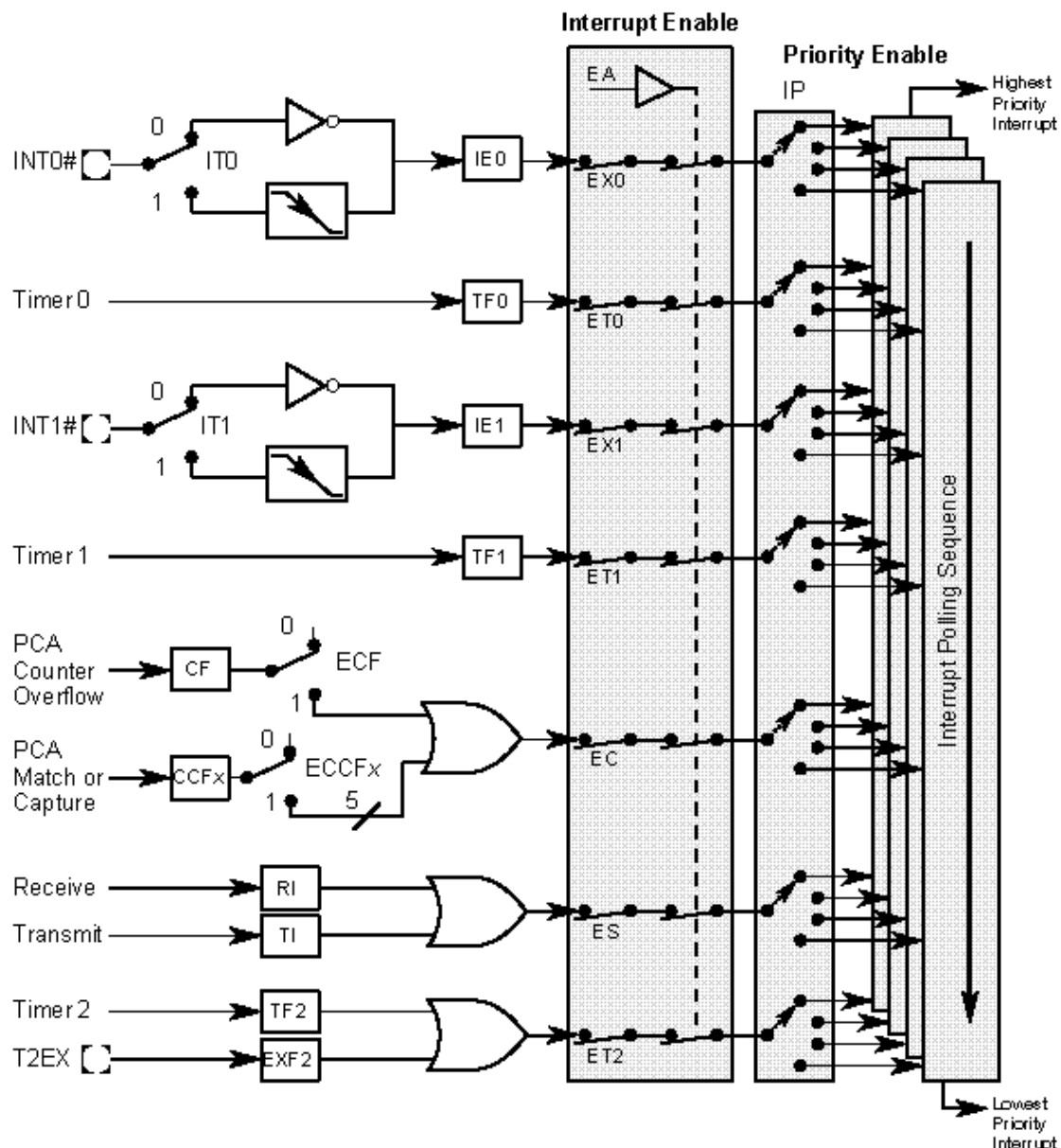
Архитектура семейства MCS-51 поддерживает двух или четырехуровневую приоритетную систему прерываний с количеством источников запросов прерывания до 15. Каждый источник имеет свой, фиксированный вектор прерывания. Под вектором прерывания, в данном случае, понимается ячейка памяти программ с фиксированным адресом, которой передается управление в случае прихода соответствующего запроса прерывания. В частности, микроконтроллеры 8XC51FX имеют 7 векторов прерывания: два внешних вывода запроса прерывания  $\overline{INT0}$  и  $\overline{INT1}$ , прерывания таймеров T0, T1 и T2, прерывание модулей "сравнения-фиксации" и прерывание последовательного порта. Перечисленные источники прерываний имеют следующие вектора прерываний.

| Источник прерывания            | Вектор прерывания |
|--------------------------------|-------------------|
| Внешний вход $\overline{INT0}$ | 0003H             |
| Таймер 0                       | 000BH             |
| Внешний вход $\overline{INT1}$ | 0013H             |
| Таймер 1                       | 001BH             |
| Последовательный порт          | 0023H             |
| Таймер 2                       | 002BH             |
| Модули "сравнения-фиксации"    | 0033H             |

Для приема внешних запросов на прерывание служат линии  $\overline{INT0}$  и  $\overline{INT1}$ , которые могут быть запрограммированы на срабатывание как по фронту, так и по уровню в зависимости от значения битов IT0 и IT1 регистра TCON.

| D7  | D6  | D5  | D4  | D3  | D2  | D1  | D0  |
|-----|-----|-----|-----|-----|-----|-----|-----|
| TF1 | TR1 | TF0 | TR0 | IE1 | IT1 | IE0 | IT0 |

Если значение бита IT0 (для вывода  $\overline{INT0}$ ) или IT1 (для вывода  $\overline{INT1}$ ) равно 0, то соответствующий запрос на прерывание возникает при нулевом уровне сигнала на данном входе (прерывание по уровню). Если же какой-либо из этих битов равен 1, то соответствующий запрос на прерывание возникает при обнаружении отрицательного перепада уровня сигнала на данном входе (прерывание по фронту). При возникновении запроса на прерывание устанавливается бит IE0 или IE1 соответственно. Если прерывание было вызвано по фронту на внешнем выводе  $\overline{INT0}$  или  $\overline{INT1}$ , то соответствующий флаг запроса на прерывание (IE0 или IE1) сбрасывается аппаратно при передаче управления обслуживающей его подпрограмме обработки прерывания. Если запрограммировано прерывание по уровню, то состояние флага запроса соответствует уровню сигнала на его внешнем выводе. Для обнаружения запроса требуемые уровни сигналов должны присутствовать на входе в течение, как минимум одного машинного цикла. Запросы прерываний от внутренних источников генерируются в соответствии с принципами работы устройств, их генерирующих.



Флаг любого прерывания, кроме того, может быть установлен программно для вызова подпрограммы его обработки.

Прерывание от каждого из источников может быть индивидуально разрешено или запрещено путем соответственно установки или сброса определенного бита в регистре IE (Interrupt Enable Register) (0A8H).

| D7 | D6 | D5  | D4 | D3  | D2  | D1  | D0  |
|----|----|-----|----|-----|-----|-----|-----|
| EA | EC | ET2 | ES | ET1 | EX1 | ET0 | EX0 |

Биты регистра IE соответствуют отдельным источникам запросов на прерывание следующим образом:

- EX0 - прерывание  $\overline{\text{INT0}}$ ;
- ET0 - прерывание таймера T0;
- EX1 - прерывание  $\overline{\text{INT1}}$ ;
- ET1 - прерывание таймера T1;
- ES - прерывание последовательного порта;
- ET2 - прерывание таймера T2;
- EC - прерывание модулей “сравнения-фиксации”.

Бит EA является глобальным битом разрешения прерываний. Если он равен 1, то прерывания, которые разрешены установкой индивидуальных флагов, действительно разрешены в системе. Если он равен 0, то все прерывания в системе запрещены, даже если отдельные флаги разрешения конкретных прерываний установлены.

Каждый источник запроса на прерывание может индивидуально быть запрограммирован на один из 4 уровней приоритета путем установки или сброса соответствующих битов в регистрах IP (0B8H) и IPH (0B7H). Регистр IP имеет следующий вид.

| D7 | D6  | D5  | D4 | D3  | D2  | D1  | D0  |
|----|-----|-----|----|-----|-----|-----|-----|
| -  | PPC | PT2 | PS | PT1 | PX1 | PT0 | PX0 |

Регистр IPH имеет следующий вид.

| D7 | D6   | D5   | D4  | D3   | D2   | D1   | D0   |
|----|------|------|-----|------|------|------|------|
| -  | PPCH | PT2H | PSH | PT1H | PX1H | PT0H | PX0H |

Биты регистров IP и IPH соответствуют отдельным источникам запросов на прерывание следующим образом:

- PX0, PX0H - прерывание  $\overline{\text{INT0}}$ ;
- PT0, PT0H - прерывание таймера T0;
- PX1, PX1H - прерывание  $\overline{\text{INT1}}$ ;
- PT1, PT1H - прерывание таймера T1;
- PS, PSH - прерывание последовательного порта;
- PT2, PT2H - прерывание таймера T2;
- PPC, PPCH - прерывание модулей “сравнения-фиксации”.

Уровень приоритета отдельного источника запроса на прерывание определяется установкой значений соответствующих битов регистров IP и IPH следующим образом.

| IPH.X | IP.X | Уровень приоритета |
|-------|------|--------------------|
| 0     | 0    | 0 (наименьший)     |
| 0     | 1    | 1                  |
| 1     | 0    | 2                  |
| 1     | 1    | 3 (высший)         |

Уровень приоритета определяет отношение между различными источниками прерываний. Прерывание с более низким приоритетом может быть, в свою очередь прервано прерыванием с более высоким приоритетом, но не может быть прервано прерыванием с более низким или равным ему приоритетом. Прерывание с самым высоким приоритетом не может быть прервано никогда. Если одновременно приходят два различных запроса на прерывание, то первым будет обслуживаться прерывание с более высоким приоритетом. Если одновременно приходят два запроса на прерывание с одинаковым приоритетом, то порядок их обслуживания определяется внутренней очередностью опроса запросов на прерывание:

1. Прерывание  $\overline{\text{INT0}}$ ;

2. Прерывание таймера T0;
3. Прерывание  $\overline{INT1}$ ;
4. Прерывание таймера T1;
5. Прерывание модулей “сравнения-фиксации”;
6. Прерывание последовательного порта;
7. Прерывание таймера T2.

Процесс обработки прерываний осуществляется следующим образом. Флаги запросов на прерывание опрашиваются в фазе S5P2 каждого машинного цикла в порядке внутренней очередности опроса. Если один или несколько флагов запроса прерывания в этот момент оказываются установленными, то в следующем машинном цикле аппаратно формируется команда LCALL, передающее управление по вектору прерывания, имеющего наивысший приоритет из тех, чьи флаги запроса были установлены и не замаскированы соответствующими битами регистра IE. Процесс вызова подпрограммы обработки прерывания блокируется на некоторое время если:

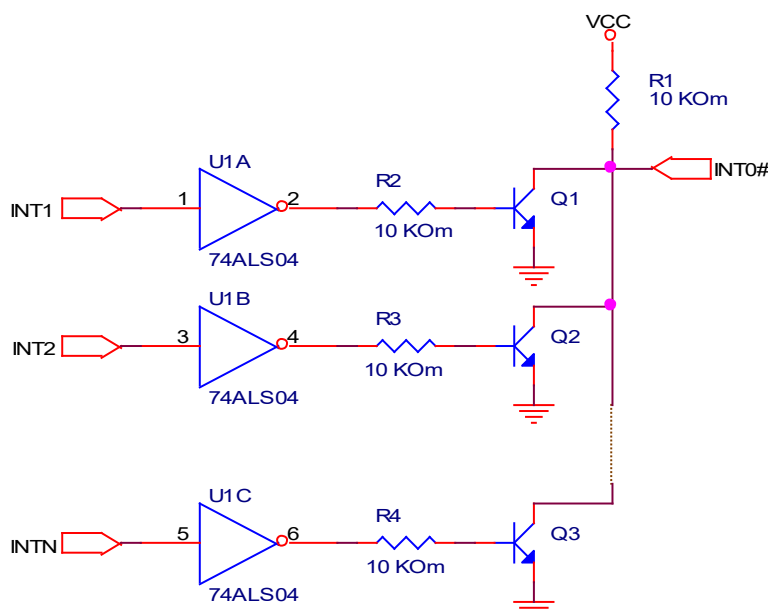
- в данный момент выполняется подпрограмма обработки прерывания с высшим или равным приоритетом;
- текущий машинный цикл не является последним машинным циклом команды, выполняемой в данный момент;
- команда, выполняемая в данный момент является командой RETI или любой командой, осуществляющей запись в регистры IE, IP или IPH.

Т.о., вызов подпрограммы обработки прерывания может осуществиться только на границе между двумя соседними командами основной программы (команда не может быть прервана, не завершившись до конца). Если команда, выполняемая в данный момент является командой RETI или любой командой, осуществляющей запись в регистры IE, IP или IPH, то, как минимум, еще одна команда, следующая за ней будет выполнена до вызова подпрограммы обработки прерываний.

Из вышеизложенного следует, что существует задержка между моментом установления флага запроса на прерывание и моментом запуска подпрограммы обслуживания этого прерывания. Эта задержка может составлять от 3 до 9 машинных циклов в зависимости от момента поступления запроса и выполняемой в этот момент команды. Если внешний вход запроса на прерывание был запрограммирован на срабатывание по уровню, то нулевой уровень на этом входе должен присутствовать до самого момента запуска подпрограммы обработки этого прерывания. Иначе прерывание не будет обслужено.

Подпрограмма обработки любого прерывания должна заканчиваться командой RETI. Эта команда помимо возвращения управления прерванной программе сообщает процессору о том, что данное прерывание больше не обслуживается и могут быть приняты к обслуживанию прерывания с таким же или более низким приоритетом.

Если существуют более чем два внешних источника запросов на прерывание, то они могут быть объединены и подключены к внешним выводам  $\overline{INT0}$  или  $\overline{INT1}$  с помощью схем с открытым коллектором.



В этом случае все объединенные источники запроса на прерывание будут делить один и тот же вектор прерывания и вызванная по этому вектору подпрограмма обработки прерывания должна определять кто конкретно был инициатором данного запроса.

## Лекция № 11

### “Семейство DSP-микроконтроллеров TMS320F28xx фирмы Texas Instruments - как представитель 32-разрядных микроконтроллеров, специализированных для управление импульсными преобразователями.

#### 1. Обобщенная структура DSP-микроконтроллеров семейства TMS320F28XX.

Семейство DSP микроконтроллеров TMS320F28XX фирмы Texas Instruments включает в себя высокопроизводительные 32-разрядные вычислительные устройства, интегрированные с широким набором интеллектуальных устройств ввода-вывода. Высокая вычислительная производительность (до 200 миллионов инструкций в секунду) позволяет с высокой точностью реализовывать в режиме реального времени сложные математически насыщенные алгоритмы управления различными объектами. Большая гамма встроенных периферийных устройств даёт возможность организовывать интерфейс с разнообразными исполнительными устройствами и датчиками обратных связей с минимальной загрузкой вычислительного устройства и минимальным временем реакции на событие.

Обобщенная функциональная схема представителей этого семейства представлена на рис.1. Ее главным элементом является центральное вычислительное устройство (CPU).

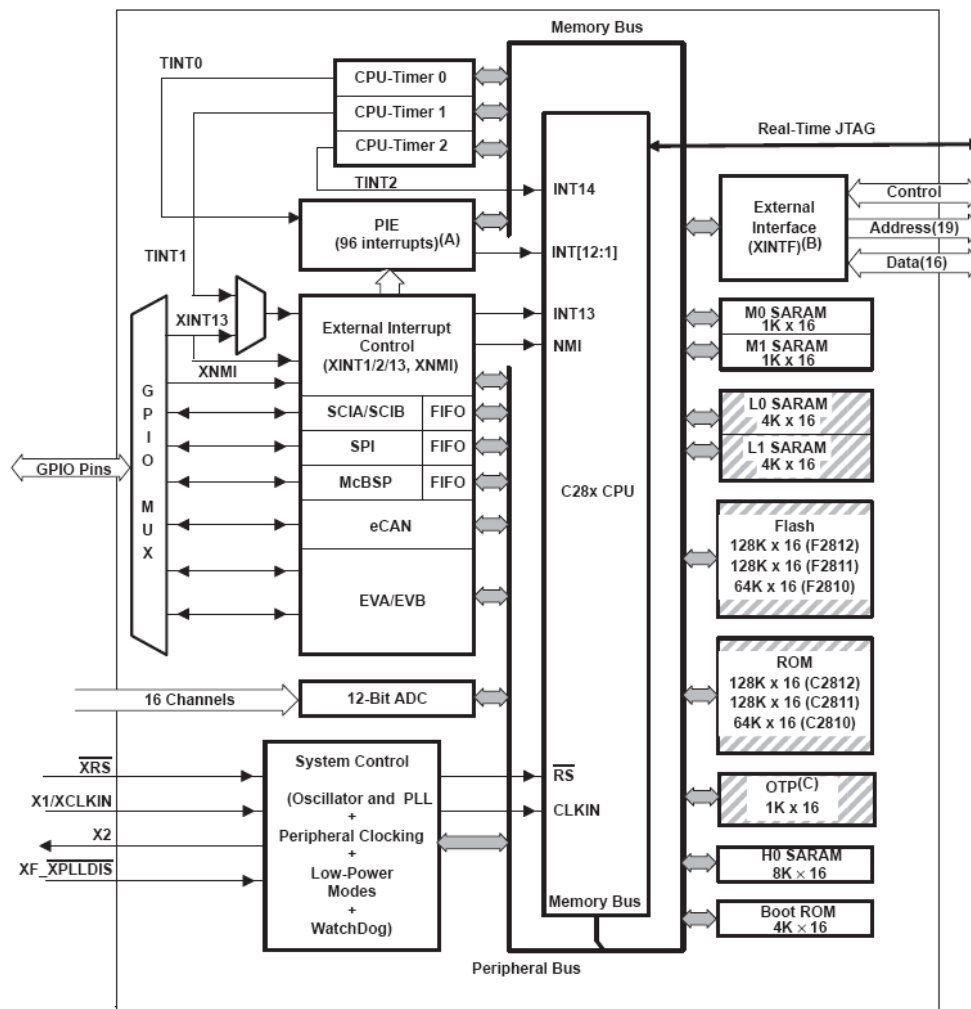


Рис.1.

Архитектурные особенности построения вычислительного ядра этого семейства объединяет в себе достоинства цифровых сигнальных процессоров (DSP – digital signal processor), RISC (Reduced Instruction Set Computer) микропроцессоров и микроконтроллеров.

Вычислительные устройства TMS320F28XX обладают модифицированной Гарвардской архитектурой, характерной для DSP микропроцессоров. Ее характерной чертой является наличие шести отдельных внутренних шин адреса – данных, что позволяет одновременно осуществлять считывание из памяти кода команды и данных, а также обратную запись результата операции в память. Это обеспечивает

выполнение большинства команд за один машинный цикл, который может достигать 5 нс. У RISC-микропроцессоров рассматриваемые устройства унаследовали развитый набор регистров и операции типа «регистр-регистр», что сокращает количество промежуточных операций пересылки данных. Характерной чертой микроконтроллеров, присутствующей у данных устройств, является возможность манипуляции с отдельными битами, что часто используется при реализации задач управления периферийным оборудованием.

## 2. Подсистема памяти DSP-микроконтроллеров семейства TMS320F28XX.

Устройства семейства TMS320F28XX могут адресовать до 4 Гигаслов памяти данных и до 4 Мегаслов памяти программ. Каждый адрес соответствует 16-битному слову. Память программ и память данных объединены в едином блоке памяти, в котором каждая ячейка может быть адресована и как ячейка памяти программ, и как ячейка памяти данных. Все адресное пространство распределено между блоками внутренней памяти, реализованными на одном кристалле с процессором, и блоками внешней памяти, доступным через внешнюю магистраль. Внутренняя память состоит из блоков Flash-памяти суммарным объемом до 256К\*16, блоков ОЗУ суммарным объемом до 34 К\*16, однократно программируемого ПЗУ объемом до 1К\*16. Помимо этого блок ПЗУ объемом до 128К\*16 содержит процедуры автоматической загрузки программы, а также таблицы и процедуры распространенных математических функций. Блоки внутренней памяти имеют механизм защиты от несанкционированного считывания.

Помимо CPU и блоков внутренней памяти на функциональной схеме присутствует большой набор встроенных устройств ввода-вывода. Три 32-разрядных таймера позволяют синхронизировать выполнение различных алгоритмов управления в реальном времени. 12 или 16-разрядный аналого-цифровой преобразователь с временем преобразования до 80 нс на канал дает возможность вводить и обрабатывать до 24 аналоговых сигналов. Один или два 6-канальных широтно-импульсных модулятора позволяют реализовывать различные законы управления силовыми преобразователями энергии. Широкий набор последовательных интерфейсов, таких, как SPI, UART, CAN, MCBSP дают возможность подключать к процессору дополнительные периферийные устройства и организовывать интерфейс с другими частями системы управления. В дополнение к этому в распоряжении разработчиков имеется до 56 портов ввода-вывода общего назначения. Все периферийные устройства, а также другие внутренние и внешние элементы системы, могут служить источниками запросов прерываний, которые обрабатываются системой приоритетного арбитража прерываний.

Для синхронизации всех составных частей микроконтроллера используется внутренний генератор синхросигнала с системой умножения частоты.

Для целей отладки на реальном оборудовании все устройства данного семейства имеют в своем составе оборудование для внутрисхемной эмуляции с интерфейсом через стандартный JTAG-порт.

3. Вычислительное ядро DSP-микроконтроллеров семейства TMS320F28XX.

3.1. Структура вычислительное ядро DSP-микроконтроллеров семейства TMS320F28XX.

Структурная схема центрального вычислительного устройства семейства TMS320F28XX представлена на рис.2.

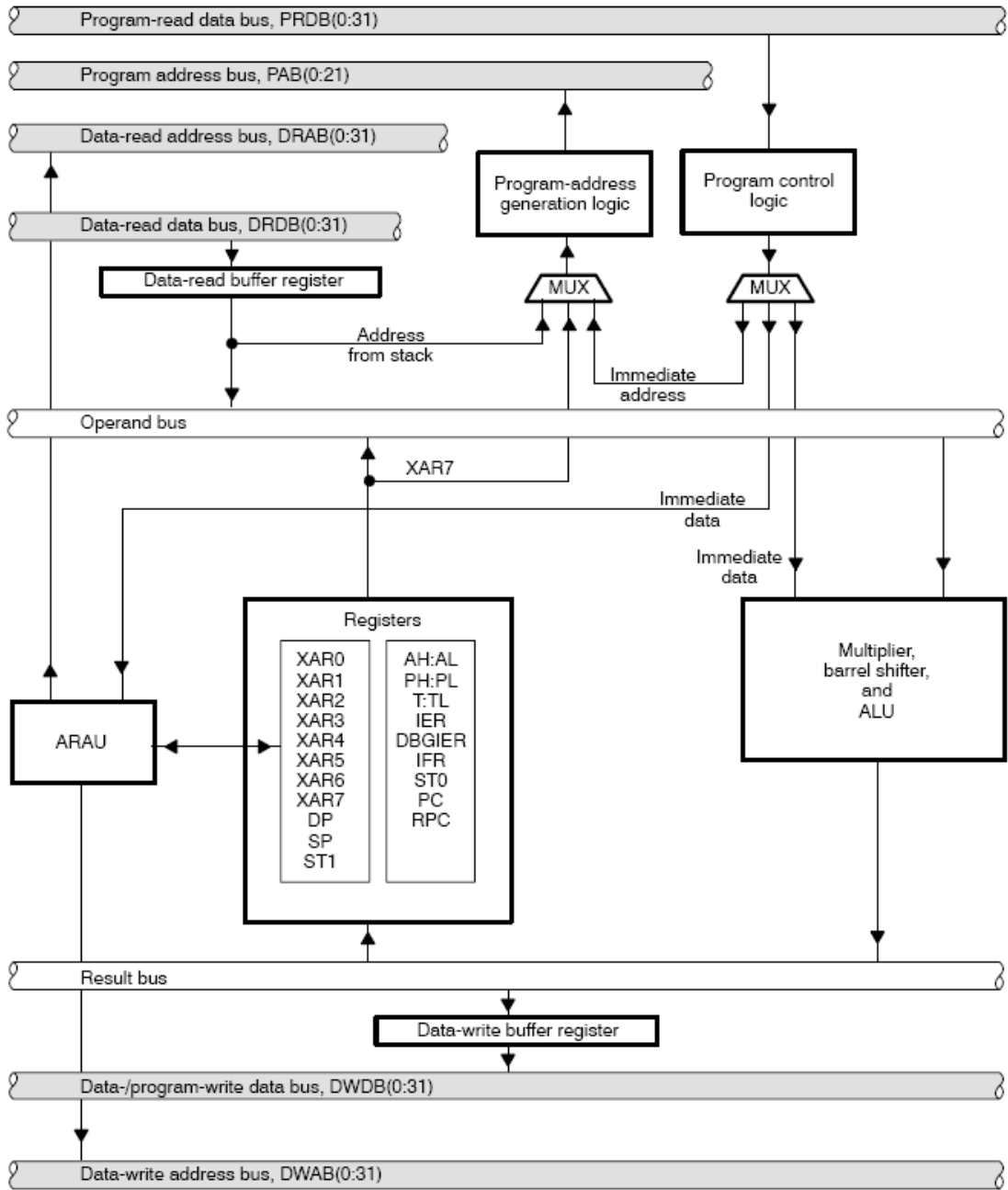


Рис.2.

3.2. Регистры вычислительного ядра DSP-микроконтроллеров семейства TMS320F28XX.

Список регистров вычислительного устройства представлен в таб.1.

| Регистр | Разрядность | Описание                  |
|---------|-------------|---------------------------|
| ACC     | 32          | Аккумулятор               |
| AH      | 16          | Старшая половина ACC      |
| AL      | 16          | Младшая половина ACC      |
| XAR0    | 32          | Регистр общего назначения |
| XAR1    | 32          | Регистр общего назначения |



|        |    |                                                   |
|--------|----|---------------------------------------------------|
| XAR2   | 32 | Регистр общего назначения                         |
| XAR3   | 32 | Регистр общего назначения                         |
| XAR4   | 32 | Регистр общего назначения                         |
| XAR5   | 32 | Регистр общего назначения                         |
| XAR6   | 32 | Регистр общего назначения                         |
| XAR7   | 32 | Регистр общего назначения                         |
| AR0    | 16 | Младшая половина XAR0                             |
| AR1    | 16 | Младшая половина XAR1                             |
| AR2    | 16 | Младшая половина XAR2                             |
| AR3    | 16 | Младшая половина XAR3                             |
| AR4    | 16 | Младшая половина XAR4                             |
| AR5    | 16 | Младшая половина XAR5                             |
| AR6    | 16 | Младшая половина XAR6                             |
| AR7    | 16 | Младшая половина XAR7                             |
| DP     | 16 | Указатель страницы данных                         |
| IFR    | 16 | Регистр флагов прерываний                         |
| IER    | 16 | Регистр разрешения прерываний                     |
| DBGIER | 16 | Регистр разрешения прерываний в отладочном режиме |
| P      | 32 | Регистр произведения                              |
| PH     | 16 | Старшая половина P                                |
| PL     | 16 | Младшая половина P                                |
| PC     | 22 | Программный счетчик                               |
| RPC    | 22 | Регистр адреса возврата                           |
| SP     | 16 | Указатель стека                                   |
| ST0    | 16 | Регистр статуса 0                                 |
| ST1    | 16 | Регистр статуса 1                                 |
| XT     | 32 | Регистр множимого                                 |
| T      | 16 | Старшая половина T                                |
| TL     | 16 | Младшая половина T                                |

Таб.1.

Регистр-аккумулятор ACC является приемником результата и/или источником операнда в большинстве вычислительных операций. Младшая и старшая половины ACC могут быть отдельно адресованы как AL и AH соответственно.

Регистровый банк XAR0-XAR7 содержит 8 регистров, которые могут использоваться как регистры-указатели либо регистры данных. Младшие половины этих регистров могут быть отдельно адресованы как AR0-AR7. При обращении к младшим половинам XAR0-XAR7 их старшие половины могут модифицироваться либо не модифицироваться в зависимости от конкретной инструкции. Старшие половины XAR0-XAR7 отдельно не адресуются.

Регистр DP используется при прямых методах адресации для указания необходимой страницы данных. При этом адресное пространство памяти данных разделяется на последовательные непересекающиеся страницы по 64 слова каждая. Для адресации нижних 4 Мегаслов необходимо, таким образом, 65536 страниц, для указания номера которых используется 16-разрядный регистр DP. Данные выше первых 4 Мегаслов адресуются с помощью других методов адресации (косвенных).

Регистр XT используется в операциях умножения для хранения 32-разрядного множимого. Младшие 16 разрядов регистра XT адресуются как регистр TL. Этот регистр может быть загружен непосредственно 16-разрядной константой, при этом старшая половина регистра XT автоматически расширяется знаком константы. Старшие 16 разрядов регистра XT адресуются как регистр T.

Регистр P предназначен для хранения 32-разрядного произведения в операциях умножения. В некоторых инструкциях перед сохранением произведения оно сдвигается вправо или влево на количество разрядов, определяемое значением поля PM регистра ST0. Младшая и старшая половины регистра P могут быть адресованы отдельно как регистры PL и PH соответственно.

Регистр PC содержит адрес инструкции, которая будет выполняться следующей.

Регистр RPC используется для сохранения адреса возврата при вызове процедуры с помощью команды LCR и возврата из процедуры с помощью команды LRETR. Другие команды вызова и возврата из процедур для сохранения адреса возврата этот регистр не используют.

Регистр SP хранит адрес текущей вершины стека, организованного в памяти данных. Поскольку этот регистр имеет только 16 разрядов, для организации стека доступны лишь первые 64 Кслов памяти данных. По умолчанию в этом регистре хранится значение 0x0400.

Регистр статуса ST0 имеет структуру, представленную на рис.3.

|          |    |    |    |    |    |    |   |   |   |   |   |   |    |     |     |
|----------|----|----|----|----|----|----|---|---|---|---|---|---|----|-----|-----|
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8 | 7 | 6 | 5 | 4 | 3 | 2  | 1   | 0   |
| OVC/OVCU |    |    |    |    |    | PM |   |   | V | N | Z | C | TC | OVМ | SXM |

Рис.3.

Поле OVC/OVCU является счетчиком переполнений регистра ACC. При выполнении операций над числами со знаком функциональность счетчика зависит от значения поля OVM. Если OVM=0, то OVC инкрементируется каждый раз, когда ACC переполняется в положительном направлении, и декрементируется, когда ACC переполняется в отрицательном направлении. Если OVM=1, то OVC не изменяется при переполнениях ACC. При выполнении операций над числами без знака OVCU инкрементируется каждый раз, когда при выполнении сложения выставлялся в 1 флаг переноса C, и декрементируется, когда флаг переноса C выставляется при выполнении вычитания.

Поле PM определяет тип операции сдвига результата операции умножения перед его помещением в регистр R. Возможные типы операций сдвига, соответствующие различным значениям поля PM представлены в таб.2.

| Значение PM | Операция сдвига Р         |
|-------------|---------------------------|
| 000         | Сдвиг на 1 разряд влево.  |
| 001         | Нет сдвига                |
| 010         | Сдвиг на 1 разряд вправо. |
| 011         | Сдвиг на 2 разряд вправо. |
| 100         | Сдвиг на 3 разряд вправо. |
| 101         | Сдвиг на 4 разряд вправо. |
| 110         | Сдвиг на 5 разряд вправо. |
| 111         | Сдвиг на 6 разряд вправо. |

Таб.2.

Бит V (флаг переполнения) выставляется в 1 если в результате операции сложения или вычитания со знаком результат не уместается в количество разрядов, соответствующее разрядности операции. Будучи установленным, этот бит остается в этом состоянии до тех пор, пока не будет сброшен с помощью системного сброса или команды условного перехода, проверяющей состояние бита V.

Бит N (флаг знака) выставляется в 1 если результат операции оказался отрицательным и сбрасывается в 0 в противоположном случае.

Бит Z (флаг нуля) выставляется в 1 если результат операции оказался равным нулю и сбрасывается в 0 в противоположном случае.

Бит C (флаг переноса) выставляется в 1 если в результате операции сложения произошел перенос из старшего разряда результата, или если в результате операции вычитания не произошел заем в старший разряд результата, и сбрасывается в 0 в противоположных случаях. Этот бит используется также в операциях сдвига.

Бит TC (тестовый флаг) используется в командах TBIT и NORM. При выполнении команды TBIT этот флаг устанавливается в 1 если тестируемый бит равен 1 и сбрасывается в 0 в противоположном случае. При выполнении команды NORM этот флаг устанавливается в 1 если в регистре ACC содержится 0 или если биты 30 и 31 этого регистра не равны друг другу, и сбрасывается в 0 в остальных случаях.

Бит OVM (флаг режима переполнения) определяет действия по модификации результата операций сложения и вычитания, в которых произошло переполнение. Если бит OVM=0, результат переполнения не модифицируется, а факт переполнения фиксируется флагом OVC. Если бит OVM=1, то в случае переполнения в регистр ACC заносится либо максимальное положительное (0x7fffffff) либо максимальное отрицательное (0x80000000) число, в зависимости от того, в какую сторону произошло переполнение.

Бит SXM (флаг расширения знака) используется в инструкциях, в которых 32-разрядный аккумулятор ACC загружается, складывается или вычитается с 16-разрядным числом. Если флаг SXM=0, то 16-разрядное число расширяется до 32-разрядного нулями (как беззнаковое). Если флаг SXM=1, то 16-разрядное число расширяется до 32-разрядного знаковым разрядом (как знаковое).

Регистр статуса ST1 имеет структуру, представленную на рис.4.

|     |    |    |    |         |    |         |       |
|-----|----|----|----|---------|----|---------|-------|
| 15  | 14 | 13 | 12 | 11      | 10 | 9       | 8     |
| ARP |    |    | XF | M0M1MAP | -  | OBJMODE | AMODE |

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|

|          |        |      |     |      |       |      |      |
|----------|--------|------|-----|------|-------|------|------|
| IDLESTAT | EALLOW | LOOP | SPA | VMAP | PAGE0 | DBGM | INTM |
|----------|--------|------|-----|------|-------|------|------|

Рис.4.

Поле ARP определяет один из регистров XAR0 – XAR7 в качестве текущего активного регистра. Этот регистр адресуется во многих командах по умолчанию. Соответствие значений поля ARP номерам выбранных активных регистров представлено в таб.3.

| ARP | Активный регистр |
|-----|------------------|
| 000 | XAR0             |
| 001 | XAR1             |
| 010 | XAR2             |
| 011 | XAR3             |
| 100 | XAR4             |
| 101 | XAR5             |
| 110 | XAR6             |
| 111 | XAR7             |

Таб.3.

Бит XF используется для изменения состояния выходного сигнала XFS. Этот сигнал присутствует только в предыдущих версиях микроконтроллеров TMS320F2XX. В микроконтроллерах TMS320F28XX этот бит конкретного назначения не имеет.

Бит M0M1MAP используется для совместимости со старыми версиями микроконтроллеров TMS320F2XX. Для микроконтроллеров TMS320F28XX этот бит должен быть всегда установлен в 1. При установке этого бита в 0 значение, записываемое по умолчанию в регистр SP после системного сброса равно 0x0000, а расположение блоков памяти M0 и M1 меняется местами.

Бит OBJMODE используется для совместимости программного обеспечения, написанного для предыдущих версий микроконтроллеров TMS320F2XX. По умолчанию после системного сброса этот бит установлен в 0, что соответствует режиму совместимости со старыми версиями микроконтроллеров. Это накладывает некоторые ограничения на использование аппаратных ресурсов новых версий микроконтроллеров. Поэтому, для использования всех функциональных возможностей микроконтроллеров TMS320F28XX, после системного сброса этот бит должен быть установлен в 1 с помощью команды «SETC OBJMODE».

Бит AMODE используется для переключения между способами адресации, используемыми в новых и старых версиях микроконтроллеров TMS320F28XX. Отличие состоит в размере страницы данных. В микроконтроллерах TMS320F28XX размер страницы равен 64 слова, а в предыдущих версиях микроконтроллеров TMS320F28XX – 128 слов. Для микроконтроллеров TMS320F28XX этот бит должен быть установлен в 0. Это значение является значением по умолчанию для этого бита после системного сброса.

Бит IDLRSTAT является флагом режима пониженного энергопотребления. Этот флаг устанавливается автоматически при выполнении инструкции IDLE и сбрасывается автоматически при системном сбросе или начале обслуживания разрешенного прерывания. Этот флаг является битом только для чтения.

Бит EALLOW предназначен для разрешения и запрещения доступа к защищенным регистрам. Такими регистрами являются некоторые конфигурационные регистры микроконтроллера, содержимое которых оказывает существенное влияние на функционирование устройства. Поэтому, эти регистры снабжены дополнительным механизмом защиты от случайного несанкционированного изменения их содержимого. Для снятия этой защиты (что необходимо для занесения в эти регистры нового содержимого) бит EALLOW должен быть установлен в 1 с помощью команды EALLOW. Для возобновления защиты содержимого этих регистров бит EALLOW должен быть сброшен в 0 с помощью команды EDIS.

Бит LOOP устанавливается автоматически в 1 во время выполнения инструкций цикла LOOPZ или LOOPNZ и сбрасывается автоматически в 0 после завершения выполнения этих инструкций.

Бит SPA устанавливается автоматически в 1 если с помощью инструкции ASP указатель стека SP был выровнен до четного адреса. Значение этого бита используется в команде NASP для восстановления первоначального значения указателя стека.

Бит VMAP определяет расположение таблицы векторов прерываний. Если VMAP=0, то процессор выбирает вектора прерываний из таблицы, расположенной в начальной области адресного пространства по адресам 0x000000 – 0x00003f. Эту область памяти занимает ОЗУ, что дает возможность программно сформировать таблицу векторов прерываний. Если VMAP=1, то процессор выбирает вектора прерываний из таблицы, расположенной в конечной области адресного пространства по адресам 0x3FFFC0 – 0x3FFFFFF. Эту область памяти занимает ПЗУ, поэтому данную таблицу изменить невозможно. Бит VMAP по умолчанию установлен в 1 после системного сброса.

Бит PAGE0 предназначен для совместимости с предыдущими версиями микроконтроллеров семейства TMS320F28XX. Если PAGE0=1 адресация памяти данных не учитывает содержимое регистра DP, т.е. адресуется всегда нулевая страница памяти. Если PAGE0=0 к смещению внутри страницы добавляется содержимое регистра SP (указателя стека). Для микроконтроллеров TMS320F28XX бит PAGE0 должен быть сброшен в 0.

Бит DBGМ определяет возможность доступа к внутренней памяти и регистрам микроконтроллера со стороны отладочного оборудования, подключенного к микроконтроллеру через JTAG-порт. Если DBGМ=1, то доступ к внутренней памяти и регистрам микроконтроллера со стороны отладочного оборудования блокирован. Если DBGМ=0, то доступ к внутренней памяти и регистрам микроконтроллера со стороны отладочного оборудования разрешен.

Бит INTM является глобальным флагом разрешения прерываний. Если INTM=0, то обслуживание маскируемых прерываний разрешено. При этом каждое из маскируемых прерываний должно быть еще разрешено индивидуально. Если INTM=1, то обслуживание маскируемых прерываний, даже разрешенных индивидуально, запрещено. Бит INTM не участвует в разрешении прерываний, если микроконтроллер находится в отладочных режимах работы.

### 3.3. Конвейер команд вычислительного ядра DSP-микроконтроллеров семейства TMS320F28XX.

Работа центрального вычислительного устройства по выполнению команд представляет из себя последовательность нескольких этапов по их считыванию из памяти и дальнейшей обработке. Все внутренние блоки вычислительного устройства, реализующие эти операции, синхронизируются системным синхросигналом. В результате, суммарная длительность выполнения каждой команды составляет несколько машинных циклов. Для увеличения быстродействия выполнение отдельных стадий обработки команд организовано параллельно. Для этого в вычислительном устройстве реализован 8-стадийный конвейер выполнения команд. На пути выполнения каждая команда проходит стадии, представленные в таб.4.

| Стадия        | Описание                                                                                                                                                                                                                                                                                                                                                                                               |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| F1 (Fetch 1)  | Адрес команды выставляется на шине адреса памяти программ                                                                                                                                                                                                                                                                                                                                              |
| F2 (Fetch 2)  | Одна или две команды считываются из памяти программ и помещаются в очередь команд.                                                                                                                                                                                                                                                                                                                     |
| D1 (Decode 1) | Определяется размер команды. Проверяется легальность команды.                                                                                                                                                                                                                                                                                                                                          |
| D2 (Decode 2) | Команда декодируется. Если в команде требуется чтение данных из памяти или запись данных в память, то их адрес декодируется. Производится модификация указателя стека и адресных регистров, если таковая необходима. Если требуется изменение последовательности выполнения команд (переход или программное прерывание), оно выполняется. Если команда доходит до этой фазы, она выполняется до конца. |
| R1 (Read 1)   | Если в команде требуется чтение данных из памяти, то их адрес выставляется на соответствующей шине адреса.                                                                                                                                                                                                                                                                                             |
| R2 (Read 2)   | Если в предыдущей фазе были адресованы данные, то они считываются из памяти.                                                                                                                                                                                                                                                                                                                           |
| E (Execute)   | Выполняется вычислительная операция, указанная в команде. Если в команде используются данные, хранящиеся в регистрах, то они считываются в начале этой стадии и записываются обратно в конце этой стадии.                                                                                                                                                                                              |
| W (Write)     | Результат выполнения операции записывается в память.                                                                                                                                                                                                                                                                                                                                                   |

Таб.4.

В обычном режиме работы команды последовательно проходят все стадии конвейера, что обеспечивает выполнение команд каждый машинный цикл. Иллюстрация работы конвейера представлена в таб.5.

| F1 | F2 | D1 | D2 | R1 | R2 | E  | W  | Циклы |
|----|----|----|----|----|----|----|----|-------|
| I1 |    |    |    |    |    |    |    | 1     |
| I2 | I1 |    |    |    |    |    |    | 2     |
| I3 | I2 | I1 |    |    |    |    |    | 3     |
| I4 | I3 | I2 | I1 |    |    |    |    | 4     |
| I5 | I4 | I3 | I2 | I1 |    |    |    | 5     |
| I6 | I5 | I4 | I3 | I2 | I1 |    |    | 6     |
| I7 | I6 | I5 | I4 | I3 | I2 | I1 |    | 7     |
| I8 | I7 | I6 | I5 | I4 | I3 | I2 | I1 | 8     |
|    | I8 | I7 | I6 | I5 | I4 | I3 | I2 | 9     |

|  |  |    |    |    |    |    |    |    |
|--|--|----|----|----|----|----|----|----|
|  |  | I8 | I7 | I6 | I5 | I4 | I3 | I0 |
|  |  |    | I8 | I7 | I6 | I5 | I4 | I1 |
|  |  |    |    | I8 | I7 | I6 | I5 | I2 |
|  |  |    |    |    | I8 | I7 | I6 | I3 |
|  |  |    |    |    |    | I8 | I7 | I4 |
|  |  |    |    |    |    |    | I8 | I5 |

Таб.5.

Однако, иногда возникают ситуации, нарушающие последовательную работу конвейера, что сказывается на скорости выполнения команд. Если в последовательности команд встречается команда перехода, то стадии конвейера F1 – D1 очищаются. Команды, дошедшие до стадии D2 и дальше, при этом продолжают выполняться. Поэтому, первая команда после команды перехода выполнится на 3 машинных цикла дольше. Эта ситуация иллюстрируется в таб.6.

Таким образом, количество исполняемых команд перехода влияет на увеличение времени выполнения программы. Работа конвейера также приостанавливается при чтении или записи данных или кодов команд, если в циклах чтения или записи используются циклы ожидания.

| F1  | F2  | D1  | D2  | R1  | R2  | E   | W   | Циклы |
|-----|-----|-----|-----|-----|-----|-----|-----|-------|
| I1  |     |     |     |     |     |     |     | 1     |
| I2  | I1  |     |     |     |     |     |     | 2     |
| I3  | I2  | I1  |     |     |     |     |     | 3     |
| I4  | I3  | I2  | I1  |     |     |     |     | 4     |
| I10 |     |     |     | I1  |     |     |     | 5     |
| I11 | I10 |     |     |     | I1  |     |     | 6     |
| I12 | I11 | I10 |     |     |     | I1  |     | 7     |
|     | I12 | I11 | I10 |     |     |     | I1  | 8     |
|     |     | I12 | I11 | I10 |     |     |     | 9     |
|     |     |     | I12 | I11 | I10 |     |     | 10    |
|     |     |     |     | I12 | I11 | I10 |     | 11    |
|     |     |     |     |     | I12 | I11 | I10 | 12    |
|     |     |     |     |     |     | I12 | I11 | 13    |
|     |     |     |     |     |     |     | I12 | 14    |

Таб.6.

Увеличение времени выполнения программы может быть также следствием определенного расположения команд в памяти. Различные команды могут занимать в памяти 16 или 32 бита. Команды могут быть расположены в памяти, как по четным, так и по нечетным адресам. Считывание команд из памяти осуществляется по 32 разряда. При этом младшая половина шины данных соответствует четному адресу, а старшая – нечетному. Таким образом, за один цикл чтения считывается одна или две команды. Исключение составляет лишь случай, когда первая команда после команды перехода занимает 32 бита и расположена по нечетному адресу. В этом случае для ее считывания необходимы два цикла чтения. В первом цикле считываются начальные 16 бит команды, расположенные по нечетному адресу, а во втором цикле считываются уже, как обычно, 32 бита, в том числе конечные 16 бит данной команды. Таким образом, время выполнения такой команды увеличивается на один (без учета циклов ожидания) машинный цикл. Для исключения такой задержки нужно использовать в качестве первой команды после перехода 16-битную команду, либо располагать ее по четному адресу.

С последовательным характером обработки команд в конвейере связана возможность конфликтов данных, используемых на разных стадиях конвейера. В вычислительном устройстве TMS320F28XX реализована защита от подобных конфликтов, которая связана с увеличением времени выполнения команд.

Если в программе следуют одна за другой команды, связанные с записью, а затем с чтением одной и той же ячейки памяти, то возникает конфликт, связанный с тем, что чтение и запись данных в память выполняются на разных стадиях работы конвейера. Поскольку чтение из памяти производится на стадиях R1 и R2, а запись в память – на стадии W, то несмотря на то, что чтение выполняется в команде, следующей за командой записи, реально, оно будет выполнено раньше, чем запись. Это приведет к алгоритмическому конфликту обработки данных. Эта ситуация иллюстрируется в таб.7.

| F1 | F2 | D1 | D2 | R1 | R2 | E | W | Циклы |
|----|----|----|----|----|----|---|---|-------|
| I1 |    |    |    |    |    |   |   | 1     |
| I2 | I1 |    |    |    |    |   |   | 2     |

|    |    |    |    |             |             |    |             |   |
|----|----|----|----|-------------|-------------|----|-------------|---|
| I3 | I2 | I1 |    |             |             |    |             | 3 |
| I4 | I3 | I2 | I1 |             |             |    |             | 4 |
| I5 | I4 | I3 | I2 | I1          |             |    |             | 5 |
| I6 | I5 | I4 | I3 | I2 (чтение) | I1          |    |             | 6 |
| I7 | I6 | I5 | I4 | I3          | I2 (чтение) | I1 |             | 7 |
| I8 | I7 | I6 | I5 | I4          | I3          | I2 | I1 (запись) | 8 |

Таб.7.

Предположим, команда I1 записывает значение в ячейку памяти данных, а команда I2 считывает значение из этой же ячейки. Из таблицы видно, что запись данных первой инструкцией осуществляется в цикле 8, а чтение данных второй командой – в циклах 6 и 7.

Для предотвращения этого конфликта механизм защиты конвейера команд автоматически вставляет после первой команды 3 цикла ожидания, что возвращает требуемую последовательность операций с памятью. Но это увеличивает время выполнения команд. Для исключения такого непродуктивного увеличения времени выполнения программы, в случаях, когда друг за другом следуют команды записи и чтения, обращающиеся к одной и той же ячейке памяти, необходимо вставлять между ними 3 неформальные команды. Это иллюстрируется в таб.8.

| F1 | F2 | D1 | D2 | R1          | R2          | E  | W           | Циклы |
|----|----|----|----|-------------|-------------|----|-------------|-------|
| I1 |    |    |    |             |             |    |             | 1     |
| I2 | I1 |    |    |             |             |    |             | 2     |
| I3 | I2 | I1 |    |             |             |    |             | 3     |
| I4 | I3 | I2 | I1 |             |             |    |             | 4     |
| I5 | I4 | I3 | I2 | I1          |             |    |             | 5     |
| I5 | I4 | I3 | I2 |             | I1          |    |             | 6     |
| I5 | I4 | I3 | I2 |             |             | I1 |             | 7     |
| I5 | I4 | I3 | I2 |             |             |    | I1 (запись) | 8     |
|    | I5 | I4 | I3 | I2 (чтение) |             |    |             | 9     |
|    |    | I5 | I4 | I3          | I2 (чтение) |    |             | 10    |
|    |    |    | I5 | I4          | I3          | I2 |             | 11    |
|    |    |    |    | I5          | I4          | I3 | I2          | 12    |

Таб.8.

Аналогичный конфликт возникает в случае последовательного выполнения записи и чтения одного и того же регистра. Запись и чтение данных из регистров осуществляется на стадиях D2 или E конвейера команд. В ситуации, аналогичной рассмотренной ранее, защитный механизм для предотвращения конфликта, также автоматически вставляет до трех циклов ожидания между командами, обращающимися к одному и тому же регистру. Для уменьшения непроизводительных затрат времени в подобных случаях рекомендуется вставлять между этими командами три 3 неформальные команды.

Несмотря на наличие механизма защиты от конфликтов данных в конвейере команд, возможны ситуации, в которых конфликты, тем не менее, возможны. Защитный механизм отслеживает конфликтные ситуации, связанные с обращениями к памяти данных и регистрам. Но аналогичные ситуации, связанные с обращением к памяти программ (с помощью команд PREAD, PWRITE) остаются неконтролируемыми. Поэтому, рекомендуется, где это возможно, заменять данные команды другими, либо учитывать конфликтные ситуации, вставляя перед командой чтения памяти программ три команды, не обращающихся к этой ячейке.

При обращении к специфическому оборудованию могут возникать ситуации, когда обращение к одной ячейке памяти оказывает влияние на содержимое другой ячейки. Такие ситуации в общем случае, не могут быть отслежены защитным механизмом защиты конвейера команд. Для защиты от конфликтов данных в подобных ситуациях в вычислительном устройстве TMS320F28XX реализован дополнительный защитный режим. Он обеспечивается заданием т.н. «защищенного» диапазона памяти данных. При записи и чтении любых ячеек этого диапазона в последовательных командах, защитный механизм вставляет между этими командами такты ожидания. «Защищенный» диапазон задается программированием регистров PROTSTART и PROTRANGE. Значение регистра PROTRANGE определяет размер «защищенного» диапазона в страницах размером 64 слова. Значение регистра PROTSTART задает начальный адрес «защищенного» диапазона в страницах размером 64 слова. Стартовый адрес должен быть кратен выбранному размеру блока.

#### 4. Система прерываний DSP-микроконтроллеров семейства TMS320F28XX.

##### 4.1. Обобщенная структура системы прерываний DSP-микроконтроллеров семейства TMS320F28XX.

Процессорное ядро TMS320F28XX включает в себя систему приоритетного арбитража прерываний. Эта система поддерживает 32 вектора прерываний. Каждый вектор представляет собой 22-разрядный адрес соответствующей процедуры обработки прерывания. Векторы расположены в последовательной таблице векторов прерываний. Каждый вектор расположен в двух 16-разрядных ячейках памяти (в ячейке с младшим адресом расположены младшие разряды адреса). Структура таблицы векторов прерываний представлена в таб.9.

| Прерывание | Адрес  |        | Приоритет          | Описание                                                   |
|------------|--------|--------|--------------------|------------------------------------------------------------|
|            | VMAP=0 | VMAP=1 |                    |                                                            |
| RESET      | 000000 | 3FFFC0 | 1 (наивысший)      | Системный сброс                                            |
| INT1       | 000002 | 3FFFC2 | 5                  | Маскируемое прерывание 1                                   |
| INT2       | 000004 | 3FFFC4 | 6                  | Маскируемое прерывание 2                                   |
| INT3       | 000006 | 3FFFC6 | 7                  | Маскируемое прерывание 3                                   |
| INT4       | 000008 | 3FFFC8 | 8                  | Маскируемое прерывание 4                                   |
| INT5       | 00000A | 3FFCA  | 9                  | Маскируемое прерывание 5                                   |
| INT6       | 00000C | 3FFCC  | 10                 | Маскируемое прерывание 6                                   |
| INT7       | 00000E | 3FFCE  | 11                 | Маскируемое прерывание 7                                   |
| INT8       | 000010 | 3FFD0  | 12                 | Маскируемое прерывание 8                                   |
| INT9       | 000012 | 3FFD2  | 13                 | Маскируемое прерывание 9                                   |
| INT10      | 000014 | 3FFD4  | 14                 | Маскируемое прерывание 10                                  |
| INT11      | 000016 | 3FFD6  | 15                 | Маскируемое прерывание 11                                  |
| INT12      | 000018 | 3FFD8  | 16                 | Маскируемое прерывание 12                                  |
| INT13      | 00001A | 3FFDA  | 17                 | Маскируемое прерывание 13                                  |
| INT14      | 00001C | 3FFDC  | 18                 | Маскируемое прерывание 14                                  |
| DLOGINT    | 00001E | 3FFDE  | 19<br>(наименьший) | Прерывание завершения передачи данных через JTAG интерфейс |
| RTOSINT    | 000020 | 3FFE0  | 4                  | Прерывание операционной системы реального времени          |
|            | 000022 | 3FFE2  | 2                  | Зарезервированный вектор                                   |
| NMI        | 000024 | 3FFE4  | 3                  | Немаскируемое прерывание                                   |
| ILLEGAL    | 000026 | 3FFE6  | -                  | Прерывание нелегальной инструкции                          |
| USER1      | 000028 | 3FFE8  | -                  | Программное прерывание 1                                   |
| USER2      | 00002A | 3FFEA  | -                  | Программное прерывание 2                                   |
| USER3      | 00002C | 3FFEC  | -                  | Программное прерывание 3                                   |
| USER4      | 00002E | 3FFEE  | -                  | Программное прерывание 4                                   |
| USER5      | 000030 | 3FFF0  | -                  | Программное прерывание 5                                   |
| USER6      | 000032 | 3FFF2  | -                  | Программное прерывание 6                                   |
| USER7      | 000034 | 3FFF4  | -                  | Программное прерывание 7                                   |
| USER8      | 000036 | 3FFF6  | -                  | Программное прерывание 8                                   |
| USER9      | 000038 | 3FFF8  | -                  | Программное прерывание 9                                   |
| USER10     | 00003A | 3FFFA  | -                  | Программное прерывание 10                                  |
| USER11     | 00003C | 3FFFFC | -                  | Программное прерывание 11                                  |
| USER12     | 00003E | 3FFFFE | -                  | Программное прерывание 12                                  |

Таб.9.

В зависимости от значения бита VMAP в регистре ST1 таблица векторов прерываний может быть расположена в начале либо в конце адресного пространства памяти программ.

Маскируемые прерывания INT1 – INT14, DLOGINT, RTOSINT могут быть разрешены или запрещены программно, путем установки или сброса соответствующего бита регистра IER. Структура этого регистра представлена на рис.5.

|      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|
| 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
| INT6 | INT7 | INT6 | INT5 | INT4 | INT3 | INT2 | INT1 |

|         |         |       |       |       |      |      |      |
|---------|---------|-------|-------|-------|------|------|------|
| 15      | 14      | 13    | 12    | 11    | 10   | 9    | 8    |
| RTOSINT | DLOGINT | INT12 | INT11 | INT10 | INT9 | INT8 | INT7 |

Рис.5.

Помимо этого все маскируемые прерывания запрещаются установкой бита INTM регистра ST1. Для того чтобы любые маскируемые прерывания обрабатывались, этот бит должен быть сброшен в 0.

Если процессор находится в отладочном режиме и остановлен, то обслуживаться будут только те маскируемые прерывания, флаги разрешения которых установлены в регистре DBGIER и в регистре IER. Значение бита INTM в этом случае игнорируются. Структура регистра DBGIER аналогична структуре регистра IER, представленной на рис.5.

#### 4.2. Алгоритмы обслуживания прерываний DSP-микроконтроллеров семейства TMS320F28XX.

Входы запросов прерываний процессора опрашиваются каждый машинный цикл. В случае обнаружения активного уровня на одном из входов запроса прерывания этот факт фиксируется установкой в 1 соответствующего бита регистра IFR, структура которого аналогична структуре регистра IER, представленной на рис.1.4. Любой бит этого регистра может быть установлен программно для вызова соответствующей процедуры обработки прерывания. После установки бита в регистре IFR, при условии разрешения этого прерывания в регистре IER и глобального разрешения маскируемых прерываний в бите INTM регистра ST1, начинается процесс запуска процедуры обработки прерывания. Этот процесс состоит из следующих этапов:

1. Сброс соответствующего бита регистра IFR.
2. Очистка части конвейера команд до фазы D2.
3. Увеличение программного счетчика PC на 1 или 2 в зависимости от длины текущей команды для формирования адреса возврата, и сохранение его во временном регистре.
4. Считывание соответствующего вектора прерывания.
5. Увеличение указателя стека SP на 1 для выравнивания его содержимого по нечетному адресу.
6. Сохранение в стеке содержимого регистров процессора в следующем порядке: ST0, T, AL, AH, PL, PH, AR0, AR1, ST1, DP, IER, DBGSTAT.
7. Сохранение в стеке адреса возврата (вначале младшая половина, затем – старшая).
8. Сброс соответствующего бита регистра IER для временного запрещения обрабатываемого прерывания.
9. Установка битов INTM и DBGM для временного запрещения всех маскируемых прерываний.
10. Сброс битов LOOP, EALLOW, IDLESTAT регистра ST1.
11. Запись считанного вектора прерывания в программный счетчик PC.

С помощью команды INTR можно программно инициировать любое из прерываний INT1 – INT14, DLOGINT, RTOSINT, NMI независимо от их текущего разрешения. При этом соответствующие биты регистра IFR не устанавливаются. С помощью команды TRAP можно программно инициировать любую из 32 возможных процедур обработки прерывания, указав в качестве аргумента номер вектора прерывания.

Прерывание NMI является немаскируемым и не может быть запрещено.

В случае декодирования несуществующей инструкции автоматически генерируется запрос прерывания ILLEGAL. Это прерывание является немаскируемым и не может быть запрещено. По сохраненному адресу возврата в процедуре обработки этого прерывания можно определить адрес некорректной инструкции.

Системный сброс процессора осуществляется подачей активного (низкого) уровня сигнала на входе RS процессора. При этом все операции процессора приостанавливаются, конвейер команд очищается, регистры процессора сбрасываются в исходное состояние. После снятия активного уровня сигнала со входа RS считывается вектор прерывания RESET и запускается процедура его обработки. Это прерывание является немаскируемым и не может быть запрещено.

#### 4.3. Обработка периферийных прерываний DSP-микроконтроллеров семейства TMS320F28XX.

У большинства контроллеров семейства TMS320F28XX имеется большое количество встроенных устройств ввода-вывода, способных формировать запросы прерываний при возникновении различных специфических ситуаций. Суммарное число источников запросов прерываний значительно превышает количество входов запросов прерываний, имеющееся у процессорного ядра. Для обеспечения возможности



обработки сигналов всех имеющихся источников запросов прерываний в микроконтроллерах семейства TMS320F28XX используется т.н. «расширитель периферийных прерываний» (Peripheral Interrupts Expansion (PIE)). Его назначение заключается в согласовании большого числа источников периферийных прерываний и ограниченного количества входов запросов прерываний процессора.

Структура блока PIE представлена на рис.6.

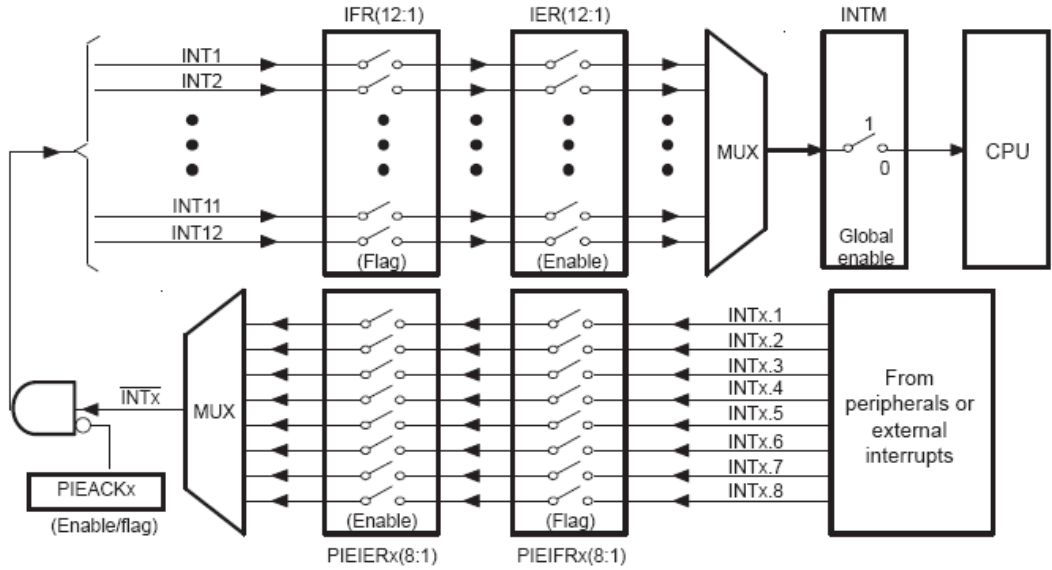


Рис.6.

Все периферийные сигналы запросов прерываний разделены на 12 групп по 8 сигналов в каждой. Каждая из выделенных групп соответствует одному входу запроса прерывания процессора INTx (x=1..12), которые обрабатываются контроллером прерываний процессора. Поступающие периферийные сигналы запросов прерываний устанавливают соответствующие флаги в регистрах PIEIFRx (x=1..12). После активации обработки конкретного прерывания его флаг в конкретном регистре PIEIFRx автоматически сбрасывается. Обработка каждого из сигналов запроса прерывания внутри группы может быть разрешена или запрещена установкой в 1 или сбросом в 0 соответствующего флага INTx.n в регистрах PIEIERx (x=1..12). Структура регистров PIEIFRx и PIEIERx представлена на рис.7.

| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
|--------|--------|--------|--------|--------|--------|--------|--------|
| INTx.8 | INTx.7 | INTx.6 | INTx.5 | INTx.4 | INTx.3 | INTx.2 | INTx.1 |

Рис.7.

При поступлении активного сигнала запроса прерывания, разрешенного к обслуживанию, автоматически устанавливается в 1 бит в регистре PIEACK, соответствующий группе, в которую входит данный сигнал. Пока сигнал в регистре PIEACK установлен в 1, другие сигналы запросов прерываний, входящие в данную группу, не могут быть приняты к обработке. Для обеспечения возможности дальнейшей обработки этих сигналов, в процедуре обработки активного прерывания флаг в регистре PIEACK должен быть сброшен в 0. Для этого в него нужно записать 1.

Структура регистра PIEACK представлена на рис.8.

| 7       | 6       | 5       | 4       | 3       | 2       | 1       | 0       |
|---------|---------|---------|---------|---------|---------|---------|---------|
| PIEACK8 | PIEACK7 | PIEACK6 | PIEACK5 | PIEACK4 | PIEACK3 | PIEACK2 | PIEACK1 |

| 11       | 10       | 9        | 8       |
|----------|----------|----------|---------|
| PIEACK12 | PIEACK11 | PIEACK10 | PIEACK9 |

Рис.8.

Управление работой блока PIE осуществляется с помощью регистра PIECTRL, структура которого представлена на рис.9.

|         |    |    |    |    |    |   |   |   |   |   |   |   |   |   |       |
|---------|----|----|----|----|----|---|---|---|---|---|---|---|---|---|-------|
| 15      | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0     |
| PIEVECT |    |    |    |    |    |   |   |   |   |   |   |   |   |   | ENPIE |

Рис.9.

Бит ENPIE активизирует работу блока PIE. Если этот бит равен 0, блок PIE выключен, и обработка прерываний осуществляется только контроллером прерываний процессорного ядра. При этом используется таблица векторов прерываний процессорного ядра. Если бит ENPIE установлен в 1, то блок PIE включен и обработка прерываний осуществляется с его использованием. При этом используется таблица векторов прерываний PIE. Поле PIEVECT содержит адрес вектора прерывания PIE, которое в данный момент обслуживается (младший бит адреса вектора прерывания игнорируется). Таблица векторов прерываний PIE расположена в оперативной памяти, начиная с адреса 0x0D00. Она должна быть инициализирована программно до активизации работы блока PIE. Инициализация этой таблицы заключается в записи по соответствующим адресам стартовых адресов процедур обработки соответствующих прерываний.

Таблица распределения отдельных источников запросов прерываний по группам с указанием адресов их векторов прерываний представлена на таб.10.

|         | INTx.8                        | INTx.7                        | INTx.6                          | INTx.5                          | INTx.4                          | INTx.3                          | INTx.2                          | INTx.1                          |
|---------|-------------------------------|-------------------------------|---------------------------------|---------------------------------|---------------------------------|---------------------------------|---------------------------------|---------------------------------|
| INT1.y  | WAKEINT<br>(LPM/WD)<br>0xD4E  | TINT0<br>(TIMER 0)<br>0xD4C   | ADCINT<br>(ADC)<br>0xD4A        | XINT2<br><br>0xD48              | XINT1<br><br>0xD46              | Reserved<br>-<br>0xD44          | SEQ2INT<br>(ADC)<br>0xD42       | SEQ1INT<br>(ADC)<br>0xD40       |
| INT2.y  | Reserved<br>-<br>0xD5E        | Reserved<br>-<br>0xD5C        | EPWM6_TZINT<br>(ePWM6)<br>0xD5A | EPWM5_TZINT<br>(ePWM5)<br>0xD58 | EPWM4_TZINT<br>(ePWM4)<br>0xD56 | EPWM3_TZINT<br>(ePWM3)<br>0xD54 | EPWM2_TZINT<br>(ePWM2)<br>0xD52 | EPWM1_TZINT<br>(ePWM1)<br>0xD50 |
| INT3.y  | Reserved<br>-<br>0xD6E        | Reserved<br>-<br>0xD6C        | EPWM6_INT<br>(ePWM6)<br>0xD6A   | EPWM5_INT<br>(ePWM5)<br>0xD68   | EPWM4_INT<br>(ePWM4)<br>0xD66   | EPWM3_INT<br>(ePWM3)<br>0xD64   | EPWM2_INT<br>(ePWM2)<br>0xD62   | EPWM1_INT<br>(ePWM1)<br>0xD60   |
| INT4.y  | Reserved<br>-<br>0xD7E        | Reserved<br>-<br>0xD7C        | Reserved<br>-<br>0xD7A          | Reserved<br>-<br>0xD78          | ECAP4_INT<br>(eCAP4)<br>0xD76   | ECAP3_INT<br>(eCAP3)<br>0xD74   | ECAP2_INT<br>(eCAP2)<br>0xD72   | ECAP1_INT<br>(eCAP1)<br>0xD70   |
| INT5.y  | Reserved<br>-<br>0xD8E        | Reserved<br>-<br>0xD8C        | Reserved<br>-<br>0xD8A          | Reserved<br>-<br>0xD88          | Reserved<br>-<br>0xD86          | Reserved<br>-<br>0xD84          | EQEP2_INT<br>(eQEP2)<br>0xD82   | EQEP1_INT<br>(eQEP1)<br>0xD80   |
| INT6.y  | SPITXINTD<br>(SPI-D)<br>0xD9E | SPIRXINTD<br>(SPI-D)<br>0xD9C | SPITXINTC<br>(SPI-C)<br>0xD9A   | SPIRXINTC<br>(SPI-C)<br>0xD98   | SPITXINTB<br>(SPI-B)<br>0xD96   | SPIRXINTB<br>(SPI-B)<br>0xD94   | SPITXINTA<br>(SPI-A)<br>0xD92   | SPIRXINTA<br>(SPI-A)<br>0xD90   |
| INT7.y  | Reserved<br>0xDAE             | Reserved<br>0xDAC             | Reserved<br>0xDAA               | Reserved<br>0xDA8               | Reserved<br>0xDA6               | Reserved<br>0xDA4               | Reserved<br>0xDA2               | Reserved<br>0xDA0               |
| INT8.y  | Reserved<br>-<br>0xDBE        | Reserved<br>-<br>0xDBC        | Reserved<br>-<br>0xDBA          | Reserved<br>-<br>0xDB8          | Reserved<br>-<br>0xDB6          | Reserved<br>-<br>0xDB4          | I2CINT2A<br>(I2C-A)<br>0xDB2    | I2CINT1A<br>(I2C-A)<br>0xDB0    |
| INT9.y  | ECAN1INTB<br>(CAN-B)<br>0xDCE | ECAN0INTB<br>(CAN-B)<br>0xDCC | ECAN1INTA<br>(CAN-A)<br>0xDCA   | ECAN0INTA<br>(CAN-A)<br>0xDC8   | SCITXINTB<br>(SCI-B)<br>0xDC6   | SCIRXINTB<br>(SCI-B)<br>0xDC4   | SCITXINTA<br>(SCI-A)<br>0xDC2   | SCIRXINTA<br>(SCI-A)<br>0xDC0   |
| INT10.y | Reserved<br>0xDDE             | Reserved<br>0xDDC             | Reserved<br>0xDDA               | Reserved<br>0xDD8               | Reserved<br>0xDD6               | Reserved<br>0xDD4               | Reserved<br>0xDD2               | Reserved<br>0xDD0               |
| INT11.y | Reserved<br>0xDEE             | Reserved<br>0xDEC             | Reserved<br>0xDEA               | Reserved<br>0xDE8               | Reserved<br>0xDE6               | Reserved<br>0xDE4               | Reserved<br>0xDE2               | Reserved<br>0xDE0               |
| INT12.y | Reserved<br>0xDFE             | Reserved<br>0xDFC             | Reserved<br>0xDFA               | Reserved<br>0xDF8               | Reserved<br>0xDF6               | Reserved<br>0xDF4               | Reserved<br>0xDF2               | Reserved<br>0xDF0               |

Таб.10.

## 5. Схемотехника DSP-микроконтроллеров семейства TMS320F28XX.

Условное обозначение микроконтроллера TMS320F28335JGF представлено на рис.11.

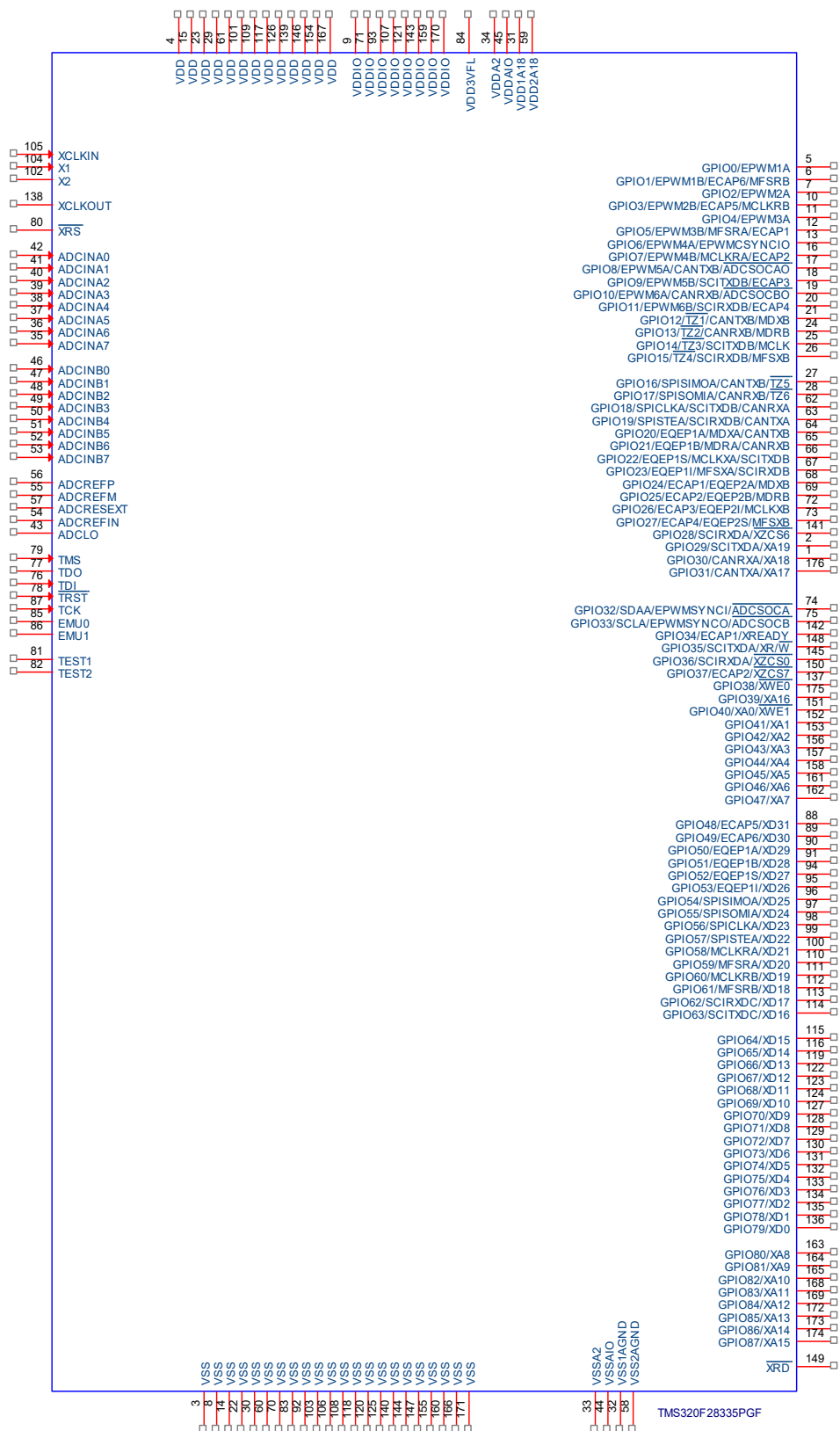


Рис.11

Для питания цифровой и аналоговой частей ядра микроконтроллера на выводы VDD и VDDA необходимо подать напряжение 1.9В. Для питания периферийных устройств и внутренней FLASH-памяти на выводы VDDIO и VDD3FL необходимо подать напряжение 3/3В.

Для запуска внутреннего тактового генератора микроконтроллера в цепь его обратной связи через выводы X1 и X2 необходимо подключить кварцевый резонатор с резонансной частотой от 20 до 35 МГц

(рис.12). Внутренняя система фазовой автоподстройки частоты PLL (Phase Locked Loop) может увеличить эту частоту вплоть до 150 МГц.

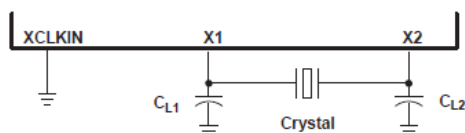


Рис.12

В случае необходимости внешней синхронизации тактовый сигнал частотой до 150 МГц и амплитудой 3.3В, сформированный отдельным генератором, может быть подан на вывод XCLKIN (рис.13).

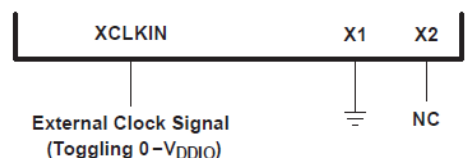


Рис.13

В микроконтроллере реализована система контроля входного тактового сигнала. Если в процессе работы обнаружится пропадание тактового сигнала внутреннего или внешнего тактового генератора, то система PLL продолжит синхронизацию ядра и периферии микроконтроллера с частотой 1 – 5 МГц.

При включении питания и запуске синхронизации во избежание непредвиденных ситуаций микроконтроллер должен удерживаться в состоянии системного сброса. Для этого на вывод XRS должен подаваться сигнал уровня логического нуля. После стабилизации питания и частоты внешней синхронизации сигнал сброса должен удерживаться ещё как минимум на протяжении 8 периодов внешнего тактового сигнала.

Для доступа к внутренним ресурсам микроконтроллера, записи программы во внутреннюю память и реализации режимов отладки предусмотрен специальный отладочный интерфейс JTAG (Joint Test Action Group). Схема подключения микроконтроллера к стандартному разъёму JTAG показана на рис.14.

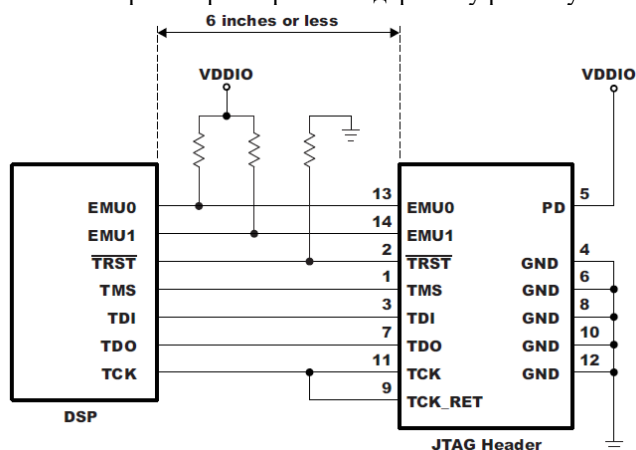


Рис.14

Микроконтроллер имеет в своём составе 88 портов ввода-вывода (GPIO0 – GPIO87). Как видно из рис.11, на этих выводах совмещены функции портов и функции других периферийных устройств. Для выбора конкретных используемых функций каждый вывод должен быть программно сконфигурирован до начала использования.

Внешняя демультиплексная магистраль микроконтроллера тоже выведена через контакты портов. Она может иметь разрядность шины данных 16 или 32 бит и адресовать до 1Мегаслов по 16 бит. Способ подключения внешних устройств к внешней магистрали с разрядностью 16 бит представлен на рис.15.

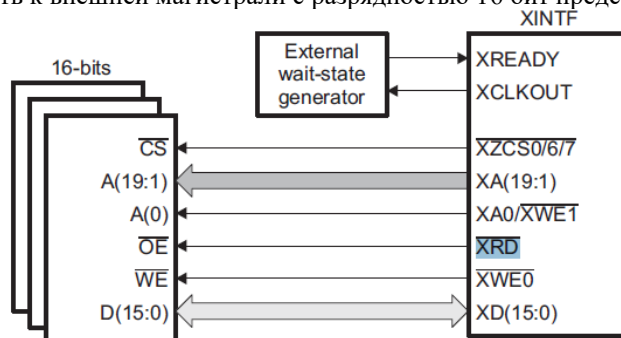


Рис.15

Способ подключения внешних устройств к внешней магистрали с разрядностью 32 бита представлен на рис.16.

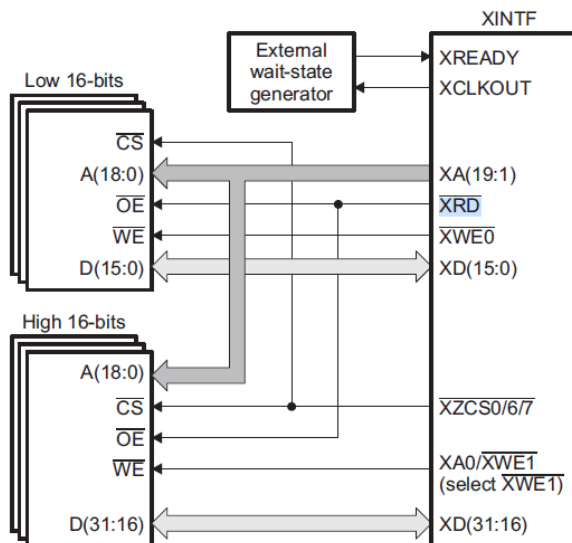


Рис.16

## 6. Система команд DSP-микроконтроллеров семейства TMS320F28XX.

### 6.1. Методы адресации DSP-микроконтроллеров семейства TMS320F28XX.

Набор команд TMS320F28XX поддерживает следующие режимы адресации:

**Прямая адресация.** При этом адресное пространство памяти данных разделяется на последовательные непересекающиеся страницы по 64 слова каждая. Для адресации 4 Мегаслов необходимо, таким образом, 65536 страниц, для указания номера которых используется 16-разрядный регистр DP. Смещение адреса внутри страницы шириной 6 бит размещается в команде.

Примеры:

**MOVW DP,#VarA** – загрузить в регистр DP номер страницы переменной VarA;

**SUB AL,@VarA** – вычесть значение переменной VarA из значения из AL и поместить сумму в AL;

**MOV @VarB, AL** – загрузить в ячейку переменной VarB содержимое регистра AL.

**Стековая адресация.** В данном случае элемент данных адресуется через регистр указателя стека SP. Этот регистр всегда указывает на следующую свободную ячейку после текущей вершины стека. Поле шириной 6 бит размещаемое в команде, может указывать смещение конечного адреса относительно текущей вершины стека.

Примеры:

**ADD AL, \*-SP[5]** – сложить 16-битное число, размещённое по адресу на 5 меньше относительно текущей вершины стека, с содержимым регистра AL и поместить сумму в AL;

**MOV \*-SP[8], AL** – сохранить 16-битное число из регистра AL по адресу на 8 меньше относительно текущей вершины стека;

**ADDL ACC, \*-SP[12]** – сложить 32-битное число, размещённое по адресу на 12 меньше относительно текущей вершины стека, с содержимым регистра ACC и поместить сумму в ACC;

**MOVL \*-SP[34], ACC** – сохранить 32-битное число из регистра AL по адресу на 34 меньше относительно текущей вершины стека.

При помещении числа на вершину стека содержимое SP авто-инкрементируется. При снятии числа с вершины стека содержимое SP авто-декрементируется.

Примеры:

**MOV \*SP++, AL** – сохранить 16-битное число из регистра AL по адресу текущей вершины стека;

**MOVL \*SP++, P** – сохранить 32-битное число из регистра P по адресу текущей вершины стека;

**ADD AL, \*--SP** – снять 16-битное число с вершины стека и сложить его с содержимым регистра AL и поместить сумму в AL;

**MOVL ACC, \*--SP** – снять 32-битное число с вершины стека и поместить его в ACC;

**PUSH ACC** – поместить содержимое регистра ACC на вершину стека;

**PUSH @AR4** – поместить содержимое регистра AR4 на вершину стека;

**PUSH \*XAR4++** – поместить содержимое ячейки памяти, которая адресуется содержимым регистра AR4, на вершину стека с последующим инкрементом XAR4;

**POP ST0** - снять содержимое регистра ST0 с вершины стека;

**POP AR1:AR0** - снять содержимое регистров AR1 и AR0 с вершины стека.

**Косвенная адресация.** В данном случае 32-разрядные регистры XAR0 – XAR7 используются в качестве указателей данных. При выполнении команды может выполняться пост-инкремент или пре/пост-декремент адреса. Смещение конечного адреса относительно базового адреса в регистре может храниться либо в 3-битном поле в команде, либо в отдельном регистре.

Примеры:

**MOVL \*XAR3++, ACC** – сохранить содержимое регистра ACC в ячейке, адрес которой хранится в XAR3, после чего содержимое XAR3 увеличить на 2;

**MOV AL, \*--XAR3** – уменьшить содержимое XAR3 на 1, затем записать содержимое ячейки, адрес которой хранится в XAR3, в регистр AL;

**MOVL \*+XAR2[AR0], ACC** – сохранить содержимое регистра ACC в ячейке, адрес которой определяется как сумма содержимого XAR2 и AR0;

**MOVL \*+XAR2[5], ACC** – сохранить содержимое регистра ACC в ячейке, адрес которой определяется как сумма содержимого XAR2 и 5;

**NOP \*, ARP2** – настроить роле ARP регистра ST1 на указание регистра XAR2;

**MOV \*, #0x4040** - сохранить число 0x4040 в ячейке, адрес которой хранится в регистре XAR2;

**MOV \*++, #0x4040** - сохранить число 0x4040 в ячейке, адрес которой хранится в регистре XAR2, после чего увеличить содержимое XAR2 на 1;

**MOVL \*--, ACC** - сохранить число из регистра ACC в ячейке, адрес которой хранится в регистре XAR2, после чего уменьшить содержимое XAR2 на 2;

**MOVL ACC, \*0++** - загрузить в регистр ACC число из ячейке, адрес которой хранится в регистре XAR2, после чего увеличить содержимое XAR2 на содержимое AR0;

**MOVL \*0--, ACC** - сохранить число из регистра ACC в ячейке, адрес которой хранится в регистре XAR2, после чего уменьшить содержимое XAR2 на содержимое AR0.

**Регистровая адресация.** В данном случае операнды команды хранятся в регистрах микроконтроллера.

Примеры:

**ADDL ACC, @XAR2** – сложить содержимое регистра ACC с содержимым регистра XAR2. Результат поместить в регистр ACC;

**MOV @ACC, XT** - Загрузить в регистр ACC содержимое регистра XT.

**Непосредственная адресация.** В данном случае 16 младших разрядов адреса операнда расположены непосредственно в команде. Остальные старшие разряды адреса считаются равными нулю. Поэтому, данный метод адресации может быть использован только внутри младших 64 КСлов. Если данный метод адресации используется в цикле, то после каждого прохода адрес инкрементируется.

Примеры:

**MOV XAR2, #Array2** – загрузить в регистр XAR2 адрес массива Array2;

**RPT #(N-1)** – повторить следующую команду N раз;

**|| MOV \*XAR2++, \*(0:Array1)** – Записать очередной элемент массива Array1 в соответствующий элемент массива Array2;

**OUT \*(IORegA), @AL** – записать по адресу ввода-вывода IORegA число из регистра AL;

**MOV XAR2, #X\_Arr** - записать в регистр XAR2 адрес массива X\_Arr;

**SPM 4** - настройка сдвига на 4 разряда влево;

**ZARA** – сброс аккумулятора ACC, регистра произведения P и счётчика переполнений OVC;

**RPT #N-1** - повторить следующую команду N раз;

**|| MAC P, \*XAR2++, 0:Coef\_Arr** – сложить содержимое аккумулятора с содержимым регистра произведения P, сдвинутым на 4 разряда влево, умножить число из ячейки памяти адресуемой регистром XAR2 на число из массива в памяти программ Coef\_Arr с последующим инкрементом обоих указателей.

## 6.2. Арифметические инструкции DSP-микроконтроллеров семейства TMS320F28XX.

Большинство арифметических и логических операций связано с регистром аккумулятора ACC или его составными частями.

Примеры:

**ADD ACC, @VarA << #6** – сложить содержимое аккумулятора с содержимым 16-битной ячейки памяти с адресом VarA, сдвинутым на 6 разрядов влево (старшие разряды расширяются либо знаковым битом, либо нулём в зависимости от SXM);

**ADDU ACC, @VarB** – сложить беззнаковое содержимое аккумулятора с содержимым 16-битной ячейки памяти с адресом VarA (старшие разряды расширяются нулём);

**ADDCL ACC, @VarC** - сложить беззнаковое содержимое аккумулятора с содержимым 32-битной ячейки памяти с адресом VarA и флагом переноса;

**SUBL ACC, @VarC** – вычесть из содержимого аккумулятора содержимое 32-битной ячейки памяти с адресом VarA;

**SUBL @VarC, ACC** – вычесть из 32-битной ячейки памяти с адресом VarA содержимое аккумулятора;

**ABS ACC** – найти модуль содержимого аккумулятора;

**MAXL ACC,@VarD** – найти максимальное из чисел, хранящихся в аккумуляторе и 32-битной ячейки памяти с адресом VarA;

**NEG ACC** – поменять знак числа, хранящегося в аккумуляторе;

**AND ACC, @VarD** – побитовое логическое «И» содержимого аккумулятора с содержимым 16-битной ячейки памяти с адресом VarA (старшие разряды расширяются нулём);

**OR ACC, #0x8000 << 4** – побитовое логическое «ИЛИ» содержимого аккумулятора с содержимым 16-битной константы 0X8000, сдвинутой на 4 разряда влево;

**MPY P,T,@Coef** – умножить 16-битное число из регистра T на 16-битное число из ячейки памяти с адресом Coef и поместить 32-битное произведение в регистр P;

**MOV XAR2, #X\_Arr** – записать в регистр XAR2 адрес массива X\_Arr;

**SPM 4** – настройка сдвига на 4 разряда влево;

**ZARA** – сброс аккумулятора ACC, регистра произведения P и счётчика переполнений OVC;

**RPT #N-1** – повторить следующую команду N раз;

**|| MAC P, \*XAR2++, 0:Coef\_Arr** – сложить содержимое аккумулятора с содержимым регистра произведения P, сдвинутым на 4 разряда влево, умножить число из ячейки памяти адресуемой регистром XAR2 на число из массива в памяти программ 0:Coef\_Arr с последующим инкрементом обоих указателей;

**IMPYL P, XT, @Cof32**

**QMPYL ACC, XT, @Coef32** – умножить 32-битное число из регистра XT на 32-битное число из ячейки памяти Coef32, записать в регистр P младшие 32 бита, а в ACC – старшие 32 бита произведения;

**SQRA \*XAR2++** – возведение в квадрат значения из ячейки памяти, адресуемой регистром XAR2 (с последующим автоинкрементом) с накоплением предыдущего значения квадрата в аккумуляторе.

### 6.3. Инструкции ветвления алгоритма DSP-микроконтроллеров семейства TMS320F28XX.

Инструкции переходов предназначены для изменения нормальной последовательности выполнения команд. Инструкция перехода может сопровождаться логическим условием. Если результат условия при данном выполнении команды определяется как «истинно», то переход выполняется, если – как «ложно», то нет. Большинство команд переходов оперируют 16-битным знаковым смещением адреса перехода относительно текущего содержимого программного счётчика PC. Только команды длинных переходов и длинных вызовов процедур оперируют абсолютными 22-разрядными адресами переходов.

Примеры:

**B L1, NEQ** – перейти на метку L1 если флаг нуля в регистре ST0 равен нулю;

**BAR L2, AR0, AR1, EQ** – перейти на метку L2 если числа в регистрах AR0 и AR1 равны друг другу;

**BANZ Loop, AR0--** – перейти на метку Loop если содержимое регистра AR0 не равно нулю с последующим декрементом AR0;

**LB L3** – длинный переход по абсолютной метке L3;

**LB \*XAR7** – длинный переход по абсолютному адресу, хранящемуся в регистре XAR7;

**LC S3** – длинный вызов процедуры по абсолютной метке S3;

**LC \*XAR7** – длинный вызов процедуры по абсолютному адресу, хранящемуся в регистре XAR7;

**LOOPZ @VarA,#0x0004** – ждать, пока логическое «И» числа из ячейки VarA и 0x0004 не станет равно нулю;

**RPT #7** – повторить следующую инструкцию 8 раз.

## Лекция № 12

# Альтернативное вычислительное устройство Control Law Accelerator.

Control Law Accelerator (CLA) представляет из себя полностью независимое программируемое вычислительное устройство, которое может работать одновременно с центральным процессорным устройством микроконтроллера (ЦПУ). Оно дает возможность распараллеливания вычислений для ускоренной реализации критических участков алгоритма, реализуемого микроконтроллером.

### 1. Структура блока CLA.

Общая структурная схема CLA представлена на рис.1.1.

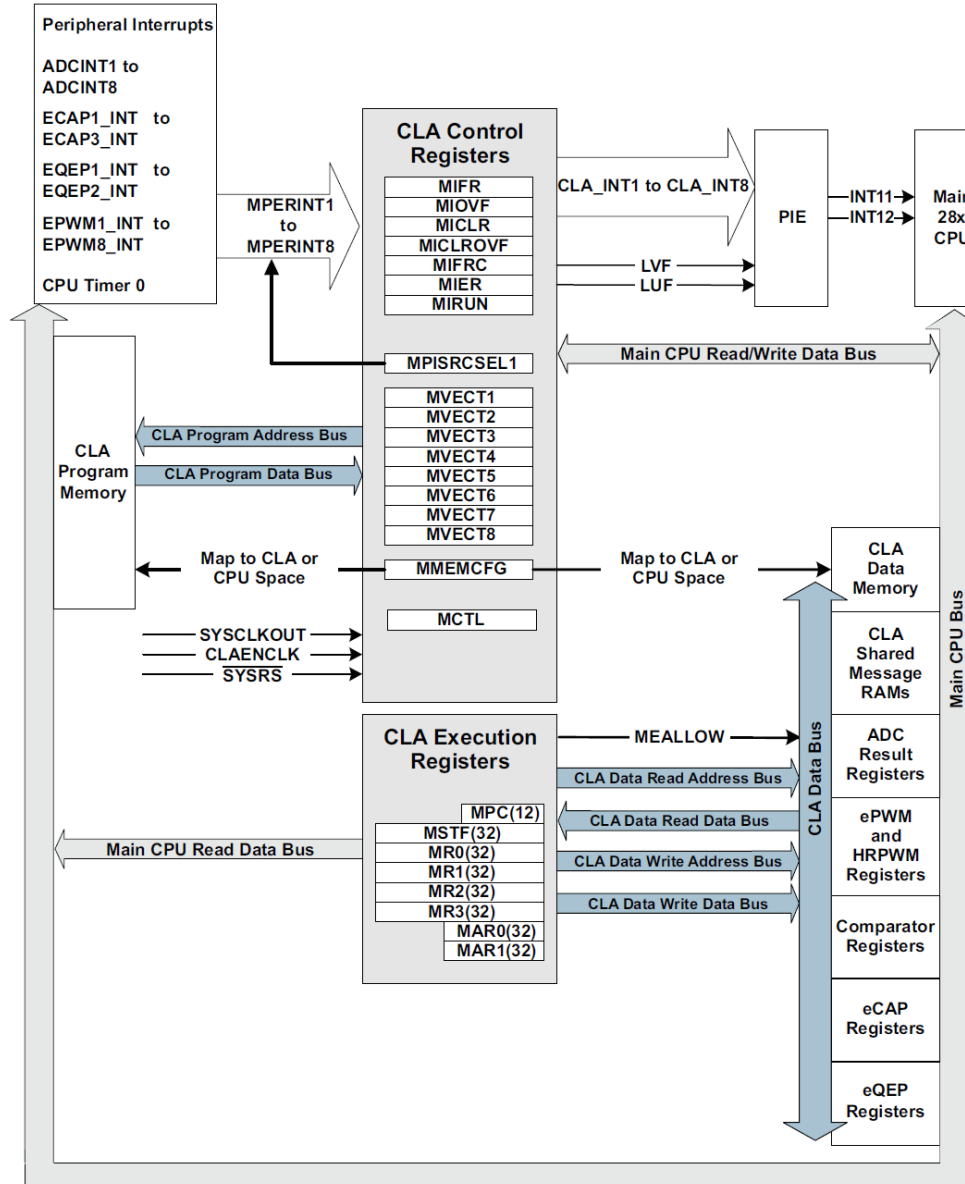


Рис.1.1

CLA тактируется с той же частотой, что и ЦПУ микроконтроллера (SYSCLKOUT), что предопределяет аналогичную производительность у обоих вычислительных устройств. CLA обладает собственными массивами памяти программ и памяти данных, выделяемых в общем блоке памяти микроконтроллера. Помимо этого, для обмена данными с ЦПУ, используются два двухпортовых однонаправленных блока памяти. С помощью одного данные передаются от ЦПУ в CLA, а с помощью другого – в обратном направлении. CLA связан с описанными массивами памяти с помощью собственной многошинной магистрали. Для увеличения вычислительной производительности в CLA реализован собственный 8-стадийный конвейер команд, аналогичный конвейеру ЦПУ. Для реализации вычислений CLA оснащен собственным набором регистров адреса и данных, среди которых 4 32-разрядных регистра данных



(MR0-MR3), 2 16-разрядных адресных регистра (MAR0, MAR1). Аналогично ЦПУ, CLA оснащен регистром статуса, в котором собраны флаги различных признаков состояния вычислений. Собственный набор команд CLA ориентирован на реализацию математически сложных вычислительных алгоритмов. CLA имеет непосредственный доступ к регистрам многих периферийных устройств микроконтроллера, таких, как модуль ШИМ, модуль захвата-сравнения, интерфейс энкодера, модуль АЦП.

## 2. Память блока CLA.

Память CLA выделяется из общего массива памяти микроконтроллера и подразделяется на память программ (Program Memory), память данных (Data Memory) и два двухпортовых однонаправленных массива ОЗУ для обмена информацией с ЦПУ (Shared Message RAMs).

Память программ CLA расположена в массиве ОЗУ микроконтроллера. Изначально этот участок памяти относится, как и остальные, к ЦПУ. Поскольку этот участок памяти является энергозависимым, то ЦПУ должно предварительно переписать программу, по которой будет функционировать CLA в эту область из другой, энергонезависимой области памяти. После того, как программа была переписана, этот блок памяти должен быть перераспределен для CLA. Это осуществляется записью единицы в битовое поле PROGE регистра MMEMCFG. После этого данный блок памяти становится доступным только со стороны CLA для считывания кодов программы. ЦПУ может получить доступ к этой области только тогда, когда CLA находится в отладочном режиме. В противном случае все попытки доступа к этому блоку памяти со стороны ЦПУ будут блокированы и возвращать нулевое значение. Шина данных памяти программ CLA имеет разрядность 32 бита. В то же время, все команды, входящие в систему команд CLA имеют размер 32 бита. Поэтому никакого дополнительного выравнивания команд в памяти не требуется.

Память данных CLA разделена на три отдельных блока, которые изначально, аналогично памяти программ, относятся к ЦПУ. Это позволяет ему осуществлять начальную инициализацию данных для программ CLA. Каждый из данных блоков по отдельности может быть перераспределен для CLA. Это осуществляется записью единицы в битовые поля RAM0E, RAM1E и RAM2E регистра MMEMCFG соответственно. После этого перераспределенные блоки становятся доступными для CLA. Возможность доступа к перераспределенным блокам памяти со стороны ЦПУ сохраняется при условии записи единицы в соответствующие битовые поля RAM0CPUE, RAM1CPUE, RAM2CPUE регистра MMEMCFG соответственно. В противном случае все попытки доступа к этим блокам памяти со стороны ЦПУ будут блокированы и возвращать нулевое значение. Шина данных памяти данных CLA имеет разрядность 16 бит. Помимо ячеек своей памяти данных CLA может осуществлять доступ к регистрам таких периферийных устройств, как блок АЦП и компараторов, блок ШИМ, блок захвата-сравнения, блок интерфейса с энкодером. Структура регистра MMEMCFG представлена на рис.2.1.

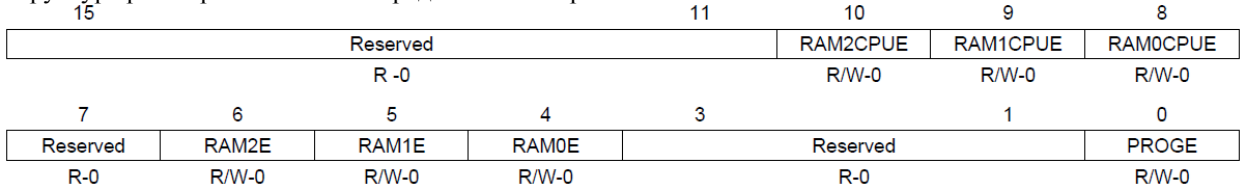


Рис.2.1

Значение конкретных битовых полей регистра MMEMCFG поясняется в таб.2.1.

| Номер бита | Название поля | Значение | Описание                                                                                                           |
|------------|---------------|----------|--------------------------------------------------------------------------------------------------------------------|
| 10         | RAM2CPUE      | 0<br>1   | Доступ к блоку памяти данных RAM2 со стороны ЦПУ когда он распределен для CLA (RAM2E=1):<br>Запрещен;<br>Разрешен. |
| 9          | RAM1CPUE      | 0<br>1   | Доступ к блоку памяти данных RAM1 со стороны ЦПУ когда он распределен для CLA (RAM1E=1):<br>Запрещен;<br>Разрешен. |
| 8          | RAM0CPUE      | 0<br>1   | Доступ к блоку памяти данных RAM0 со стороны ЦПУ когда он распределен для CLA (RAM0E=1):<br>Запрещен;<br>Разрешен. |
| 6          | RAM2E         | 0<br>1   | Распределение блока памяти данных RAM2:<br>Для ЦПУ;<br>Для CLA.                                                    |

|   |       |        |                                                                  |
|---|-------|--------|------------------------------------------------------------------|
| 5 | RAM1E | 0<br>1 | Распределение блока памяти данных RAM1:<br>Для ЦПУ;<br>Для CLA.  |
| 4 | RAM0E | 0<br>1 | Распределение блока памяти данных RAM0:<br>Для ЦПУ;<br>Для CLA.  |
| 0 | PROGE | 0<br>1 | Распределение блока памяти программ CLA:<br>Для ЦПУ;<br>Для CLA. |

Таб.2.1

Помимо памяти программ и памяти данных в распоряжении CLA имеются два небольших двухпортовых однонаправленных блока ОЗУ (Message RAM), предназначенных для обмена информацией между CLA и ЦПУ. Первый из них CLA to CPU Message RAM доступен со стороны CLA по чтению и по записи, а со стороны ЦПУ – только по чтению. Он предназначен для передачи информации из CLA в ЦПУ. Второй – CPU to CLA Message RAM доступен со стороны ЦПУ по чтению и по записи, а со стороны CLA – только по чтению. Он используется для передачи информации от ЦПУ в CLA.

Некоторые регистры периферийных устройств, к которым имеется доступ со стороны CLA, обладают механизмом защиты от несанкционированной записи. Этот механизм со стороны ЦПУ осуществляется с помощью флага EALLOW в регистре статуса ST1. Аналогичный механизм защиты этих регистров имеется и со стороны CLA. Флаг MEALLOW в регистре MSTF, будучи сброшенным в ноль запрещает запись новых значений в защищенные регистры периферийных устройств со стороны CLA.

Структура регистра MSTF представлена на рис.2.2.

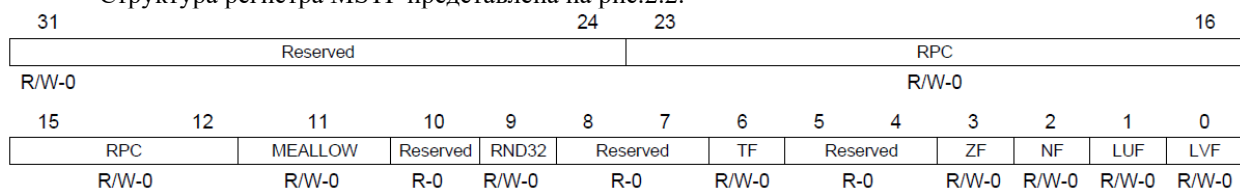


Рис.2.2

Значение конкретных битовых полей регистра MSTF поясняется в таб.2.2.

| Номер бита | Название поля | Значение | Описание                                                                                                                                                                                      |
|------------|---------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 23-16      | RPC           |          | Поле для сохранения адреса возврата из процедуры.                                                                                                                                             |
| 11         | MEALLOW       | 0<br>1   | Запись новых значений в защищенные регистры периферийных устройств:<br>Запрещена;<br>Разрешена.                                                                                               |
| 9          | RND32         | 0<br>1   | Метод округления результата арифметической операции с плавающей точкой:<br>Усечение;<br>Округление.                                                                                           |
| 6          | TF            | 0<br>1   | Результат тестируемого в последней команде условия:<br>Ложь;<br>Истина.                                                                                                                       |
| 3          | ZF            | 0<br>1   | Результат последней операции:<br>Не равен нулю;<br>Равен нулю.                                                                                                                                |
| 2          | NF            | 0<br>1   | Результат последней операции:<br>Больше или равен нулю;<br>Меньше нуля.                                                                                                                       |
| 1          | LUF           | 0<br>1   | Флаг отрицательного переполнения результата:<br>Отрицательное переполнение зафиксировано;<br>Отрицательное переполнение не зафиксировано.<br>Может служить источником запроса прерывания ЦПУ. |
| 0          | LVF           |          | Флаг положительного переполнения результата:                                                                                                                                                  |

|  |  |        |                                                                                                                                                     |
|--|--|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
|  |  | 0<br>1 | Положительное переполнение зафиксировано;<br>Положительное переполнение не<br>зафиксировано.<br>Может служить источником запроса<br>прерывания ЦПУ. |
|--|--|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------|

Таб.2.2

### 3. Запуск задач блока CLA.

Программное функционирование CLA организовано по принципу аппаратно или программно иницируемых задач. Всего возможна реализация до восьми задач, каждая из которых может быть запущена либо программно ЦПУ, либо аппаратно по сигналам, формируемым определенными периферийными устройствами микроконтроллера. Причем каждая из задач имеет свои конкретные программно выбираемые сигналы, выступающие в роли источника запуска. Ими могут служить сигналы прерывания ШИМ, АЦП, блока сравнения-фиксации, интерфейса энкодера, процессорного таймера 0. Настойка конкретного источника запуска для каждой из задач осуществляется программированием битовых полей регистра MPISRCSEL1, структура которого представлена на рис.3.1.

|            |            |            |            |    |    |    |    |
|------------|------------|------------|------------|----|----|----|----|
| 31         | 28         | 27         | 24         | 23 | 20 | 19 | 16 |
| PERINT8SEL | PERINT7SEL | PERINT6SEL | PERINT5SEL |    |    |    |    |
| R/W-0      | R/W-0      | R/W-0      | R/W-0      |    |    |    |    |
| 15         | 12         | 11         | 8          | 7  | 4  | 3  | 0  |
| PERINT4SEL | PERINT3SEL | PERINT2SEL | PERINT1SEL |    |    |    |    |
| R/W-0      | R/W-0      | R/W-0      | R/W-0      |    |    |    |    |

Рис.3.1

Значение конкретных битовых полей регистра MPISRCSEL1 поясняется в таб.3.1.

| Номер бита | Название поля | Значение                                             | Описание                                                                                                                                        |
|------------|---------------|------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-28      | PERINT8SEL    | 0000<br>0010<br>0100<br>0101<br>1000<br>1001<br>1010 | Выбор аппаратного источника запуска задачи 8:<br>ADCINT8;<br>CPU Timer 0;<br>EQEP1_INT;<br>EQEP2_INT;<br>ECAP1_INT;<br>ECAP2_INT;<br>ECAP3_INT; |
| 27-24      | PERINT7SEL    | 0000<br>0010<br>0100<br>0101<br>1000<br>1001<br>1010 | Выбор аппаратного источника запуска задачи 7:<br>ADCINT7;<br>EPWM7_INT;<br>EQEP1_INT;<br>EQEP2_INT;<br>ECAP1_INT;<br>ECAP2_INT;<br>ECAP3_INT;   |
| 23-20      | PERINT6SEL    | 0000<br>0010<br>0100<br>0101<br>1000<br>1001<br>1010 | Выбор аппаратного источника запуска задачи 6:<br>ADCINT6;<br>EPWM6_INT;<br>EQEP1_INT;<br>EQEP2_INT;<br>ECAP1_INT;<br>ECAP2_INT;<br>ECAP3_INT;   |
| 19-16      | PERINT5SEL    | 0000<br>0010<br>0100<br>0101<br>1000<br>1001         | Выбор аппаратного источника запуска задачи 5:<br>ADCINT5;<br>EPWM5_INT;<br>EQEP1_INT;<br>EQEP2_INT;<br>ECAP1_INT;<br>ECAP2_INT;<br>ECAP3_INT;   |

|       |            |                                                      |                                                                                                                                               |
|-------|------------|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
|       |            | 1010                                                 |                                                                                                                                               |
| 15-12 | PERINT4SEL | 0000<br>0010<br>0100<br>0101<br>1000<br>1001<br>1010 | Выбор аппаратного источника запуска задачи 4:<br>ADCINT4;<br>EPWM4_INT;<br>EQEP1_INT;<br>EQEP2_INT;<br>ECAP1_INT;<br>ECAP2_INT;<br>ECAP3_INT; |
| 11-8  | PERINT3SEL | 0000<br>0010                                         | Выбор аппаратного источника запуска задачи 3:<br>ADCINT3;<br>EPWM3_INT;                                                                       |
| 7-4   | PERINT2SEL | 0000<br>0010                                         | Выбор аппаратного источника запуска задачи 2:<br>ADCINT2;<br>EPWM2_INT;                                                                       |
| 3-0   | PERINT1SEL | 0000<br>0010                                         | Выбор аппаратного источника запуска задачи 1:<br>ADCINT1;<br>EPWM1_INT;                                                                       |

Таб.3.1

Помимо аппаратного возможен программный запуск любой из задач CLA со стороны ЦПУ. Он может быть осуществлен двумя путями. Первый заключается в записи единицы в соответствующий бит регистра MIFRC, структура которого представлена на рис.3.2.

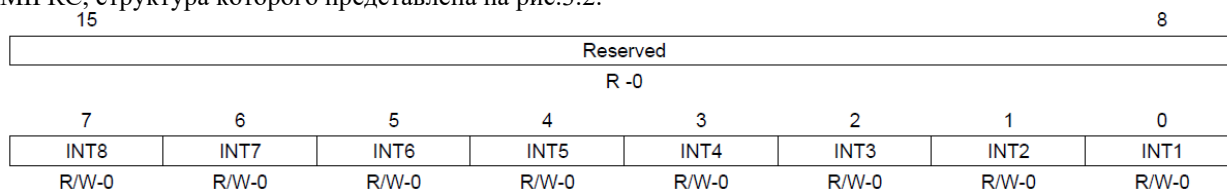


Рис.3.2

Запись в этот регистр можно осуществить непосредственно, либо с помощью команды IACK #t. Каждый бит аргумента t отвечает за запуск соответствующей задачи при установке его в единицу. Таким образом, одновременно можно сформировать запрос на запуск нескольких задач. Использование команды IACK имеет преимущество перед непосредственной записью в регистр MIFRC, поскольку избавляет от необходимости предварительного снятия защиты регистра с помощью команды EALLOW. Однако, для разрешения использования команды IACK необходимо предварительно разрешить ее использование, установив в единицу бит IACKE в регистре MCTL.

#### 4. Арбитраж задач блока CLA.

В каждый момент времени может быть активна только одна из восьми возможных задач CLA. Для возможности арбитража одновременных запросов активизации нескольких задач каждая из них наделяется приоритетом, который убывает по мере увеличения номера задачи. Т.о., наивысший приоритет имеет задача 1, а наименьший – задача 8. Если в период активизации одной из задач поступает сигнал активизации другой задачи, то независимо от приоритета вновь поступившего запроса, он будет ждать своей активизации до момента завершения текущей активной задачи. Механизм вложенных задач у CLA невозможен.

Для того, чтобы задача, в случае поступления ее запроса, могла быть активирована, ее обслуживание должно быть разрешено установкой в единицу соответствующего бита в регистре MIER, структура которого аналогична представленной на рис.1.63.

Все поступившие запросы на запуск задач фиксируются в регистре MIFR, структура которого аналогична представленной на рис.1.63. После завершения очередной задачи следующей активируется задача с наивысшим приоритетом из разрешенных к обслуживанию задач, чей запрос в регистре MIFR на данный момент является активным. При этом соответствующий бит в регистре MIFR автоматически сбрасывается в ноль. До момента активации задачи, ее запрос можно принудительно снять записью единицы в соответствующий бит регистра MICLR, структура которого аналогична представленной на рис.1.63. Если очередной запрос активизации определенной задачи поступит в момент, когда бит, соответствующий этой задаче, в регистре MIFR еще не сброшен, автоматически выставляется бит переполнения в регистре MIOVF, структура которого аналогична представленной на рис.1.63. Будучи установленным, бит в этом регистре может быть сброшен только программно записью единицы в соответствующий бит регистра MICLR0VF. Номер текущей активной задачи индицируется установкой в единицу соответствующего бита регистра MIRUN, структура которого аналогична представленной на рис.1.63.

## 5. Инициализация блока CLA.

Каждая из задач может быть расположена где угодно в пределах памяти программ CLA и занимать любой доступный объем памяти. Расположение первой команды каждой из задач фиксируется в соответствующем регистре вектора задачи MVECT1 – MVECT8. В каждый из этих регистров записывается 12-разрядное смещение первой команды соответствующей задачи относительно адреса начала памяти программ CLA в терминах общего адресного пространства микроконтроллера.

Каждая из задач завершается командой MSTOP.

После окончания выполнения текущей активной задачи выставляется соответствующий запрос на прерывания ЦПУ, сигнализирующий последнему о завершении данной задачи. В процедуре обработки этого прерывания ЦПУ должен быть сброшен флаг периферийного устройства, инициировавший активацию данной задачи CLA.

Инициализация CLA должна состоять из следующих этапов:

1. Программный код, реализующий задачи CLA, копируется из места хранения в область ОЗУ, соответствующую памяти программ CLA;
2. Если необходимо, в области ОЗУ, соответствующей памяти данных CLA, инициализируются переменные для задач CLA;
3. Включается синхронизация CLA в регистре PCLKCR3;
4. Стартовые адреса задач CLA записываются в регистры MVECT1 –MVECT8;
5. Для каждой задачи выбирается сигнал периферийного устройства, инициирующий ее запрос активации в регистре MPISRCSEL1. Если планируется только программная активация задачи, то в соответствующем битовом поле выбирается комбинация, не перечисленная в таб.3;
6. В случае необходимости, для разрешения программной активации задач CLA с помощью команды IACK, устанавливается в единицу бит IACHE в регистре MCTL;
7. Используемые блоки памяти данных CLA переназначаются для CLA установкой в единицу соответствующих битов RAMxE в регистре MMEMCFG. Для сохранения возможности доступа к этой памяти со стороны ЦПУ необходимо установить соответствующие биты RAMxCPUe в регистре MMEMCFG. После переназначения памяти для CLA перед доступом к ней необходимо подождать не менее двух периодов системного синхросигнала SYSCLOCKOUT;
8. Память программ CLA переназначается для CLA установкой в единицу бита PROGE в регистре MMEMCFG;
9. Инициализируется таблица векторов прерываний PIE записью адресов процедур обработки соответствующих прерываний;
10. Используемые задачи CLA разрешаются к обслуживанию установкой в единицу соответствующих битов в регистре MIER.

Сброс CLA в случае необходимости, возможно осуществить двумя способами. Полный сброс CLA (Hard Reset) осуществляется записью единицы в бит HARDRESET регистра MCTL. При полном сбросе все регистры и внутренние устройства CLA переходят в исходное состояние. После этого необходима полная реинициализация CLA. Частичный сброс CLA (Soft Reset) осуществляется записью единицы в бит SOFTRESET регистра MCTL. При этом останавливается текущая активная задача, сбрасывается ее флаг в регистре MIRUN, и очищаются все биты разрешения задач CLA в регистре MIER. Для повторного разрешения задач CLA после частичного сброса, перед записью нового значения в регистр MIER необходимо подождать не менее одного периода системного синхросигнала SYSCLOCKOUT.

Структура регистра MCTL представлена на рис.5.1.

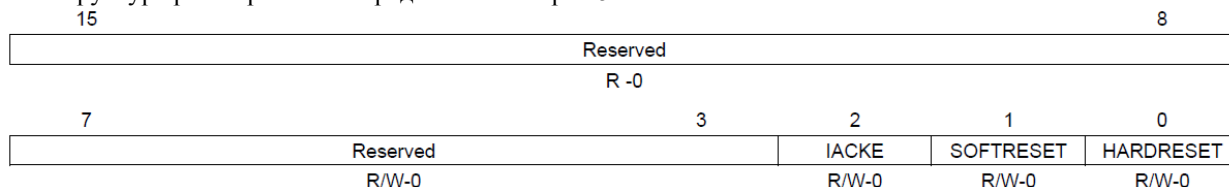


Рис.5.1

Значение конкретных битовых полей регистра MCTL поясняется в таб.5.1.

| Номер бита | Название поля | Значение | Описание                                 |
|------------|---------------|----------|------------------------------------------|
| 2          | IACHE         | 0<br>1   | Команда IACK<br>Запрещена;<br>Запрещена. |

|   |           |        |                                      |
|---|-----------|--------|--------------------------------------|
| 1 | SOFTRESET | 0<br>1 | Частичный сброс CLA<br>Нет;<br>Есть. |
| 0 | HARDRESET | 0<br>1 | Полный сброс CLA<br>Нет;<br>Есть.    |

Таб.5.1

## 6. Конвейер блока CLA.

Для повышения вычислительной производительности CLA оснащен восьми-стадийным конвейером команд, который имеет следующие стадии выполнения команды:

- F1 (Fetch 1). Адрес считывания команды выставляется на шину адреса программ CLA;
- F2 (Fetch 2). Команда считывается с помощью шины данных программ;
- D1 (Decode 1). Считанная команда декодируется;
- D2 (Decode 2). Если в команде используются данные, то их адрес декодируется. Модифицируются значения адресных регистров при использовании косвенной постинкрементной адресации. Если команда является командой условного перехода, то условие перехода проверяется на этой стадии;
- R1 (Read 1). Адрес данных выставляется на шину адреса чтения памяти данных.
- R2 (Read 2). Данные считываются с помощью шины данных чтения памяти данных.
- EXE (Execute). Команда выполняется. Осуществляется прямая непосредственная загрузка адресных регистров.
- W (Write). Результат операции записывается по месту назначения с использованием шины адреса и шины данных записи памяти данных.

Все стадии конвейера выполняются последовательно. Каждая стадия конвейера выполняется в течение одного машинного цикла. Из-за последовательной реализации стадий выполнения команды возможны конфликтные ситуации при последовательной записи и считывании одной и той же ячейки памяти данных. Считывание данных осуществляется на стадиях R1 и R2, а запись – на стадии W. Если в первой команде осуществляется запись в ячейку памяти данных нового значения, а в команде, непосредственно следующей за ней – чтение из этой же ячейки, то будут прочитаны устаревшие данные. Конвейер команд ЦПУ обладает механизмом защиты, который автоматически останавливает его на три машинных цикла для предотвращения описанного конфликта данных. Конвейер команд CLA не обладает подобным механизмом защиты. Поэтому, во избежание подобных конфликтов между командами, осуществляющими последовательную запись и чтение одной и той же ячейки памяти в коде программы необходимо вставлять не менее трех команд, которые не обращаются к этой ячейке памяти данных.

Еще одно отличие конвейера CLA от конвейера ЦПУ заключается в том, что конвейер CLA не очищается при выполнении команд условного перехода, вызова или возврата из процедур. При выполнении команды условного перехода проверка условия перехода осуществляется на стадии D2. Поэтому последняя команда, которая может изменить условие перехода, должна располагаться как минимум за три команды до данной команды перехода. Помимо этого, независимо от того выполнен ли действительно условный переход, вызов или возврат из процедуры, обязательно выполнятся три команды, непосредственно следующие за данной командой. Это происходит потому, что на момент проверки условия перехода на стадии D2 эти три команды уже успели занять три предыдущие стадии конвейера.

Загрузка адресных регистров непосредственным значением или значением, считанным из памяти, осуществляется на стадии EXE. В то же время декодирование адреса данных выполняется на стадии D2. Поэтому, три команды, следующие за командой загрузки адресного регистра, не должны использовать содержимое этого регистра для обращения к памяти. В противном случае будет использовано старое содержимое этого регистра.

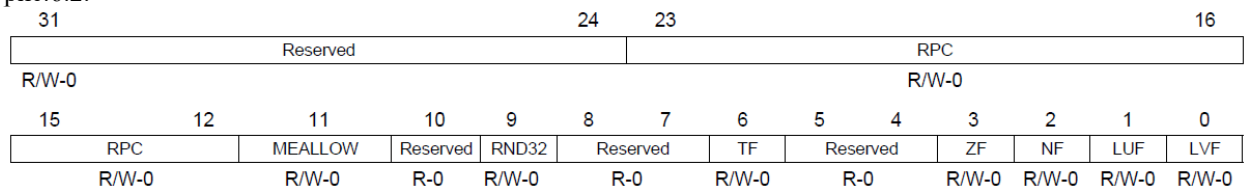
Конвейер CLA останавливается с помощью команды MDEBUGSTOP. После нее CLA переходит в отладочный режим, в котором возможен доступ к ресурсам CLA со стороны отладочного интерфейса микроконтроллера. В этом режиме возможна пошаговая отладка программы CLA. Команда MDEBUGSTOP и команда завершения задачи CLA MSTOP не должны быть размещены ближе, чем за три команды, или через три команды от любой команды условного перехода, вызова или возврата из процедуры. Для пошаговой отладки команд условного перехода, вызова или возврата из процедуры команда MDEBUGSTOP должна быть расположена как минимум за четыре команды до них.

Если блок АЦП микроконтроллера настроен на режим ранней выработки сигнала запроса прерывания, то задача CLA, инициируемая этим сигналом, может считывать результат преобразования АЦП не раньше, чем в восьмой команде задачи. Иначе, к моменту считывания регистра результата АЦП еще не успеет обновиться.

Если при считывании очередной команды из памяти программ считанный код оказывается некорректным, то конвейер останавливается на стадии D2 и выставляется запрос прерывания ЦПУ, соответствующий выполняемой в данный момент задаче. Бит в регистре MIRUN, соответствующий данной задаче, остается при этом установленным.

|          |    |     |   |
|----------|----|-----|---|
| 15       | 12 | 11  | 0 |
| Reserved |    | MPC |   |
| R-0      |    | R-0 |   |

Регистр MSTF содержит набор флагов текущего состояния CLA. Его структура представлена на рис.6.2.



Значение конкретных битовых полей регистра MSTF поясняется в таб.6.1.

| Номер бита | Название поля | Значение | Описание                                                                                                                                                                                                              |
|------------|---------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 23-12      | RPC           |          | Адрес возврата из процедуры. Используется в командах вызова и возврата из процедуры.                                                                                                                                  |
| 11         | MEALLOW       | 0<br>1   | Флаг защиты регистров периферийных устройств от записи новых данных со стороны CLA:<br>Установлен;<br>Не установлен.                                                                                                  |
| 9          | RND32         | 0<br>1   | Способ округления результатов арифметических операций CLA:<br>Усечение;<br>Округление.                                                                                                                                |
| 6          | TF            | 0<br>1   | Тестовый флаг. Модифицируется при тестировании различных признаков результатов вычислительных операций CLA. Используется в командах условного перехода и вызова процедуры. Тестируемое условие:<br>Ложно;<br>Истинно. |
| 3          | ZF            | 0<br>1   | Флаг нуля. Результат вычислительной операции:<br>Не равен нулю;<br>Равен нулю.                                                                                                                                        |
| 2          | NF            | 0<br>1   | Флаг знака. Результат вычислительной операции:<br>Не отрицательный;<br>Отрицательный.                                                                                                                                 |
| 1          | LUF           | 0<br>1   | Флаг обратного переполнения результата вычислительной операции. Будучи однажды установленным, сбрасывается только программно.<br>Нет обратного переполнения;<br>Есть обратное переполнение.                           |
| 0          | LVF           | 0<br>1   | Флаг переполнения результата вычислительной операции. Будучи однажды установленным, сбрасывается только программно.<br>Нет переполнения;<br>Есть переполнение.                                                        |

7. Арбитраж блока CLA и центрального процессорного устройства.

При одновременном доступе к ресурсам CLA и регистрам периферийных устройств со стороны CLA и ЦПУ для предотвращения конфликтов данных реализуется арбитраж запросов данных. Он основан на

анализе приоритетов различных запросов. В зависимости от того, к какому ресурсу осуществляется обращение, приоритеты различных запросов расставляются по разному.

При обращении к CLA to CPU Message RAM приоритеты расставляются следующим образом:

1. Запись со стороны CLA (наивысший приоритет).
2. Запись в отладочном режиме со стороны ЦПУ.
3. Чтение со стороны ЦПУ.
4. Чтение со стороны CLA (низший приоритет).

При обращении к CPU to CLA Message RAM приоритеты расставляются следующим образом:

1. Чтение со стороны CLA (наивысший приоритет).
2. Запись со стороны ЦПУ.
3. Чтение со стороны ЦПУ (низший приоритет).

При обращении к памяти программ CLA приоритеты расставляются следующим образом:

1. Чтение команды со стороны CLA (наивысший приоритет).
2. Запись в отладочном режиме со стороны ЦПУ.
3. Чтение в отладочном режиме со стороны ЦПУ. (низший приоритет).

При обращении к памяти данных CLA приоритеты расставляются следующим образом:

1. Запись со стороны CLA (наивысший приоритет).
2. Запись со стороны ЦПУ.
3. Чтение со стороны ЦПУ.
4. Чтение со стороны CLA (низший приоритет).

При одновременном обращении к регистрам периферийных устройств со стороны CLA и ЦПУ запросы CLA будут иметь высший приоритет по сравнению с запросами со стороны ЦПУ.

При одновременном поступлении нескольких запросов в первую очередь будет выполняться запрос с наивысшим приоритетом.



## Лекция № 13

# “Микроконтроллеры с ядром ARM Cortex-M3/M4/M4F/M7/M7F - как представитель 32-разрядных RISC микроконтроллеров общего применения.

## 1. Введение

Расширение рынка портативных автономных цифровых устройств побудило появление и интенсивное развитие микропроцессорной архитектуры, называемой RISC (Reduced Instruction Set Computer). Основными принципами организации RISC-микропроцессоров являются:

- одинаковая длина кода для большинства команд;
- реализация минимально-достаточного набора методов адресации путем разделения команд для обработки данных и обращения к памяти. Обращение к памяти идёт только через команды load и store, а все прочие команды ограничены внутренними регистрами;
- большое количество регистров общего назначения;
- каждая команда выполняется за один машинный цикл;
- трехадресный формат команд;
- вычислительные команды только типа «регистр-регистр».

Это упростило организацию процессоров: позволило командам иметь фиксированную длину, упростило конвейеры и изолировало логику, имеющую дело с задержками при доступе к памяти, только в двух командах. В результате подобные микропроцессоры стали потреблять гораздо меньше энергии в пересчете на одну вычислительную операцию.

Одним из лидеров на рынке RISC-микропроцессоров является фирма ARM (Advanced RISC Machines). Она разработала целый ряд микропроцессорных ядер, предназначенных для использования как в простейших, так и в сложных высокопроизводительных системах.

Семейство ARM-Cortex-M представляет 32-разрядные микропроцессоры с тактовой частотой до 600 МГц, ориентированные на построение цифровых систем управления, работающих в режиме реального времени. Микроконтроллеры с такими ядрами выпускаются ведущими мировыми производителями, такими, как ST Microelectronics, Texas Instruments, Analog Devices. Выпускаются подобные микроконтроллеры и отечественными компаниями, такими, как Миландр и НИИЭТ.

Особенности отдельных типов процессорных ядер ARM-Cortex-M представлены в таб.1.

| Ядро | Характеристики                                                                                                                                                                                                           |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| M3   | 32-разрядный RISC процессор с 3-ступенчатым конвейером. Расширенная система команд Thumb-2, объединяющая 16-разрядные и 32-разрядные команды. Аппаратная реализация умножения и деления.                                 |
| M4   | Добавлены команды цифровой обработки сигналов: умножение с накоплением 32-разрядных вещественных чисел с фиксированной точкой, насыщение.                                                                                |
| M4F  | Добавлены аппаратный сопроцессор вычислений с плавающей точкой однократной точности (32 разряда) и соответствующие команды процессора.                                                                                   |
| M7   | Добавлены двойной суперскалярный 6-ступенчатый конвейер с динамическим предсказанием переходов, конвейер для операций умножения, кэш-память программ и данных, память прямого подключения (TCM – Tightly Coupled Memory) |
| M7F  | Добавлены аппаратный сопроцессор вычислений с плавающей точкой однократной и двойной точности.                                                                                                                           |

Таб.1

## 2. Обобщенная структура процессоров ARM-Cortex-M3/M4/M4F/M7/M7F.

Все рассматриваемые процессорные ядра имеют единую архитектуру ARMv7-M. Производительность процессорных ядер ARM\_Cortex-M3/M4/M4F равна 1.25 DMIPS/МГц (миллионов инструкций в секунду на стандартном тесте Dhrystone). Производительность процессорных ядер ARM\_Cortex-M7/M7F равна 2.14 DMIPS/МГц. Они построены по Гарвардской архитектуре и имеют систему внутренних интерфейсов для одновременной реализации операций с данными и с кодом программы.

У ядер ARM\_Cortex-M3/M4/M4F все шины адреса и данных имеют разрядность 32 бита. Это дает возможность адресовать до 4 Гбайт информации. К числу внутренних интерфейсов ядра относятся следующие:

- ICode Bus. Предназначен для чтения кодов команд из внутреннего ППЗУ. Соответствует протоколу AHB Lite;
- DCode Bus. Предназначен для считывания константных данных из внутреннего ППЗУ. Соответствует протоколу AHB Lite;
- System Bus. Предназначен для чтения и записи информации из внутреннего ОЗУ, встроенных периферийных устройств и внешней памяти. Одновременный доступ к информации, хранящейся в указанных регионах невозможен. Соответствует протоколу AHB Lite. Для связи с периферийными устройствами используются мосты AHB Lite - APB;
- Private Peripheral Bus (PPB). Предназначен для обмена информацией с собственными периферийными устройствами процессора, такими, как контроллер прерываний NVIC, системный таймер SysTick, блок защиты памяти MPU, система отладки и пр. Соответствует протоколу APB.

В областях внутреннего и внешнего ОЗУ могут располагаться как элементы обрабатываемых данных, так и исполняемые коды программы. Однако, при считывании кодов команд из ОЗУ невозможны одновременные манипуляции с данными, поскольку и для одного, и для другого используется одна и та же шина. Аналогично, через шину System адресуются регистры внутренних и внешних периферийных устройств.

Протоколы AHB (Advanced High Performance Bus), AHB Lite, APB (Advanced Peripheral Bus) являются собственными разработками фирмы ARM, соответствующими стандарту AMBA (Advanced Microcontroller Bus Architecture).

Структура внутренних шин процессора STM32 фирмы ST Microelectronics представлена на рис.1.

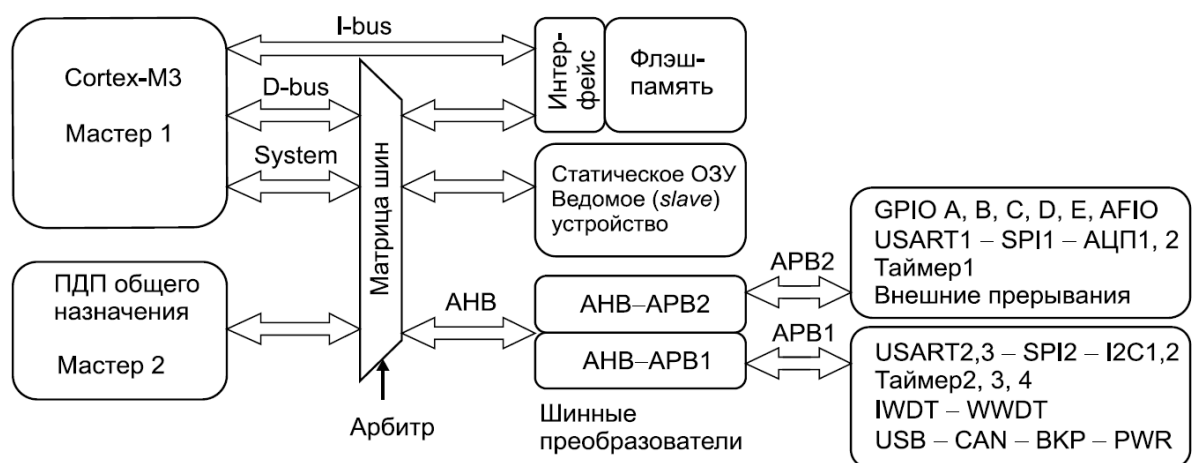


Рис.1

Структура внутренних шин процессора 1986BEXX фирмы Миландр представлена на рис.2.

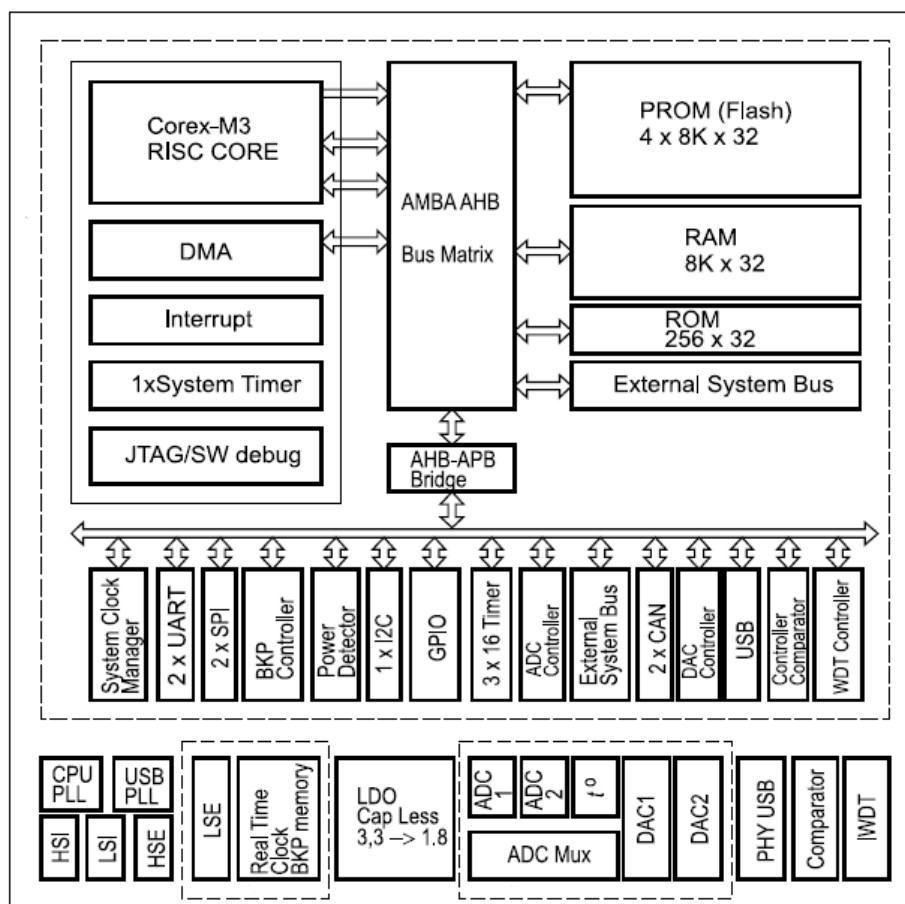


Рис.2

Структура внутренних шин процессора K1921BK01 фирмы НИИЭТ представлена на рис.3.

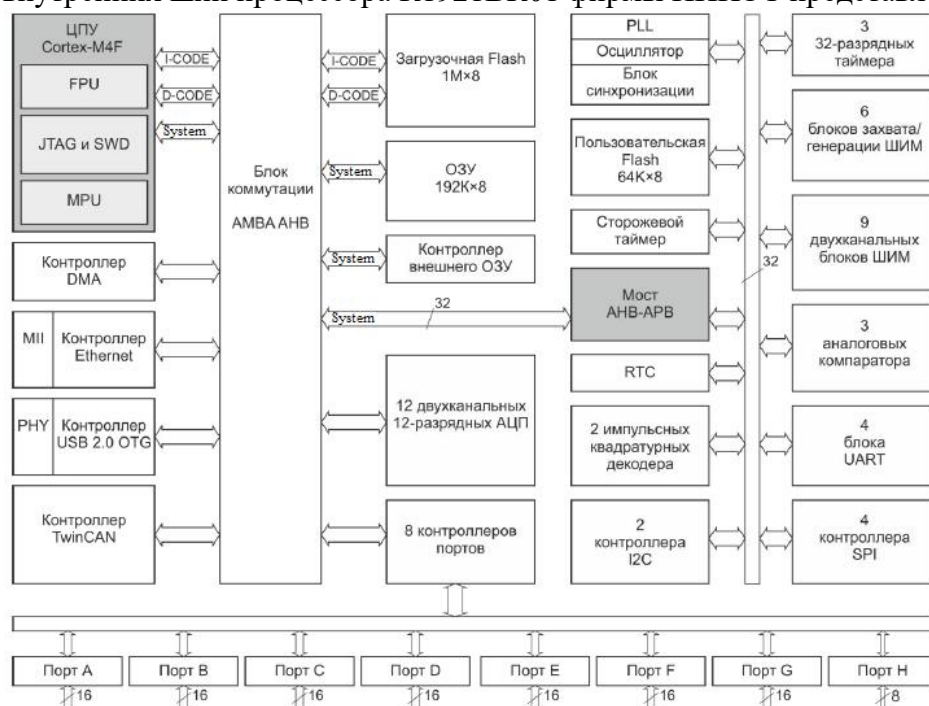


Рис.3

Как видно из приведённых рисунков, различные периферийные устройства микроконтроллеров обмениваются информацией с процессорным ядром посредством нескольких шин APB через специальные мосты AHB-APB. Частота синхронизации каждой шины APB задаётся отдельно специальными делителями частоты для достижения оптимальных характеристик подключённых к данным шинам периферийных устройств.

Ядра ARM Cortex-M7/M7F в качестве базовой шины используют высокопроизводительную 64-разрядную шину AXI (Advanced Extensible Interface), которая через матрицу шин связана с блоком памяти и периферийными узлами процессора. Для ускорения операций с командами и данными эта шина оснащена кэш-памятью команд и кэш-памятью данных объемом от 4 до 64 Кбайт. Кроме того, у этих ядер две дополнительные шины прямого подключения ICTM и DTSM, через которые непосредственно к ядру, минуя матрицу шин могут быть подключены дополнительные модули памяти программ и данных. Обмен с этими модулями будет производиться за кратчайшее время в силу отсутствия дополнительных задержек и арбитража, присущих матрице шин. В этих модулях памяти могут храниться критические по быстродействию участки кода и данные.

На рис.4 представлена структура внутренних шин контроллера семейства STM32F74XX фирмы ST Microelectronics, построенного на базе ядра ARM Cortex-M7F. Он имеет кэш-память программ и кэш-память данных по 4 Кбайта каждая. По шине ICTM к ядру подключён модуль быстрого ОЗУ программ объемом 16 Кбайт, а по шине DCTM - модуль быстрого ОЗУ данных объемом 64 Кбайта.

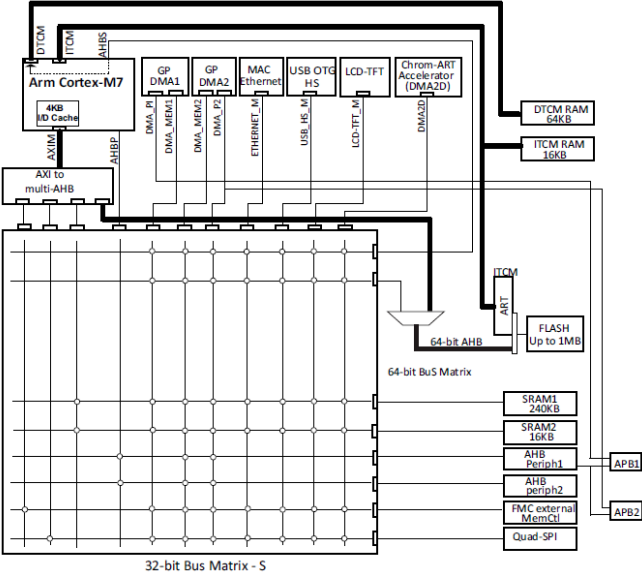


Рис.4

Системный сброс, инициализирующий либо всё устройство, либо определённую его часть может осуществляться при в следующих ситуациях:

- Включение питания (Power On Reset (POR)). Осуществляется с помощью внутреннего монитора питания микроконтроллера;
- Активный уровень на внешнем выводе системного сброса (External Reset);
- Программный сброс (Software Reset). Осуществляется установкой определённых битов в регистре AIRCR;
- Срабатывание сторожевого таймера (WatchDog Reset).

В процессорных ядрах ARM-Cortex-M3/M4/M4F/M7/M7F поддерживается несколько видов сбросов. В зависимости от конкретного вида сброса инициализируется либо всё устройств, либо его определённая часть. Различные виды сброса описаны в таб.2.

| Вид сброса                  | Сигнал сброса | Описание                                                                                                                                 |
|-----------------------------|---------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Сброс при включении питания | PORESETn      | Выполняется при включении питания. Сбрасываются процессорное ядро, все периферийные устройства и система отладки.                        |
| Сброс системы               | SYSRESETn     | Выполняется при установке флага SYSRESETREQ регистра AIRCR. Сбрасываются процессорное ядро, периферийные устройства без системы отладки. |

|                    |           |                                                                                                               |
|--------------------|-----------|---------------------------------------------------------------------------------------------------------------|
| Сброс процессора   | VECTRESET | Выполняется при установке флага VECTRESET регистра AIRCR. Сбрасываются процессорное ядро без системы отладки. |
| Сброс отладки JTAG | nTRST     | Выполняется при установке сигнала nTRST отладочного интерфейса JTAG. Сбрасывается система отладки.            |

Таб.2

### 3. Система памяти процессоров ARM-Cortex-M3/M4/M4F/M7/M7F.

Все процессоры ARM-Cortex-M3/M4/M4F/M7/M7F имеют унифицированную карту распределения адресов памяти. Она представлена в таб.3.

| Диапазон адресов             | Вид памяти                       | Объём     | Тип памяти       | Шина                         | Назначение                                                                                                                                      |
|------------------------------|----------------------------------|-----------|------------------|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| 0xffffffff<br>0xe0100000     | Определяется производителем      | 511 Мбайт | Device           | System                       | Определяется производителем                                                                                                                     |
| 0xe00fffff<br><br>0xe0000000 | Собственная периферия процессора | 1 Мбайт   | Strongly Ordered | PPB (Private Peripheral Bus) | Регистры периферийных устройств процессора: контроллер прерываний NVIC; системный таймер SysTick; блок защиты памяти MPU; система отладки и пр. |
| 0xdf000000<br>0xa0000000     | Внешняя периферия                | 1 Гбайт   | Device           | System                       | Регистры внешних периферийных устройств                                                                                                         |
| 0x9fffffff<br>0x60000000     | Внешнее ОЗУ                      | 1 Гбайт   | Normal           | System                       | Данные и программный код                                                                                                                        |
| 0x5fffffff<br>0x40000000     | Внутренняя периферия             | 0.5 Гбайт | Device           | System                       | Регистры внутренних периферийных устройств                                                                                                      |
| 0x3fffffff<br>0x20000000     | Внутреннее ОЗУ                   | 0.5 Гбайт | Normal           | System                       | Данные и программный код                                                                                                                        |
| 0x1fffffff<br>0x00000000     | Внутреннее ППЗУ                  | 0.5 Гбайт | Normal           | ICode DCode                  | Хранение кодов программы и констант                                                                                                             |

Таб.3

Микроконтроллеры разных фирм производителей и разных семейств могут иметь различные объёмы, меньшие указанных, при соблюдении указанных границ регионов.

Отдельные области памяти обладают различными свойствами по сохранению очередности доступа к информации, хранящейся в них. Эти особенности отражены в колонке «Тип памяти» таб.2. Указанные типы характеризуются следующими свойствами:

- **Normal.** Процессор имеет право изменить очередность доступа к памяти в этой области, выполнять упреждающее чтение и буферирование данных при записи (запись выполняется из буфера в то время, как процессор выполняет другую команду);
- **Device.** Процессор сохраняет предписанную очередность доступа к памяти в этой области, буферирование данных при записи возможно;
- **Strongly Ordered.** Процессор строго соблюдает предписанную очередность доступа к памяти в этой области, буферирование данных при записи невозможно.

Первый мегабайт адресного пространства области внутреннего ОЗУ (0x20000000 – 0x200fffff) и первый мегабайт адресного пространства области внутренних периферийных устройств (0x40000000 – 0x400fffff) являются побитно адресуемыми. Это означает, что можно программно обратиться к любому отдельному биту внутри любого байта указанного адресного пространства независимо от остальных семи бит этого байта. Это позволяет избежать реализации последовательности «чтение – модификация - запись» при необходимости изменения отдельного бита. Данный метод битовой адресации называется bit-banding. Для его реализации в адресном пространстве выделяется область (bit-band alias), объёмом в 32 раза больше области побитовой адресации (bit-band region). При обращении к младшему биту любого слова из bit-band alias на самом деле происходит обращение к одному из битов bit-band region. Соответствие битов из bit-band region и слов из bit-band alias для внутреннего ОЗУ представлено в таб.4, а для внутренних периферийных устройств – в таб.5.

| Биты bit-band region | Слова bit-band alias |
|----------------------|----------------------|
| 0x20000000 бит[0]    | 0x22000000 бит[0]    |
| 0x20000000 бит[1]    | 0x22000004 бит[0]    |
| 0x20000000 бит[2]    | 0x22000008 бит[0]    |
| ...                  | ...                  |
| 0x20000000 бит[31]   | 0x2200007c бит[0]    |
| ...                  | ...                  |
| 0x200ffffc бит[31]   | 0x23fffffc бит[0]    |

Таб.4

| Биты bit-band region | Слова bit-band alias |
|----------------------|----------------------|
| 0x40000000 бит[0]    | 0x42000000 бит[0]    |
| 0x40000000 бит[1]    | 0x42000004 бит[0]    |
| 0x40000000 бит[2]    | 0x42000008 бит[0]    |
| ...                  | ...                  |
| 0x40000000 бит[31]   | 0x4200007c бит[0]    |
| ...                  | ...                  |
| 0x400ffffc бит[31]   | 0x43fffffc бит[0]    |

Таб.5

Каждый отдельный адрес памяти соответствует одному байту информации. 32-битное слово располагается в памяти в четырёх последовательных ячейках. При этом по младшему адресу располагается младший байт. Это соответствует прямому порядку расположения данных (little endian).

#### **4. Вычислительное ядро процессоров ARM-Cortex-M3/M4/M4F/M7/M7F.**

Процессоры ARM-Cortex-M3/M4/M4F/M7/M7F могут находиться в одном из двух режимов:

- пользовательский режим (User Mode, Thread Mode);
- режим исключений (Handler Mode).

Эти режимы отличаются уровнем доступа к пространству управления системой (System Control Space (SCS) - диапазон адресов 0xe000e000 – 0xe000efff). В этой области расположены регистры управления контроллера прерываний NVIC, модуля защиты памяти MPU, системного таймера SYSTICK и компоненты системы отладки. При привилегированном уровне доступа обращение к этому пространству разрешено, а при пользовательском уровне – заблокировано. В пользовательском режиме процессор может иметь как привилегированный, так и пользовательский уровень доступа. В частности, после системного сброса процессор автоматически переходит в пользовательский режим с привилегированным уровнем доступа к SCS. В режиме исключений процессор всегда имеет привилегированный уровень доступа SCS. Он переходит в этот режим при возникновении исключений или прерываний. Конкретно уровнем доступа к SCS управляет бит 0 регистра CONTROL (nPRIV). Если программа работает на привилегированном уровне доступа, то она может переключиться на пользовательский уровень, записав в этот бит 1. Программа, работающая на пользовательском уровне, не может сама переключиться на привилегированный уровень. Она должна использовать для этого соответствующий обработчик исключения.

Операционный блок ядра ARM-Cortex-M3/M4/M4F включает в себя три независимых вычислительных устройства:

- арифметико-логическое устройство, реализующее операции сложения, вычитания, сдвига и логических функций с возможностью выполнять SIMD-инструкции (Single Instruction Multiple Data);
- аппаратный умножитель, реализующий умножение 32-разрядных чисел со знаком и без знака (в ядре M4 добавлена операция умножения с накоплением);
- аппаратный делитель, реализующий деление 32-разрядных чисел со знаком и без знака.

В операционный блок ядра ARM-Cortex-M4F добавлен сопроцессор для операций над числами с плавающей точкой одинарной точности.

Операционный блок ядра ARM-Cortex-M7/M7F включает в себя три независимых вычислительных устройства:

- два арифметико-логические устройства, одно из которых может выполнять SIMD-инструкции (Single Instruction Multiple Data);
- аппаратный конвейерный умножитель, реализующий операцию умножения с накоплением за один цикл;
- аппаратный делитель, реализующий деление 32-разрядных чисел со знаком и без знака.

В операционный блок ядра ARM-Cortex-M7F добавлен конвейерный сопроцессор для операций над числами с плавающей точкой одинарной и двойной точности.

Перечисленные вычислительные устройства выполняет операции над данными, хранящимися исключительно в 16 регистрах общего назначения, представленных в таб.6.

| Регистр | Альтернативное обозначение | Описание                                                                                                                                                                                                                                                  |
|---------|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| R0      | -                          | Младший банк регистров общего назначения. Могут быть использованы в любых командах в качестве операндов- источников или операндов приёмников.                                                                                                             |
| R1      | -                          |                                                                                                                                                                                                                                                           |
| R2      | -                          |                                                                                                                                                                                                                                                           |
| R3      | -                          |                                                                                                                                                                                                                                                           |
| R4      | -                          |                                                                                                                                                                                                                                                           |
| R5      | -                          |                                                                                                                                                                                                                                                           |
| R6      | -                          |                                                                                                                                                                                                                                                           |
| R7      | -                          |                                                                                                                                                                                                                                                           |
| R8      | -                          | Старший банк регистров общего назначения. Есть ограничения на использование в некоторых командах.                                                                                                                                                         |
| R9      | -                          |                                                                                                                                                                                                                                                           |
| R10     | -                          |                                                                                                                                                                                                                                                           |
| R11     | -                          |                                                                                                                                                                                                                                                           |
| R12     | -                          |                                                                                                                                                                                                                                                           |
| R13     | SP (Stack Pointer)         | Указатель стека. Содержит банк из двух регистров: <ul style="list-style-type: none"> <li>- MSP (Main Stack Pointer) – указатель стека процесса пользователя;</li> <li>- PSP (Process Stack Pointer) – указатель стека обработчиков исключений.</li> </ul> |
| R14     | LR (Link Register)         | Регистр связи. Содержит адрес возврата в вызывающую программу после вызова подпрограммы.                                                                                                                                                                  |
| R15     | PC (Program Counter)       | Программный счётчик. Всегда содержит адрес команд, следующей в памяти за выполняемой командой                                                                                                                                                             |

Таб.6

Разрядность всех регистров – 32 бита.

Регистры R0 – R12 могут быть использованы для хранения операндов или их адресов. Младший банк регистров R0 – R7 может быть использован во всех 32-разрядных и 16-разрядных командах. Старший банк регистров R8 – R12 может быть использован во всех 32-разрядных и в некоторых 16-разрядных командах. Состояние регистров обоих банков после сброса может быть любым.

Регистр R13 выполняет роль указателя стека (Stack Pointer (SP)). В данных процессорах реализуется модель «полного убывающего стека». Это означает, что указатель стека указывает на последнее число, записанное в стек, и декрементируется перед новой операцией записи в стек. Декремент осуществляется сразу на 4 поскольку запись в стек и извлечение из стека осуществляется всегда 32-битными словами. При снятии числа с вершины стека оно считывается из ячейки, на которую указывает регистр SP, после чего регистр инкрементируется на 4. В зависимости от режима работы процессора регистр R13 может отображать один из двух указателей стека в зависимости от значения бит 1 регистра CONTROL:

- MSP (Main Stack Pointer (SP\_main)) – основной указатель стека. Используется в режиме исключений всегда и в пользовательском режиме, если бит 1 регистра CONTROL равен 0;

- PSP (Process Stack Pointer (SP\_process)) – указатель стека процесса. Используется в пользовательском режиме, если бит 1 регистра CONTROL равен 1.

По умолчанию после окончания системного сброса используется основной указатель стека MSP. Его начальное содержимое считывается аппаратно после сброса из ячейки памяти с адресом 0x00000000. Это гарантирует нормальную конфигурацию стека и возможность программного обработки исключений сразу после сброса. В пользовательском режиме тоже можно использовать основной указатель стека MSP. Однако, при желании разделить стеки



пользовательского режима и режима исключений это можно сделать, записав 1 в 1 регистра CONTROL (SPSEL) при наличии привилегированного уровня доступа к SCS.

Регистр R14 используется как регистр связи (Link Register – LR). В него помещается адрес возврата при вызове невложенных процедур. Это сокращает время доступа к адресу возврата по сравнению с традиционным использованием для этой цели стека. При использовании вложенных процедур для сохранения адресов возврата, как и в большинстве процессоров, необходимо использовать стек для сохранения содержимого регистра LR.

Регистр R15 является программным счётчиком (Program Counter – PC). Его содержимое определяет адрес команды, которая должна быть выполнена после той команды, которая выполняется в данный момент. С учётом наличия конвейера команд этот адрес соответствует команде, которая уже считана в конвейер и прошла две первые его стадии – выборку и дешифрацию. При выполнении 32-разрядной команде содержимое PC инкрементируется на 4, а при выполнении 16-разрядной команды – на 2. При программной записи в этот регистр нового числа происходит безусловный переход по адресу, соответствующему записываемому числу. Записываемый адрес должен быть выровнен по границе 16-разрядного или 32-разрядного слова (кратен 2 или 4). Это означает, что младший бит адреса перехода всегда должен быть равен нулю. Однако, на самом деле, младший бит адреса перехода, записываемого в регистр PC, должен обязательно быть равен единице, поскольку он используется для индикации используемого набора команд Thumb-2. Если этот бит окажется равным нулю, то процессор попытается переключиться в состояние ARM, что вызовет генерацию исключения использования (Usage Fault). Несмотря на единичное значение младшего бита записываемого в PC адреса, переход будет выполнен по правильному адресу с автоматическим обнулением младшего бита.

Ядро ARM Cortex-M4F дополнительно к другим ядрам содержит аппаратный сопроцессор для выполнения операций над числами с плавающей точкой одинарной разрядности (32 бита). Вычислительные операции с двойной точностью реализуются только программно с помощью соответствующих процедур. Аппаратный сопроцессор ядра ARM Cortex-M7F может выполнять операции над числами с плавающей точкой одинарной и двойной разрядности (64 бита). Эти сопроцессоры имеют свой регистровый банк, состоящий из 32 регистров данных одинарной точности S0 – S31. Каждая смежная пара этих регистров S(2n) и S(2n+1) образует один регистр двойной точности D(n). Например, регистры S0 и S1 образуют регистр двойной точности D0 (S0 – младшая половина, S1 – старшая половина), регистры S2 и S3 – регистр двойной точности D1 и т.д. Таким образом, регистровый банк может содержать до 16 регистров двойной точности.

Помимо основного регистрового банка окружение процессора включает регистр статуса программы xPSR (Program Status Register). С точки зрения программного доступа он разделяется на три регистра:

- регистр статуса программы приложения APSR (Application Program Status Register);
- регистр статуса прерываний и исключений IPSR (Interrupt Program Status Register);
- регистр статуса выполнения программы EPSR (Execution Program Status Register).

Структура регистра статуса программы приложения представлена на рис.5.

|           |           |           |           |           |                |                |               |
|-----------|-----------|-----------|-----------|-----------|----------------|----------------|---------------|
| <b>31</b> | <b>30</b> | <b>29</b> | <b>28</b> | <b>27</b> | <b>26 - 20</b> | <b>19 - 16</b> | <b>15 - 0</b> |
| N         | Z         | C         | V         | Q         |                | GE[3:0]        |               |

Рис.5

Бит N (флаг знака) выставляется в 1 если результат операции оказался отрицательным и сбрасывается в 0 в противоположном случае.

Бит Z (флаг нуля) выставляется в 1 если результат операции оказался равным нулю и сбрасывается в 0 в противоположном случае.

Бит C (флаг переноса) выставляется в 1 если в результате операции сложения произошел перенос из старшего разряда результата, или если в результате операции вычитание не произошел заем в старший разряд результата и сбрасывается в 0 в противоположных случаях.

Бит V (флаг переполнения) выставляется в 1 если в результате операции сложения или вычитания со знаком результат не уместается в количество разрядов, соответствующее разрядности операции.

Бит Q (флаг насыщения) выставляется в 1 если в результате операции с насыщением произошло положительное или отрицательное насыщение результата и сбрасывается в 0 в противоположных случаях.

Биты GE[3:0] имеются только в ядрах ARM Cortex-M4/M4F/M7/M7F. Они выставляются в множественных инструкциях (SIMD – Single Instruction Multiple Data) для индикации свойств результатов в отдельных байтах или полусловах.

Структура регистра статуса прерываний и исключений IPSR представлена на рис.6.

|        |            |   |   |   |   |   |   |   |   |
|--------|------------|---|---|---|---|---|---|---|---|
| 31 - 9 | 8          | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|        | ISR Number |   |   |   |   |   |   |   |   |

Рис.6

В поле ISR\_Number отображается номер исключения, которое обрабатывается в данный момент.

Структура регистра статуса выполнения программы EPSR представлена на рис.7.

|         |        |    |    |         |        |    |    |    |    |    |       |
|---------|--------|----|----|---------|--------|----|----|----|----|----|-------|
| 31 - 27 | 26     | 25 | 24 | 23 - 16 | 15     | 14 | 13 | 12 | 11 | 10 | 9 - 0 |
|         | ICI/IT |    | T  |         | ICI/IT |    |    |    |    |    |       |

Рис.7

В случае возникновения исключения во время выполнения команды групповой загрузки или сохранения в поле ICI/IT запоминается номер следующего незагруженного (несохраненного) регистра для продолжения групповой операции с прерванного места после завершения обработки исключения. Таким образом, допускается прерывание «длинных» команд для сокращения времени реакции на исключение. Аналогично, если исключение возникнет внутри т.н. IT-блока команд условного выполнения, текущее состояние блока сохраняется в ICI/IT поле, что позволит корректно завершить блок после обработки исключения. Если в IT-блок входила команда групповой загрузки или сохранения, то она будет полностью прекращено, а после завершения обработки исключения запущена заново.

Флаг T является признаком командного состояния Thumb. Для описываемых процессоров он должен всегда быть установлен в 1. При попытке сброса этого флага в 0 будет сгенерировано исключения использования (Usage Fault).

Структура регистра управления CONTROL представлена на рис.8.

|        |            |            |
|--------|------------|------------|
| 31 - 2 | 1          | 0          |
|        | CONTROL[1] | CONTROL[0] |

Рис.8

Бит CONTROL[0] определяет уровень доступа к пространству управления системой (System Control Space (SCS) в пользовательском режиме. Если этот бит равен нулю, то уровень доступа привилегированный, а если единице, то пользовательский. В режиме исключений уровень доступа всегда привилегированный. Это бит доступен для записи только в режиме исключений. Для изменения этого бита в пользовательском режиме необходимо сгенерировать прерывание и изменить этот бит в обработчике этого прерывания.

Бит CONTROL[1] определяет используемый регистр указателя стека в пользовательском режиме. Если этот бит равен нулю, то используется MSP, а если единице, то PSP. В режиме исключений всегда используется MSP. Поэтому в режиме исключений этот бит всегда сброшен в 0. Это бит доступен для записи только в пользовательском режиме с привилегированным уровнем доступа к SCS. Изменить данный бит можно не только записью в регистр, но и изменив 2-й бит регистра LR при возврате из обработчика исключения.

## 5. Конвейер команд процессоров ARM-Cortex-M3/M4/M4F/M7/M7F.

Как и большинство современных процессоров рассматриваемые ядра осуществляют выполнение последовательности инструкций с помощью «конвейера команд». Это позволяет исполнять большинство инструкций за один машинный цикл. В ядрах ARM Cortex-M4/M4F реализован трёхстадийный конвейер, разделяющий весь процесс выполнения инструкции на стадии: выборка, дешифрация, выполнение.

На первой стадии команда (или команды) считывается из памяти и помещается в буфер конвейера команд. Если считываемая команда имеет длину 16 бит, то возможно считывание сразу двух команд по 32-разрядной шине ICode, т.е. предварительную выборку команды. Предвыбранная команда помещается в буфер, рассчитанный на четыре 32-битных слова. На второй стадии считанная команда дешифрируется и подготавливается к выполнению. В это время следующая команда считывается либо из памяти программ, либо из буфера команд. На третьей стадии осуществляется выполнение команды и запись результата. В это время следующая команда дешифрируется, а следующая за ней считывается. Работа конвейера команд иллюстрируется на рис.9.



Рис.9

У ядер ARM-Cortex-M7/M7F реализован суперскалярный конвейер, способный обрабатывать две команды одновременно (рис.10). Две ветки конвейера способны при благоприятных условиях одновременно загрузить два из набора следующих операционных устройств, входящих в блок обработки данных DPU (Data Processing Unit):

- два устройства загрузки – сохранения (LSU - Load Store Unit);
- два арифметико-логических устройства (ALU – Arithmetic Logic Unit);
- конвейерный умножитель с накоплением (MAC – Multiply Accumulate Unit);
- блок ветвления алгоритма;
- сопроцессор операций с плавающей точкой (FPU \_Float Point Unit) при наличии.

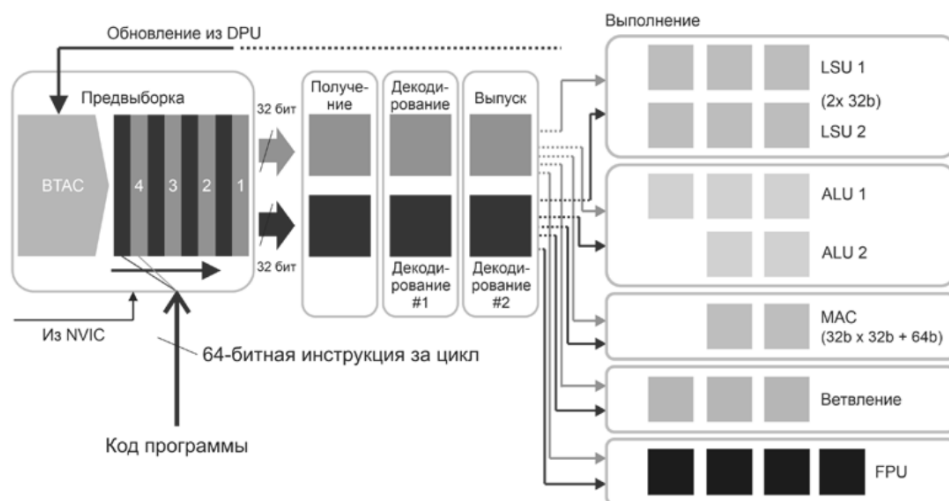


Рис.10

При выполнении команды перехода необходимо очистить и перезагрузить конвейер, что снижает его эффективность. Для сокращения описанных издержек конвейер реализует логику динамического предсказания переходов. Предвыбранные инструкции по целевому адресу перехода помещаются в специальный кэш-буфер ВТАС (Branch Target Address Cache). При выполнении команды косвенного перехода упреждающую выборку сделать невозможно и перезагрузка конвейера неизбежна.

Помимо этого, при необходимости условных переходов данные процессорные ядра поддерживают условно выполняемые команды в блоках IF-THEN-ELSE, которые не требуют перезагрузки конвейера. Можно выполнить подряд до четырёх таких команд предварительно сформировав актуальные значения флагов условий перехода.

## **6. Система прерываний процессоров ARM-Cortex-M3/M4/M4F/M7/M7F.**

Процессорные ядра ARM-Cortex-M3/M4/M4F поддерживают приоритетную обработку нескольких системных исключений и до 240 прерываний. Под исключением понимается нештатная ситуация, возникшая в процессе работы процессорного ядра. Обработкой сигналов запросов исключений и прерываний занимается контроллер вложенных векторных прерываний NVIC (Nested Vectored Interrupt Controller), интегрированный в процессорное ядро. На каждое прерывание или исключение процессор реагирует запуском соответствующей программы обработки прерывания (исключения). Начальные адреса этих программ должны быть записаны в таблицу векторов прерываний (исключений). После включения питания и системного сброса эта таблица должна быть расположена в памяти программ, начиная с нулевого адреса. После запуска программ размещение этой таблицы может быть изменено с помощью регистра VTOR, расположенного по адресу 0xE000ED08. Структура этого регистра представлена на рис.11.

|                |           |               |              |
|----------------|-----------|---------------|--------------|
| <b>31 - 30</b> | <b>29</b> | <b>28 - 7</b> | <b>6 - 0</b> |
|                | TBLBASE   | TBLOFF        |              |

Рис.11

Поле TBLBASE задает расположение таблицы векторов в памяти программ (0) или в памяти данных (1). В поле TBLOFF записывается смещение начального адреса таблицы относительно начала памяти программ или данных соответственно.

Структура таблицы векторов прерываний и исключений представлена в таб.7.

| Номер вектора | Смещение адреса | Приоритет            | Запрос        | Описание                      |
|---------------|-----------------|----------------------|---------------|-------------------------------|
| 18 - 255      | 0x48 - 0x3FC    | Программируемый (≥0) | IRQ2 – IRQ239 | Прерывание 2 - Прерывание 239 |
| -17           | 0x44            | Программируемый (≥0) | IRQ1          | Прерывание 1                  |
| 16            | 0x40            | Программируемый (≥0) | IRQ0          | Прерывание 0                  |
| 15            | 0x3C            | Программируемый (≥0) | SYSTICK       | Прерывание системного таймера |
| 14            | 0x38            | Программируемый (≥0) | PendSV        | Вызов системной службы        |
| 13            | 0x34            | Зарезервировано      |               |                               |
| 12            | 0x30            | Программируемый (≥0) | Debug Monitor | Вызов отладочного монитора    |
| 11            | 0x2C            | Программируемый (≥0) | SVCall        | Вызов супервизора             |
| 7 - 10        | 0x1C - 0x28     | Зарезервировано      |               |                               |

|   |      |                                 |                        |                                              |
|---|------|---------------------------------|------------------------|----------------------------------------------|
| 6 | 0x18 | Программируемый<br>( $\geq 0$ ) | Usage<br>Fault         | Исключение отказа<br>использования           |
| 5 | 0x14 | Программируемый<br>( $\geq 0$ ) | Bus Fault              | Исключение отказа<br>шины                    |
| 4 | 0x10 | Программируемый<br>( $\geq 0$ ) | Mem<br>Manage<br>Fault | Исключение отказа<br>модуля защиты<br>памяти |
| 3 | 0x0C | -1                              | Hard Fault             | Исключение<br>тяжёлого отказа                |
| 2 | 0x08 | -2                              | NMI                    | Немаскируемое<br>прерывание                  |
| 1 | 0x04 | -3 (наивысший)                  | Reset                  | Системный сброс                              |
| 0 | 0x00 |                                 | MSP                    | Начальное значение<br>указателя стека<br>MSP |

Таб.7

Первым элементом представленной таблицы является начальное значение основного указателя стека MSP. Сразу после сигнала системного сброса это значение аппаратно считывается из памяти и помещается в регистр указателя стека MSP. Это дает возможность сохранения на стеке контекста и адреса возврата если сразу после начала работы возникнет какое-либо исключение.

Следующий вектор в таблице - вектор системного сброса Reset. Здесь должен быть записан адрес процедуры начальной инициализации процессора. Она активизируется каждый раз после сигнала системного сброса.

Немаскируемое прерывание NMI вместе с системным сбросом являются единственными прерываниями, обработку которых невозможно запретить с помощью соответствующей маски. В различных версиях кристаллов с описываемым ядром ко входу NMI подключаются, как правило, сигналы прерываний критически важных узлов системы. Например, в микроконтроллерах STM32 к этому входу подключён аварийный сигнал блока контроля системного тактового генератора. У микроконтроллера вход NMI выведен на внешний контакт микросхемы.

Системные исключения предусмотрены для быстрой реакции микропроцессора на возникшие нештатные ситуации.

Отказ шины Bus Fault выставляется во время осуществления обмена данными по какой-либо из шин процессора при получении сигнала ошибки. Это может произойти:

- во время предварительной выборки команды из памяти программ (Prefetch Abort);
- во время обращения к данным (Data Abort);
- во время записи данных в стек в начале обработки прерывания (Stacking Error);
- во время чтения данных со стека в конце обработки прерывания (Unstacking Error).

Причиной сигнала ошибки может быть:

- попытка обращения по несуществующему адресу;
- неготовность целевого устройства к обмену данными;
- неподдерживаемая разрядность обмена данными;
- защита целевого адреса от записи на непривилегированном уровне.

Для разрешения обработчика исключения Bus Fault необходимо записать единицу во флаг BUSFAULTENA регистра SHCSR.

Причина конкретного отказа шины фиксируется в 8-битном регистре BFSR, расположенного по адресу 0xE000ED29. Его структура представлена на рис.12.

|           |     |        |          |             |           |         |
|-----------|-----|--------|----------|-------------|-----------|---------|
| 7         | 6-5 | 4      | 3        | 2           | 1         | 0       |
| BFARVALID |     | STKERR | UNSTKERR | IMPRECISERR | PRECISERR | IBUSERR |

Рис.12

Смысл представленных флагов поясняется в таб.8.

| Флаг        | Описание                           |
|-------------|------------------------------------|
| IBUSERR     | Ошибка доступа к команде           |
| PRECISERR   | «Точная» ошибка доступа к данным   |
| IMPRECISERR | «Неточная» ошибка доступа к данным |
| UNSTKERR    | Ошибка чтения со стека             |
| STKERR      | Ошибка записи в стек               |
| BFARVALID   | Корректность содержимого регистра  |

Таб.8

«Неточная» ошибка соответствует операции обмена данными, которая могла выполняться несколько тактов тому назад. «Точная» ошибка соответствует последней завершенной операции обмена данными. Нулевое значение флага BFARVALID подтверждает корректность текущего содержимого регистра.

Для сброса любого флага в регистре кроме BFARVALID необходимо записать в него единицу.

Отказ модуля защиты памяти Mem Menage Fault возникает при операции с памятью, не соответствующей текущим установкам модуля защиты памяти MPU (Memory Protection Unit). Примерами подобных ситуаций могут быть:

- обращение на пользовательском уровне к региону памяти, допускающему обращение только на привилегированном уровне;
- попытка записи в область памяти, допускающую только чтение;
- обращение к областям памяти, не заданным в текущей конфигурации MPU.

Для разрешения обработчика исключения Mem Menage Fault необходимо записать единицу во флаг MEMFAULTENA регистра SHCSR.

Причина конкретного отказа шины фиксируется в 8-битном регистре MMFSR, расположенного по адресу 0xE000ED28. Его структура представлена на рис.13.

|           |     |         |           |   |          |          |
|-----------|-----|---------|-----------|---|----------|----------|
| 7         | 6-5 | 4       | 3         | 2 | 1        | 0        |
| MMARVALID |     | MSTKERR | MUNSTKERR |   | DACCVIOL | IACCVIOL |

Рис.13

Смысл представленных флагов поясняется в таб.9.

| Флаг      | Описание                          |
|-----------|-----------------------------------|
| IACCVIOL  | Ошибка доступа к команде          |
| DACCVIOL  | Ошибка доступа к данным           |
| MUNSTKERR | Ошибка чтения со стека            |
| MSTKERR   | Ошибка записи в стек              |
| MMARVALID | Корректность содержимого регистра |

Таб.9

Для сброса любого флага в регистре кроме MMARVALID необходимо записать в него единицу.

Отказ использования Usage Fault выставляется при возникновении проблем выполнения команд. Причинами отказа использования могут являться:

- некорректный код считанной команды;
- попытка переключения из состояния THUMB в состояние ARM, которое не поддерживается в ядрах ARM-Cortex-M3/M4/M4F/M7/M7F;
- некорректное значение регистра связи LR при выходе из прерывания;

- обращение к не выровненным данным при реализации команд групповой пересылки данных;

- делении на ноль с фиксированной точкой;

Для разрешения обработчика исключения Usage Fault необходимо записать единицу во флаг USGFAULTENA регистра SHCSR.

Причина конкретного отказа шины фиксируется в 16-битном регистре UFSR, расположенного по адресу 0xE000ED2A. Его структура представлена на рис.14.

|         |           |           |       |      |       |          |            |
|---------|-----------|-----------|-------|------|-------|----------|------------|
| 15 - 10 | 9         | 8         | 7 - 4 | 3    | 2     | 1        | 0          |
|         | DIVBYZERO | UNALIGNED |       | NOCF | INVPC | INVSTATE | UNDEFINSTR |

Рис.14

Смысл представленных флагов поясняется в таб.10.

| Флаг       | Описание                                                  |
|------------|-----------------------------------------------------------|
| UNDEFINSTR | Некорректная команда                                      |
| INVSTATE   | Попытка переключения в состояние ARM                      |
| INVPC      | Некорректное значение LR при выходе из прерывания         |
| NOCF       | Попытка выполнить команду сопроцессора при его отсутствии |
| UNALIGNED  | Обращение к не выровненным данным                         |
| DIVBYZERO  | Деление на ноль                                           |

Таб.10

Для сброса любого флага в регистре необходимо записать в него единицу.

Случайная попытка перехода в состояние ARM может произойти при загрузке в счётчик команд PC числа с нулевым младшим битом.

Тяжёлый отказ Hard Fault возникает если при выставлении отказа использования, отказа модуля защиты памяти или отказа шины запуск программ обработки этих исключений невозможен. Это может произойти если обработка соответствующего исключения запрещена регистре SHCSR или если в данный момент уже обрабатывается исключение с большим или равным приоритетом. Помимо этого, тяжёлый отказ возникает в случае отказа шины при чтении вектора прерывания из таблицы векторов или сбое в системе отладки.

Причина конкретного тяжёлого отказа фиксируется в регистре HFSR, расположенного по адресу 0xE000ED2C. Его структура представлена на рис.15.

|          |        |        |        |   |
|----------|--------|--------|--------|---|
| 31       | 30     | 29 - 2 | 1      | 0 |
| DEBUGEVT | FORCED |        | VECTBL |   |

Рис.15

Смысл представленных флагов поясняется в таб.11.

| Флаг     | Описание                                                       |
|----------|----------------------------------------------------------------|
| VECTBL   | Ошибка чтения вектора прерывания                               |
| FORCED   | Отказ шины, отказ модуля защиты памяти или отказ использования |
| DEBUGEVT | Отказ вызван отладочным событием                               |

Таб.11

Выставление флага VECTBL может быть вызвано отказом шины Bus Fault во время чтения вектора прерывания или некорректной настройкой смещения таблицы векторов.

Выставление флага DEBUGEVT может быть вызвано некорректным выполнением команды программной точки останова BKPT.

Если во время выполнения обработчика исключения Hard Fault, обработчика NMI или системного сброса будет выставлен запрос другого системного исключения, то процессор перейдет в состояние блокировки (LockUp). При этом в счётчик команд PC будет

загружен адрес 0xFFFFFFFFX и процессор будет постоянно считывать команду, расположенную по этому фиксированному адресу. Если блокировка произошла во время выполнения обработчика исключения Hard Fault, то процессор может реагировать только на запрос немаскируемого прерывания NMI, поскольку его уровень приоритета (-2) выше уровня приоритета Hard Fault (-1). Но после завершения обработчика NMI процессор снова вернётся в режим блокировки. Из состояния блокировки процессор может быть стабильно выведен только с помощью системного сброса.

Исключения SVCcall и PendSV предназначены для вызова специальной программы «супервизора» или «системной службы». В этих программах могут быть реализованы различные сервисные функции по управлению системой. В случае исключения SVCcall оно вызывается командой SVC #num8, где num – 8-битный номер вызываемой функции. Этот номер может быть определён в процедуре обработки исключения. Для этого нужно прочитать со стека сохранённое значение счётчика команд, считать код команды по этому адресу и маскируя в нём лишние биты вычленив переданный параметр. Если на пользовательском уровне используется указатель стека PSP, то предварительно нужно определить в каком из стеков сохранён счётчик команд. Это можно сделать в процедуре обработки исключения на основании содержимого регистра связи LR.

Команду SVC нельзя использовать внутри самой процедуры обработки исключения SVCcall потому, что приоритет нового исключения будет равен приоритету выполняющегося исключения. При попытке это сделать будет выставлен запрос исключения Hard Fault. Исключение PendSV предназначено для выполнения функций, аналогичных исключению SVCcall, но, в отличие от последнего его выполнение может быть отложено без генерации исключения Hard Fault. Поэтому, оно может быть использовано в обработчиках более или равно приоритетных исключений. Вызывается оно не командой, а установкой в единицу бита PENDSVSET регистра ICSR.

Исключение Debug Monitor вызывается событиями системы отладки и предназначены для реализации различных отладочных функций. При возникновении отладочных событий, таких, как точки останова, процессор не останавливается полностью (состояние “Halt”), а переходит к обработке исключения Debug Monitor. Это обеспечивает продолжение функционирования других блоков контроллера (в том числе периферийных устройств) в этом состоянии.

Для разрешения обработчика исключения Debug Monitor необходимо записать единицу во флаг MON\_EN регистра DEMCR. Программная установка бита MON\_PEND приведёт к генерации запроса этого исключения.

Прерывания SYSTICK выставляются системным таймером, который входит в состав контроллера NVIC. Таймер программируется для формирования запросов прерывания через определённые интервалы времени в моменты своего антипереполнения. Для разрешения обработки прерываний системного таймера необходимо записать единицу во флаг TICKINT регистра SYST\_CSR.

Для разрешения или запрещения обработки аппаратных исключений, а также контроля их текущего состояния используется регистр SHCSR (0xE000ED24), структура которого представлена в таб.12.

| Номер бита | Обозначение    | Описание                              |
|------------|----------------|---------------------------------------|
| 18         | USGFAULTENA    | Разрешение обработки Usage Fault      |
| 17         | BUSFAULTENA    | Разрешение обработки Bus Fault        |
| 16         | MEMFAULTENA    | Разрешение обработки Mem Menage Fault |
| 15         | SVCALLPENDEd   | Исключение SVCcall отложено           |
| 14         | BUSFAULTPENDEd | Исключение Bus Fault отложено         |
| 13         | MEMFAULTPENDEd | Исключение Mem Menage Fault отложено  |
| 12         | USGFAULTPENDEd | Исключение Usage Fault отложено       |
| 11         | SYSTICKACT     | Исключение SYSTICK активно            |
| 10         | PENDSVACT      | Исключение PendSV активно             |



|   |             |                                     |
|---|-------------|-------------------------------------|
| 8 | MONITORACT  | Исключение Debug Monitor активно    |
| 7 | SVCALLACT   | Исключение SVCALL активно           |
| 3 | USGFAULTACT | Исключение Usage Fault активно      |
| 1 | BUSFAULTACT | Исключение Bus Fault активно        |
| 0 | MEMFAULTACT | Исключение Mem Manage Fault активно |

Таб.12

Программное управление отложенным состоянием исключений NMI, SYSTICK и контроль текущего состояния системы обработки исключений осуществляется с помощью регистра ICSR (0xE000ED04), структура которого представлена в таб.13.

| Номер бита | Обозначение   | Описание                                                  |
|------------|---------------|-----------------------------------------------------------|
| 31         | NMIPENDSET    | Исключение NMI отложено                                   |
| 28         | PENDSVSET     | Исключение PendSV отложено                                |
| 27         | PENDSVCLR     | Запись 1 очищает флаг отложенного исключения PendSV       |
| 26         | PENDSTSET     | Исключение SYSTICK отложено                               |
| 25         | PENDSTCLR     | Запись 1 очищает флаг отложенного исключения SYSTICK      |
| 23         | ISRPREEMPT    | Отложенное прерывание станет активным на следующем шаге   |
| 22         | ISRPENDING    | Имеется отложенное внешнее прерывание                     |
| 21 - 12    | VECTORPENDING | Номер отложенного исключения с наибольшим приоритетом     |
| 11         | RETTOBASE     | Устанавливается в 1 при выполнении обработчика исключения |
| 9 - 0      | VECTACTIVE    | Номер обрабатываемого в данный момент исключения          |

Таб.13

Остальные векторы прерываний IRQ0 – IRQ239 обслуживают прерывания различных периферийных устройств микроконтроллера. Конкретный набор этих периферийных устройств и распределение их сигналов запроса прерывания по конкретным векторам определяется фирмой – производителем и может отличаться для различных вариантов устройств. Для разрешения обработки конкретного прерывания необходимо записать единицу в соответствующий бит одного из регистров NVIC\_ISERx (где x – номер регистра). Регистр NVIC\_ISER0 (0xE000E100) предназначен для прерываний IRQ0 – IRQ31, регистр NVIC\_ISER1 (0xE000E104) - предназначен для прерываний IRQ32 – IRQ63 и т.д. Младшие биты в регистрах соответствуют меньшим номерам прерываний. Для запрещения обработки конкретного прерывания необходимо записать единицу в соответствующий бит одного из регистров NVIC\_ICERx (0xE000E180 + x\*4, где x – номер регистра) с аналогичной организацией. Такой отдельный механизм разрешения и запрещения прерываний позволяет управлять отдельным прерыванием независимо от разрешения других прерываний.

Если какой-либо из выставленных запросов прерывания не может быть обслужен немедленно поскольку его обработка запрещена или в данный момент обслуживается прерывание с равным или более высоким приоритетом, то выставляется соответствующий флаг отложенного прерывания в одном из регистров NVIC\_ISPRx (0xE000E200 + x\*4, где x – номер регистра). Любой такой флаг можно установить программно. Для этого в соответствующий бит нужно записать единицу. Для программного сброса флагов отложенного прерывания необходимо записать единицу в соответствующий бит одного из регистров NVIC\_ICPRx (0xE000E280 + x\*4, где x – номер регистра). Эти регистры имеют структуру, аналогичную регистрам разрешения и запрещения обработки прерываний.

Программное формирование прерываний может быть реализовано также с помощью регистра STIR (0xE000EF00). Его структура представлена в таб.14.

| Номер бита | Обозначение | Описание                       |
|------------|-------------|--------------------------------|
| 8 - 0      | INTID       | Номер генерируемого прерывания |

Таб.14

Запись в поле INTID числа приведет к установке флага отложенного прерывания с этим номером в одном из регистров NVIC\_ISPRx.

Как только процессор приступает к обработке конкретного прерывания оно начинает считаться активным. Если процесс его обработки будет прерван более высокоприоритетным запросом. Однако, первое прерывание продолжает считаться активным. Активный статус конкретного прерывания можно отслеживать по соответствующим флагам активности прерываний, собранным в регистрах активного состояния прерываний NVIC\_IABRx (0xE000E300 + x\*4, где x – номер регистра). Эти регистры имеют структуру, аналогичную регистрам разрешения и запрещения обработки прерываний. После завершения процедуры обработки прерывания соответствующий флаг активного состояния сбрасывается.

Для одновременного запрещения всех исключений, кроме NMI и Hard Fault нужно записать единицу в регистр PRIMASK. При этом текущий уровень приоритета меняется на нулевой (наивысший программируемый уровень), что блокирует все исключения с равным и более низким приоритетом. Для одновременного запрещения всех исключений, кроме NMI нужно записать единицу в регистр FAULTMASK. При этом текущий уровень приоритета меняется на -1, что приводит к блокировке даже исключения Hard Fault.

Блокировка обработки всех исключений с приоритетом ниже либо равным определенному может быть реализована с помощью 8-битного регистра BASEPRI. После записи в него числа больше нуля обработка всех исключений с таким же или меньшим приоритетом будет запрещена.

Запись в регистры PRIMASK, FAULTMASK и BASEPRI разрешена только на привилегированном уровне доступа.

Уровень приоритета любого исключения или прерывания кроме Reset, NMI и Hard Fault может быть задан программно. Он задается в регистре приоритета соответствующего исключения PRI\_X (где X – номер исключения) в диапазоне от 0 до 128. Это 8 – разрядные регистры, структура которых представлена на рис.16.

|           |     |  |     |   |              |     |   |
|-----------|-----|--|-----|---|--------------|-----|---|
| 7         | ... |  | N+1 | N | N-1          | ... | 0 |
| Приоритет |     |  |     |   | Субприоритет |     |   |

Рис.16

Все биты каждого регистра приоритета разделены на две группы. Старшие 0 -7 бит регистра задают приоритет данного исключения. Начавшаяся обработка исключений с меньшим приоритетом может быть прервана другим исключением с большим приоритетом. Чем меньше число, записанное в эту группу бит, тем больше приоритет данного исключения. Самый высший возможный приоритет соответствует нулю, записанному в эту группу бит. Оставшаяся часть младших бит этого регистра задаёт т.н. «субприоритет» данного исключения. Он влияет на определение порядка обслуживания нескольких исключений с равным уровнем приоритета. Исключение с более высоким субприоритетом (меньшим числом, записанным в эту группу бит) будет обслужено в первую очередь. При одинаковом уровне приоритета и субприоритета у нескольких исключений первым будет обслужено исключение с меньшим номером. Количество бит, выделяемое под задание субприоритета определяется полем PRIGROUP регистра AIRCR (0xE000SD0C), структура которого представлена в таб.15.

| Номер бита | Обозначение   | Описание                                                                             |
|------------|---------------|--------------------------------------------------------------------------------------|
| 31 - 16    | VECTKEY       | Ключ записи. При записи в регистр должен быть равен 0x05FA, иначе запись блокируется |
| 15         | ENDIANNESS    | Порядок расположения байтов данных: 0 – прямой; 1 - обратный                         |
| 10 - 8     | PRIGROUP      | Количество бит для субприоритета исключения                                          |
| 2          | SYSRESETREQ   | Запрос на системный сброс устройства                                                 |
| 1          | VECTCLRACTIVE | Очистка информации о состоянии исключений                                            |
| 0          | VECTRESET     | Запрос на сброс процессора за исключением внешней периферии и компонентов отладки    |

Таб.15

Распределение количества бит для задания приоритета и субприоритета исключения представлено в таб.16.

| Значение PRIGROUP | Поле приоритета | Поле субприоритета |
|-------------------|-----------------|--------------------|
| 0                 | 7 – 1 (7 бит)   | 0 (1 бит)          |
| 1                 | 7 – 2 (6 бит)   | 1 - 0 (2 бита)     |
| 2                 | 7 – 3 (5 бит)   | 2 - 0 (3 бита)     |
| 3                 | 7 – 4 (4 бит)   | 3 - 0 (4 бита)     |
| 4                 | 7 – 5 (3 бит)   | 4 - 0 (5 бит)      |
| 5                 | 7 – 6 (2 бит)   | 5 - 0 (6 бит)      |
| 6                 | 7 (1 бит)       | 6 - 0 (7 бит)      |
| 7                 | 0 бит           | 7 - 0 (8 бит)      |

Таб.16

После системного сброса в поле PRIGROUP записан ноль.

В начале обработки исключения процессор аппаратно выполняет:

- сохранение контекста на стеке;
- чтение вектора обрабатываемого прерывания;
- изменение используемого в данный момент стека на MSP;
- загрузку в регистр связи LR специального значения EXC\_RETURN;
- запись в счётчик команд считанного вектора прерывания.

Во время сохранения контекста на используемом в данный момент стеке сохраняется содержимое регистров xPSR, PC, LR, R12, R3, R2, R1, R0. В процессорном ядре ARM Cortex M4F, содержащем сопроцессор операций с плавающей точкой, возможно дополнительное сохранение контекста сопроцессора (FPSCR, S15 – S0). Для этого необходимо установить в единицу бит 2 регистра CONTROL (FPCA). В первую очередь на стеке сохраняются регистры xPSR и PC. Сразу после этого начинается выборка первых команд процедуры обработки прерывания параллельно с продолжением сохранения контекста.

На момент начала обработки исключения процессор мог находиться либо в пользовательском режиме, либо в режиме исключений. Соответственно, в этот момент активным мог быть либо указатель стека PSP, либо MSP. Контекст будет сохранён на том стеке, чей указатель оказался активным на момент начала сохранения. После окончания сохранения контекста активным в любом случае становится указатель MSP.

В регистр связи LR заносится код, EXC\_RETURN, который будет использоваться при возврате из процедуры обработки исключения для определения из какого режима процессора данная процедура была вызвана. Такой метод возврата позволяет обойтись в системе команд без специальной команды выхода из процедуры обработки исключения. Эта процедура может завершаться как обычная подпрограмма. Когда при выходе из процедуры обработки исключения восстанавливая контекст процессор загружает эти биты в PC, это даст понять ядру, что операция не является обычной, а означает завершение

обработки прерывания. В результате запускается последовательность возврата из исключения.

Структура кода EXC\_RETURN изображена на рис.17.

| 31 - 4     | 3              | 2             | 1 | 0                                |
|------------|----------------|---------------|---|----------------------------------|
| 0xFFFFFFFF | Режим возврата | Стек возврата | 0 | Состояние процессора (1 – Thumb) |

Рис.17

Возможные значения кода EXC\_RETURN представлены в таб.17.

| Значение EXC_RETURN | Описание                                                                            |
|---------------------|-------------------------------------------------------------------------------------|
| 0xFFFFFFFF1         | Возврат в режим исключений, без плавающей точки, активный указатель стека MSP       |
| 0xFFFFFFFF9         | Возврат в пользовательский режим, без плавающей точки, активный указатель стека MSP |
| 0xFFFFFFFFD         | Возврат в пользовательский режим, без плавающей точки, активный указатель стека PSP |
| 0xFFFFFFE1          | Возврат в режим исключений, с плавающей точкой, активный указатель стека MSP        |
| 0xFFFFFE9           | Возврат в пользовательский режим, с плавающей точкой, активный указатель стека MSP  |
| 0xFFFFFED           | Возврат в пользовательский режим, с плавающей точкой, активный указатель стека PSP  |

Таб.17

При выходе из процедуры обработки исключения с учётом значения EXC\_RETURN, записанного в регистр связи LR, производятся следующие действия:

- переключение активного указателя стека;
- обратное восстановление контекста, снятого с активного стека;
- сброс бита активности обработанного исключения;
- снятие со стека адреса возврата и передача ему управления с учётом режима, записанного в EXC\_RETURN.

Если при выходе из процедуры обработки исключения процессор обнаруживает, что как минимум одно исключение находится в состоянии отложенного, то вместо восстановления и нового сохранения контекста он сразу переходит к обработке отложенного исключения. Такой механизм называется «цепочечной» (tail chaining) обработкой исключений. Это позволяет сократить временные интервалы между последовательными процедурами обработки исключений. Более того, если во время сохранения контекста при входе в процедуру обработки какого-либо исключения процессор обнаружит, что поступил запрос другого исключения с более высоким приоритетом чем у первого, то после завершения сохранения контекста будет запущена процедура обработки второго исключения и только после её завершения процессор без восстановления контекста вернётся к обработчику первого исключения.

Стандартная задержка обработки прерывания (интервал времени от момента выставления запроса до начала выполнения первой команды его обработчика) у процессоров с описываемыми ядрами составляет 12 тактов процессора. Цепочечная обработка сокращает эту задержку до 6 тактов. Задержка может быть увеличена за счёт тактов ожидания используемой памяти.

Если запрос исключения приходит во время выполнения многотактовой команды (например, команды групповой загрузки ли сохранения), то она может быть

приостановлена с сохранением в поле ICI регистра EPSR номера следующего необслуженного регистра, и возобновлена после завершения обработки исключения. Если групповая команда является условно выполняемой, то она обрывается, а после завершения обработки исключения выполняется полностью заново.

Если на момент выставления запроса исключения происходит незавершённый обмен данными по одной из шин, то процессор будет ожидать его завершения. При возникновении аварийной ситуации в процессе этого обмена будет выставлен запрос исключения отказа шины Bus Fault и если его приоритет окажется выше приоритета предварительно выставленного запроса, то исключение Bus Fault будет обслужено первым.

## 7. Системный таймер SYSTICK процессоров ARM-Cortex-M3/M4/M4F/M7/M7F.

В процессорное ядро включён 24-разрядный таймер, который может генерировать запросы прерывания SYSTICK. Эти запросы могут использоваться для отслеживания различных временных интервалов и моментов. Таймер синхронизируется либо системным тактовым сигналом процессора, либо другим источником синхронизации, который выбирается разработчиком кристалла. Например, у контроллеров семейства STM32 часто этим другим источником синхронизации является системный тактовый сигнал процессора после деления его частоты на 8. Выбор одного из двух источников синхронизации определяется значением бита CLKSOURCE регистра SYST\_CSR. У некоторых производителей альтернативный источник синхронизации может отсутствовать. Об этом сообщает флаг NOREF регистра SYST\_CALIB.

Каждым импульсом сигнала синхронизации внутреннее значение таймера уменьшается на единицу. При достижении нулевого значения после следующего импульса синхронизации в таймер записывается предустановленное значение, хранящееся в регистре SYST\_RVR (0xE000E014). В этот же момент выставляется запрос прерывания SYSTICK если это разрешено битом TICKINT регистра SYST\_CSR. Запускается работа таймера установкой в единицу бита ENABLE регистра SYST\_CSR. Структура регистра SYST\_CSR представлена в таб.18.

| Номер бита | Обозначение | Описание                                                                                                                                                                 |
|------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 16         | COUNTFLAG   | Выставляется в 1 при достижении таймером нулевого значения. Сбрасывается в 0 при чтении регистра SYST_CSR или при программной записи нового значения в регистр SYST_CVR. |
| 2          | CLKSOURCE   | 0 – альтернативный тактовый сигнал;<br>1 – тактовый сигнал процессора.                                                                                                   |
| 1          | TICKINT     | 0 – прерывания SYSTICK запрещены;<br>1 – прерывания SYSTICK разрешены;                                                                                                   |
| 0          | ENABLE      | 0 – таймер выключен;<br>1 – таймер включён.                                                                                                                              |

Таб.18

Текущее значение таймера всегда можно прочитать из регистра SYST\_CVR (0xE000E018).

Многие производители кристаллов калибруют системный таймер для облегчения получения прерываний с интервалом 10 мс. Для этого необходимо записать в регистр перезагрузки SYST\_RVR значение, хранящееся в поле TENMS регистра SYST\_CALIB. Корректное значение этого калибровочного числа подтверждается нулевым значением флага SKEW регистра SYST\_CALIB. Структура этого регистра показана в таб.19.

| Номер бита | Обозначение | Описание                              |
|------------|-------------|---------------------------------------|
| 31         | NOREF       | 0 – альтернативный синхросигнал есть; |

|        |       |                                                                            |
|--------|-------|----------------------------------------------------------------------------|
|        |       | 1 – альтернативного синхросигнала нет;                                     |
| 30     | SKEW  | 0 – калибровочное значение верное;<br>1 – калибровочное значение неверное; |
| 23 - 0 | TENMS | Калибровочное значение для интервала 10 мс.                                |

Таб.19

При желании использовать альтернативный источник синхронизации или калибровочное значение необходимо вначале убедиться в их наличии и корректности по соответствующим флагам этого регистра.

## 8. Модуль защиты памяти процессоров ARM-Cortex-M3/M4/M4F/M7/M7F.

Процессорные ядра ARM-Cortex-M3/M4/M4F/M7/M7F могут опционально содержать модуль защиты памяти (Memory Protection Unit – MPU). Он позволяет защитить отдельные регионы памяти от несанкционированного доступа. Эта защита осуществляется с помощью:

- запрета доступа к данным в определённом регионе памяти;
- запрета доступа к данным в определённом регионе памяти в пользовательском режиме;
- запрета записи новых данных в определённый регион памяти;
- запрета считывания команд из определённого региона памяти.

Помимо этого, MPU позволяет назначать отдельным регионам памяти специальные атрибуты, влияющие на их разделяемость, буферизируемость и кэшируемость. Под разделяемостью понимается разрешение доступа к определённому региону памяти со стороны нескольких активных устройств (например, процессор и контроллер прямого доступа к памяти). Буферизируемость означает, что записываемые данные могут накапливаться в промежуточных буферах и записываться в память позже, уже после окончания выполнения соответствующей команды. Кэшируемость определяет, может ли информация, считываемая из памяти, помещаться в кэш и извлекаться по возможности именно оттуда. Процессорные ядра ARM-Cortex-M3/M4/M4F не содержат кэш-памяти, но для совместимости с более старшими моделями эта возможность предусмотрена. Ядра ARM-Cortex-M7/M7F имеют кэш-память программ и кэш-память данных объёмом от 4 до 64 Кбайт каждая.

Для реализации описанных функций может быть выделено до 8 регионов памяти, которые могут пересекаться между собой. При обращении к области памяти, которая не входит ни в один из выделенных регионов или доступ к которой запрещен параметрами её региона, будет вырабатываться запрос исключения MemManage Fault. Если область памяти принадлежит сразу нескольким выделенным регионам, то параметры доступа к ней определяются настройками региона с наибольшим номером.

Данный модуль является опциональным, поэтому перед его использованием необходимо убедиться в его наличии. Это определяется содержимым регистра MPU\_TYPE (0xE000ED90). Его структура представлена на рис.18.

|                |                |               |          |
|----------------|----------------|---------------|----------|
| <b>31 - 24</b> | <b>23 - 16</b> | <b>15 - 8</b> | <b>0</b> |
|                | IREGION        | DREGION       | SEPARATE |

Рис.18

Поле IREGION определяет число поддерживаемых регионов для кода. Оно всегда содержит 0. Поле DREGION определяет число поддерживаемых регионов для данных. Оно всегда содержит 8. Флаг SEPARATE определяет единый или отдельные регионы задаются для кода и данных. Для данного типа MPU этот бит всегда содержит 0 (единый регион).

Для включения MPU необходимо установить в единицу флаг ENABLE в регистре MPU\_CTRL (0xE000ED94). Структура этого регистра представлена на рис.19.

|               |            |          |          |
|---------------|------------|----------|----------|
| <b>31 - 3</b> | <b>2</b>   | <b>1</b> | <b>0</b> |
|               | PRIVDEFENA | HFNMIENA | ENABLE   |

Рис.19

Флаг HFNMIENA при единичном значении разрешает использование MPU в обработчиках немаскируемого прерывания NMI и исключения Hard Fault. При нулевом значении этого флага при выполнении указанных обработчиков MPU блокируется. Если флаг PRIVDEFENA сброшен, то при попытке любого доступа к области памяти, не входящей ни в один из заданных регионов при включённом MPU, будет вызывать запрос исключения MemManage Fault. Если этот флаг установлен, то области, не вошедшие ни в один из заданных регионов, объявляются «фоновой областью» и становятся доступными для привилегированного доступа, но попытки доступа к ним на пользовательском уровне будут блокироваться и вызывать запрос исключения MemManage Fault.

Для объявления контролируемого региона памяти необходимо задать его номер, базовый адрес, размер и свойства. Номер конфигурируемого региона задаётся в первую очередь с помощью регистра MPU\_RNR (0xE00ED98). Его структура показана на рис.20.

|               |              |
|---------------|--------------|
| <b>31 - 3</b> | <b>2 - 0</b> |
|               | REGION       |

Рис.20

Перед заданием свойств региона в поле REGION необходимо записать его номер. Базовый адрес региона задаётся в поле ADDR регистра MPU\_RBAR (0xE00ED9C), структура которого изображена на рис.21.

|               |                |          |              |
|---------------|----------------|----------|--------------|
| <b>31 - N</b> | <b>N-1 - 5</b> | <b>4</b> | <b>3 - 0</b> |
| ADDR          |                | VALID    | REGION       |

Рис.21

Поскольку минимальный размер региона – 32 байта, то поле ADDR может занимать максимум 27 бит. При увеличении размера региона ширина этого поля уменьшается. В поле REGION можно записать номер региона вместо его задания в регистре MPU\_RNR. Корректность номера региона подтверждает единичное значения флага VALID. При нулевом значении этого флага номер региона берётся и регистра MPU\_RNR.

Размер региона и его атрибуты задаются в регистре MPU\_RASR (0xE00EDA0). Его структура представлена в таб.20.

| Номер бита | Обозначение | Описание                                                                              |
|------------|-------------|---------------------------------------------------------------------------------------|
| 28         | XN          | Запрет чтения команд:<br>0 – чтение команд разрешена;<br>1 – чтение команд запрещена; |
| 26 - 24    | AP          | Права доступа                                                                         |
| 21 - 19    | TEX         | Расширение типа региона                                                               |
| 18         | S           | Разделяемость региона:<br>0 – неразделяемый;<br>1 - разделяемый.                      |
| 17         | C           | Кэшируемость региона:<br>0 – некешируемый;<br>1 – кэшируемый.                         |
| 16         | B           | Буферизируемость региона:<br>0 – небуферизируемый;<br>1 – буферизируемый.             |
| 15 - 8     | SRD         | Запрет субрегионов:<br>0 – субрегион разрешён;<br>1 – субрегион запрещён.             |
| 5 - 1      | SIZE        | Размер региона                                                                        |

|   |        |                                                                     |
|---|--------|---------------------------------------------------------------------|
| 0 | ENABLE | Разрешение региона:<br>0 – регион запрещён;<br>1 – регион разрешён. |
|---|--------|---------------------------------------------------------------------|

Таб.20

Поле SIZE определяет размер региона в соответствии с формулой  $(32 \text{ байта}) * 2^{(\text{SIZE}-3)}$ . Таким образом, можно задать размер региона от 32 байт до 4 Гбайт. Поле SRD позволяет разбить регион на 8 равных субрегионов с разрешением каждого из них. Если субрегион разрешён, то его свойства совпадают со свойствами всего региона. Если регион запрещён, но при этом входит в другой разрешённый регион, то его свойства определяются свойствами этого региона. Если запрещённый субрегион не входит ни в один из других разрешённых регионов, то обращение к нему будет заблокировано и вызовет запрос исключения MemMenage Fault. Использование субрегионов возможно только при размере региона более 128 байт.

Поле AP задаёт права доступа к информации, хранящейся в регионе в соответствии с таб.21.

| AP | Привилегированный доступ           | Пользовательский доступ            |
|----|------------------------------------|------------------------------------|
| 0  | Доступ запрещён                    | Доступ запрещён                    |
| 1  | Чтение и запись разрешены          | Доступ запрещён                    |
| 2  | Чтение и запись разрешены          | Чтение разрешено, запись запрещена |
| 3  | Чтение и запись разрешены          | Чтение и запись разрешены          |
| 4  | Непредсказуемо                     | Непредсказуемо                     |
| 5  | Чтение разрешено, запись запрещена | Доступ запрещён                    |
| 6  | Чтение разрешено, запись запрещена | Чтение разрешено, запись запрещена |
| 7  | Чтение разрешено, запись запрещена | Чтение разрешено, запись запрещена |

Таб.21

Атрибут XN задаёт возможность чтения команд из данного региона. Если он установлен, то выполнение команды из данного региона вызовет запрос исключения MemMenage Fault.

Атрибут B (Bufferable) указывает, могут ли записываемые в этот регион данные накапливаться в буферах процессора и физически записываться позже, уже после формального окончания выполнения команды.

Атрибут S (Shareable) определяет, является ли этот регион разделяемым между несколькими активными устройствами (например процессором и контроллером прямого доступа к памяти).

Атрибут C (Cacheable) задаёт, может ли информация, считываемая из памяти и записываемая в неё, помещаться в кэш и извлекаться по возможности именно оттуда. Ядра ARM-Cortex-M7/M7F имеют кэш-память программ и кэш-память данных объёмом от 4 до 64 Кбайт каждая. После системного сброса оба типа кэш-памяти являются отключёнными. Для их включения необходимо установить биты IC (кэш программ) или DC (кэш данных) регистра CCR (Configuration and Control Register). Процессорные ядра ARM-Cortex-M3/M4/M4F не содержат контроллера кэш-памяти, но для совместимости с более старшими моделями этот атрибут здесь задаётся и совместно с другими атрибутами и полем TEX определяет тип данного устройства памяти согласно таб.22.

| TEX | C | B | Описание                    | Тип памяти       | Разделяемость |
|-----|---|---|-----------------------------|------------------|---------------|
| 000 | 0 | 0 | Строго упорядоченная память | Strongly Ordered | Разделяемая   |
| 000 | 0 | 1 | Разделяемое устройство      | Device           | Разделяемая   |



|     |   |   |                                                                                 |                          |                          |
|-----|---|---|---------------------------------------------------------------------------------|--------------------------|--------------------------|
| 000 | 1 | 0 | Внешний и внутренний кэши сквозной записи, без размещения записываемых данных   | Normal                   | Определяется атрибутом S |
| 000 | 1 | 1 | Внешний и внутренний кэши обратной записи, без размещения записываемых данных   | Normal                   | Определяется атрибутом S |
| 001 | 0 | 0 | Внешний и внутренние кэши отключены                                             | Normal                   | Определяется атрибутом S |
| 001 | 1 | 0 | Определяется реализацией                                                        | Определяется реализацией | Определяется реализацией |
| 001 | 1 | 1 | Внутренний и внешний кэши с обратной записью, с размещением записываемых данных | Normal                   | Определяется атрибутом S |
| 010 | 0 | 0 | Неразделяемые устройства                                                        | Device                   | Неразделяемая            |
| 1BB | A | A | Кэшируемая память:<br>B – внешний кэш;<br>A – внутренний кэш.                   | Normal                   | Определяется атрибутом S |

Таб.22

В последней строке параметры A и B определяют политику внешней и внутренней кэш-памяти в соответствии с таб.23.

| A | B | Политика кэш-памяти                                             |
|---|---|-----------------------------------------------------------------|
| 0 | 0 | Кэш-память отключена                                            |
| 0 | 1 | Обратная запись с размещением записываемых данных при промахе.  |
| 1 | 0 | Сквозная запись без размещения записываемых данных при промахе. |
| 1 | 1 | Обратная запись без размещения записываемых данных при промахе. |

Таб.23

Политика кэш-памяти «со сквозной записью» (“Write Through”) означает, что при записи данных они записываются и в кэш-память, и в основную память. Политика кэш-памяти «с обратной записью» (“Write Back”) означает, что данные при записи попадают только в кэш-память. Реальная запись в основную память откладывается до тех пор, пока кэш не переполнится и не потребуется место для новых данных. Как правило, в кэш-памяти всегда используется политика «Read Allocate» — то есть по промаху (Cache Miss) на чтение данные забираются из основной памяти и размещаются в кэш-памяти. Аналогично, при промахе на запись данные могут размещаться в кэш-памяти («Write Allocate») или не размещаться («No Write Allocate»).

Для контроллеров с процессорным ядром ARM-Cortex-M3/M4/M4F/M4/M4F в большинстве случаев из всех перечисленных вариантов типов памяти можно ограничиться следующими, перечисленными в таб.24.

| Назначение                                     | Тип памяти       | TEX | C | B | S |
|------------------------------------------------|------------------|-----|---|---|---|
| Flash-ПЗУ                                      | Normal           | 000 | 1 | 0 | 0 |
| Внутреннее ОЗУ                                 | Normal           | 000 | 1 | 0 | 1 |
| Внешнее ОЗУ                                    | Normal           | 000 | 1 | 1 | 1 |
| Собственные периферийные устройства процессора | Strongly Ordered | 000 | 0 | 0 | 1 |
| Прочие периферийные устройства                 | Device           | 000 | 0 | 1 | 1 |

Таб.24

Как правило, при конфигурировании MPU для реализации программной системы необходимо определить следующие регионы памяти:

- регион привилегированного программного кода, работающего в режиме исключений и недоступного в пользовательском режиме, содержащий системные функции, обработчики исключений, включая начальную таблицу векторов исключений и прерываний;

- регион пользовательского программного кода, содержащий остальные программные модули;
- регион привилегированных элементов данных, включая основной стек;
- регион пользовательских данных, включая стек процесса (если он используется);
- регион привилегированных битовых данных (Bit-Banding);
- регион пользовательских битовых данных;
- регион привилегированной периферии, в том числе, если необходимо, регион битовых данных периферии;
- регион пользовательской периферии, в том числе, если необходимо, регион битовых данных периферии.

Если количество необходимых регионов превышает 8, то привилегированные регионы можно объявить неявно с помощью «фоновой области», установив флаг PRIVDEFENA в регистре MPU\_CTRL.

При выключенном MPU все области памяти становятся доступны как на привилегированном, так и на пользовательском уровне, за исключением области System Control Space, которая доступна только на привилегированном уровне. В этой области расположены регистры управления контроллера прерываний NVIC, модуля защиты памяти MPU, системного таймера SYSTICK и компоненты системы отладки. При попытке доступа к этой области на пользовательском уровне будет выставлен запрос исключения MemManage Fault.

## **9. Схемотехника DSP-микроконтроллеров с ядром ARM-Cortex-M3/M4/M4F/M7/M7F.**

Условное обозначение микроконтроллера семейства STM32F373RXXX фирмы ST Microelectronics представлена на рис.22. Эти микроконтроллеры имеют процессорное ядро ARM-Cortex-M4F и работают с тактовой частотой до 72 МГц. Они обладают внутренней FLASH-памятью объемом до 256 Кбайт, внутренним ОЗУ объемом 32 Кбайта, а также широким набором встроенных периферийных устройств, таких, как:

- 16-канальный 12-разрядный АЦП;
- три 16-разрядных сигма-дельта АЦП;
- три 12-разрядных ЦАП;
- два 32-разрядных таймера с функциями ШИМ, быстрый ввод, быстрый вывод;
- шесть 16-разрядных таймеров с функциями ШИМ, быстрый ввод, быстрый вывод;
- отладочный интерфейс JTAG и SWD;
- 52 порта ввода/вывода;
- три последовательных порта UART;
- контроллеры интерфейсов:
- CAN (протокол 2.0b) с двумя портами ввода-вывода;
- I2C с поддержкой частоты передачи данных более 1 МГц (два порта);
- SPI (три порта);
- USB 2.0 Device с физическим уровнем PHY.

Для питания цифровой и аналоговой частей микроконтроллера на выводы VDD1, VDD2, VDDA и VDDSD необходимо подать напряжение 3.3В. Для питания внутренних часов реального времени на вывод VBAT нужно подать напряжение 1.65 – 3.6В. Выводы VREF+ и VREFSD+ предназначены для подачи внешнего опорного напряжения встроенного аналого-цифрового преобразователя (АЦП), цифро-аналогового

преобразователя (ЦАП) и сигма-дельта АЦП. Возможна также работа данных устройств от внутренних источников опорного напряжения.

При включении питания и запуске синхронизации во избежание непредвиденных ситуаций микроконтроллер должен удерживаться в состоянии системного сброса. При каждом включении питания внутренний монитор питания микроконтроллера вырабатывает сигнал PORESETn, удерживающий устройство в состоянии системного сброса до тех пор, пока напряжение питания не стабилизируется на нормальном уровне. Этот сигнал выдаётся на вывод NRST микроконтроллера для сброса внешних элементов системы. Сигнал системного сброса SYSRESETn можно подать извне, установив нулевой сигнал на выводе NRST. В обоих случаях внутренний генератор сформирует импульс сброса длительностью не менее 20 мкс.

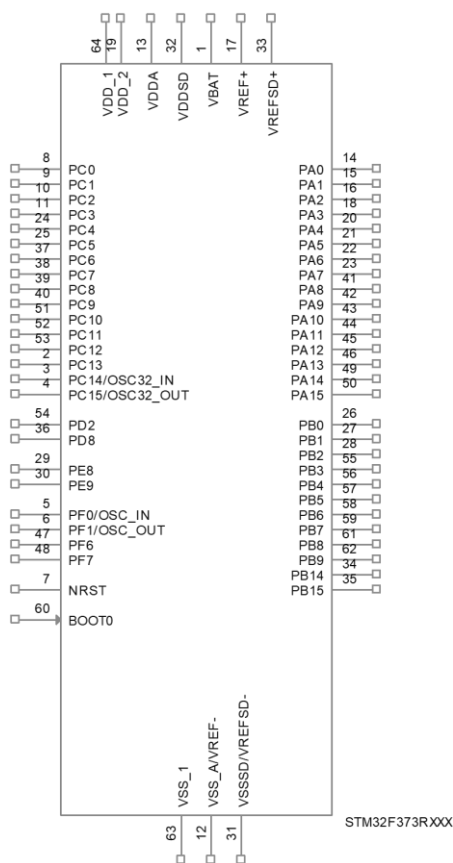


Рис.22

Синхронизация микроконтроллера может осуществляться от следующих источников:

- внутренний генератор 8 МГц (выбирается по умолчанию после сброса);
- внутренний генератор с внешним кварцевым резонатором 4 – 32 МГц;
- внешний тактовый генератор до 32 МГц.

Семы подключения внутреннего тактового генератора с внешним кварцевым резонатором представлена на рис.23.

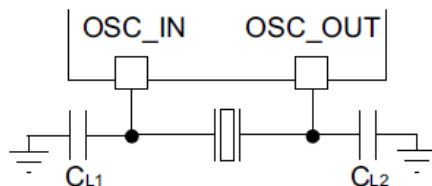


Рис.23

Семы подключения внешнего тактового генератора представлена на рис.24.

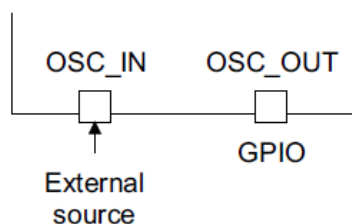


Рис.24

Внутренние часы реального времени синхронизируются с помощью отдельного резонатора по схеме, аналогичной рис.21, через выводы OSC32\_IN, OSC32\_OUT. Стандартная частота этого резонатора равна 32768 Гц.

Для доступа к внутренним ресурсам, записи программы во внутреннюю память и реализации режимов отладки микроконтроллер поддерживает стандартный отладочный интерфейс JTAG (Joint Test Action Group), а также упрощенный отладочный интерфейс SW (Serial Wire), в котором число сигналов уменьшено до двух. Схема подключения микроконтроллера к стандартному разъёму JTAG/SW показана на рис.25.

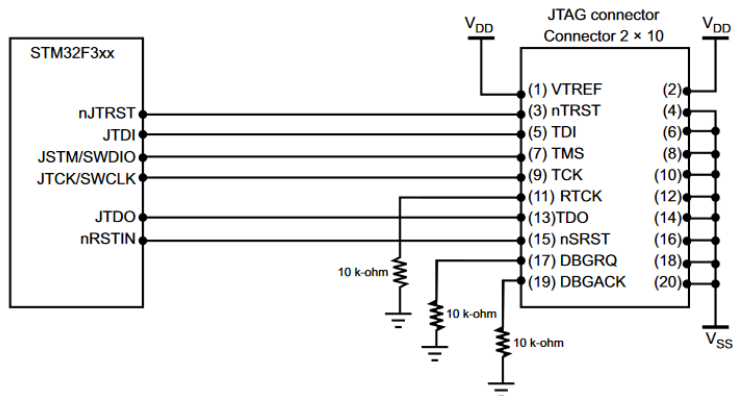


Рис.25

Состояние вывода BOOT0 и бита BOOT1 в регистре FLASH\_OBR (0x4002201C), зафиксированное по четвертому импульсу системного синхросигнала после окончания сигнала системного сброса, определяет режим старта программы в соответствии с таб.25.

| BOOT1 | BOOT0 | Режим старта                   |
|-------|-------|--------------------------------|
| x     | 0     | Основная FLASH память программ |
| 0     | 1     | Системная память программ      |
| 1     | 1     | ОЗУ                            |

Таб.25

Основной режим старта реализуется при нулевом уровне на выводе BOOT0. В этом случае процессор стартует, как обычно, с нулевого адреса. При единичном уровне на выводе BOOT0 могут быть реализованы альтернативные способы старта. В частности, при единичном уровне бита BOOT1 адресное пространство основной FLASH-памяти, начинающееся с нулевого адреса, переадресуется в область ОЗУ, начинающуюся с адреса 0x20000000. Этот способ старта используется при отладке программного обеспечения для сохранения ресурса FLASH-памяти. В этом случае отлаживаемая программа загружается и стартует из ОЗУ. При нулевом уровне бита BOOT1 после сброса запускается системная программа загрузки, записанная в области неперепрограммируемого ПЗУ, расположенной начиная с адреса 0x1FFFD800. Это позволяет осуществить загрузку программы во внутреннюю FLASH-память через имеющиеся в микроконтроллере интерфейсы UART или USB без отладочного оборудования.

На рис.26 изображено условное обозначение микроконтроллера K1921BK01T фирмы НИИЭТ.

Эти микроконтроллеры имеют процессорное ядро ARM-Cortex-M4F и работают с тактовой частотой до 125 МГц. Они обладают внутренней FLASH-памятью объёмом 1 Мбайт, пользовательской FLASH-памятью объёмом 64 КБайта (используется для хранения настроек пользователя), внутренним ОЗУ объёмом 192 КБайта, а также широким набором встроенных периферийных устройств, таких, как:

- двенадцать двухканальных 12-разрядных АЦП с режимами цифрового компаратора для каждого из каналов (равно или больше, равно или меньше, попадание в диапазон, выход из диапазона) и функцией автоматического запуска модулей ШИМ по событию «окончание преобразования»;
- три аналоговых компаратора с функцией автоматического запуска модулей ШИМ по событиям сравнения «равно или больше» и «равно или меньше»;
- девять модулей ШИМ, шесть из которых поддерживают режим высокого разрешения (возможность изменения длительности импульсов на величину менее периода тактового сигнала);
- два импульсных квадратурных декодера, используемых для обработки сигналов датчиков положения ротора в высокопроизводительных системах для определения положения, направления и скорости вращения;
- шесть модулей захвата/сравнения;
- три 32-разрядных таймера;
- отладочный интерфейс JTAG и SWD;
- семь 16-разрядных и один 8-разрядный порт ввода/вывода с отдельно программируемыми мультиплексированными выводами общего назначения;
- четыре последовательных порта UART;
- контроллеры интерфейсов:
- CAN (протокол 2.0b) с двумя портами ввода-вывода;
- I2C с поддержкой частоты передачи данных более 1 МГц (два порта);
- SPI (четыре порта);
- USB 2.0 Device/Host с физическим уровнем PHY;
- Ethernet 10/100 Мбит/с с интерфейсом MII.

Для питания цифровой и аналоговой частей ядра микроконтроллера на выводы VDD, TVDD\_ADC1, TVDD\_ADC2, TVDD\_PLL необходимо подать напряжение 1.8В. Для питания внутренних периферийных устройств на остальные выводы питания нужно подать напряжение 3.3В.

Внешний сигнал системного сброса подается на вход RST. Выход EXTRST предназначен для вывода внутреннего сигнала сброса контроллера наружу.

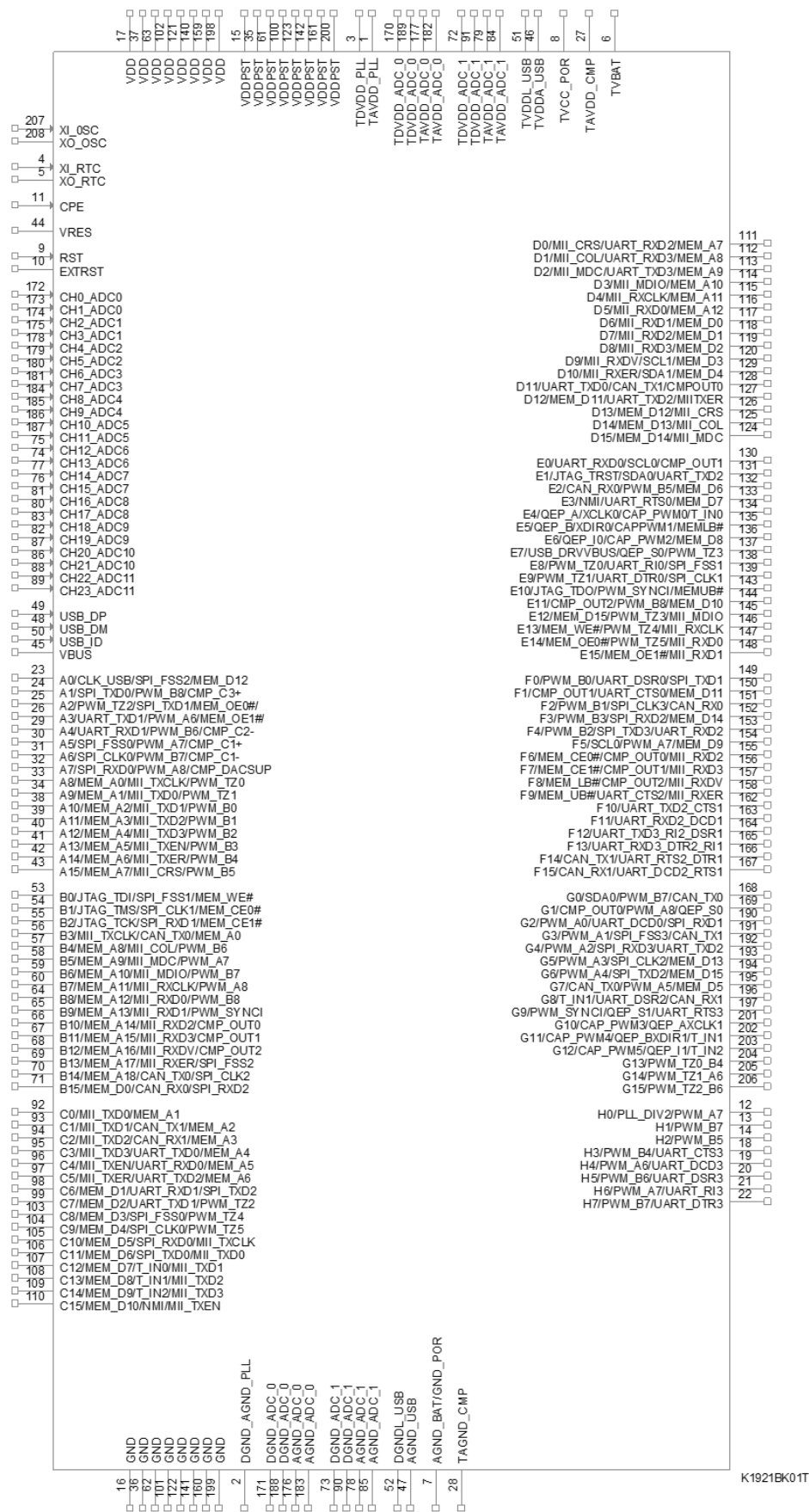


Рис.26

Синхронизация микроконтроллера может осуществляться от следующих источников:

- внутренний генератор 3.5 – 6.5 МГц (CPE = 0);
- внутренний генератор с внешним кварцевым резонатором 1 – 100 МГц (CPE = 1).

Выбор источника синхронизации осуществляется по уровню сигнала на выводе CPE.

Схема подключения внутреннего тактового генератора с внешним кварцевым резонатором аналогична представленной на рис.23 с использованием выводов XI\_OSC, XO\_OSC. Внутренние часы реального времени синхронизируются с помощью отдельного резонатора по схеме, аналогичной рис.23, через выводы XI\_RTC, XO\_RTC. Стандартная частота этого резонатора равна 32768 Гц.

Подключение отладочных средств осуществляется по схеме, аналогичной представленной на рис.25.

После сброса микроконтроллера автоматически производится считывание загрузочных настроек из информационного блока пользовательской FLASH-памяти. Далее, в зависимости от настроек производится загрузка из выбранного источника.

Микроконтроллер может загружаться из трех источников:

- основной области FLASH -памяти;
- особого защищенного блока основной FLASH -памяти;
- внешней памяти.

Защищенный блок FLASH - памяти объемом 8 КБайт, который в случае выбора соответствующего способа загрузки отображается на начальную область адресного пространства, может быть использован для хранения программы загрузчика через один из имеющихся интерфейсов.

Условное обозначение микроконтроллера семейства STM32F745ZX фирмы ST Microelectronics представлена на рис.27. Эти микроконтроллеры имеют процессорное ядро ARM-Cortex-M7F и работают с тактовой частотой до 216 МГц. Они обладают внутренней FLASH-памятью объемом до 1 Мбайт, внутренним ОЗУ объемом 320 КБайта (включая 64 Кбайта TCMD и 16 Кбайт TCM1 памяти), а также широким набором встроенных периферийных устройств, таких, как:

- три 12-разрядных АЦП суммарно на 24 канала;
- два 12-разрядных ЦАП;
- два 32-разрядных таймера с функциями ШИМ, быстрый ввод, быстрый вывод;
- до 18 16-разрядных таймеров с функциями ШИМ, быстрый ввод, быстрый вывод;
  
- отладочный интерфейс JTAG и SWD;
- 168 порта ввода/вывода;
- четыре последовательных порта UART;
- два интерфейса CAN (протокол 2.0b);
- до 4 интерфейсов I2C;
- до 6 интерфейсов SPI;
- интерфейс USB 2.0 Device с физическим уровнем PHY;
- Ethernet MAC контроллер.

Для питания цифровой и аналоговой частей микроконтроллера на выводы VDD1, VDD2, VDDA и VDDSD необходимо подать напряжение 3.3В. Для питания внутренних часов реального времени на вывод VBAT нужно подать напряжение 1.65 – 3.6В. Выводы VREF+ предназначены для подачи внешнего опорного напряжения встроенного аналого-цифрового преобразователя (АЦП), цифро-аналогового преобразователя (ЦАП). Возможна также работа данных устройств от внутренних источников опорного напряжения.

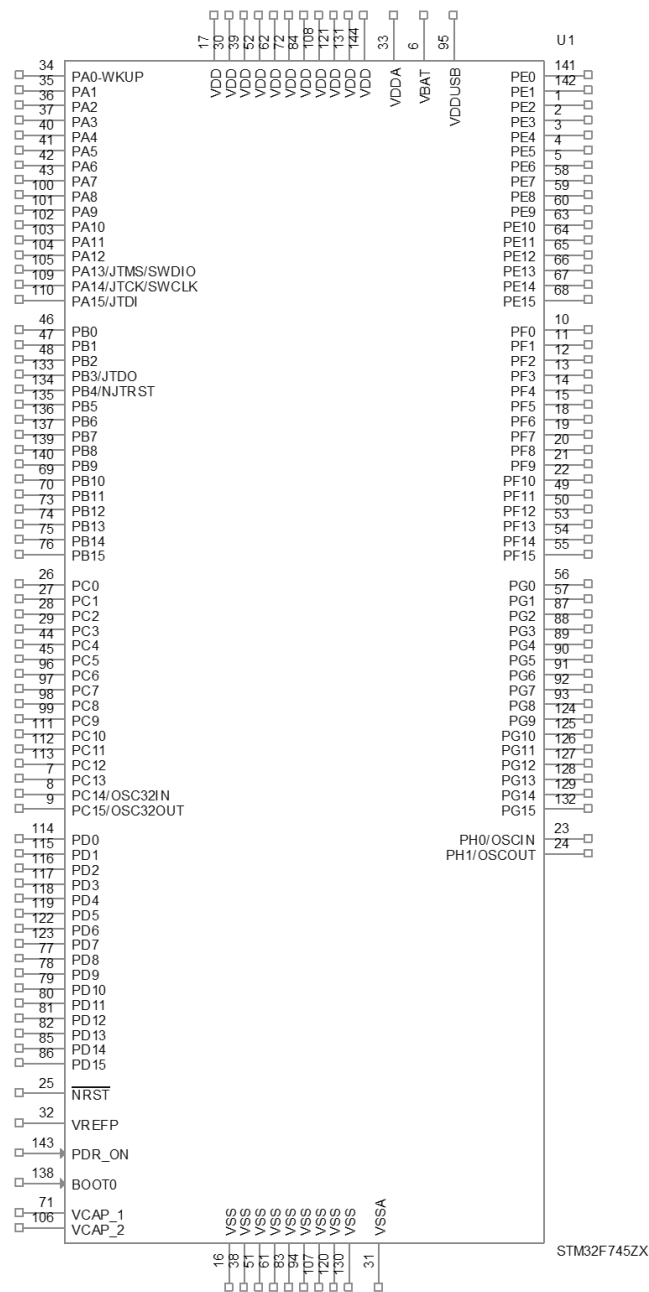


Рис.27

## 10. Система отладки процессоров ARM-Cortex-M3/M4/M4F M7/M7F.

Система отладки позволяет реализовывать следующие функции:

- пошаговое выполнение программы;
- аппаратные точки останова (до 8);
- программные точки останова;
- доступ к содержимому регистров и ячеек памяти даже во время работы процессора;
- аппаратные точки наблюдения (до 4);
- трассировка команд;
- трассировка данных.

Система отладки описываемых ядер построена по архитектуре CoreSight, позволяющей реализовывать все вышеуказанные функции. Связь отладочного модуля с процессорным ядром и системой памяти осуществляется через специальный интерфейс DAP (Debug Access Port). Однако, внешний порт отладки поддерживает стандартные отладочные протоколы JTAG и SWD. Связь между внешним портом отладки и



внутренним интерфейсом DAP осуществляется через соответствующие мосты. Протокол JTAG реализуется с помощью 4 сигналов:

- сигнал синхронизации JTCLK (JTAG Clock);
- сигнал выбора JTMS (JTAG Mode Select);
- последовательный вход данных JTDI (JTAG Data Input);
- последовательный выход данных JTDO (JTAG Data Output).

Опционально предусмотрен пятый сигнал JNTRST для сброса модуля отладки.

В протоколе SWD число сигналов сокращено до двух:

- сигнал синхронизации SWCLK (SWD Clock);
- двунаправленный сигнал данных SWDIO (SWD Data Input – Output).

Опционально предусмотрен третий сигнал TRACESWO для реализации асинхронной трассировки.

Выполнение отладочных функций возможно с помощью двух режимов.

Первый из них это режим останова процессора. В этом режиме останавливается выполнение команд процессором и системный таймер SYSTICK. Для возможности перехода в этот режим должен быть установлен флаг C\_DEBUGEN в регистре DHCSR контроллера NVIC. Это можно сделать только через интерфейс DAP, т.е. с помощью отладчика. Если этот флаг установлен, то для перехода режим останова необходимо установить флаг C\_HALT в этом регистре. Данный флаг может быть установлен либо аппаратно системой отладки, либо программно с помощью команды BKPT.

Второй режим отладки реализуется с помощью исключения монитора отладки. В этом режиме выполнение отлаживаемой программы переключается на выполнение процедуры обработки исключения Debug Monitor, которая выполняет требуемые отладочные действия. Системный таймер SYSTICK в этом режиме продолжает работать. Прерывания с более высоким приоритетом, чем у исключения Debug Monitor, будут продолжать обрабатываться. Это обеспечит продолжение управления внешним оборудованием, поскольку приоритет исключения Debug Monitor является программируемым.

Аппаратные точки останова реализуются с помощью специальных компараторов, которые адрес текущей исполняемой команды или содержимое ячейки памяти с заранее заданными значениями. При обнаружении совпадения процессор переводится в один из отладочных режимов.

Программные точки останова реализуются с помощью команды BKPT, которая аналогично предыдущему случаю переводится в один из отладочных режимов. Эта команда подменяет в памяти отслеживаемую команду и при попадании на неё выполняется останов. Если при этом был вызван обработчик исключения Debug Monitor, то при его завершении процессор возвращается по адресу отслеживаемой команды (которая к этому моменту должна вернуться на место), а не по следующему адресу чтобы данная команда могла выполняться. Если отслеживаемая команда расположена во Flash-памяти, то для неё возможна реализация только аппаратной точки останова.

Какой из двух отладочных режимов будет активирован при возникновении отслеживаемого события, зависит от того, разрешён ли режим останова и разрешена ли обработка исключения Debug Monitor.

С помощью аппаратных компараторов наблюдения можно реализовывать точки наблюдения и трассировку команд и данных.

## 11. Система команд процессоров ARM-Cortex-M3/M4/M4F M4/M4F.

### 11.1 Общие сведения о системе команд.

Изначально процессоры фирмы ARM имели 32-разрядную систему команд, которая так и называлась (ARM). Однако, в дальнейшем для увеличения плотности кода подмножество этой системы команд, выполняющее обработку 16-разрядных данных, было выделено в отдельную 16-разрядную систему команд Thumb. Процессоры поддерживали обе системы команд и могли динамически в процессе работы переключаться между состояниями ARM и Thumb. Это переключение осуществлялось записью нуля или единицы в младший бит программного счётчика PC. Обе системы команд модифицировались в процессе эволюции процессорных ядер. Наконец, с появлением архитектуры v7, к которой принадлежат описываемые ядра, появилась единая система команд Thumb-2, которая является расширением системы команд Thumb за счёт добавления 32-разрядных команд. Все описываемые ядра поддерживают только систему команд Thumb-2, поэтому необходимость в переключении между состояниями ARM и Thumb отпала. Все процессорные ядра начиная с версии v7 работают только в состоянии Thumb. Попытка перейти в состояние ARM записью нуля в младший бит программного счётчика что вызовет генерацию запроса исключения использования (Usage Fault). Несмотря на единичное значение младшего бита записываемого в PC адреса, переход будет выполнен по правильному адресу с автоматическим обнулением младшего бита. Новая система команд предопределила и новых синтаксис языка ассемблера, который называется UAL (Unified Assembler Language). В новом синтаксисе влияние команды на обновление флагов состояния N, C, Z, V в регистре APRS определяется суффиксом S в конце мнемоники операции. Например:

ADD R0,R0,R1 ; R0 = R0 + R1 без изменения флагов

ADD S R0,R0,R1 ; R0 = R0 + R1 с изменением флагов

Состояние этих флагов используется в командах условного перехода и условного выполнения для определения условий выполнения и указываются в виде соответствующих суффиксов, перечисленных в таб.26.

| Суффиксы | Условие                      | Комбинация флагов                          |
|----------|------------------------------|--------------------------------------------|
| EQ       | Равно                        | Z=1                                        |
| NE       | Не равно                     | Z=0                                        |
| CS/HS    | Беззнаковое больше или равно | C=1                                        |
| CC/LO    | Беззнаковое меньше           | C=0                                        |
| MI       | Меньше нуля                  | N=1                                        |
| PL       | Больше или равно нулю        | N=0                                        |
| VS       | Переполнение                 | V=1                                        |
| VC       | Нет переполнения             | V=0                                        |
| HI       | Беззнаковое больше           | C=1 и Z=0                                  |
| LS       | Беззнаковое меньше или равно | C=0 и Z=1                                  |
| GE       | Знаковое больше или равно    | (N=1 и V=1) или<br>(N=0 и V=0)             |
| LT       | Знаковое меньше              | (N=1 и V=0) или<br>(N=0 и V=1)             |
| GT       | Знаковое больше              | (Z=0 и N=1 и V=1) или<br>(Z=0 и N=0 и V=0) |
| LE       | Знаковое меньше или равно    | Z=1 или<br>(N=1 и V=0) или<br>(N=0 и V=1)  |
| AL       | Всегда                       | любая                                      |

Таб.26

Система команд Thumb-2 включает в себя 16-разрядные и 32-разрядные команды. Зачастую, одно и то же действие можно выполнить с помощью команд разной разрядности. Для явного указания разрядности команды используются суффиксы: .N – для 16-разрядной команды и .W – для 32-разрядной команды. Например:

ADDS.N R0,#1 ; R0 = R0 + 1 с изменением флагов (команда 16 бит)  
ADDS.W R0,#1 ; R0 = R0 + 1 с изменением флагов (команда 32 бита)

## 11.2 Используемые методы адресации.

Система команд Thumb-2 поддерживает следующие режимы адресации:

**Регистровая адресация.** Операнд расположен в одном из регистров общего назначения. Содержимое регистра после считывания и перед использованием может быть сдвинуто заданным образом. Количество битов сдвига указывается либо непосредственно (1 – 32 бита), либо через косвенный регистр.

Примеры.

ADD R3, R2, R1 ; R3 = R2 + R1  
SUB R4,R5,R9,LSR #2 ; R4 = R5 - (R9 >> 2)  
ORR R0, R10, R2, ROR R7 ; R0 = R10 OR (R2 ROR R7)

**Непосредственная адресация.** Значение операнда расположено непосредственно в команде. В качестве операнда может быть использована любая константа, которая может быть получена из 8-битного числа сдвигом влево на фиксированное число разрядов в пределах 32 бит или мультипликацией подряд или через байт (0xABABABAB, 0xAB00AB00, 0x00AB00AB).

Пример.

SUB R3,R2,#960 ; R3 = R2 – 960 (60 << 4)

Для загрузки 8-битной константы в регистр используется 16-битная команда.

Пример.

MOVS R0,#55

Для загрузки 16-битной константы в регистр используется 32-битная команда.

Пример.

MOVW.W R0, #0x1234 ; R0 = 0x1234

Загрузка 32-битной константы реализуется с помощью двух 32-битных команд.

Пример.

MOVW.W R1, #1234 ; R0 = 0x1234  
MOVT.W R1, #5678 ; R0H = 0x5678, R0 = 0x56781234

Ещё один способ загрузки 32-битной константы в регистр заключается в использовании псевдокоманд LDR или ADR. В этом случае ассемблер размещает константу в памяти программ в специальном «пуле литералов» и использует команду загрузки регистра с использованием косвенной адресации со смещением относительно программного счётчика PC. Пул литералов должен отстоять не дальше, чем 4096 от этой команды.

Пример.

LDR R3, =0x12345678

Если загружаемая константа является адресом в области памяти программ, то в ней младший бит будет автоматически установлен в единицу, что соответствует режиму Thumb. В остальных случаях константа останется без изменения.

Пример.

LDR r4, = address1 ; R4 = (address1 | 0x1)

...

address1: MOV R0, R1

**Базовая адресация со смещением.** Операнд расположен в ячейке памяти. Базовый адрес операнда расположен в одном из регистров общего назначения. Смещение физического адреса относительно базового либо указано в команде в виде константы, либо содержится в другом регистре общего назначения.

Примеры.

STR R0, [R5, #15] ;  $\text{memory}(\text{R5} + 15) = \text{R0}$   
LDR R1, [R6, -R7] ;  $\text{R1} = \text{memory}(\text{R6} - \text{R7})$   
LDR R2, [R7, R2, LSL #2] ;  $\text{R2} = \text{memory}(\text{R7} + (\text{R2} \ll 2))$   
LDR R3, [PC, #40] ;  $\text{R3} = \text{memory}(\text{PC} + 40)$

### 11.3 Команды пересылки данных.

Для пересылки данных между регистрами общего назначения используются команды MOV и MVN. Последняя побитно инвертирует пересылаемые данные.

Примеры.

MOV R0, R7 ;  $\text{R0} = \text{R7}$   
MVN R6, R4 ;  $\text{R6} = \text{not R4}$   
MOVS R0, #55  
MOVW.W R0, #0x1234 ;  $\text{R0} = 0x1234$   
MOVW.W R1, #1234 ;  $\text{R0} = 0x1234$   
MOVT.W R1, #5678 ;  $\text{R0H} = 0x5678, \text{R0} = 0x56781234$

Для пересылки данных между регистром общего назначения и регистром специального назначения XPSR, CONTROL, PRIMASK, FAULTMASK, BASEPRI используются команды MRS и MSR.

Примеры.

MRS R0, PSR ;  $\text{R0} = \text{XPSR}$   
MSR CONTROL, R2 ;  $\text{CONTROL} = \text{R2}$

Операции со стеком осуществляются с помощью команд PUSH и POP.

Примеры.

PUSH {R0, R1, R2 – R4, LR} ; save context and return address  
POP {R0, R1, R2 – R4, PC} ; context restore and return

Загрузка данных из памяти в регистр общего назначения осуществляется командой LDR и её модификациями. Обратная задача – сохранение данных из регистра общего назначения в памяти выполняется командой STR и её модификациями.

Примеры.

LDR R7, [R9] ;  $\text{R7} = \text{mem}(\text{R9})$   
STR R0, [R1, -R7] ;  $\text{mem}(\text{R1} - \text{R7}) = \text{R0}$   
LDR.W R8, [R1, #100] ;  $\text{R8} = \text{mem}(\text{R1} + 100)$   
LDRNE R1, [R0, #30]! ; if (NE) { $\text{R1} = \text{mem}(\text{R0} + 30)$ ,  $\text{R0} = \text{R0} + 30$ }  
STRB.W R2, [R5], #10 ;  $\text{mem}(\text{R5}) = \text{R2}$ ,  $\text{R5} = \text{R5} + 10$   
LDRB.W R0, [R7] ;  $\text{R0} = \text{membyte}(\text{R7}) \& 0x000000ff$   
LDRSH.W R0, [R5] ;  $\text{R0} = \text{mem}(\text{R5}) \& 0xSSSSffff$ , S – sign  
LDRD.W R0, R1, [R2, #40]! ;  $\text{R0} = \text{mem}(\text{R2} + 40)$ ,  $\text{R1} = \text{mem}(\text{R2} + 44)$ ,  $\text{R2} = \text{R2} + 40$

Для загрузки и сохранения сразу нескольких регистров общего назначения используются команды групповой загрузки и сохранения.

Примеры.

LDMIA.W R7, {R1-R3, R10, R11} ;  $\text{R1-R3, R10, R11} = \text{mem}(\text{R7} + i*4)$   
STMIA.W R0!, {R2, R3, R4-R7} ;  $\text{mem}(\text{R0} + i*4) = \text{R2, R3, R4-R7}$ ,  $\text{R0} = \text{R0} + 24$   
LDMDB.W R7, {R1-R3, R10, R11} ;  $\text{R1-R3, R10, R11} = \text{mem}(\text{R7} - (i+1)*4)$ , Decrement before  
STMDB.W R0!, {R2, R3, R4-R7} ;  $\text{mem}(\text{R0} + (i+1)*4) = \text{R2, R3, R4-R7}$ ,  $\text{R0} = \text{R0} - 24$ , Decrement before

### 11.4 Команды монопольного доступа.

Регулирование доступа к разделяемым ресурсам осуществляется обычно с помощью т.н. «семафоров». Если у семафора имеется только один «эксклюзивный» владелец, то такой семафор называется мьютексом (Mutual Exclusion – взаимное исключение). Для

реализации семафоров и мьютексов в рассматриваемых ядрах используются команды монопольного доступа.

Для разделяемого ресурса формируется флаг блокировки, хранящийся в определённой ячейке памяти. При обращении к этому ресурсу процесс должен вначале проверить статус блокировки ресурса с помощью чтения данного флага. Если ресурс окажется заблокированным, то этот процесс не имеет права сейчас обращаться к нему и должен ждать его освобождения. Если же ресурс окажется свободным, то данный процесс должен его заблокировать перед началом непосредственного обращения к нему. Это делается с помощью записи блокирующего значения во флаг блокировки. Но, поскольку это делается отдельной командой, то есть вероятность, что в промежутке времени между чтением и записью флага блокировки другой процесс (например процедура обработки прерывания) уже успел заблокировать этот ресурс раньше. В результате может возникнуть конфликтная ситуация разделения ресурса. Для исключения возможности подобных конфликтов при обращении к флагу блокировки используются команды монопольного доступа LDREX, STREX. Первая команда аналогична обычной команде LDR, загружающей в регистр текущее значение флага блокировки из ячейки памяти. Но, вдобавок она выставляет аппаратный флаг монопольного доступа. Вторая пытается сохранить в этой ячейке новое значение из регистра. Если аппаратный флаг монопольного доступа установлен, то сохранение завершается успешно и аппаратный флаг монопольного доступа сбрасывается. Об этом команда сообщает записью нуля в регистр статуса операции, указанный в ней. Если окажется, что аппаратный флаг монопольного доступа сброшен, то данные не сохраняются в ячейке и в регистр статуса будет записана единица, что сигнализирует об аварийном завершении операции. Если в промежутке времени между монопольным чтением и монопольной записью другой процесс успеет заблокировать устройство, то при выполнении команды монопольной записи флаг монопольного доступа окажется сброшенным и команда монопольной записи завершится аварийно. Это будет означать, что данный процесс должен подождать до освобождения устройства. В частности, этот флаг сбрасывается аппаратно в начале обработки любого исключения. Пример процедуры блокировки устройства представлен ниже.

LockDevice\_Func:

; Parameter: R0 – Mutex Address.

PUSH {R1, R2, LR} ; Save Context and Return Value

TryToLock:

LDREX R1,[R0] ; Exclusive Read of Mutex to R2

CMP R1,#0 ; Is Device Locked?

BNE TryToLock; Device is Locked

DeviceIsNotLocked:

MOV R1,#1 ; R0 = Mutex Value

STREX R2,R1,[R0] ; Attempt of Exclusive Write of Mutex

CMP R2, #0 ; Success?

BNE TryToLock; Failure

LockDeviceASucceed:

DMB ; Data Memory Barrier

POP {R1, R2, PC} ; Return

Для принудительного сброса аппаратного флага монопольного доступа используется команда CLREX. Она может быть использована если какой-либо процесс начал операцию монопольного доступа командой LDREX, но решил её на завершить.

## 11.5 Команды барьерной синхронизации.

Наличие конвейера команд и сложная структура системы памяти привели к необходимости в некоторых случаях осуществлять синхронизацию работы процессора и памяти для исключения конфликтов данных.

Команда DMB (Data Memory Barrier) предписывает завершить все операции с памятью должны быть завершены до нового обращения к памяти. В частности, эта команда должна выполняться в конце блокировки устройства при реализации монопольного доступа.

Команда DSB (Data Synchronization Barrier) обеспечивает завершение всех операций с памятью до выполнения следующей команды. Такая синхронизация необходима если результат следующих команд зависит от содержимого памяти, модифицированной непосредственно перед ними. В частности, это касается программной модификации таблицы векторов прерываний.

Команда ISB (Instruction Synchronization Barrier) приводит завершению всех предыдущих команд и к очищению конвейера команд до выполнения следующей команды. Данную команду рекомендуется вставлять после каждой модификации содержимого регистра CONTROL. Также, это может потребоваться при реализации самомодифицирующегося кода.

## 11.6 Команды обработки данных.

Все команды обработки данных используют операнды, расположенные в регистрах общего назначения или константы. Результат операции тоже помещается в регистр общего назначения. Ниже приводятся примеры различных команд обработки данных.

### Команды сложения.

ADD.W R0, R1, R2 ;  $R0 = R1 + R2$ , Без установки флагов.

ADDCS.W R0, R1, #25 ;  $R0 = R1 + 25 + C$ , C установкой флагов.

### Команды вычитания.

SUBS.W R2, R1, #12 ;  $R2 = R1 - 12$ , C установкой флагов.

SBC.W R3, R5, R0 ;  $R3 = R5 - R0 - \text{Borrow}$  (Borrow - инверсия C)

RSB.W R7, R4, #77 ;  $R7 = 77 - R4$  (Обратное вычитание)

### Команды сравнения.

CMP.W R1, R8 ; Сравнение R1 и R8 с установкой флагов

CMP.W R0, R7, LSL #2 ; Сравнение R0 и R7, сдвинутого на 2 разряда влево

CMN R0, R1 ; Вычисляет  $R0 - (-R1)$  и обновляет флаги

CMN R0, #0x12 ; Вычисляет  $R0 - (-0x12)$  и обновляет флаги

TST R0, R1 ; Вычисляет  $R0 \text{ AND } R1$  и обновляет флаги

TST R0, #0x12 ; Вычисляет  $R0 \& 0x12$  и обновляет флаги

### Команды знакового и беззнакового умножения.

MUL.W R0, R1, R2 ;  $R0 = 31 \text{ LSB of } R1 * R2$  (одинаково для знаковых и беззнаковых)

UMULL R0, R1, R2, R3 ;  $(R1, R0) = R2 * R3$  (как беззнаковые целые)

SMULL R0, R1, R2, R3 ;  $(R1, R0) = R2 * R3$  (как знаковые целые)

### Команды знакового и беззнакового умножения с накоплением.

MLA.W R1, R2, R5, R7 ;  $R1 = R7 + (31 \text{ LSB of } R1 * R2)$

MLS.W R1, R2, R5, R7 ;  $R1 = R7 - (31 \text{ LSB of } R1 * R2)$

UMLAL R0, R1, R2, R3 ;  $(R1, R0) = (R1, R0) + R2 * R3$  (как беззнаковые целые)

SMLAL R0, R1, R2, R3 ;  $(R1, R0) = (R1, R0) + R2 * R3$  (как знаковые целые)

### Команды знакового и беззнакового деления.

UDIV.W R4, R3, R1 ;  $R4 = R3 / R1$  (как беззнаковые целые)

SDIV.W R3, R2, R1 ;  $R3 = R2 / R1$  (как знаковые целые)

### Команды логических функций.

AND.W R7, R1, #0xff ;  $R7 = R1 \& 0xff$

OR.W R7, R1, R0 ;  $R7 = R1 | R0$

BIC.W R4, R5, #0xf ;  $R4 = R5 \& (\sim 0xf)$  (сброс битов)  
ORN.W R5, R7, R1 ;  $R5 = R7 | (\sim R1)$  (установка битов)  
EOR.W R4, R0, 0xf0 ;  $R4 = R0 \wedge 0xf0$

#### **Команды сдвига.**

ASR.W R0, R0, #4 ;  $R0 = R0 \gg 4$  (сдвиг вправо с расширением знака)  
LSR.W R1, R1, R2 ;  $R0 = R0 \gg R2$  ( 0 – 255) (сдвиг вправо без расширения знака)  
LSL.W R1, R1, #4 ;  $R0 = R0 \ll 4$  (сдвиг влево)  
ROR.W R2, R1, R3 ; R2 = циклический сдвиг R1 на количество бит в R3 (0 – 255 бит)

#### **Команды расширения знака.**

SXTB R0, R1 ; R0 = расширение знака байта R1  
SXTH R0, R1 ; R0 = расширение знака полуслова (15 – 0) R1

#### **Команды перестановки байтов и битов.**

REV R0, R1 ; R0 = обратная перестановка байтов R1  
REV16 R0, R1 ; R0 = обратная перестановка полуслов R1  
RBIT.W R0, R1 ; R0 = обратная перестановка бит R1

#### **Команды операций с битовыми полями.**

BFC.W R4, #4, #8 ; сброс 8 бит, начиная с позиции 4 в R4  
BFI.W R4, R5, #4, #8 ; копирование 8 младших бит R5 в позиции (11 – 4) R4  
CLZ.W R7, R5 ; R7 = количество лидирующих нулей R5 (без знакового бита)

#### **Команды знакового и беззнакового насыщения.**

USAT.W R1, #16, R0 ; R1 = беззнаковое насыщение R0 (предел 0xffff (16 bits))  
SSAT.W R1, #16, R0 ; R1 = беззнаковое насыщение R0 (предел 0x7fff (16 bits))

#### **Дополнительные команды ядер ARM Cortex-M4/M4F.**

В ядрах Cortex-M4/M4F добавлены множественных инструкциях (SIMD – Single Instruction Multiple Data) и дополнительные DSP (Digital Signal Processing) инструкции для упрощения реализации алгоритмов цифровой обработки сигналов. Ниже приводятся примеры подобных команд.

#### **Команды параллельного сложения и вычитания.**

SADD8 R4, R0, R5  
Побайтное сложение со знаком 4-х байт из R0 с соответствующими байтами из R5. Байтовые суммы размещаются в соответствующих байтах R4  
SHSUB16 R2, R1, R0  
Побайтное вычитание со знаком 2-х полуслов из R0 из соответствующих полуслов из R1. Суммы делятся на 2 и размещаются в соответствующих полусловах R2.  
SASX R0, R4, R5  
Добавляет к старшему полуслову в R4 младшее полуслово в R5 и записывает результат в старшее полуслово регистра R0. Вычитает из младшего полуслова в регистре R4 старшее полуслово в регистре R5 и сохраняет результат в младшем полуслове регистра R0.  
USUB8 R4, R1, R2  
Параллельное вычитание 4-х пар байт без знака (из байтов в регистре R1 байтов в регистре R2). Установка флагов GE[3:0].  
SEL r4, r1, r2  
Загрузка в регистр R4 максимальных значений из каждой пары байт в соответствии с флагами GE[3:0].  
SEL r5, r2, r1  
Загрузка в регистр R5 минимальных значений из каждой пары байт в соответствии с флагами GE[3:0].  
USADA8 R4, R0, R5, R2  
Вычитает из байтов первого операнда R0 байты второго операнда R5, получая четыре разности. Суммирует разности по модулю и добавляет к аккумулятору в регистре R2. Сохраняет результат в регистре R4.

### **Команды параллельного умножения.**

**SMUAD R0, R4, R5**

Двойное умножение знаковых полуслов исходных операндов в регистрах R4 и R5 младшего на младшее (bottom x bottom) старшего на старшее (top x top). Сложение двух 32-разрядных произведений. Сохранение результата в регистре R0.

**SMUSDx R4, R5, R3**

Двойное умножение знаковых полуслов исходных операндов в регистрах R5 и R3 перекрестное младшего R5 на старшее R3 (bottom x top) старшего R5 на младшее R3 (top x bottom). Вычитание из первого произведения второго. Сохранение результата в регистре R4.

**SMLAD R10, R2, R1, R5**

Знаковое умножение двух полуслов первого регистра-операнда R2 на знаковые полуслова второго регистра-операнда R1. Добавление обоих произведений к значению в аккумуляторе R5 и сохранение результата в регистре назначения R10.

**SMLADX R0, R2, R4, R6**

Перекрестное умножение старшего полуслова R2 на младшее полуслово R4 и младшего полуслова R2 на старшее полуслово R4. Добавление обоих произведений к R6. Сохранение накопленной суммы в R0.

### **Дополнительные команды цифровой обработки сигналов.**

**SMMLAR R6, R2, R1, R4**

Умножает слова со знаком(32-разряда) в первом R2 и втором R1 операндах, получая 64-разрядное произведение. Округляет результат, добавляя 0x80000000. Выделяет старшие 32 бита произведения (округленные). Добавляет к аккумулятору R4. Сохраняет сумму в регистре назначения R6.

**SMMLSR R3, R6, R2, R7**

Умножает слова со знаком(32-разряда) в первом R6 и втором R2 операндах, получая 64-разрядное произведение. Округляет результат, добавляя 0x80000000. Выделяет старшие 32 бита произведения (округленные). Вычитает из значения в аккумуляторе R7. Сохраняет результат в регистре R3.

### **Команды обработки данных с плавающей точкой ядра ARM Cortex-M4F/M7.**

Ядро ARM Cortex-M4F содержит дополнительно сопроцессор операций над числами с плавающей точкой одинарной точности. Примеры команд этого сопроцессора приведены ниже.

**VABS.F32 S4, S6**

Вычисление абсолютного значения числа в формате с плавающей точкой в регистре S6 и сохранение результата в регистре S4.

**VNEG.F32 S3, S4**

Изменить знак числа в регистре-источнике S4 и сохранить результат в регистре S3.

**VSQRT.F32 S5, S6**

Вычисляет корень квадратный из числа в регистре-источнике S6 и сохраняет результат в регистре-приемнике S5.

**VADD.F32 S4, S6, S7**

Сложение двух чисел с плавающей точкой в регистрах исходных операндов S6 и S7. Сохранение суммы в регистре S4.

**VSUB.F32 S4, S6, S7**

Вычитание двух чисел с плавающей точкой в регистрах исходных операндов S6 и S7. Сохранение разности в регистре S4.

**VMUL.F32 S3, S6, S7**

Умножение двух чисел с плавающей точкой в регистрах исходных операндов S6 и S7. Сохранение произведения в регистре S3.

**VDIV.F32 S3, S10, S11**

Деление двух чисел с плавающей точкой в регистрах исходных операндов S10 и S11. Сохранение частного в регистре назначения S3.



VMLA.F32 S3, S5, S7

Умножение чисел с плавающей точкой в регистрах S5 и S7. Аккумуляция произведения в регистре S3.

VMLS.F32 S2, S10, S11

Умножение чисел с плавающей точкой в регистрах S10 и S11. Вычитание произведения из содержимого регистра S2. Сохранение разности в регистре S2.

VFMA.F32 S3, S5, S7

Умножение чисел с плавающей точкой в регистрах S5 и S7 с параллельным накоплением в регистре S3. Округление накопленной суммы.

VFMS.F32 S2, S10, S11

Умножение чисел с плавающей точкой в регистрах S10 и S11 с параллельным вычитанием из регистра S2. Округление разности.

VCMP.F32 S4, S2

Сравнение двух чисел с плавающей точкой в регистрах S4 и S2.

VCMP.F32 S4, #0.0

Сравнение одного числа с плавающей точкой в регистре S4 с нулем.

VCVT.F32.S32 S5, S7

Преобразование 32-разрядного целого числа со знаком из регистра S7 в число с плавающей точкой однократной точности и сохранение в регистре S5.

VCVT.S32.F32 S3, S8

Преобразование 32-разрядного числа в формате с плавающей точкой из регистра S8 в 32-разрядное целое со знаком и сохранение в регистре S3.

VCVT.F32.S32 S5, S7, #24

Преобразование 32-разрядного числа со знаком в формате с фиксированной точкой (8.24) из регистра S7 в формат с плавающей точкой и сохранение в регистре S5.

VCVT.S32.F32 S3, S5, #16

Преобразование 32-разрядного числа с плавающей точкой из регистра S5 в 32-разрядное число со знаком в формате с фиксированной точкой (16.16) и сохранение в регистре S3.

Сопроцессор ядра ARM Cortex-M7 может выполнять все вышеприведённые команды как с одинарной, так и с двойной точностью. Разница в командах, оперирующих числами с одинарной или с двойной точностью состоит в суффиксе (F32 или F64) и в используемых регистрах (S-регистры или D-регистры). Примеры команд с двойной точностью приведены ниже.

VABS.F64 D0, D2

Вычисление абсолютного значения числа в формате с плавающей точкой в регистре D2 и сохранение результата в регистре D0.

VSQRT.F64 D5, D6

Вычисляет корень квадратный из числа в регистре-источнике D6 и сохраняет результат в регистре-приемнике D5.

VADD.F64 D4, D6, D7

Сложение двух чисел с плавающей точкой в регистрах исходных операндов D6 и D7. Сохранение суммы в регистре D4.

VMUL.F64 D3, D6, D7

Умножение двух чисел с плавающей точкой в регистрах исходных операндов D6 и D7. Сохранение произведения в регистре D3.

VDIV.F64 D3, D10, D11

Деление двух чисел с плавающей точкой в регистрах исходных операндов D10 и D11. Сохранение частного в регистре назначения D3.

VMLA.F64 D3, D5, D7

Умножение чисел с плавающей точкой в регистрах D5 и D7. Аккумуляция произведения в регистре D3.

### **Команды ветвления алгоритма.**

Эти команды предназначены для изменения нормальной последовательности выполнения команд. Команда перехода может сопровождаться логическим условием. Если результат условия при данном выполнении команды определяется как «истинно», то переход выполняется, если – как «ложно», то нет. Примеры команд ветвления приведены ниже.

**B loopA**

Переход по метке loopA. Максимальное смещение  $\pm 16\text{MB}$ .

**BX R0**

Переход по адресу, находящемуся в регистре R0.

**BLE Target**

Переход на метку Target при выполнении условия LE (меньше или равно). Максимальное смещение  $\pm 1\text{MB}$ .

**BEQ.W Target ;** Переход на метку Target при выполнении условия EQ (равно). Максимальное смещение  $\pm 16\text{MB}$ .

**BL Func\_A ;** Вызов функции Func\_A с сохранением адреса возврата в регистре связи LR. Максимальное смещение  $\pm 16\text{MB}$ .

**BLX R0**

Косвенный вызов подпрограммы по адресу, находящемуся в регистре R0, с сохранением адреса возврата в регистре связи LR.

**CBZ R5, Target**

Переход вперед на метку Target, если содержимое R5 равно 0. Регистр исходного операнда может быть только одним из младших регистров от R0 до R7. Переход возможен только вперед на расстояние от 4-х до 130 байт.

**CBNZ R0, target**

Переход вперед на метку Target, если содержимое R5 не равно 0. Регистр исходного операнда может быть только одним из младших регистров от R0 до R7. Переход возможен только вперед на расстояние от 4-х до 130 байт.

**TBB [R0, R1]**

Переход вперед по адресу, определяемому, как сумма текущего содержимого программного счётчика PC и смещения, заданного в элементе таблицы 8-разрядных смещений, с начальным адресом в R0 и номером в R1. Прочитанное в этом элементе таблицы смещение перед суммированием с содержимым PC умножается на два.

**TBN [PC, R1, LSL #1]**

Переход вперед по адресу, определяемому, как сумма текущего содержимого программного счётчика PC и смещения, заданного в элементе таблицы 16-разрядных смещений, расположенной непосредственно за этой командой, с номером в R1. Поскольку каждый элемент таблицы занимает два байта, то индекс умножается на два сдвигом на один разряд влево. Прочитанное в этом элементе таблицы смещение перед суммированием с содержимым PC умножается на два.

### **Условно выполняемые команды.**

В простых случаях условного ветвления алгоритма, когда одна из ветвей требует выполнения всего нескольких команд, вместо действительного ветвления можно использовать т.н. «условно выполняемые» команды. Это облегчает работу конвейера команд и улучшает читаемость кода. Для реализации условного выполнения команд используется т.н. ИТ-блок (IF – THEN). Такой блок может содержать до четырёх условно выполняемых команд. Он начинается с команды ИТ, в которой указывается одно из условий, перечисленных в таб.26, а также количество условно выполняемых команд в блоке. Первая команда после команды ИТ должна обязательно выполняться при истинности указанного условия. Наличие этой команды обязательно. Следующие три команды в блоке не являются обязательными и могут выполняться либо при истинности, либо при ложности указанного условия. Пример ИТ-блока приведён ниже.

|               |                                   |
|---------------|-----------------------------------|
| CMP R1, R2    | ; Если R1 < R2 (меньше),          |
| ITTEE LT      | ; то выполняются команды 1 и 2    |
|               | ; (на это указывает символ T),    |
|               | ; иначе выполняются команды 3 и 4 |
|               | ; (на это указывает символ E)     |
| SUBLT.W R2,R1 | ; 1-я команда                     |
| LSRLT.W R2,#1 | ; 2-я команда                     |
| SUBGE.W R1,R2 | ; 3-я команда (условие GE         |
|               | ; противоположно условию LT)      |
| LSRGE.W R1,#1 | ; 4-я команда                     |

Количество символов Т и Е в мнемонике команды IT должно соответствовать количеству условно выполняемых команд в блоке.

Если во время выполнения IT-блока возникнет запрос на исключение, то текущее состояние блока будет зафиксировано в поле ICI регистра PSR. После завершения обработки исключения выполнение IT-блока будет продолжено. Если запрос на исключение возникнет во время выполнения многотактной условно выполняемой команды (например, команды групповой загрузки или сохранения), то эта команда будет прервана и полностью выполнена заново после завершения обработки исключения.

#### **Вспомогательные команды.**

Кроме описанных классов в систему команд входят инструкции, выполняющие различные сервисные функции. Примеры таких команд приведены ниже.

CPSID i

Запрещение прерываний и конфигурируемых обработчиков ошибок. Команда записывает единицу в регистр PRIMASK.

CPSIE i

Разрешение прерываний и конфигурируемых обработчиков ошибок. Команда записывает ноль в регистр PRIMASK.

CPSID f

Запрещение прерываний и конфигурируемых обработчиков ошибок. Команда записывает единицу в регистр FAULTMASK.

CPSIE f

Разрешение прерываний и конфигурируемых обработчиков ошибок. Команда записывает ноль в регистр FAULTMASK.

BKPT 0x10

Точка останова с параметром 0x10.

SVC 0x32

Запрос обработчика исключения супервизора SVCall.

NOP

Нет операции. Команда не может использоваться для реализации программной задержки. Процессор может удалить ее с конвейера команд, прежде чем она достигнет стадии выполнения. Может использоваться исключительно в качестве шаблона, вместо которого в последующем можете вставить нужную инструкцию.

## **12. Литература.**

1. Джозеф Ю. Ядро Cortex-M3 компании ARM. Полное руководство/ Ю. Джозеф; пер. с англ. А.В. Евстафеева. – М.: Додэка-XXI, 2012. – 552 с. ил. (Мировая электроника).
2. Козаченко В.Ф., Анучин А.С., Алямки Д.И., Жарков А.А., Лашкевич М.М., Савкин Д.И., Шпак Д.М. Практический курс микропроцессорной техники на базе процессорных ядер ARM-Cortex-M3/M4/M4F. – М.: Издательство МЭИ, 2019.
3. ARM®v7-M Architecture Reference Manual, Arm Limited, 2014.

4. ARM v7-M Architecture Application Level Reference Manual, Arm Limited, 2006.
5. ARM Cortex-M4 Processor Revision r0p1 Technical Reference Manual. Arm Limited, 2015.
6. ARM Cortex-M7 Processor Revision r0p2 Technical Reference Manual. Arm Limited, 2014.